查找

查找

顺序查找

二叉排序树

平衡二叉树

B 树与 B+ 树

散列表

查找的基本概念

查找表

是由一组具有相同数据类型的数据元素组成的集合，这些元素通常保存在一张结构化的数据表中，用于支持对其中元素的查找操作。

可以把查找表理解成一个“装着数据的仓库”，就像超市货架那样。你想找某个商品（数据），就要在这张“表”中查一查它在哪儿。而查找表的主要作用，就是为了方便你“找东西”。

关键字

数据元素中能唯一标识该元素的属性值，查找时就是通过关键字来定位对应的数据元素的。

关键字就像“身份证号”或“商品条码”，它是用来区分每个数据的“标签”。比如你要查找学生信息，就得知道学号；要找图书，就要用ISBN；这些就是关键字。

查找

在查找表中根据给定的“关键字”找出对应数据元素的过程。

查找就像“翻书找页码”或者“在通讯录里找某个人的电话”。你给出一个线索（关键字），系统就帮你从表里把它找出来，告诉你在哪。查得快慢、准不准，就是我们后面要学习的重点。

平均查找长度

是指在一种查找方法下，查找一个元素所需要比较的关键字的平均次数，是衡量查找算法效率的一个重要指标。它通常分为两种情况来讨论：查找成功和查找失败。

$$ASL_{success} = \sum_{i=1}^{n} P_i C_i$$

如果表中有 n 个元素，第 i 个元素被查到的概率为 P_i，查找它需要比较 C_i 次关键字

平均查找长度

假设每个记录的查找概率相等 $P_i = 1 / n$

查找第 i 个元素需要比较的次数为 C_i

$$ASL_{success} = \sum_{i=1}^{n} P_i C_i = P_1C_1 + P_2C_2 + \cdots + P_nC_n$$

$$= \frac{1}{n} \times 1 + \frac{1}{n} \times 2 + \cdots + \frac{1}{n} \times n$$

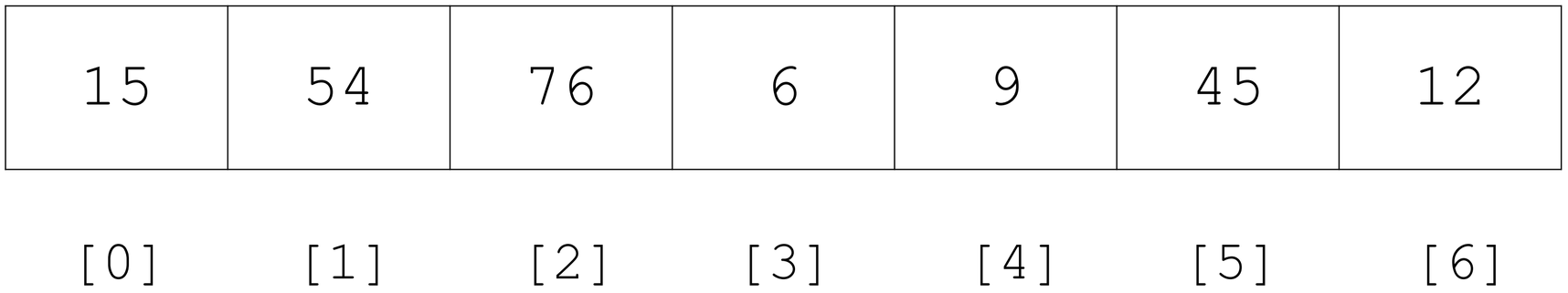
$$= \frac{1}{n}(1 + 2 + 3 + \cdots + n)$$

$$= \frac{1}{n} \times \frac{n(n+1)}{2}$$

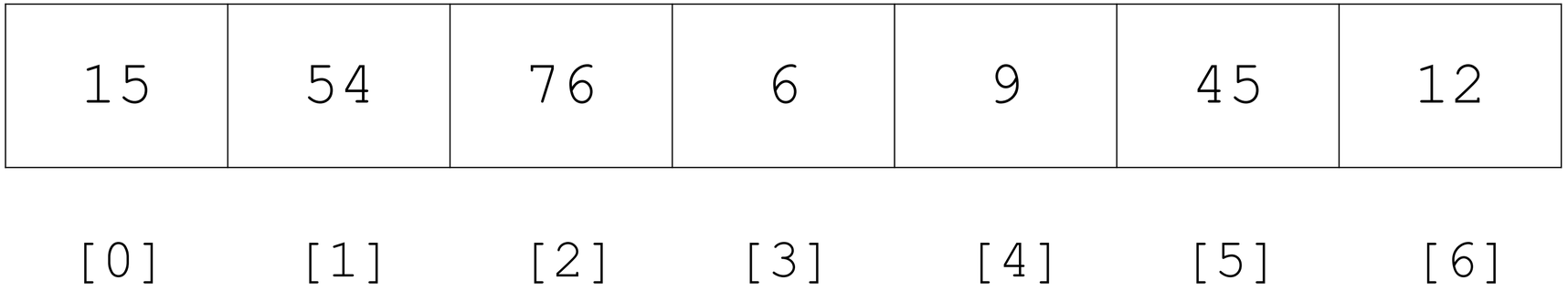
$$= \frac{n+1}{2}$$

$$ASL_{fail} = n$$

线性表的查找-顺序查找

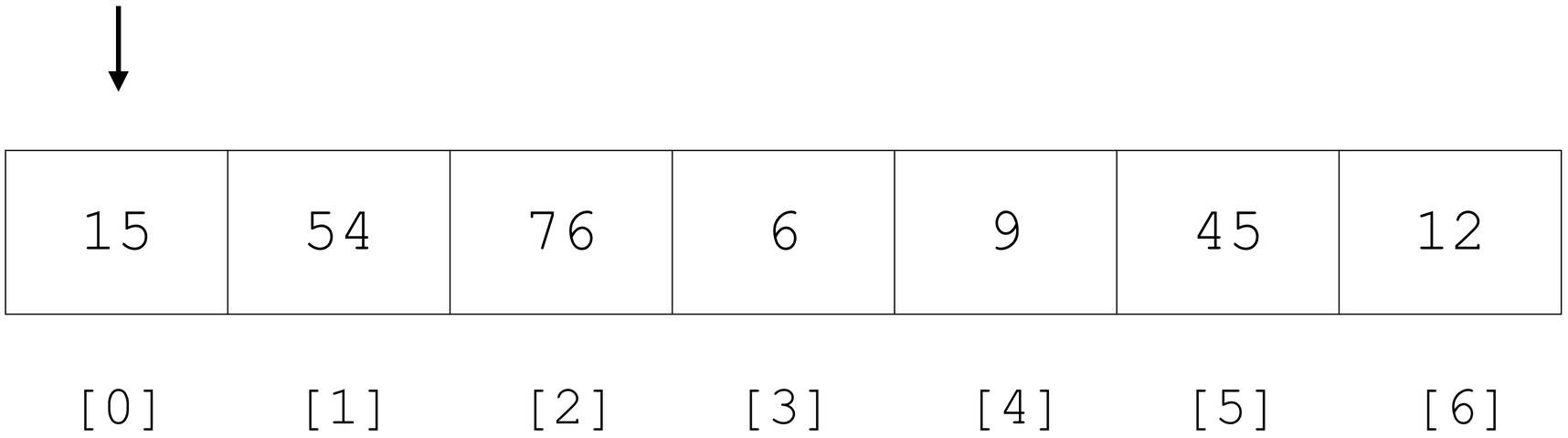


线性表的查找-顺序查找



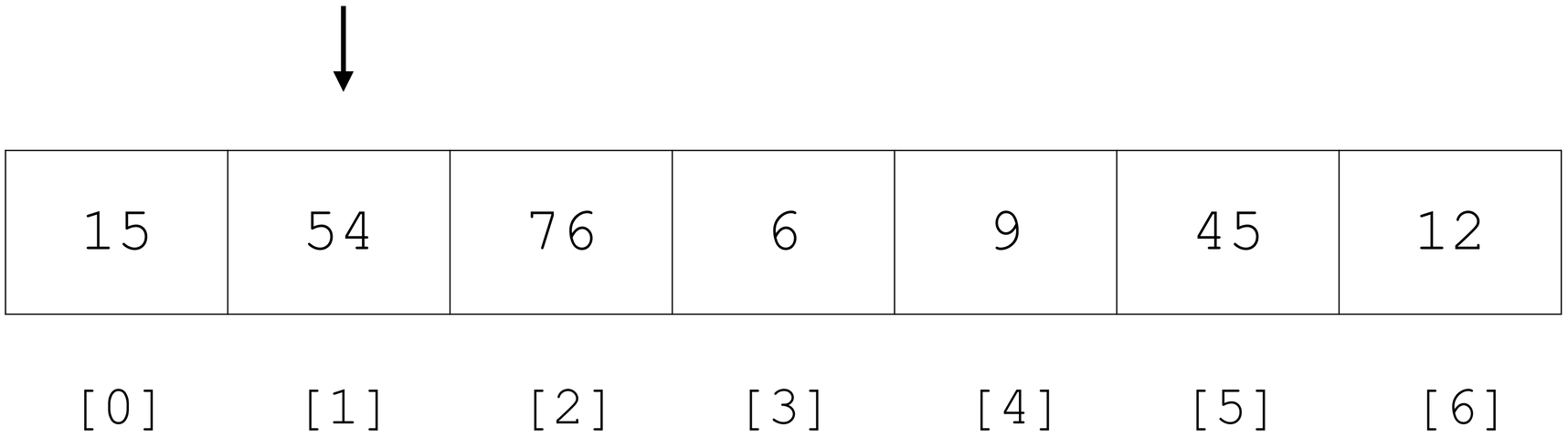
6？

线性表的查找-顺序查找



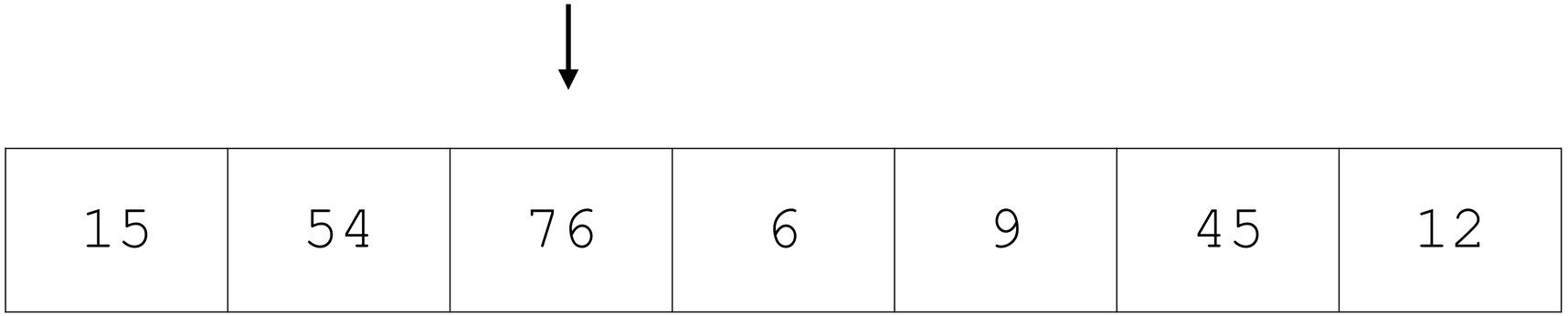
6？

线性表的查找-顺序查找



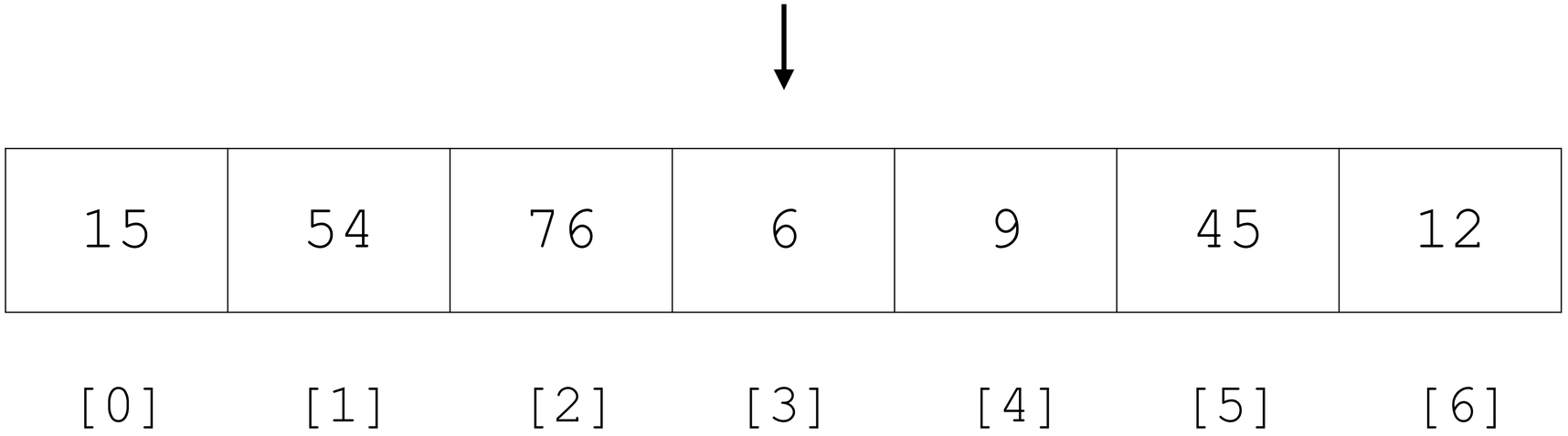
6？

线性表的查找-顺序查找



6？

线性表的查找-顺序查找



示例代码：查找 / test1.c

线性表的查找-折半查找（二分查找）

折半查找的前提是：查找表必须是有序的，这一点非常关键。无序的数据使用折半查找是完全没有意义的。

线性表的查找-折半查找（二分查找）

1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 判断

2	13	22	38	46	58	66	77	85	98
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

线性表的查找-折半查找（二分查找）

1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 判断

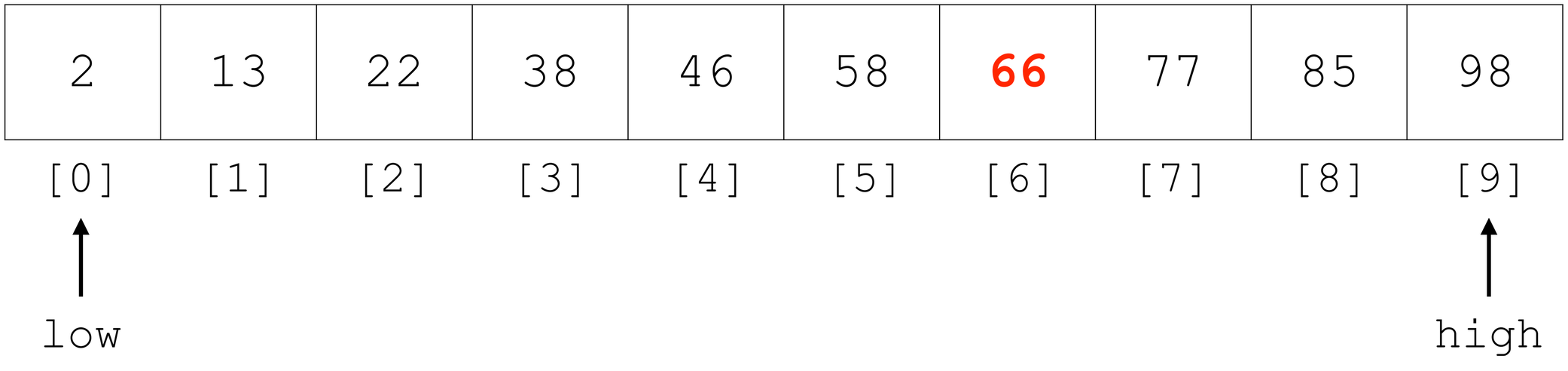
2	13	22	38	46	58	**66**	77	85	98
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

线性表的查找-折半查找（二分查找）

1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 判断

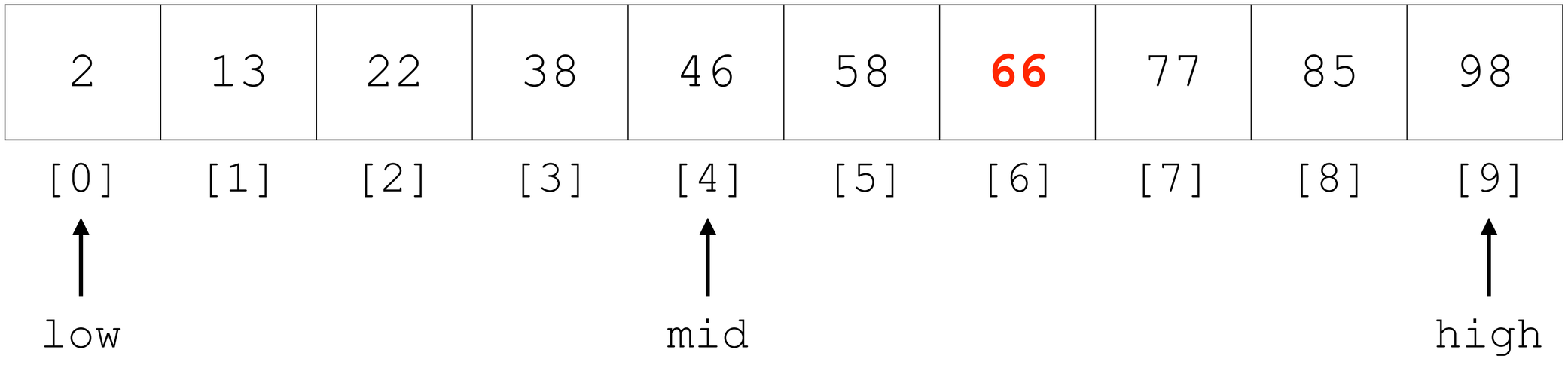


线性表的查找-折半查找（二分查找）

1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 判断



线性表的查找-折半查找（二分查找）

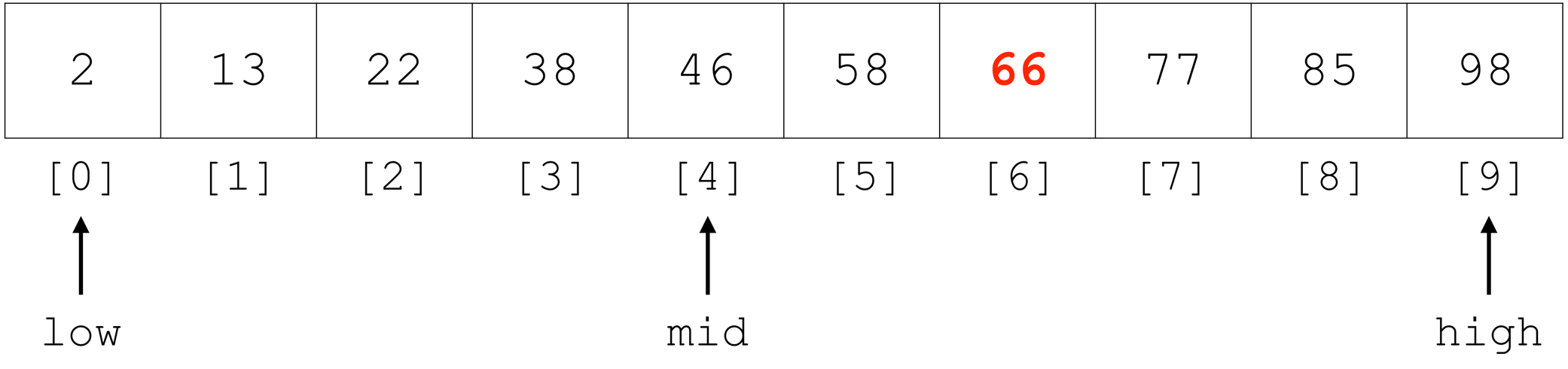
1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 如果关键字等于a[mid]，查找成功，返回下标

3. 如果关键字小于a[mid]，则在左半部分继续查找，令 high = mid - 1

3. 如果关键字大于a[mid]，则在右半部分继续查找，令 low = mid + 1



线性表的查找-折半查找（二分查找）

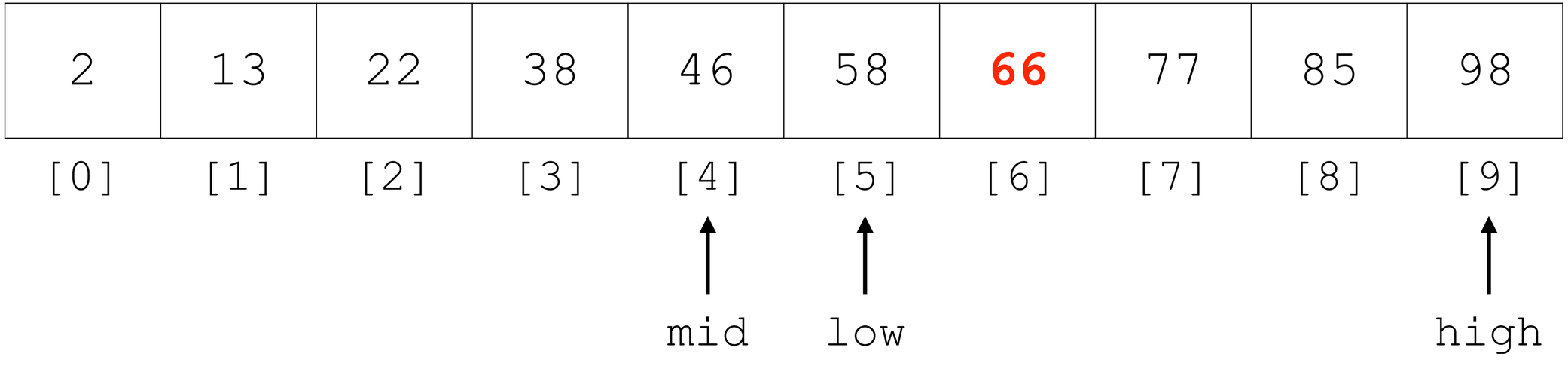
1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 如果关键字等于a[mid]，查找成功，返回下标

3. 如果关键字小于a[mid]，则在左半部分继续查找，令 high = mid - 1

3. 如果关键字大于a[mid]，则在右半部分继续查找，令 low = mid + 1



线性表的查找-折半查找（二分查找）

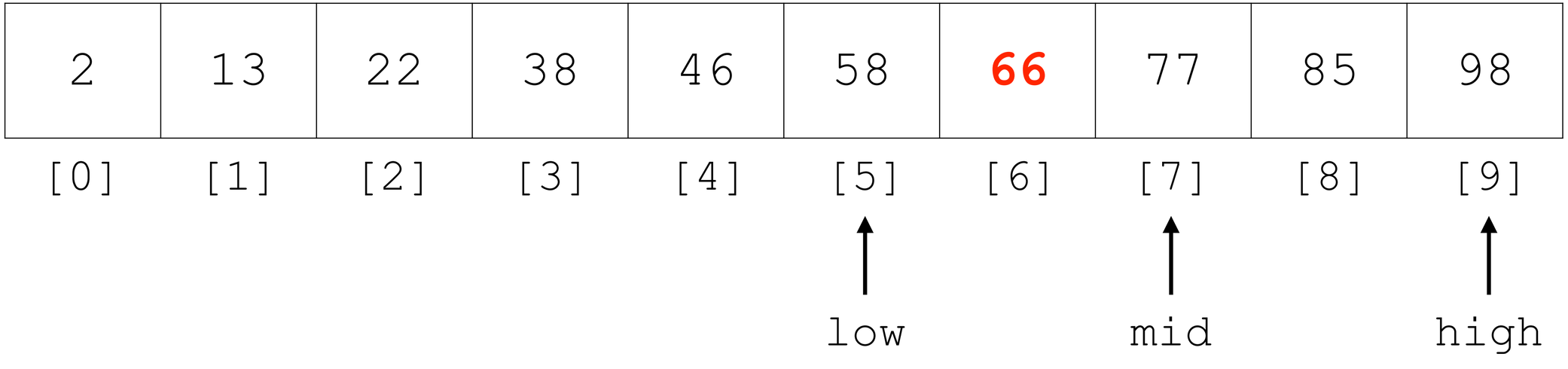
1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 如果关键字等于a[mid]，查找成功，返回下标

3. 如果关键字小于a[mid]，则在左半部分继续查找，令 high = mid - 1

3. 如果关键字大于a[mid]，则在右半部分继续查找，令 low = mid + 1



线性表的查找-折半查找（二分查找）

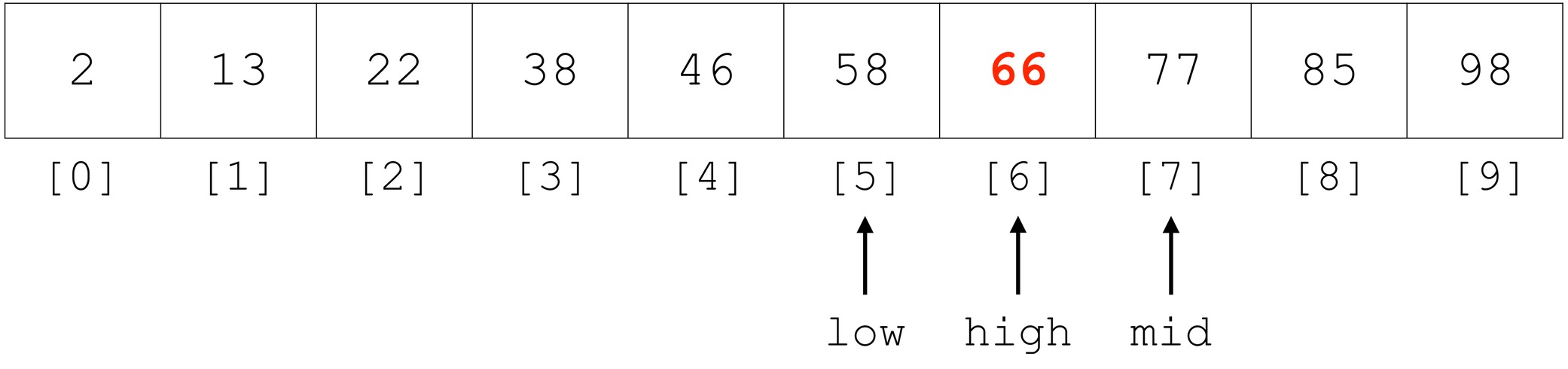
1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 如果关键字等于a[mid]，查找成功，返回下标

3. 如果关键字小于a[mid]，则在左半部分继续查找，令 high = mid - 1

3. 如果关键字大于a[mid]，则在右半部分继续查找，令 low = mid + 1



线性表的查找-折半查找（二分查找）

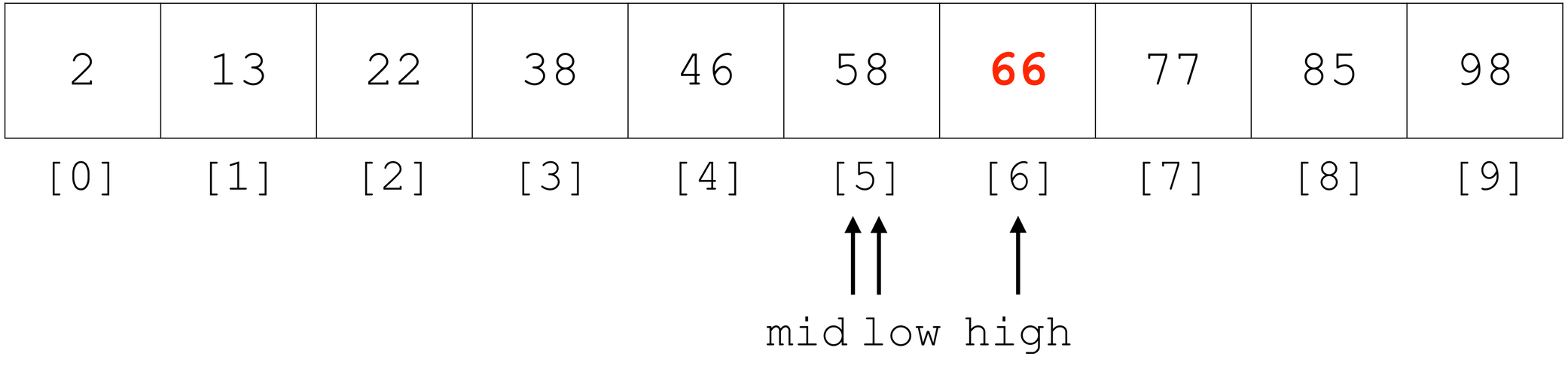
1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 如果关键字等于a[mid]，查找成功，返回下标

3. 如果关键字小于a[mid]，则在左半部分继续查找，令 high = mid - 1

3. 如果关键字大于a[mid]，则在右半部分继续查找，令 low = mid + 1



线性表的查找-折半查找（二分查找）

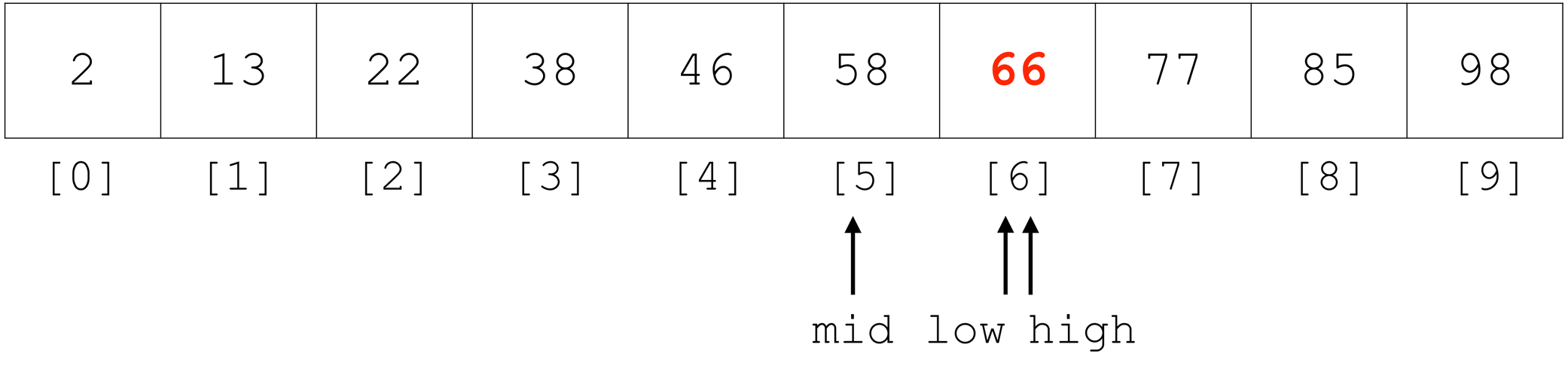
1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 如果关键字等于a[mid]，查找成功，返回下标

3. 如果关键字小于a[mid]，则在左半部分继续查找，令 high = mid - 1

3. 如果关键字大于a[mid]，则在右半部分继续查找，令 low = mid + 1



线性表的查找-折半查找（二分查找） 示例代码：查找 / test2.c

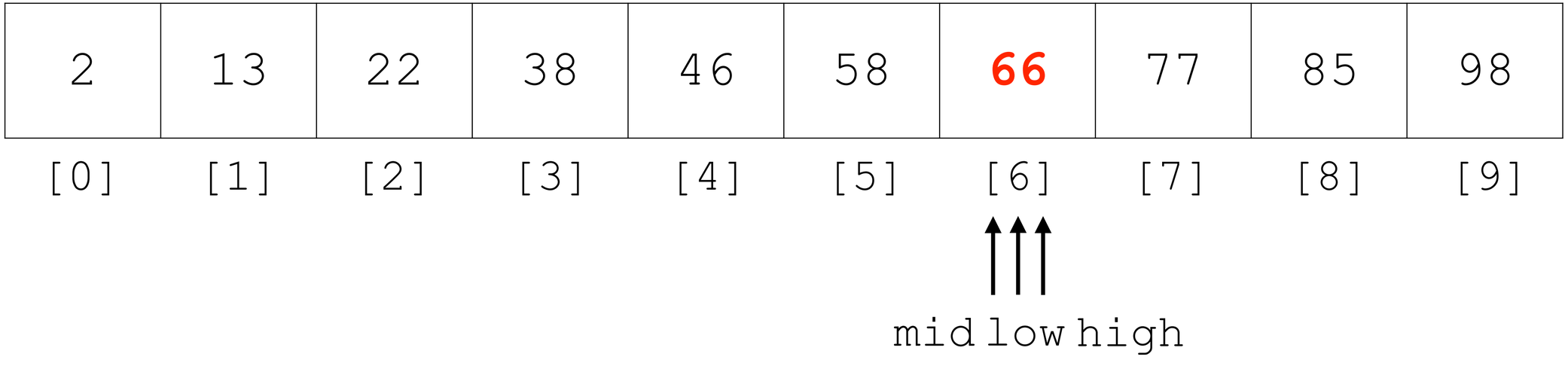
1. 假设查找表的起始下标为low，结束下标为high

2. 计算中间下标：mid = (low + high) / 2（向下取整，例如4.5，取4）

3. 如果关键字等于a[mid]，查找成功，返回下标

3. 如果关键字小于a[mid]，则在左半部分继续查找，令 high = mid - 1

3. 如果关键字大于a[mid]，则在右半部分继续查找，令 low = mid + 1



顺序查找效率较低，而折半查找又依赖于有序的顺序表结构，且插入与删除操作较为麻烦。于是我们开始思考，有没有一种查找结构，既能保持较高的查找效率，又能在动态插入和删除时保持结构的有序性?

二叉排序树

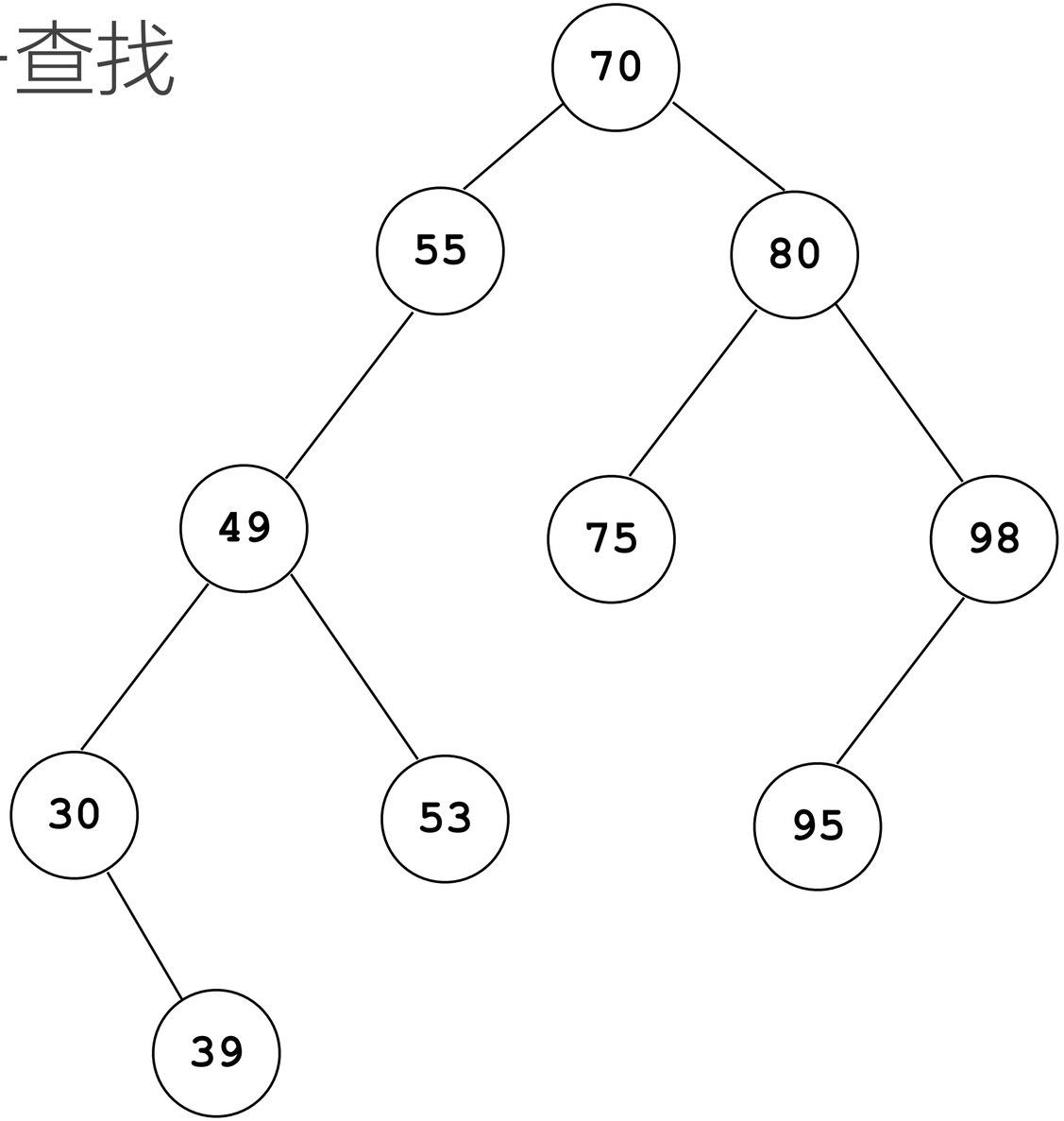
二叉排序树（Binary Search Tree，BST）是一种特殊的二叉树，它通过巧妙的节点排列方式，实现了“查找效率高、插入删除灵活”的目标，它满足以下特性：

1. 对于树中任意一个节点，其左子树上所有节点的值都小于该节点的值

2. 对于树中任意一个节点，其右子树上所有节点的值都大于该节点的值，并且左右子树本身也都是一棵二叉排序树

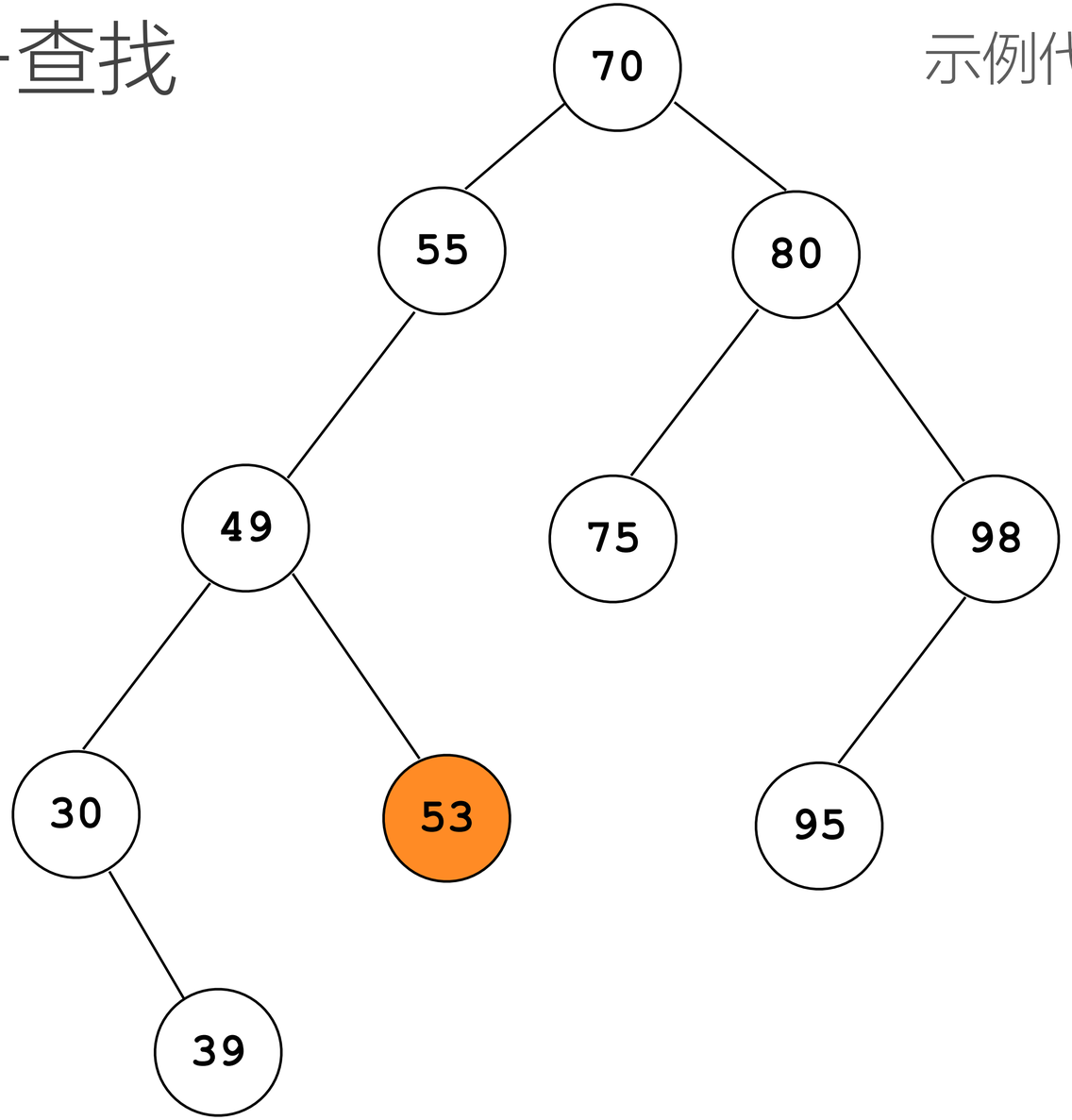
注意：构造二叉排序树的目的，并不是为了排序，而是为了提高查找、插入以及删除关键字的速度。

二叉排序树-查找

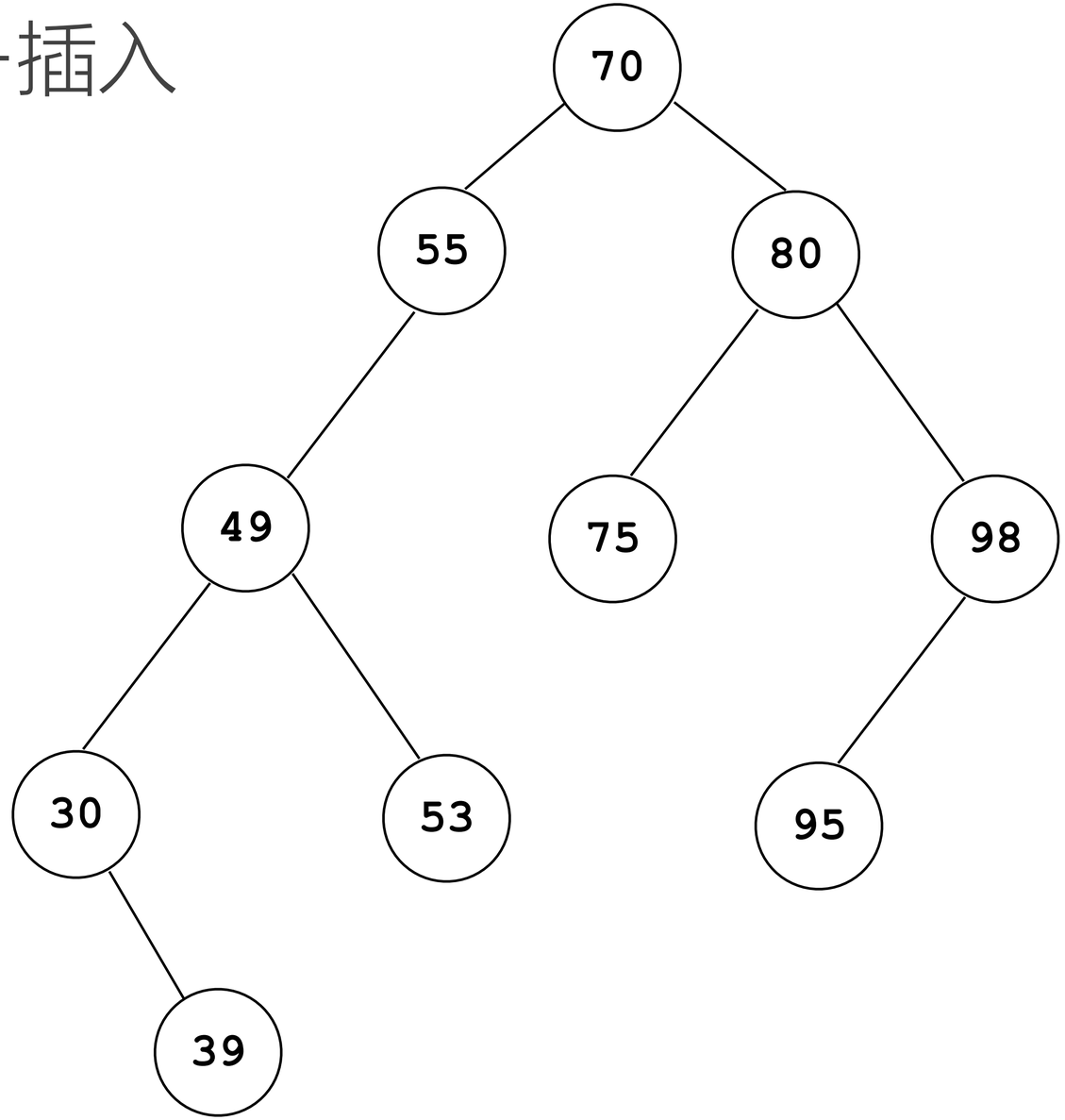


二叉排序树-查找

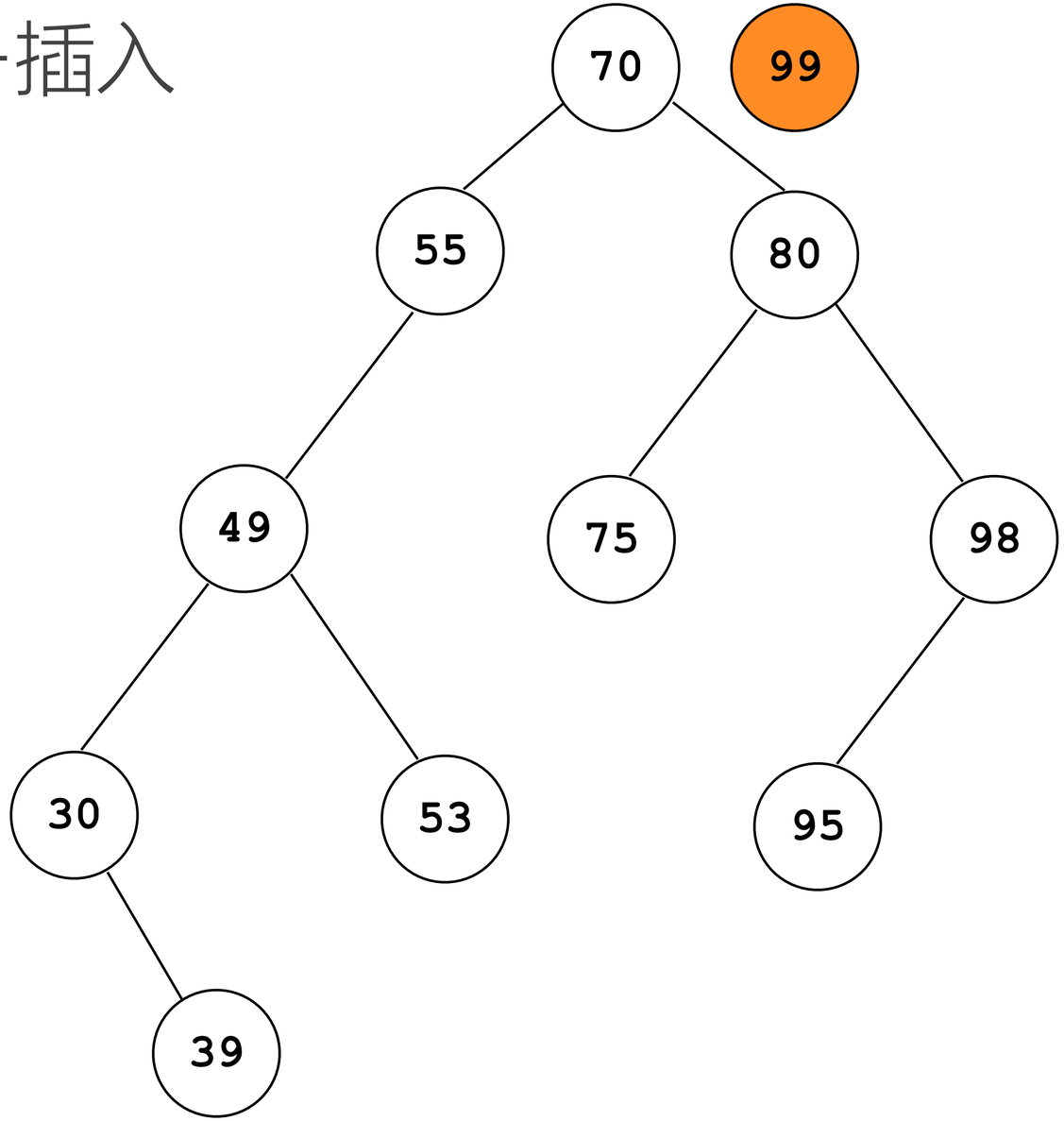
示例代码：查找 / test3.c



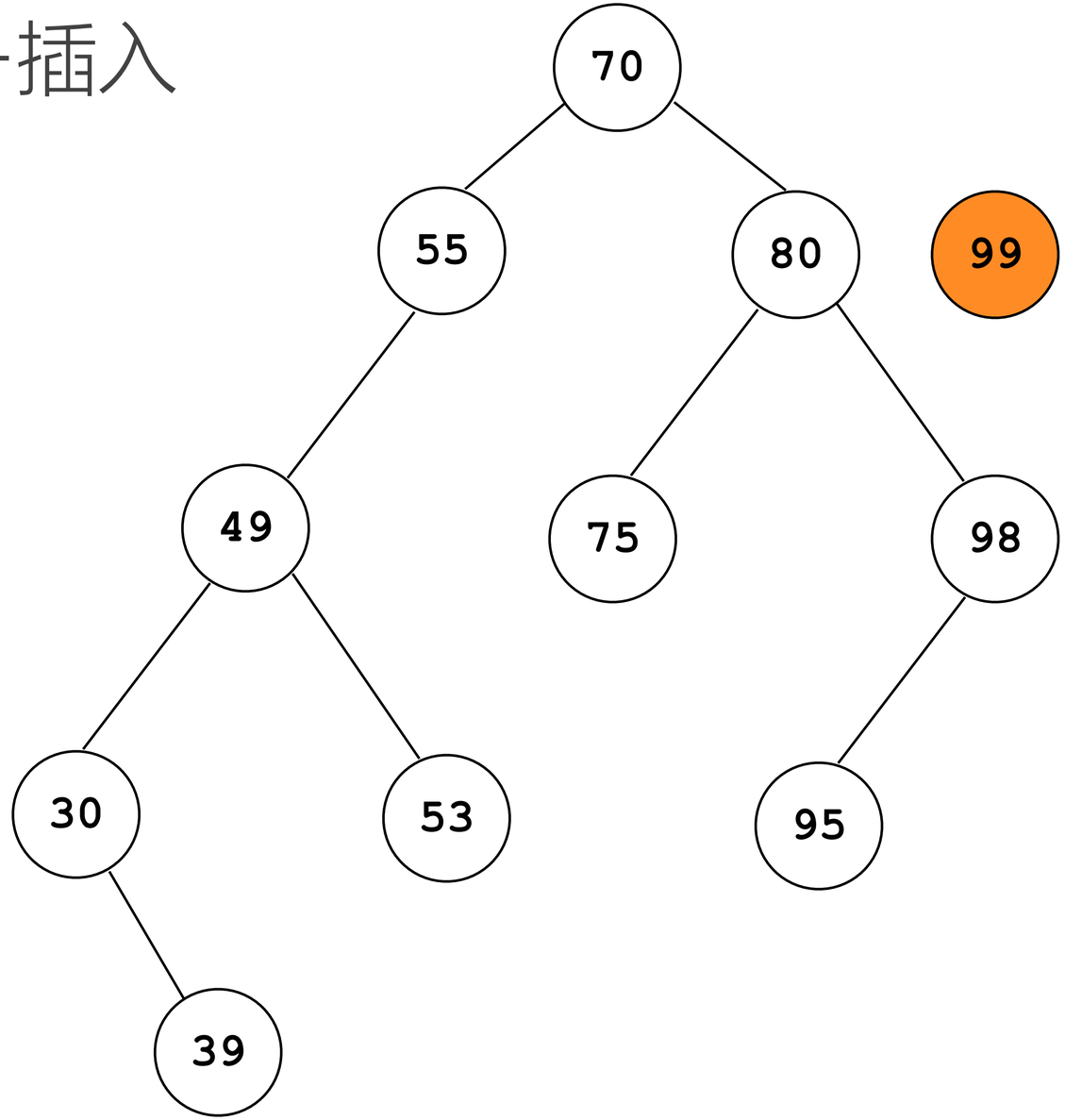
二叉排序树-插入



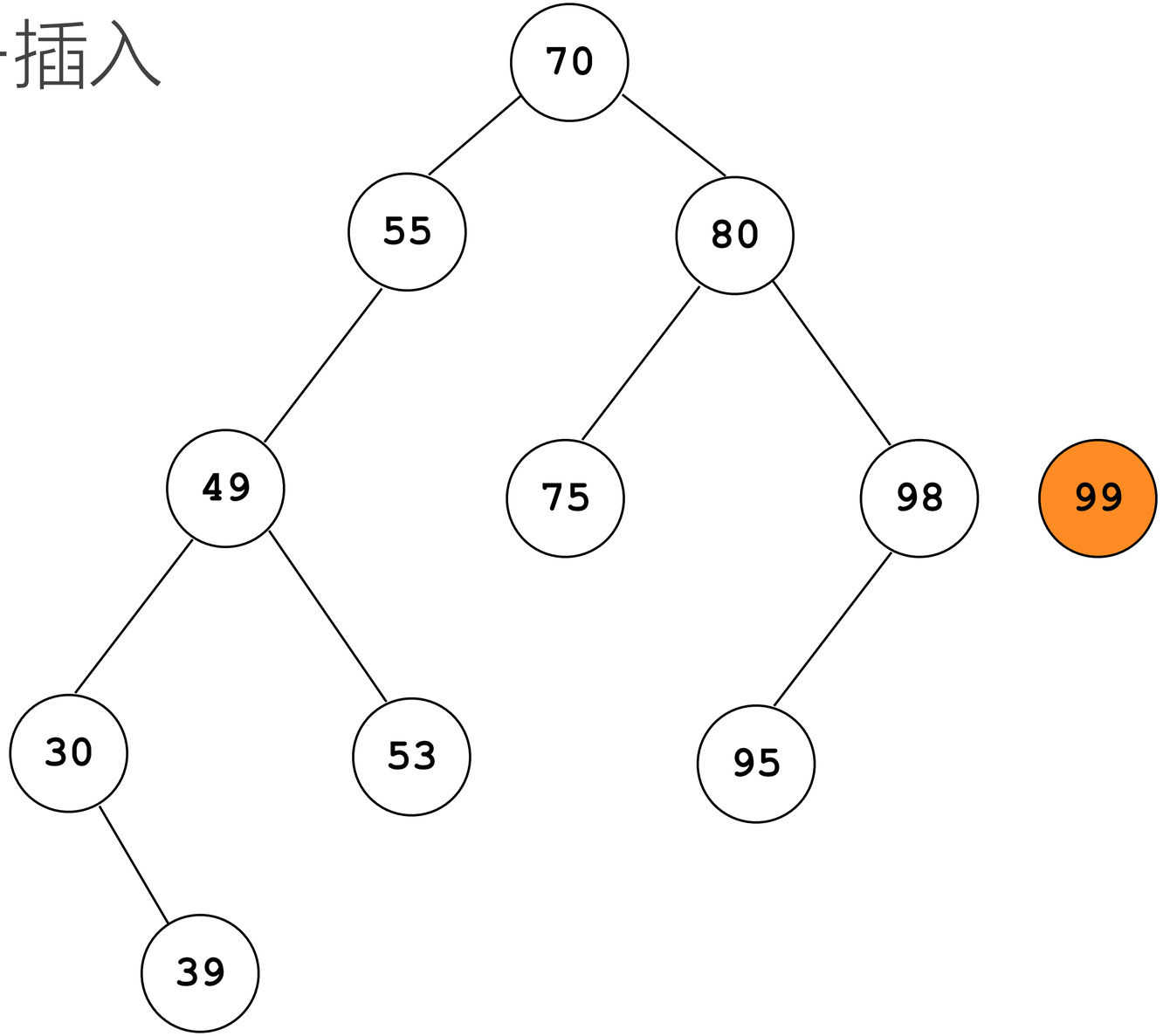
二叉排序树-插入



二叉排序树-插入

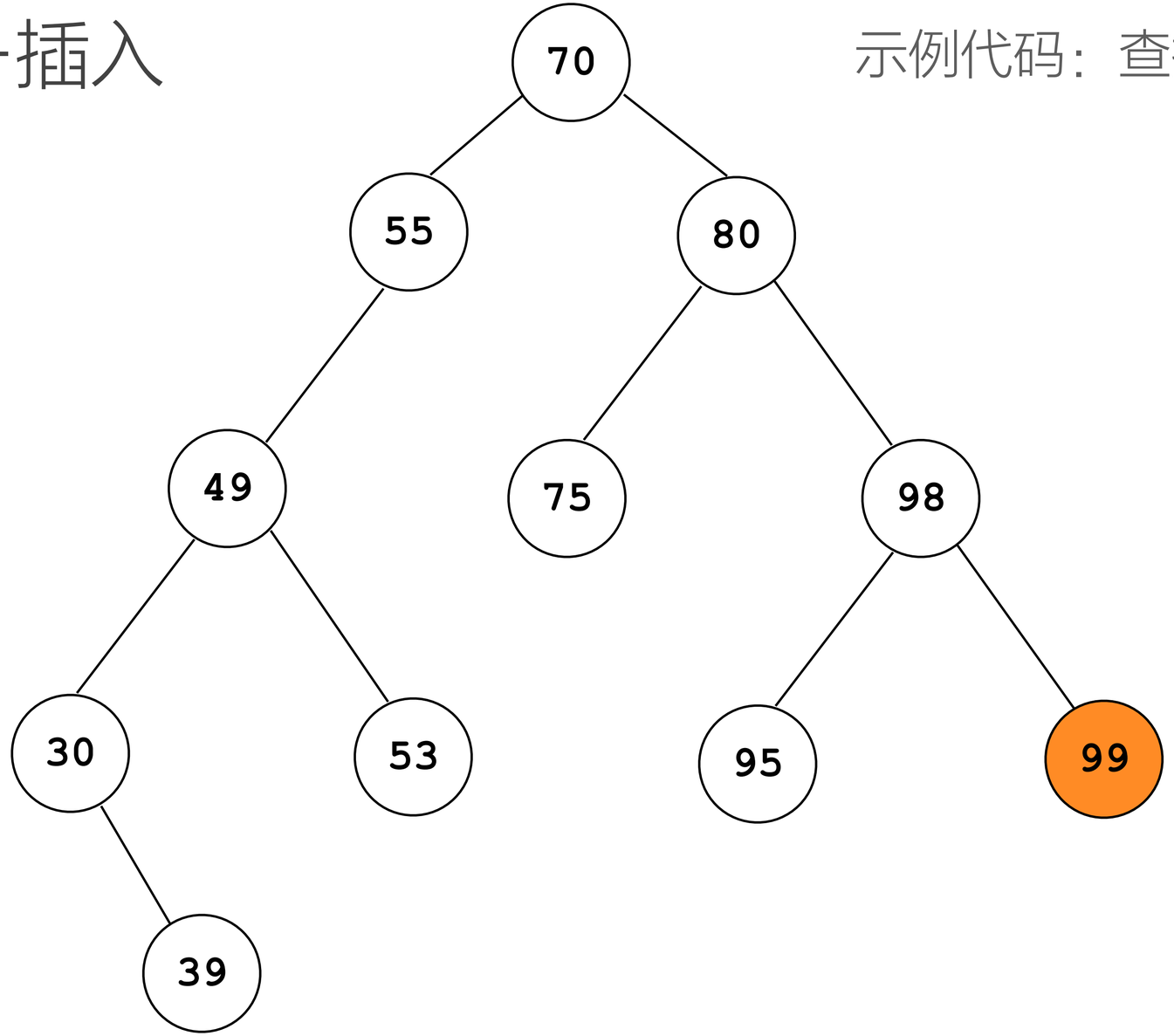


二叉排序树-插入

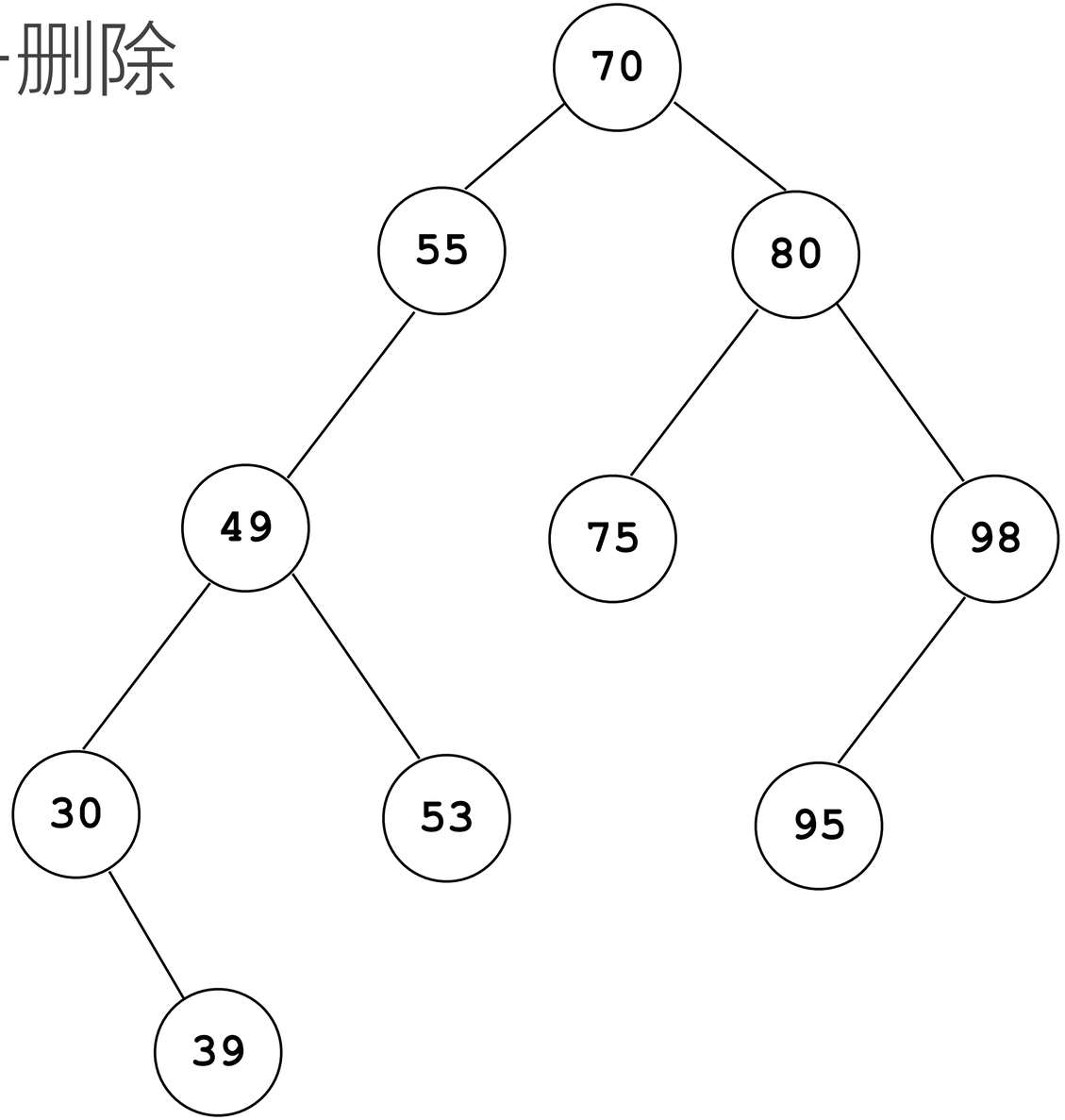


二叉排序树-插入

示例代码：查找 / test4.c

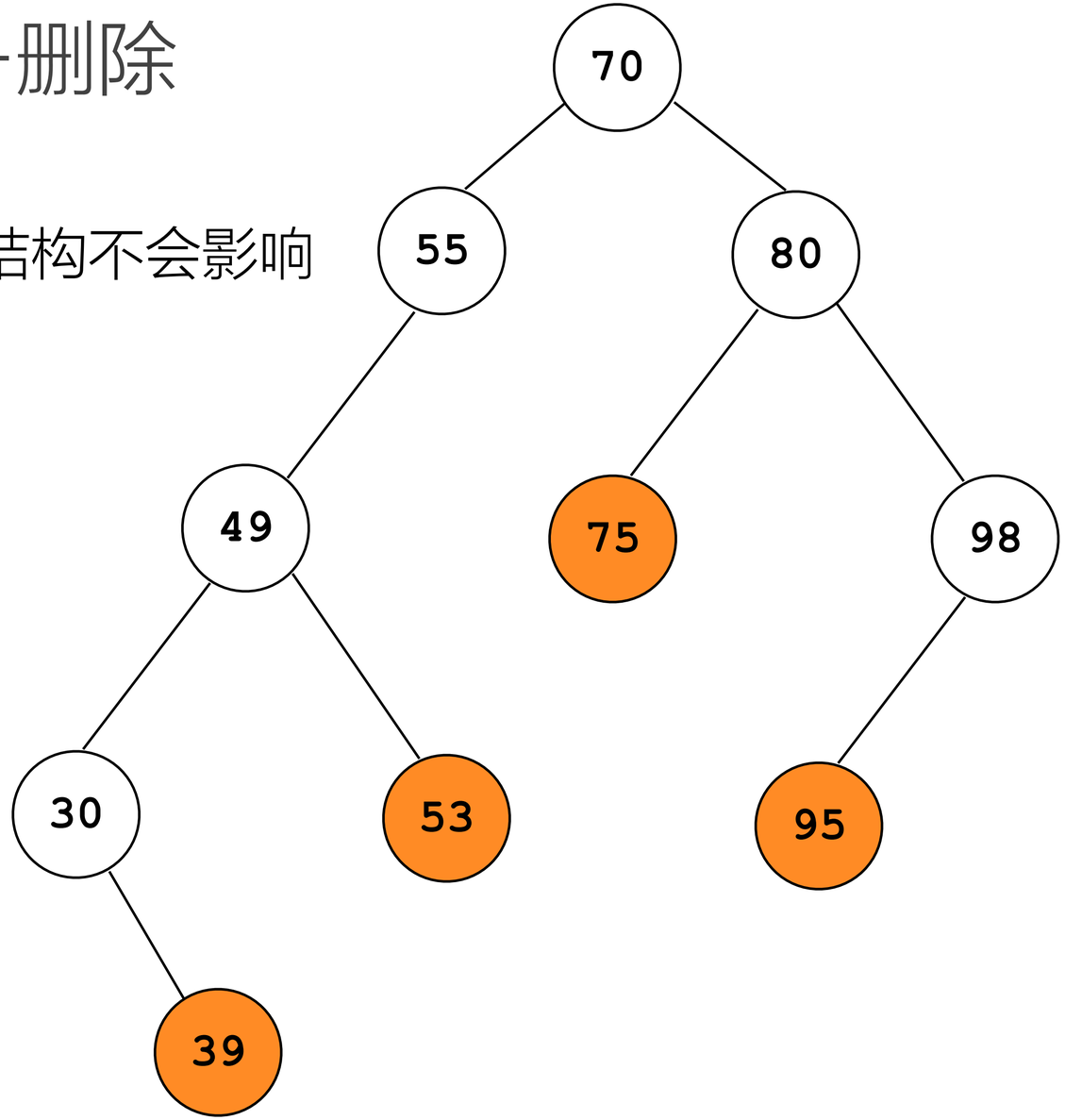


二叉排序树-删除



二叉排序树-删除

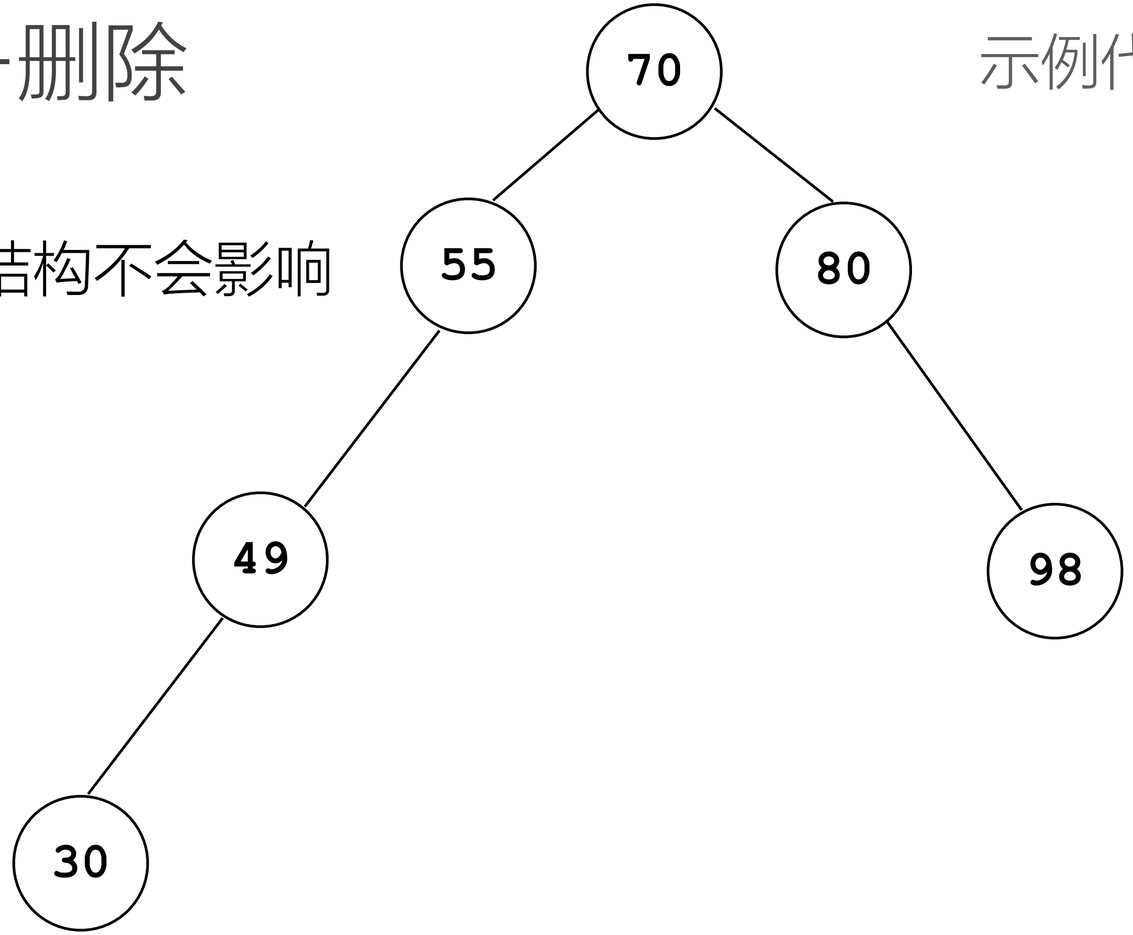
叶子结点直接删，对结构不会影响



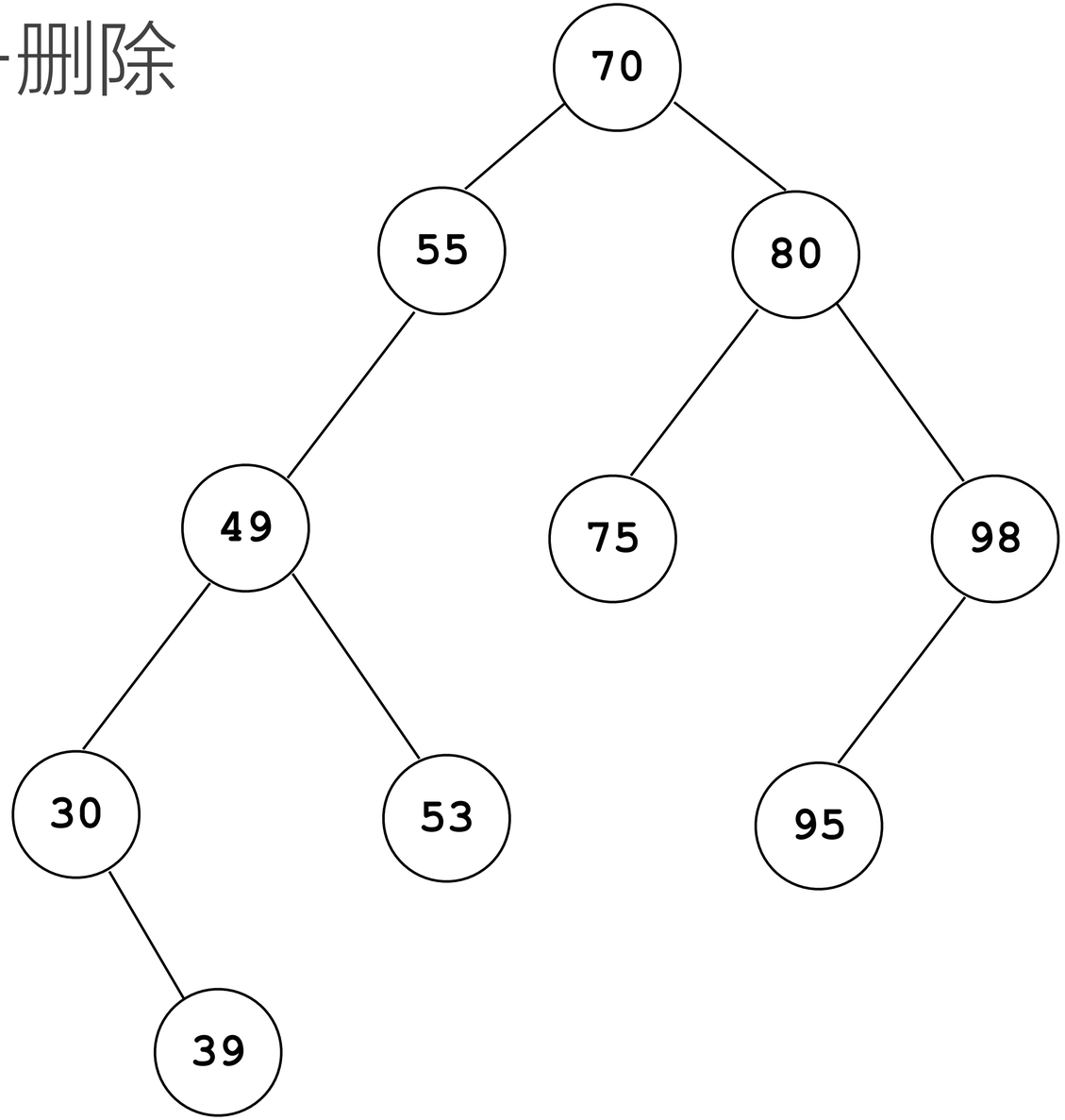
二叉排序树-删除

示例代码：查找 / test5.c

叶子结点直接删，对结构不会影响



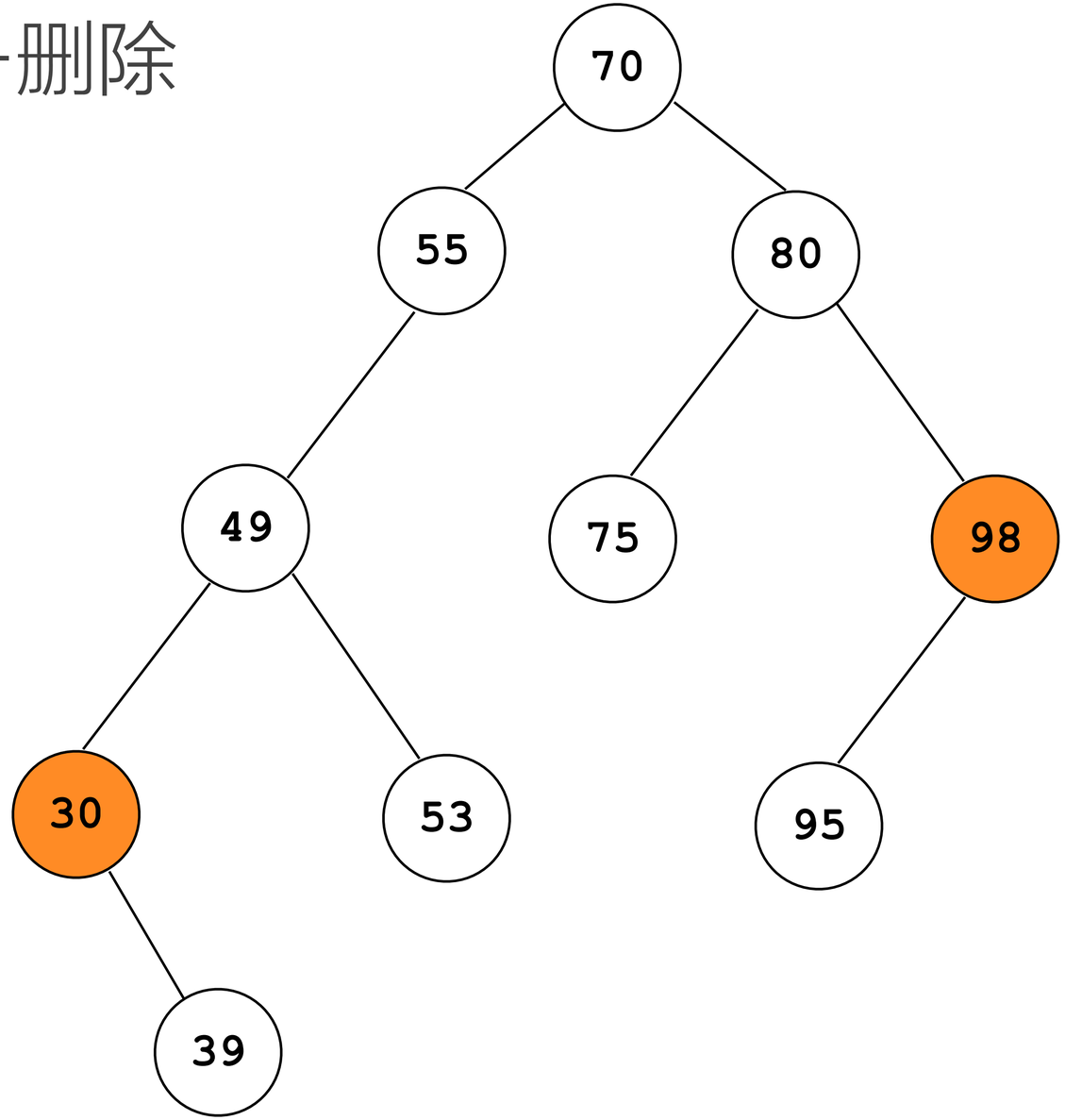
二叉排序树-删除



二叉排序树-删除

有一个孩子的结点

删除后“子承父业”

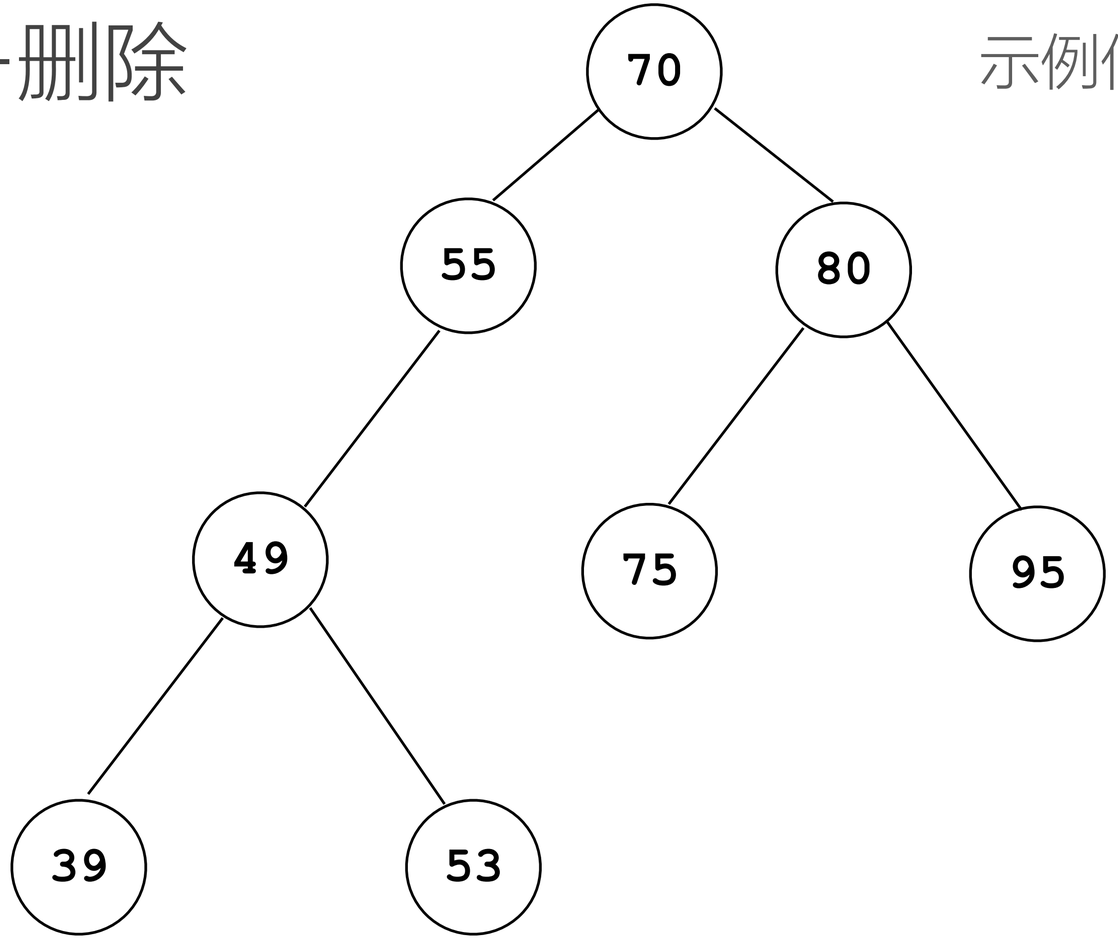


二叉排序树-删除

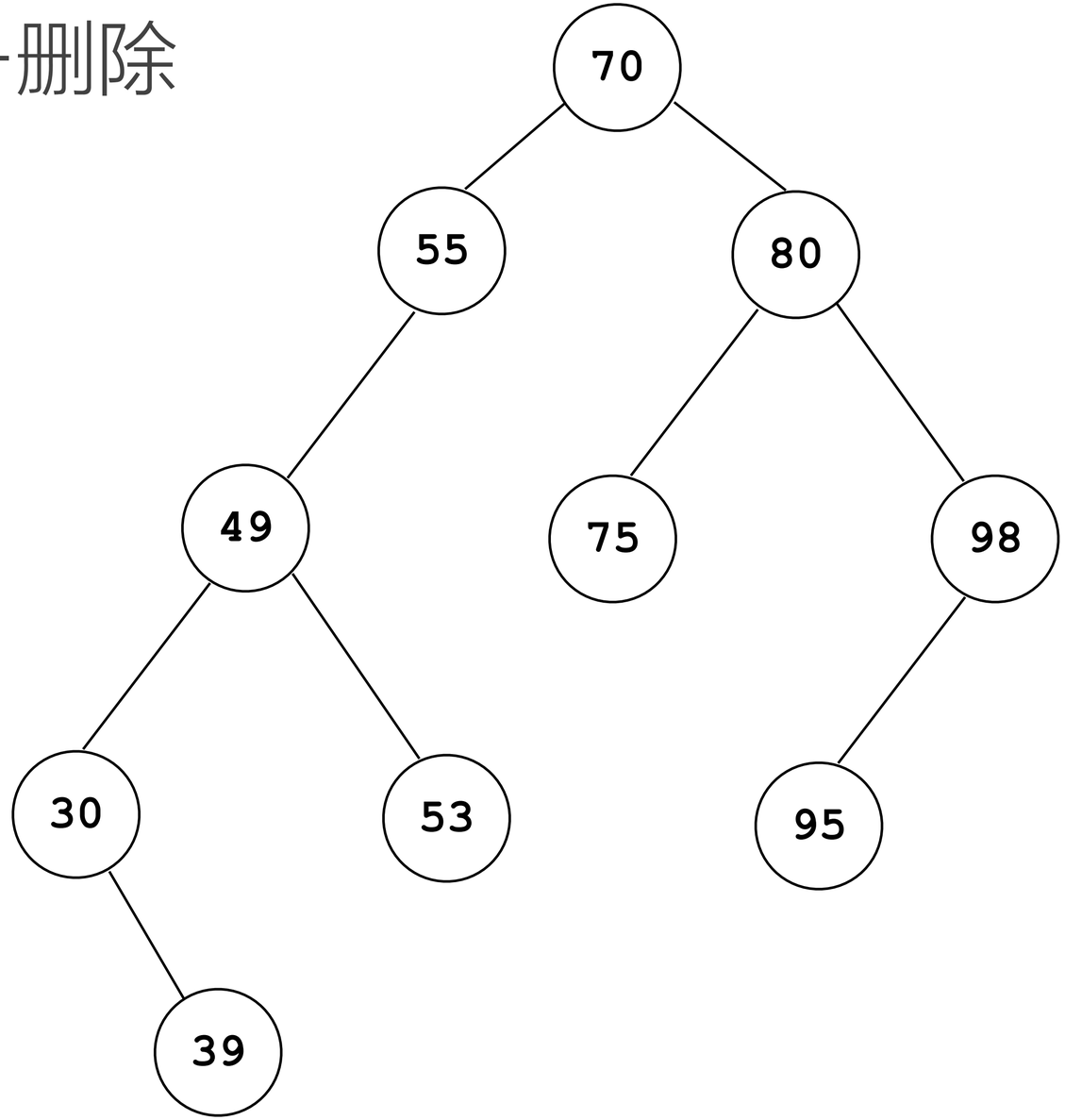
示例代码：查找 / test5.c

有一个孩子的结点

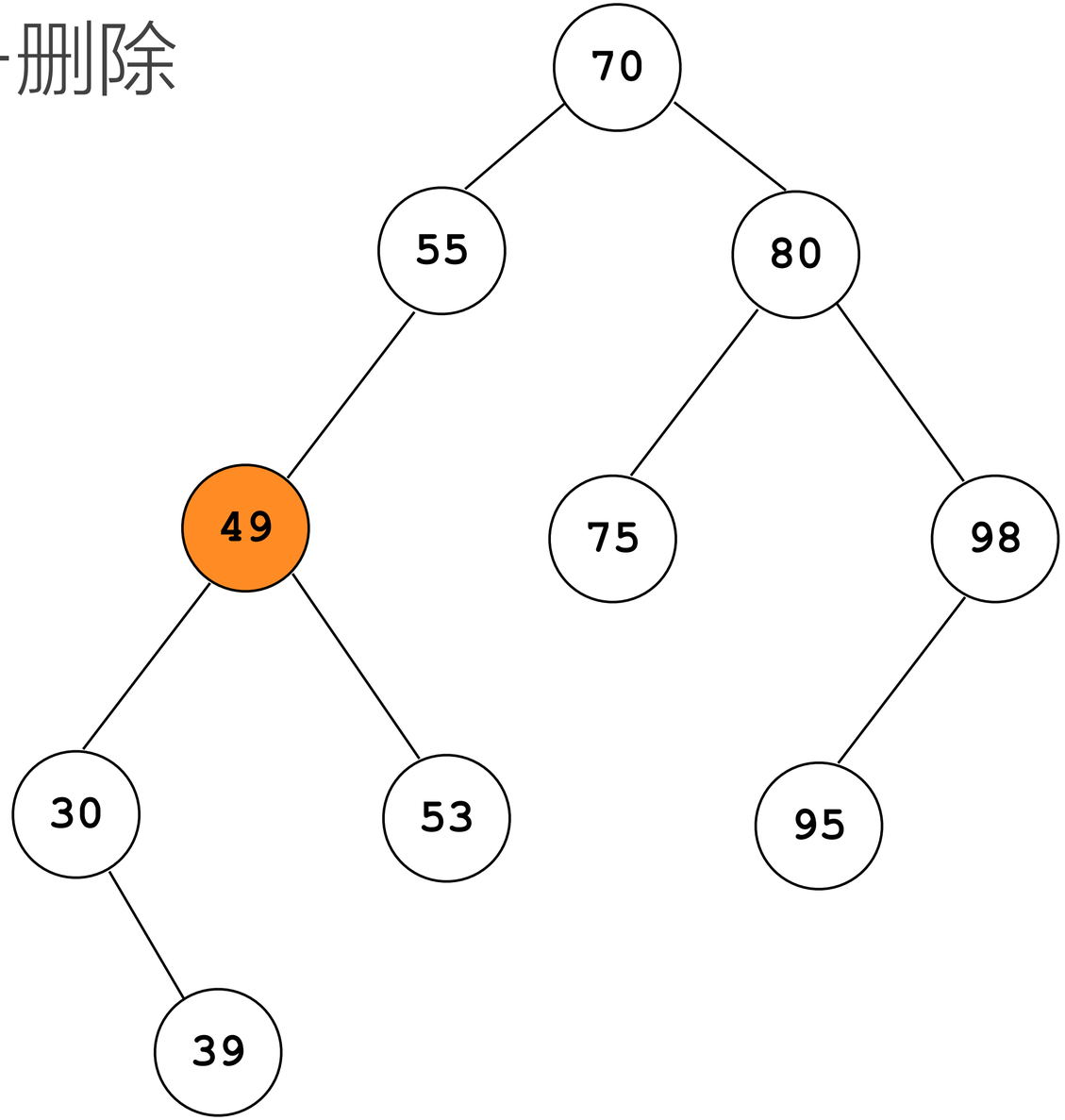
删除后“子承父业”



二叉排序树-删除



二叉排序树-删除

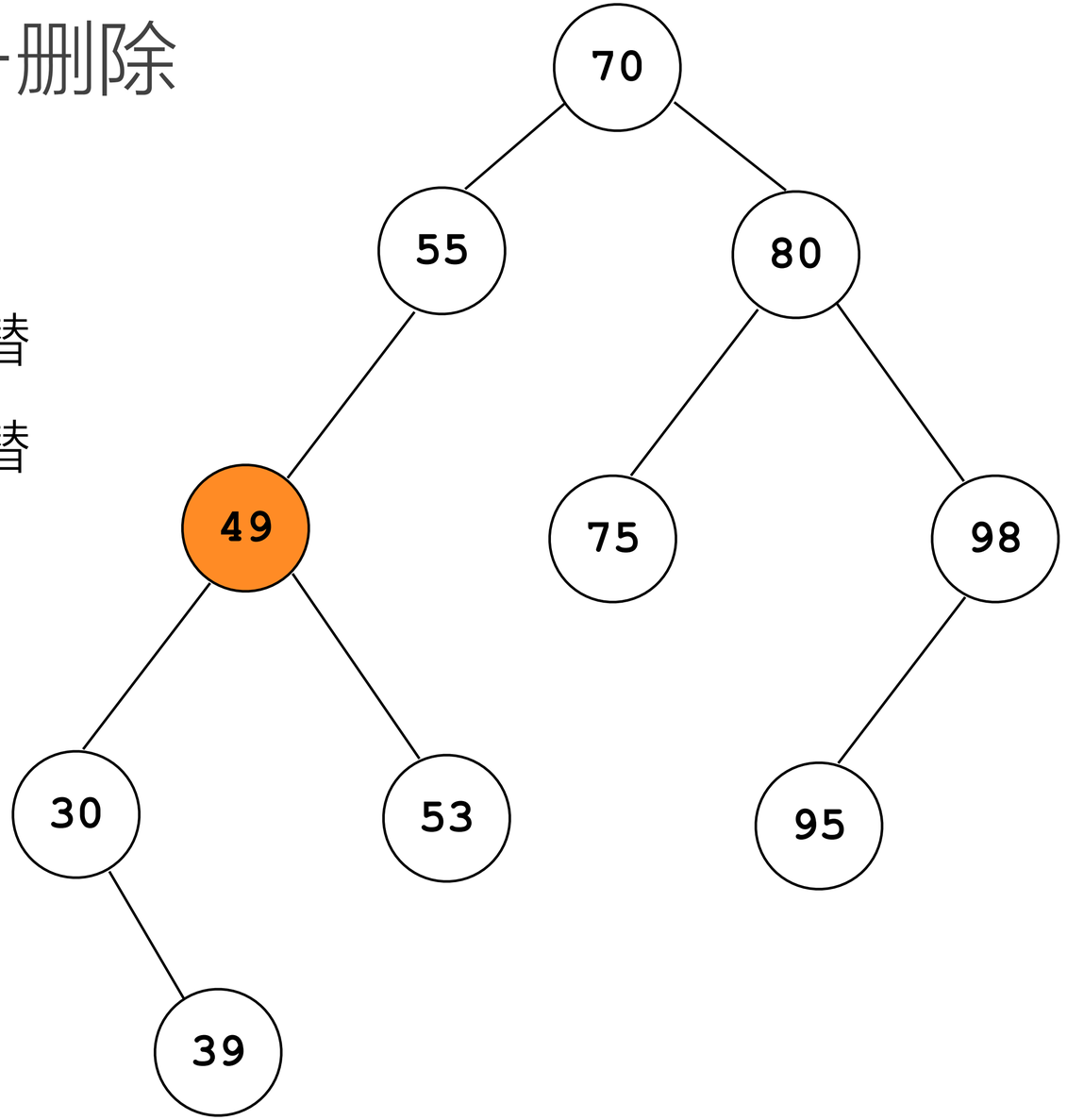


二叉排序树-删除

两个孩子的结点

右子树中最小结点代替

左子树中最大结点代替

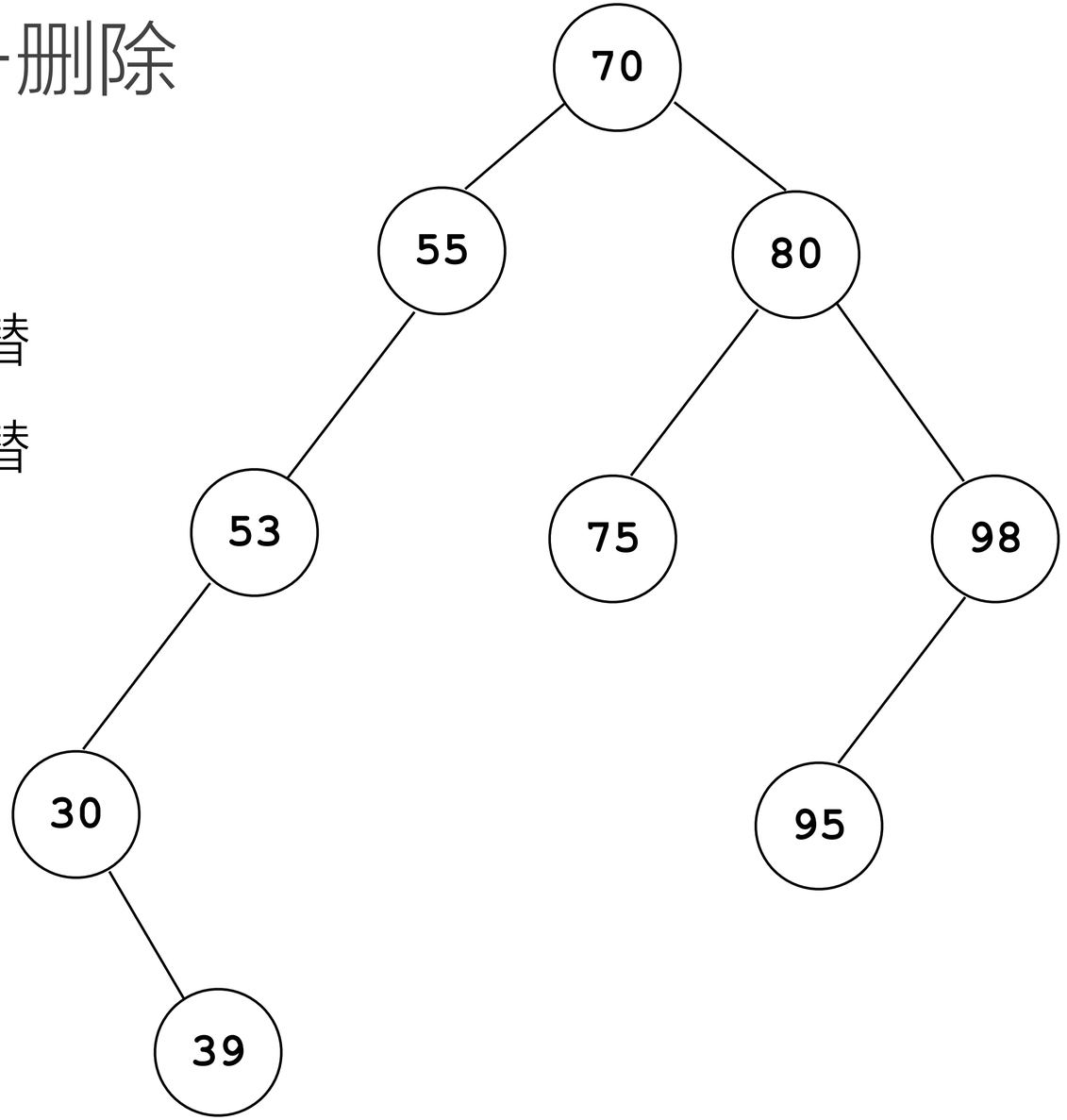


二叉排序树-删除

两个孩子的结点

右子树中最小结点代替

左子树中最大结点代替

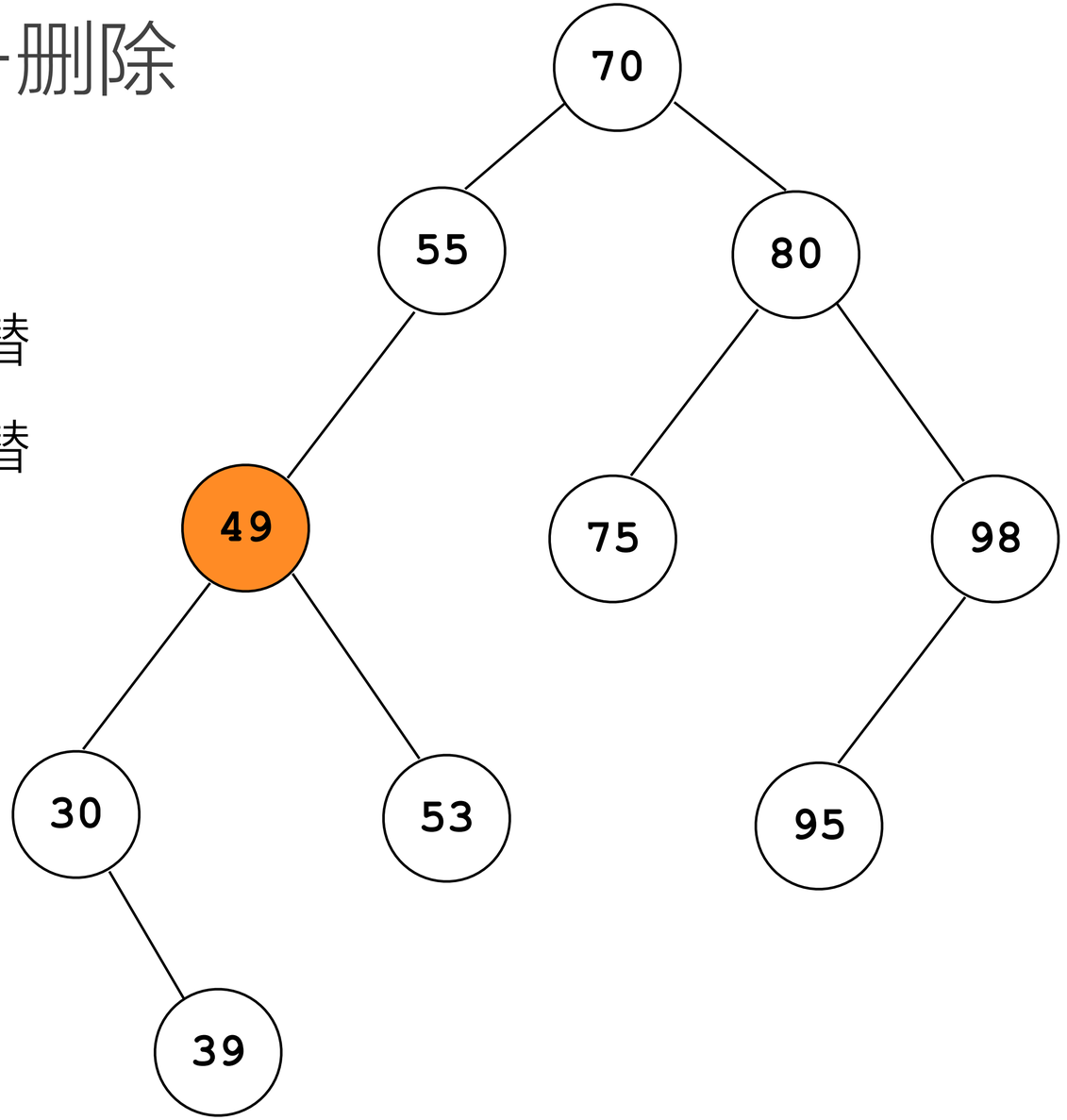


二叉排序树-删除

两个孩子的结点

右子树中最小结点代替

左子树中最大结点代替



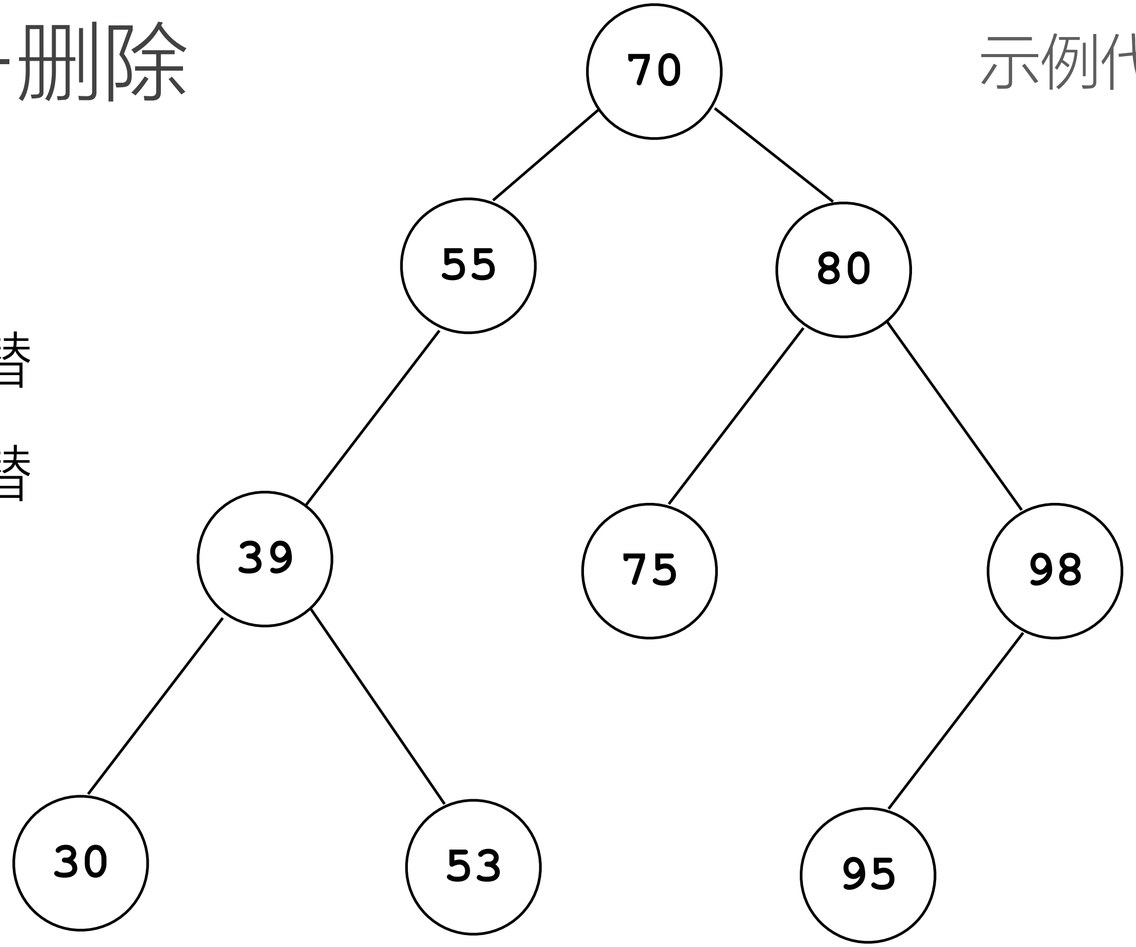
二叉排序树-删除

示例代码：查找 / test5.c

两个孩子的结点

右子树中最小结点代替

左子树中最大结点代替



二叉排序树-删除

情况一：删除的结点是叶子节点（没有左右孩子）

直接将其从树中移除即可，不影响其它结构

情况二：删除的结点只有一个孩子（左孩子或右孩子）

让其唯一的子节点“顶替”它的位置，保持排序特性不变

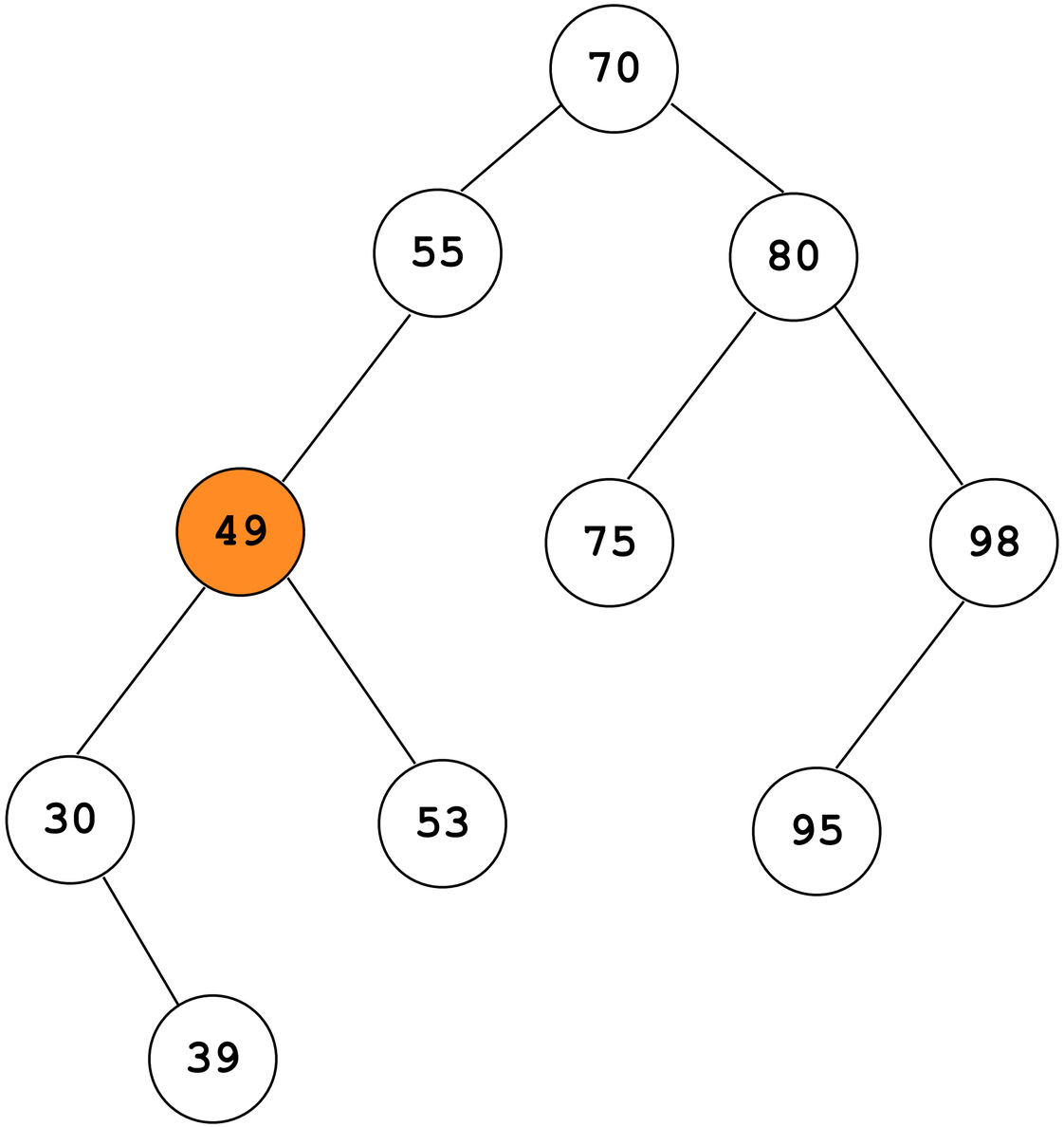
情况三：删除的结点有两个孩子

使用右子树中最小的结点代替，或使用左子树中最大的结点代替

```c
// 情况3：左右子树都有，找左子树中最大的节点替换
temp = *d;
record = (*d)->lchild;
while(record->rchild != NULL)
{
    temp = record;
    record = record->rchild;
}

// 替换当前值
(*d)->data = record->data;

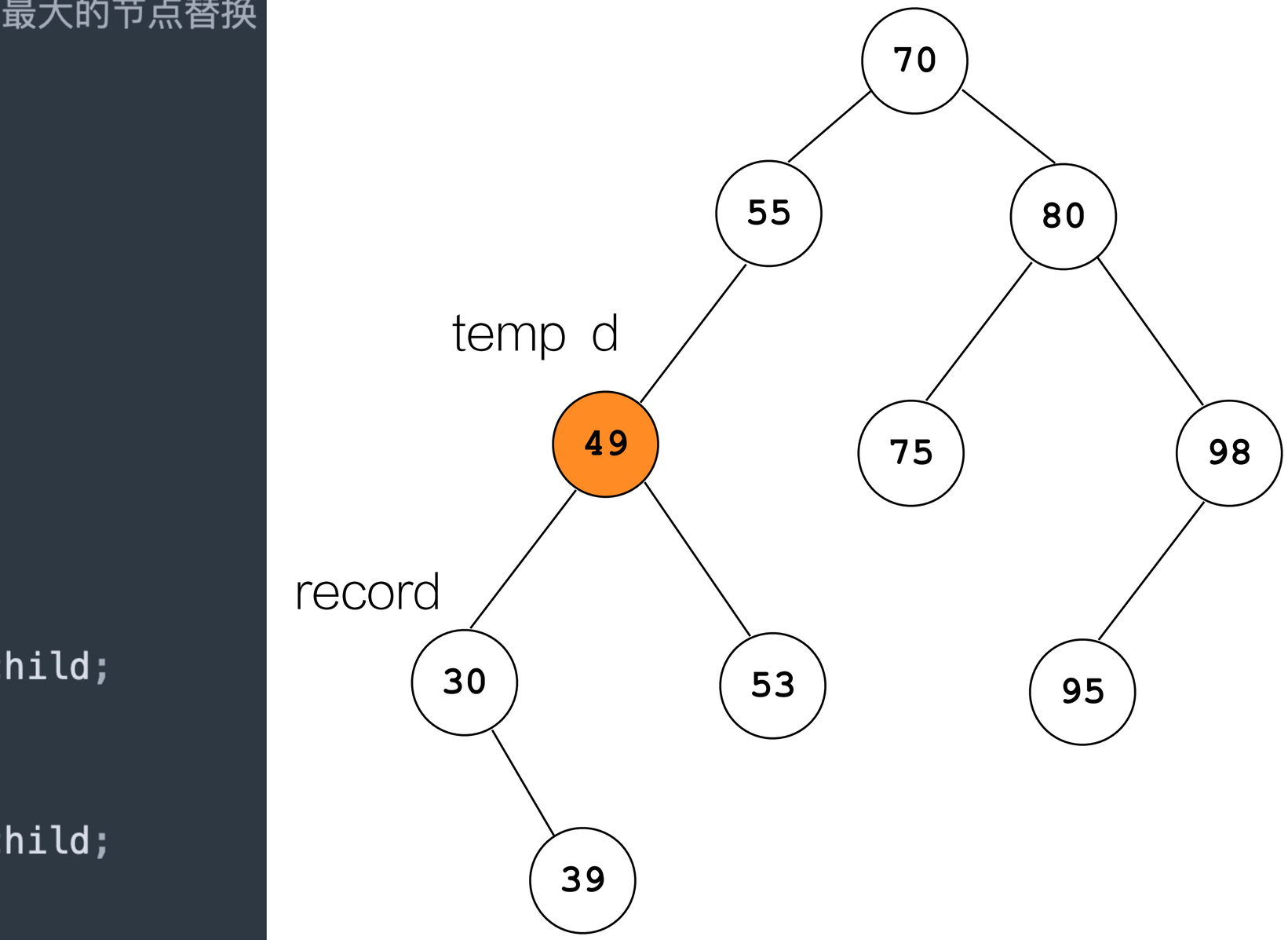
// 删除替代节点
if (temp != *d)
{
    temp->rchild = record->lchild;
}
else
{
    temp->lchild = record->lchild;
}
free(record);
```



```c
// 情况3：左右子树都有，找左子树中最大的节点替换
temp = *d;
record = (*d)->lchild;
while(record->rchild != NULL)
{
    temp = record;
    record = record->rchild;
}

// 替换当前值
(*d)->data = record->data;

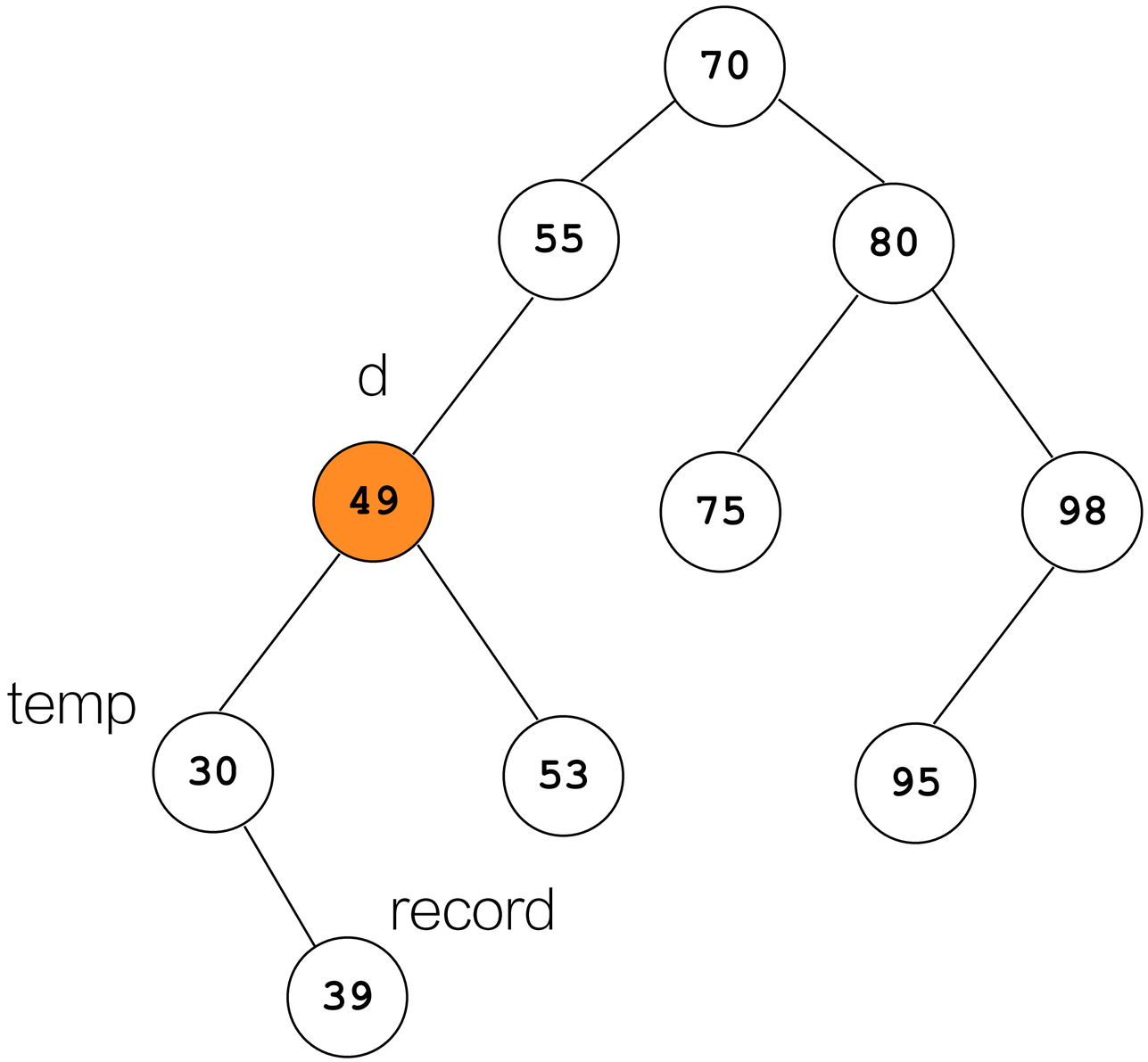
// 删除替代节点
if (temp != *d)
{
    temp->rchild = record->lchild;
}
else
{
    temp->lchild = record->lchild;
}
free(record);
```



```c
// 情况3：左右子树都有，找左子树中最大的节点替换
temp = *d;
record = (*d)->lchild;
while(record->rchild != NULL)
{
    temp = record;
    record = record->rchild;
}

// 替换当前值
(*d)->data = record->data;

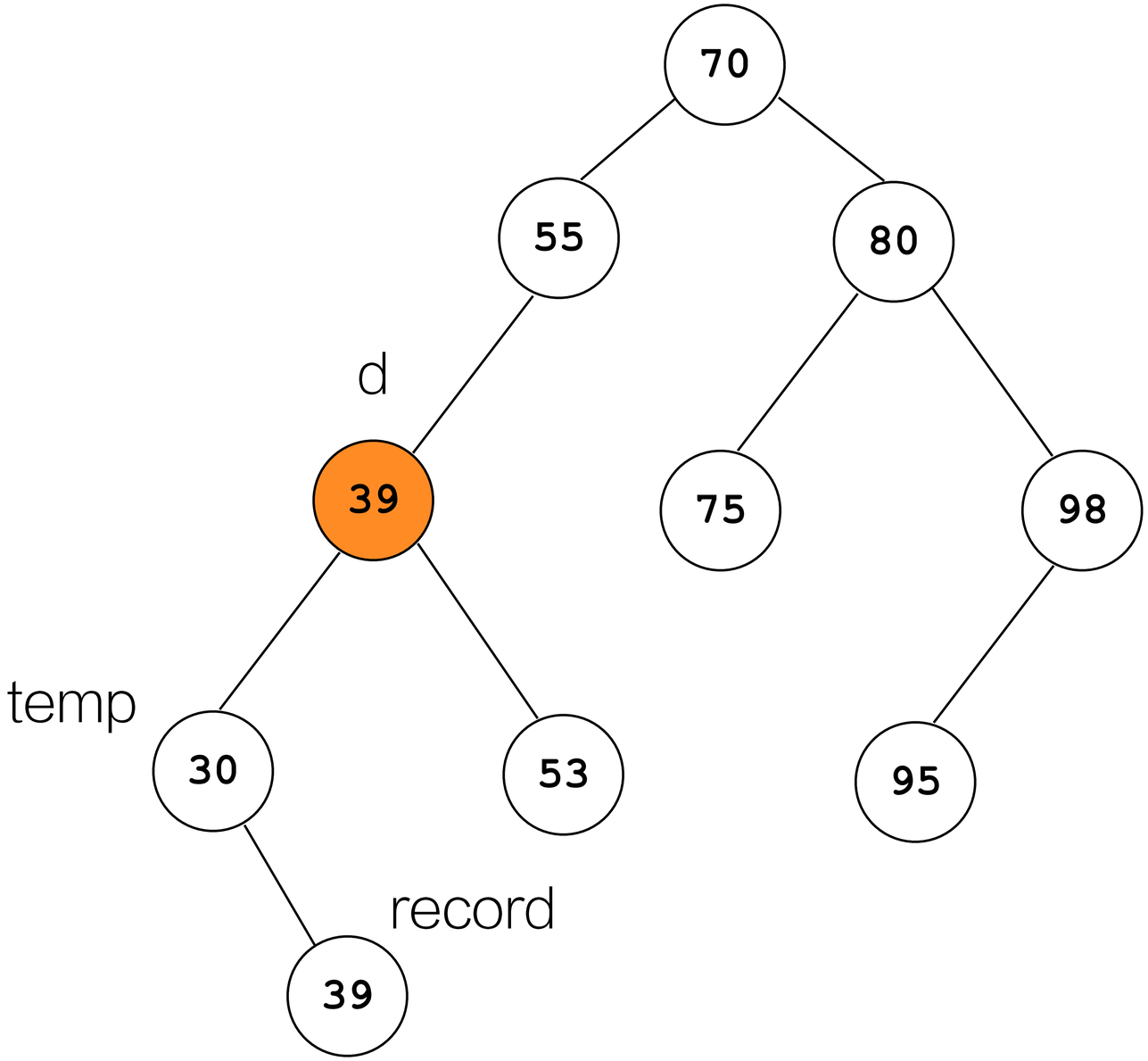
// 删除替代节点
if (temp != *d)
{
    temp->rchild = record->lchild;
}
else
{
    temp->lchild = record->lchild;
}
free(record);
```



```c
// 情况3：左右子树都有，找左子树中最大的节点替换
temp = *d;
record = (*d)->lchild;
while(record->rchild != NULL)
{
    temp = record;
    record = record->rchild;
}

// 替换当前值
(*d)->data = record->data;

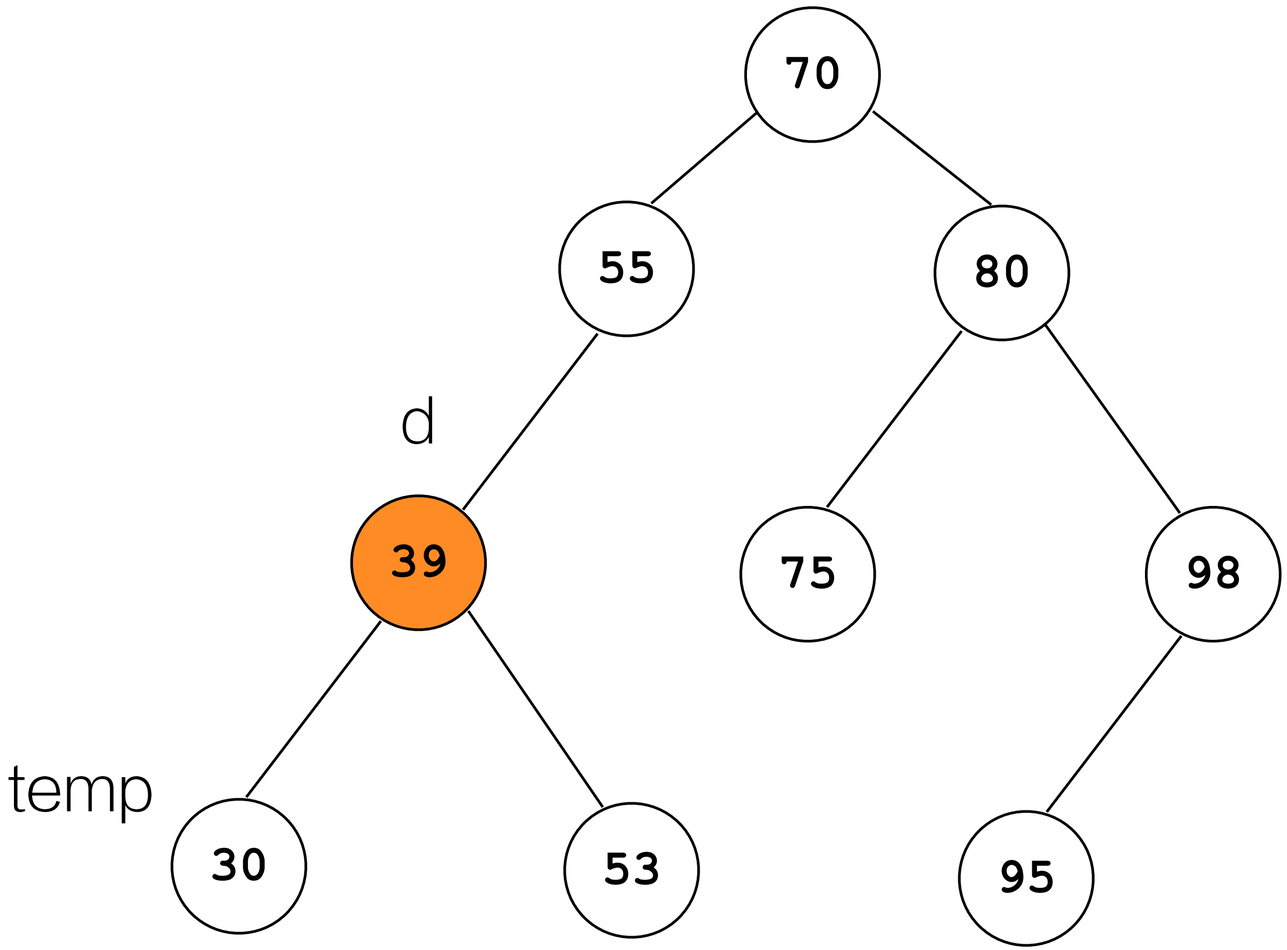
// 删除替代节点
if (temp != *d)
{
    temp->rchild = record->lchild;
}
else
{
    temp->lchild = record->lchild;
}
free(record);
```



```c
// 情况3：左右子树都有，找左子树中最大的节点替换
temp = *d;
record = (*d)->lchild;
while(record->rchild != NULL)
{
    temp = record;
    record = record->rchild;
}

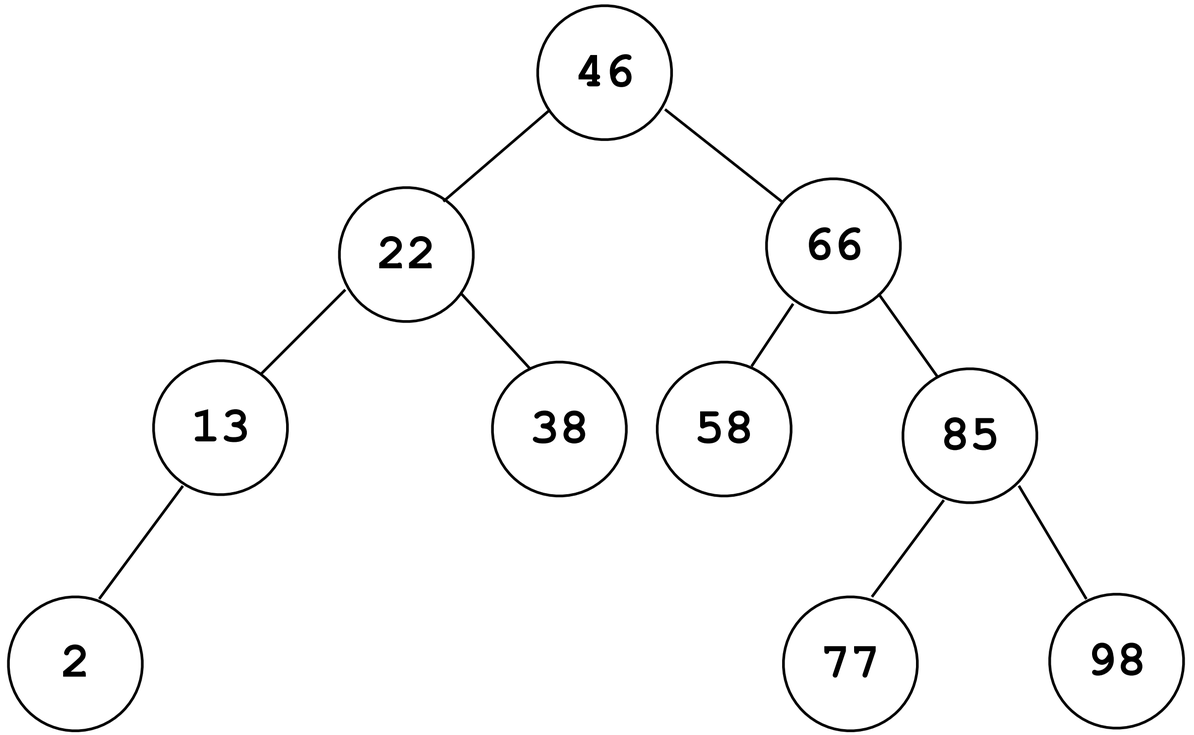
// 替换当前值
(*d)->data = record->data;

// 删除替代节点
if (temp != *d)
{
    temp->rchild = record->lchild;
}
else
{
    temp->lchild = record->lchild;
}
free(record);
```



折半查找判定树

2	13	22	38	46	58	66	77	85	98



折半查找与二叉排序树的时间复杂度

折半查找与二叉排序树的时间复杂度

具有 n 个结点的完全二叉树的深度为$\lfloor \log_2 n \rfloor + 1$

折半查找与二叉排序树的时间复杂度

具有 n 个结点的完全二叉树的深度为$\lfloor \log_2 n \rfloor + 1$

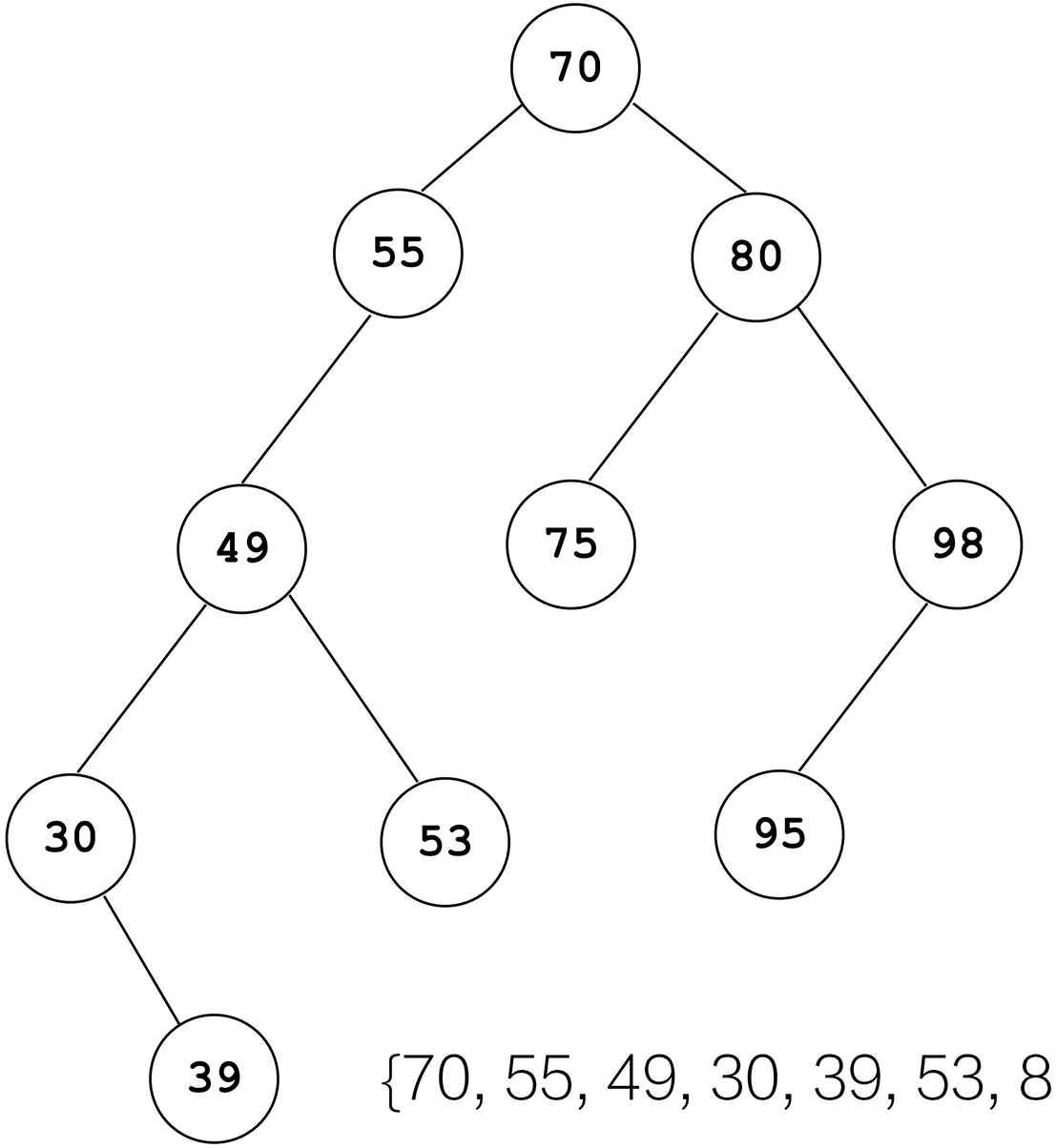
折半查找的时间复杂度：$O(\log_2 n)$

二叉排序树查找的时间复杂度：$O(n)$

折半查找与二叉排序树的时间复杂度

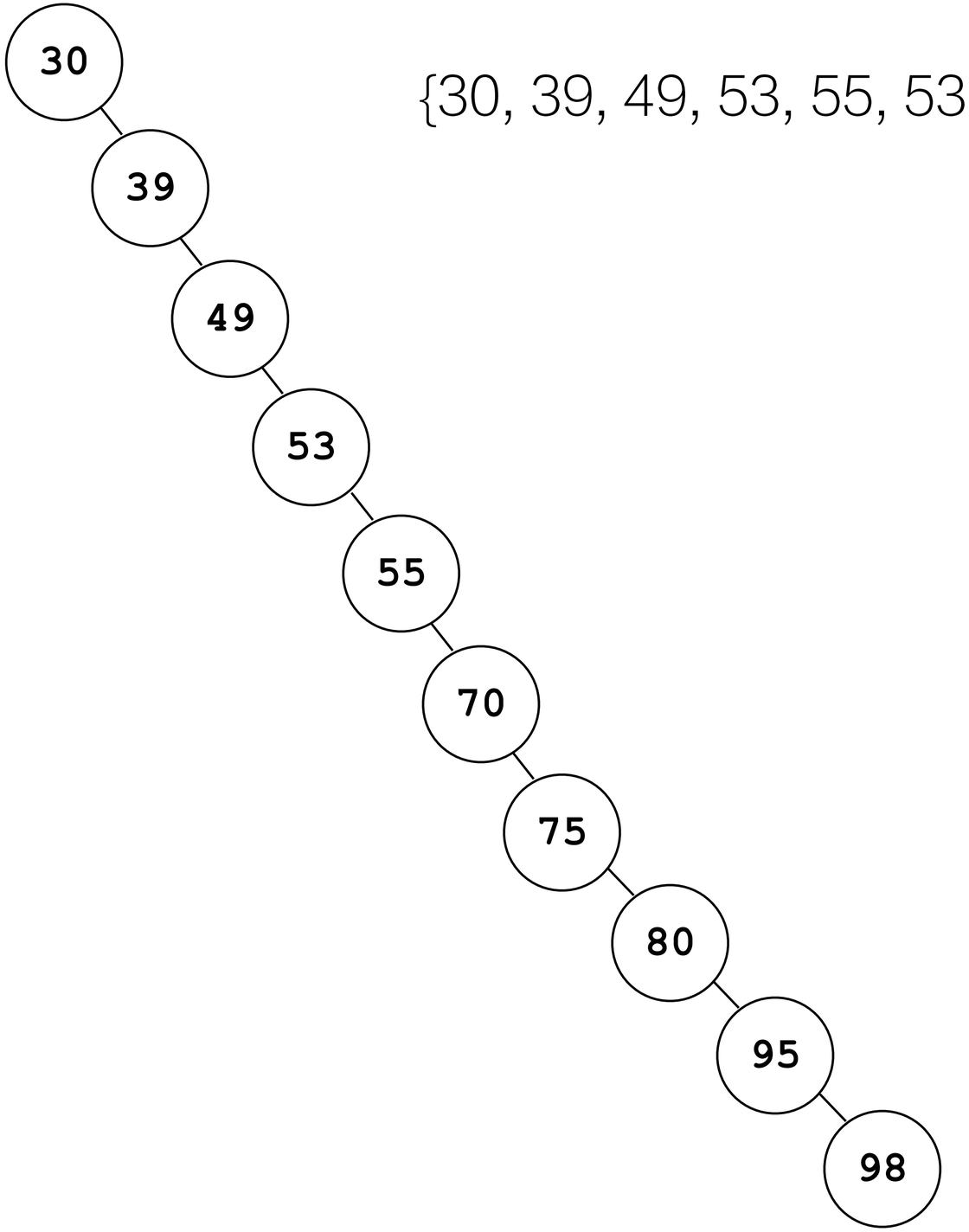
具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$

折半查找的时间复杂度：$O(\log_2 n)$



{70, 55, 49, 30, 39, 53, 80, 75, 98, 95}

{30, 39, 49, 53, 55, 53, 70, 75, 80, 95, 98}



如果一棵树“偏得厉害”，查找的过程就像沿着一条细长的枝条不断往下爬，效率很低。如果这棵树是“匀称”的，查找路径就更短，每次都能接近目标，效率高。为了解决这个问题，平衡二叉树（Balanced Binary Search Tree）应运而生。



AVL 树



平衡二叉树

平衡二叉树（Balanced Binary Tree）：又称 AVL 树，是一种二叉排序树，任意一个节点的左子树与右子树的高度之差的绝对值不超过 1。

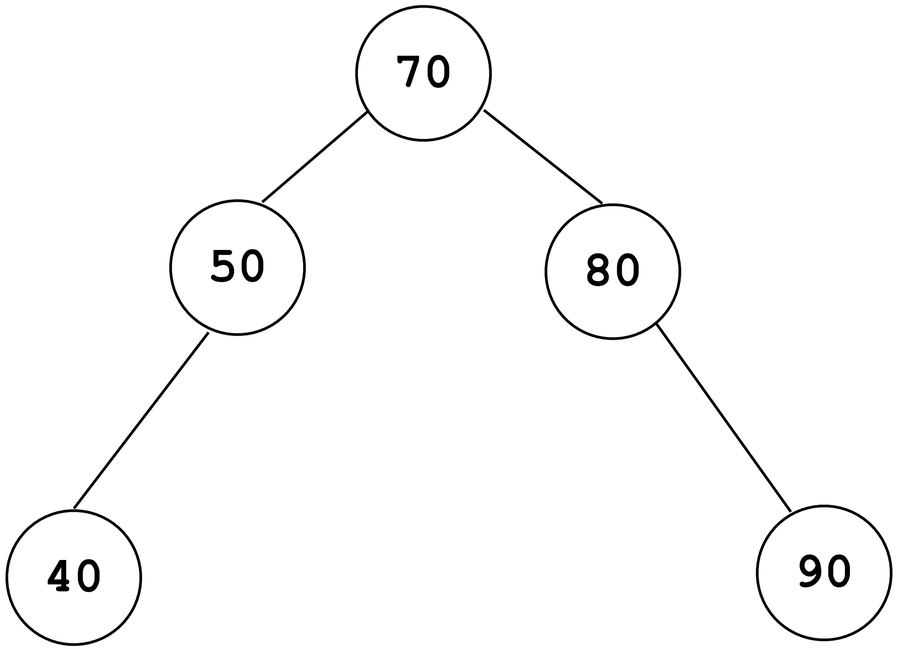
平衡二叉树

平衡二叉树（Balanced Binary Tree）：又称 AVL 树，是一种二叉排序树，任意一个节点的左子树与右子树的高度之差的绝对值不超过 1。

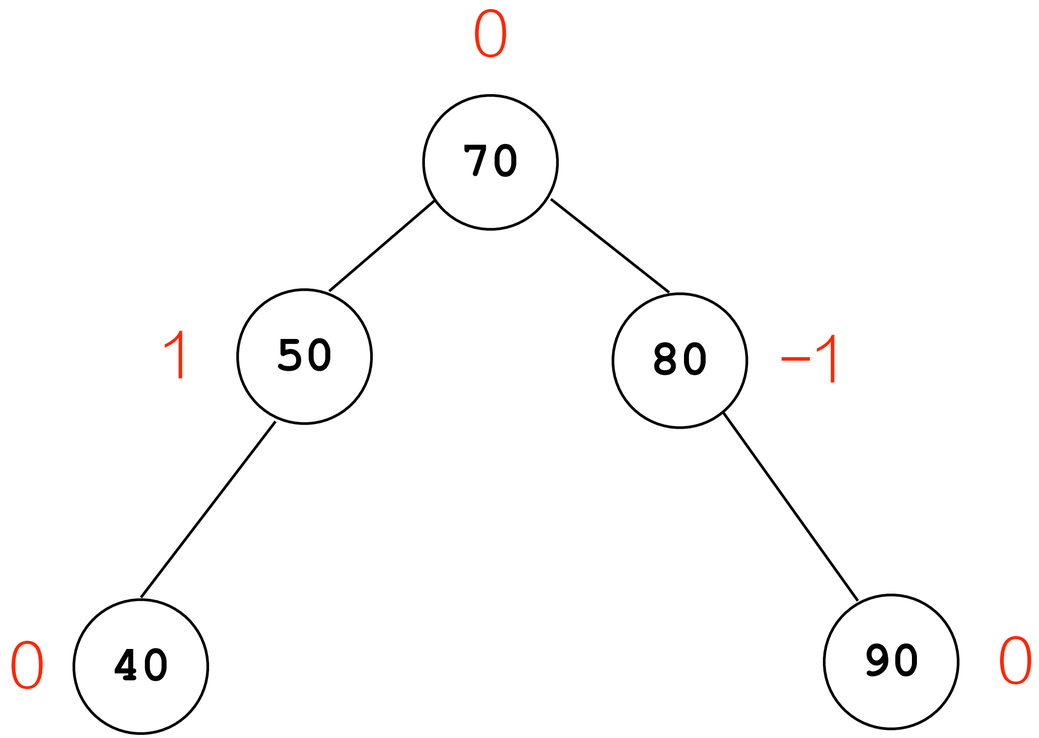
平衡因子（Balance Factor）：是衡量节点平衡程度的指标。

平衡因子 = 左子树的高度 - 右子树的高度

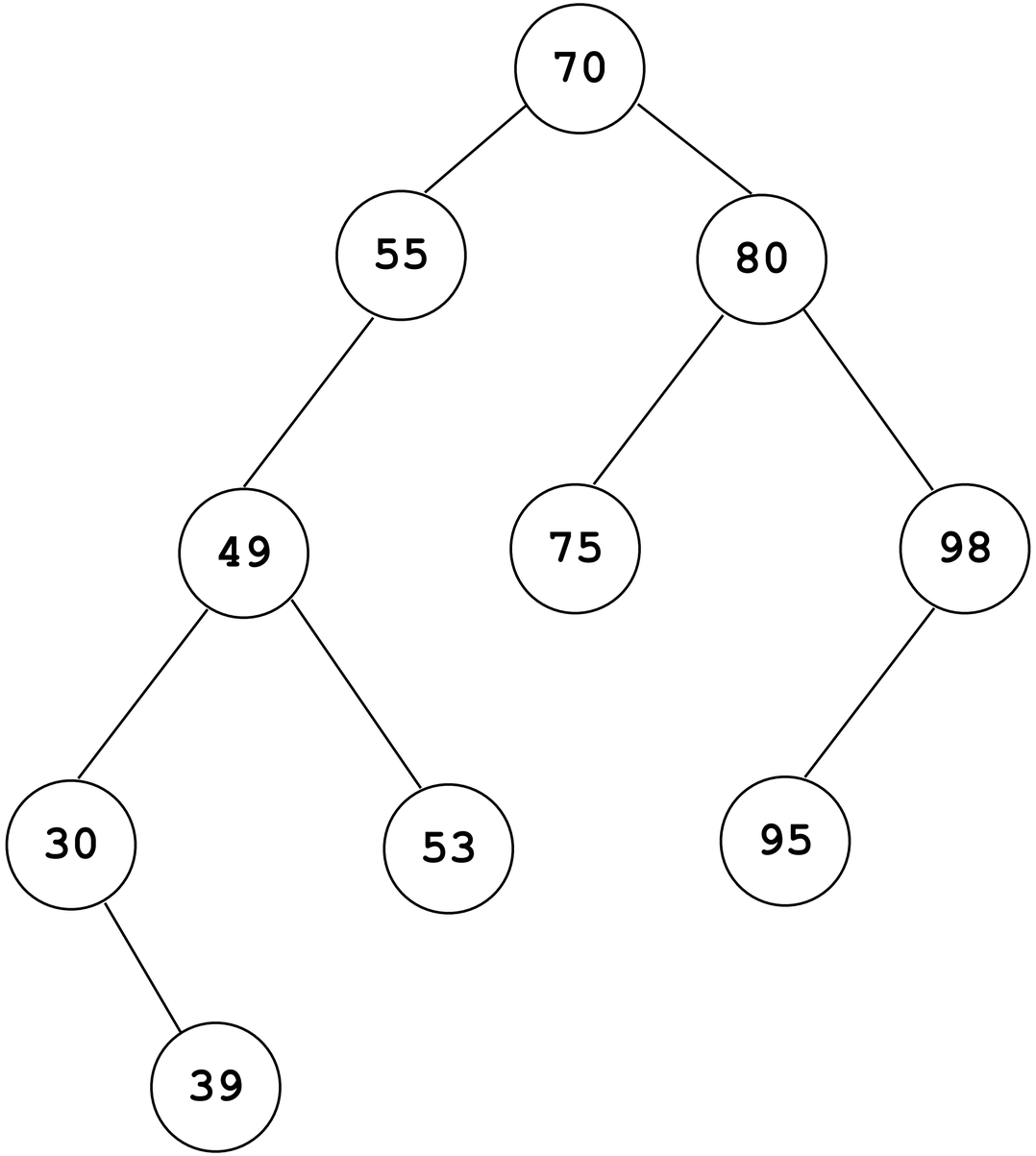
平衡二叉树



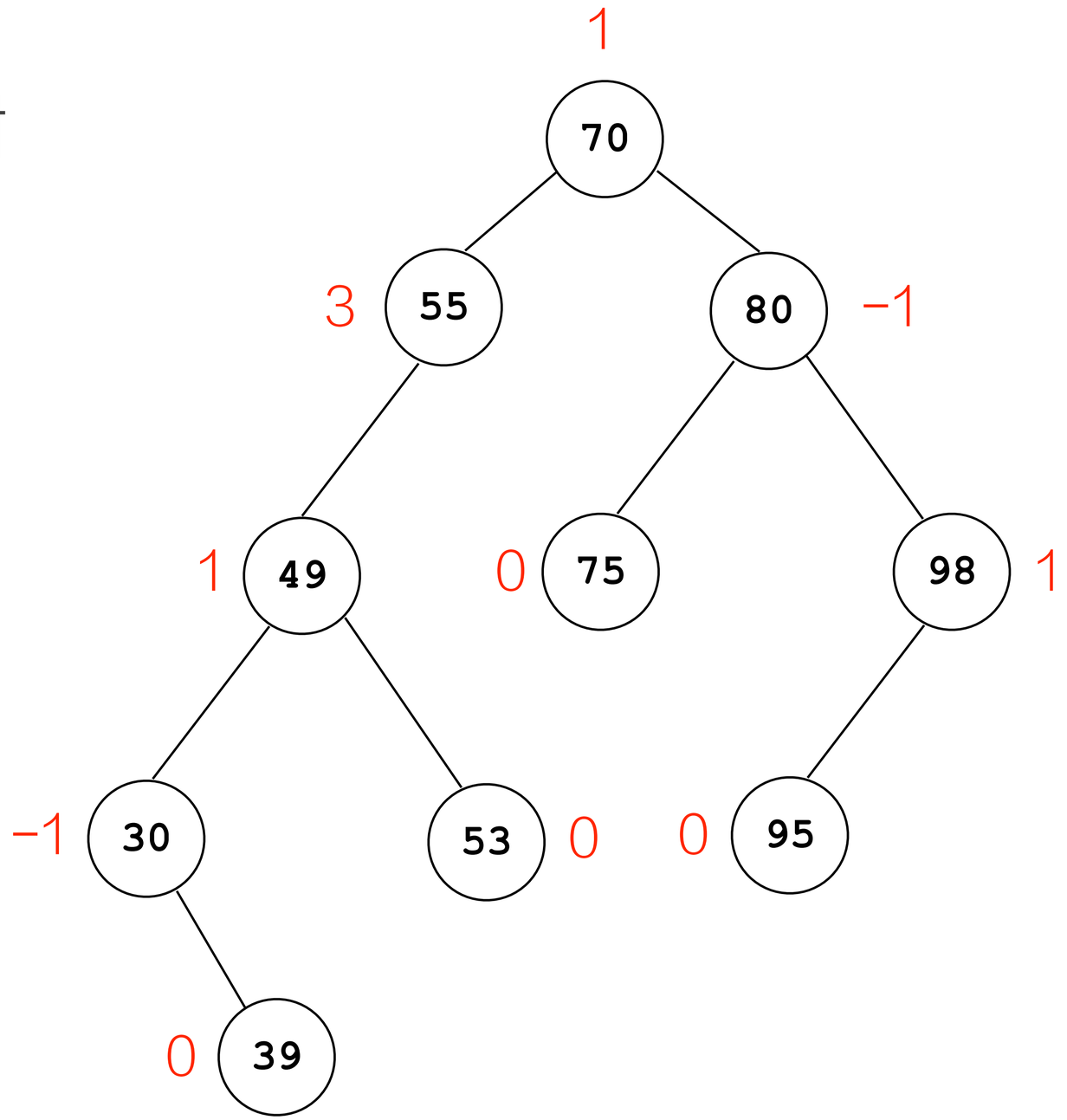
平衡二叉树



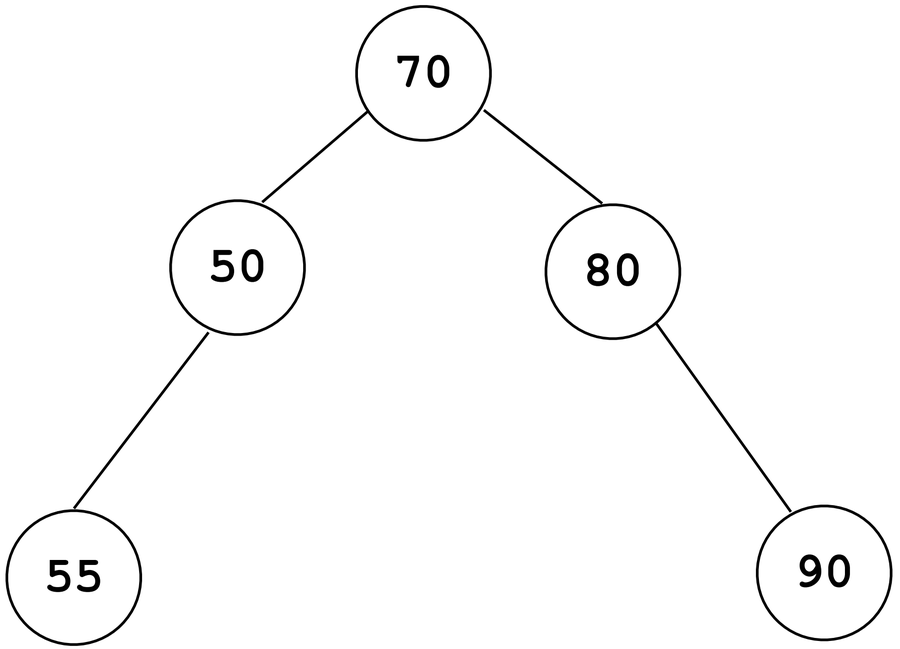
平衡二叉树



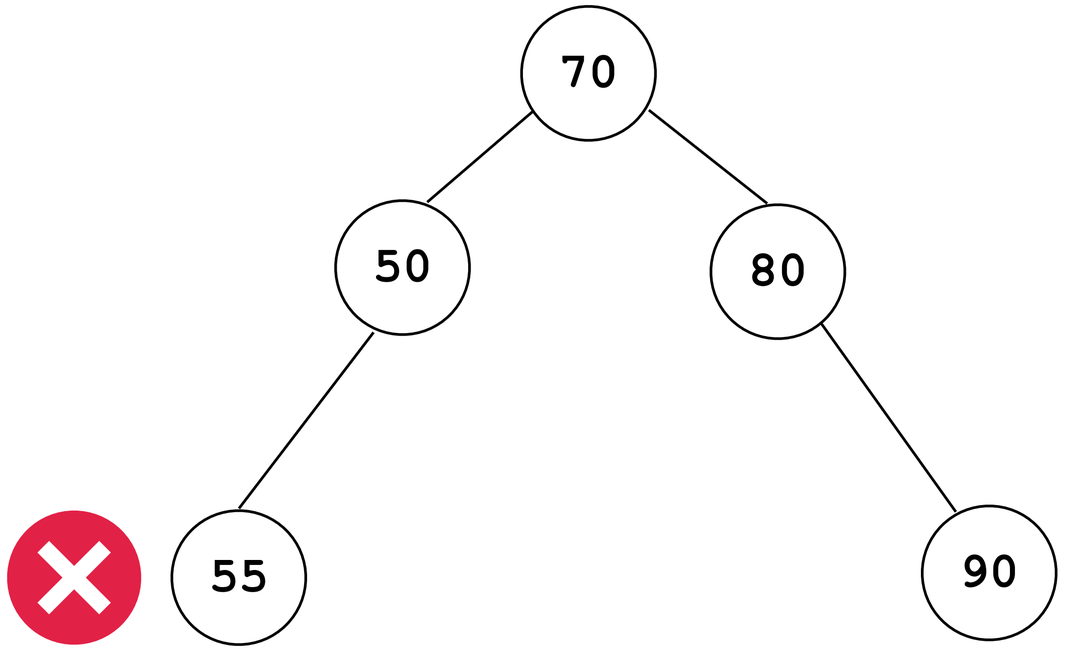
平衡二叉树



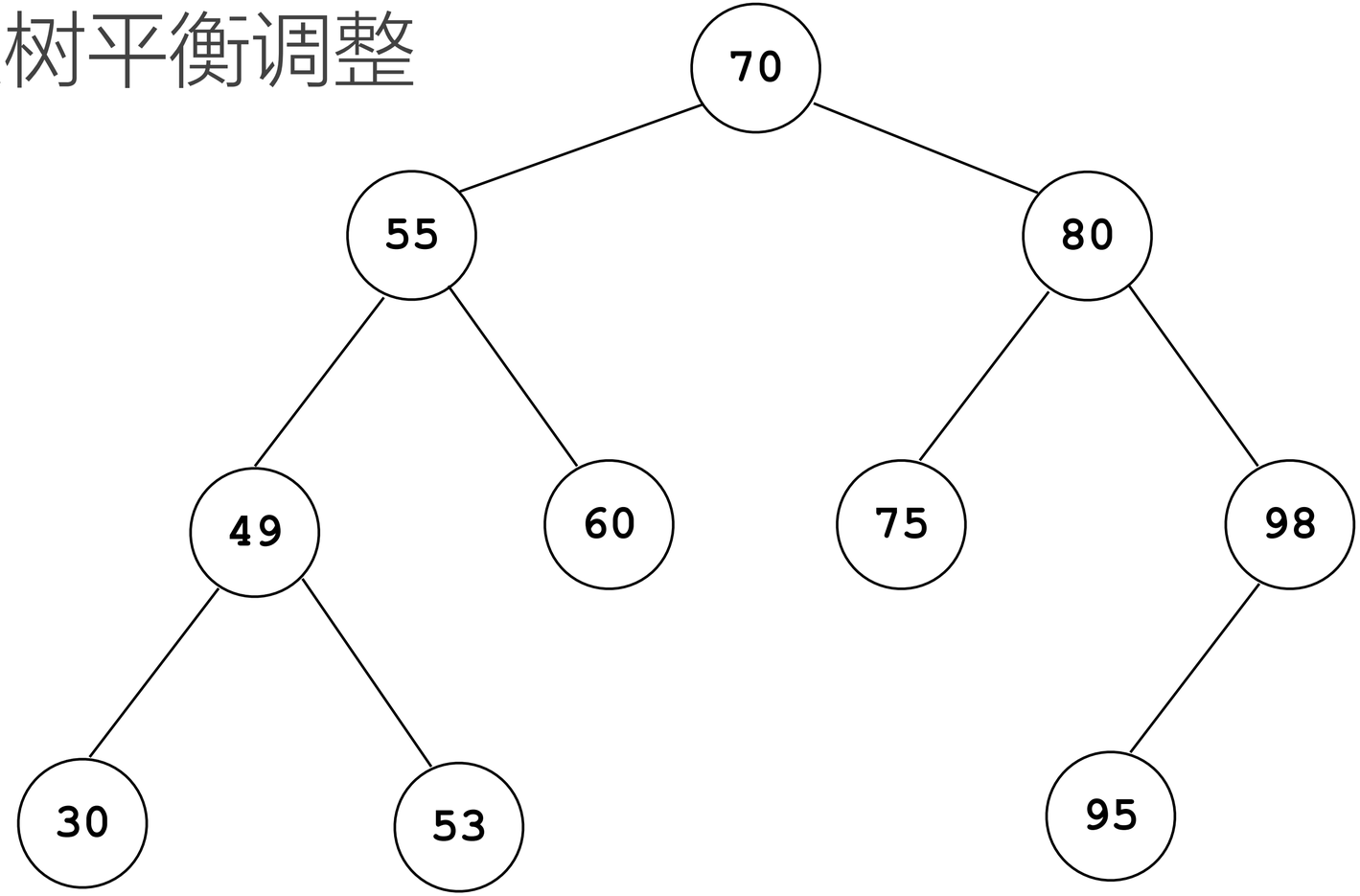
平衡二叉树



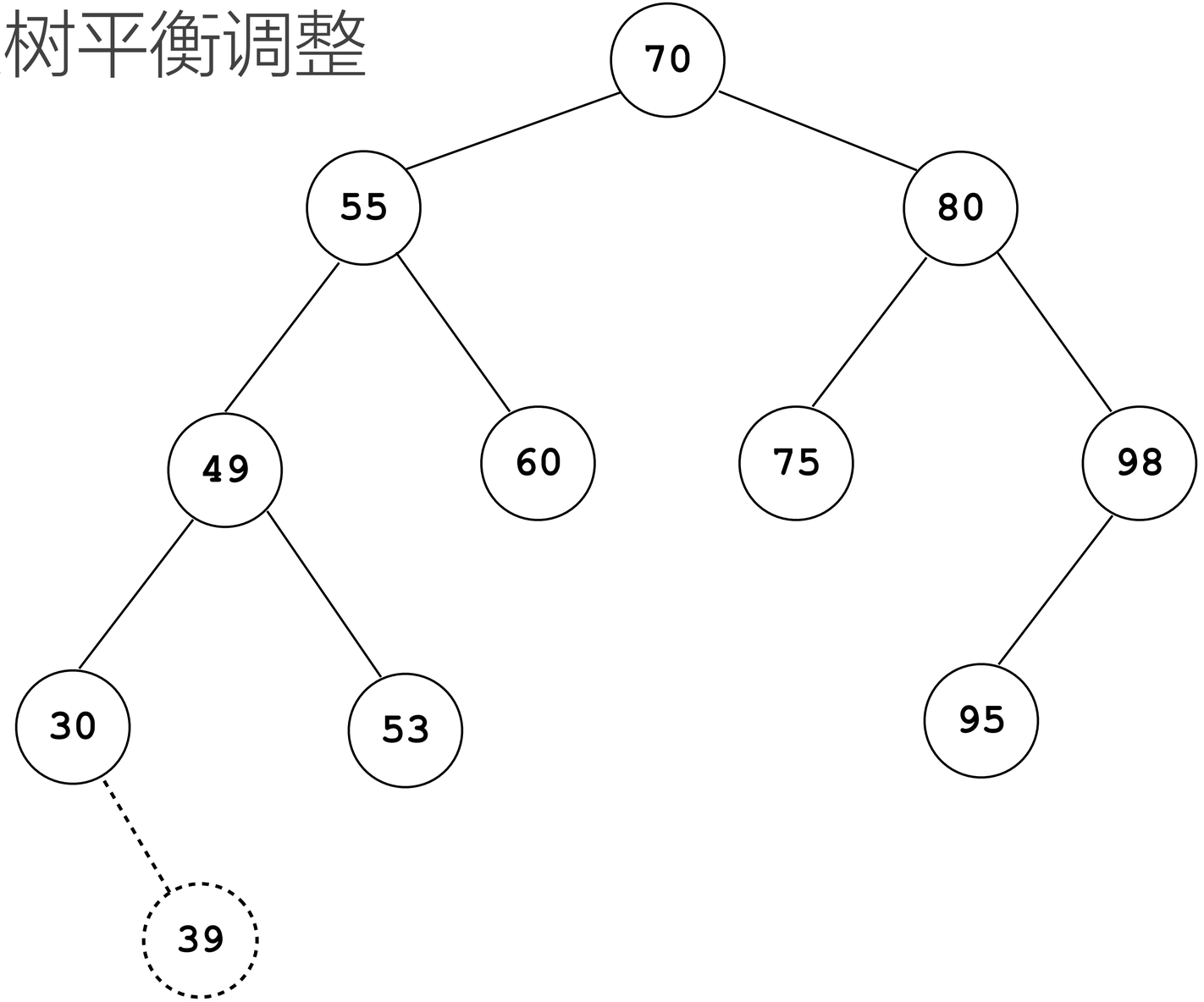
平衡二叉树



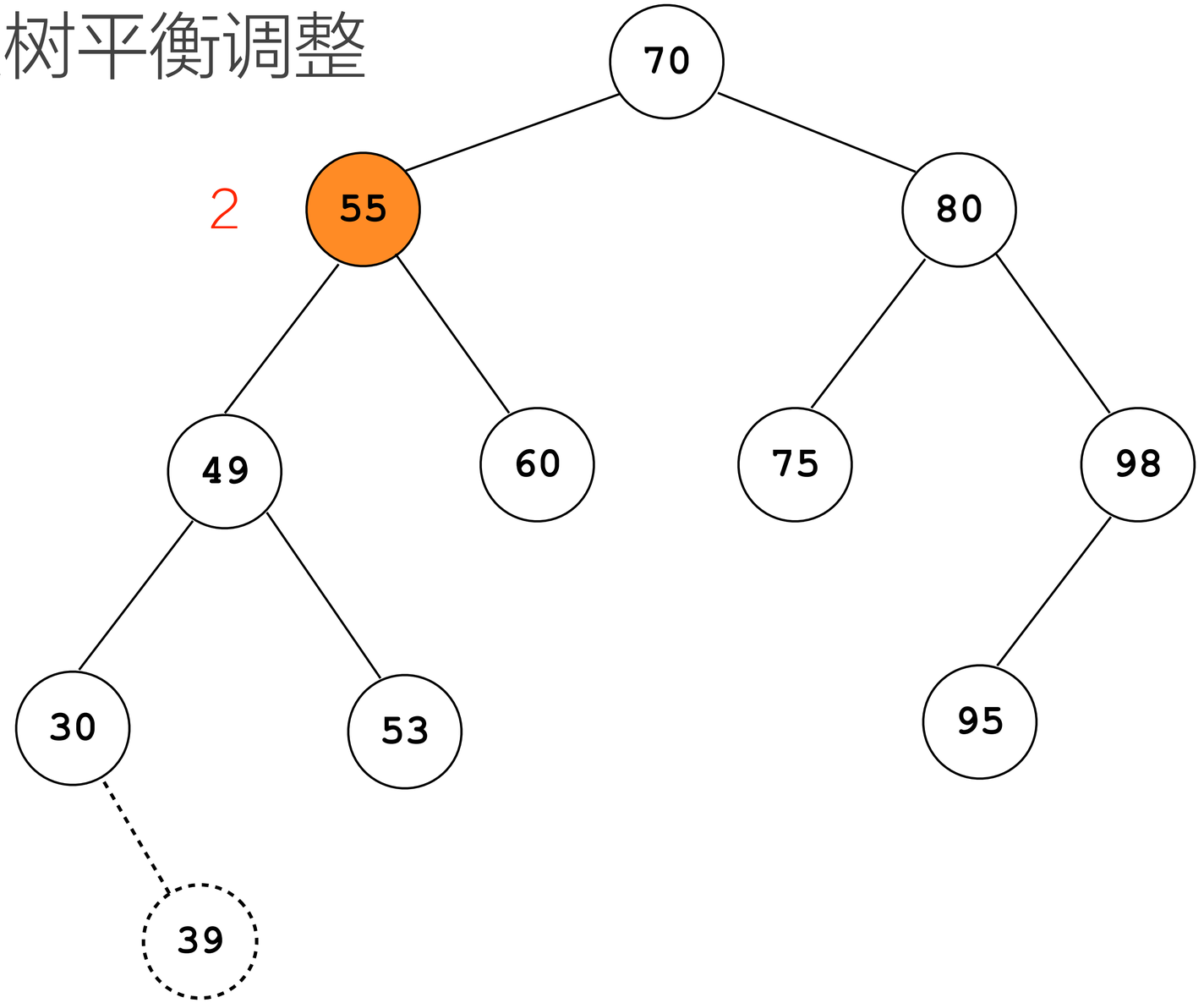
平衡二叉树平衡调整



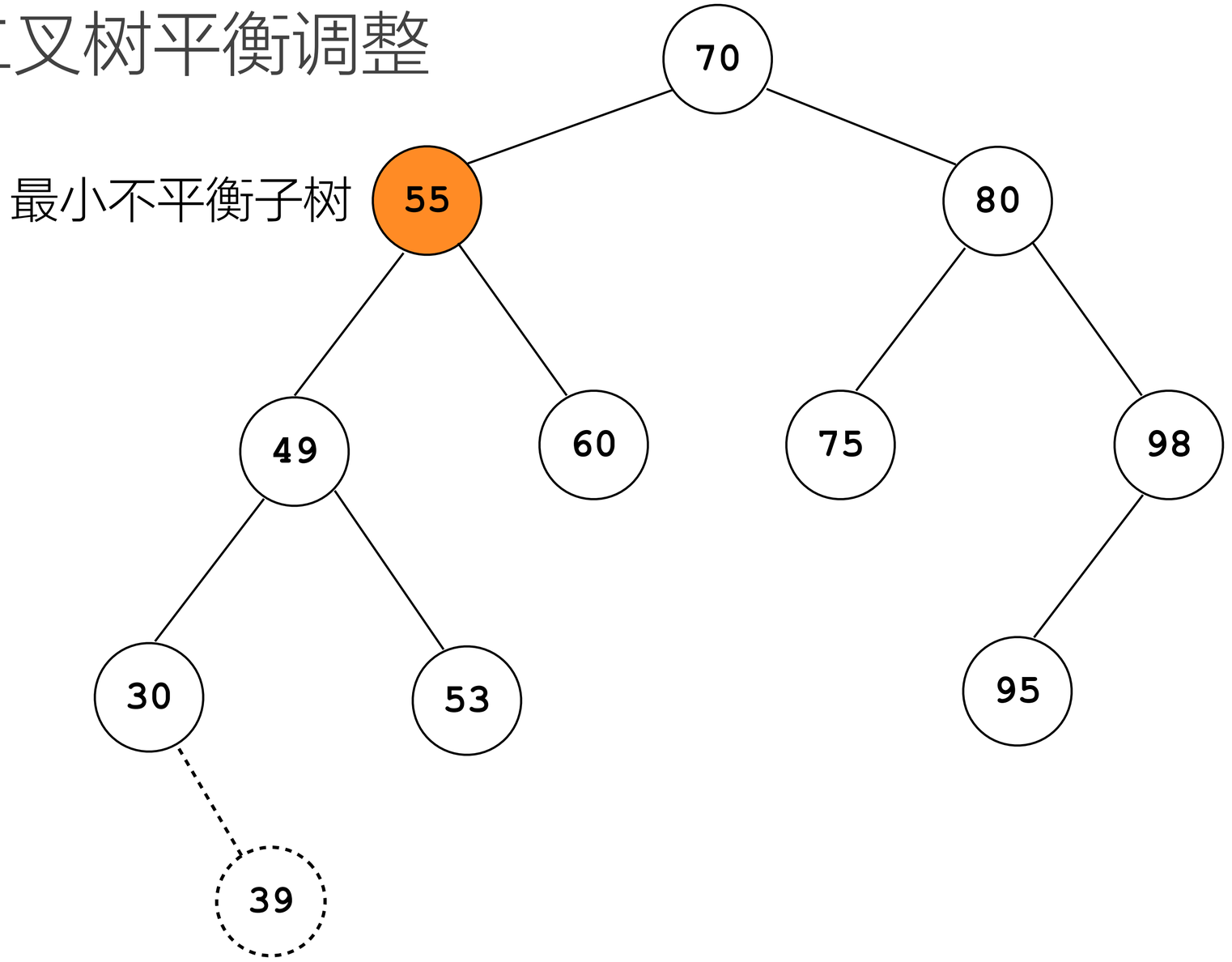
平衡二叉树平衡调整



平衡二叉树平衡调整



平衡二叉树平衡调整



平衡二叉树平衡调整

最小不平衡子树：从插入点向上回溯，第一个失衡的点，对失衡的平衡二叉树进行调整时，需要先找到最小不平衡子树。

平衡二叉树平衡调整

最小不平衡子树：从插入点向上回溯，第一个失衡的点，对失衡的平衡二叉树进行调整时，需要先找到最小不平衡子树。

LL 型失衡

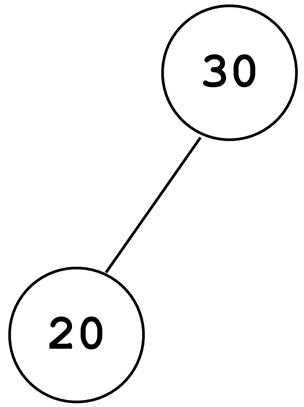
RR 型失衡

LR 型失衡

RL 型失衡

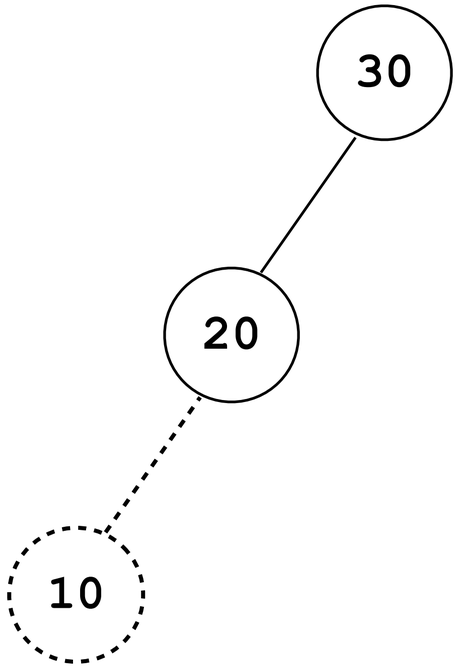
平衡二叉树平衡调整

LL 型失衡：插入在左子树的左子树，向右旋转。



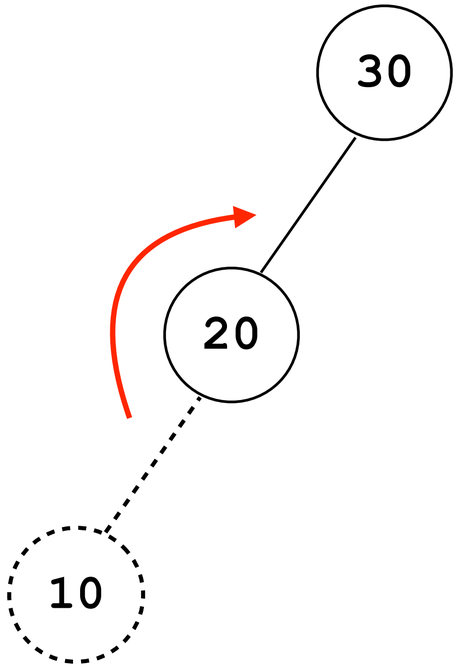
平衡二叉树平衡调整

LL 型失衡：插入在左子树的左子树，向右旋转。



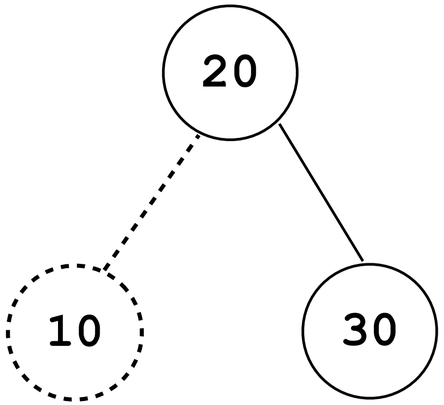
平衡二叉树平衡调整

LL 型失衡：插入在左子树的左子树，向右旋转。



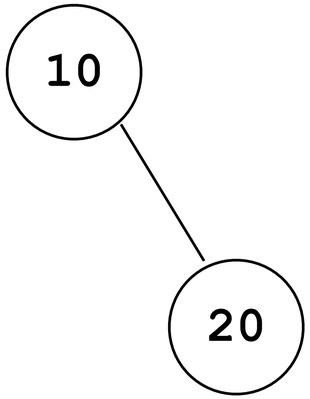
平衡二叉树平衡调整

LL 型失衡：插入在左子树的左子树，向右旋转。



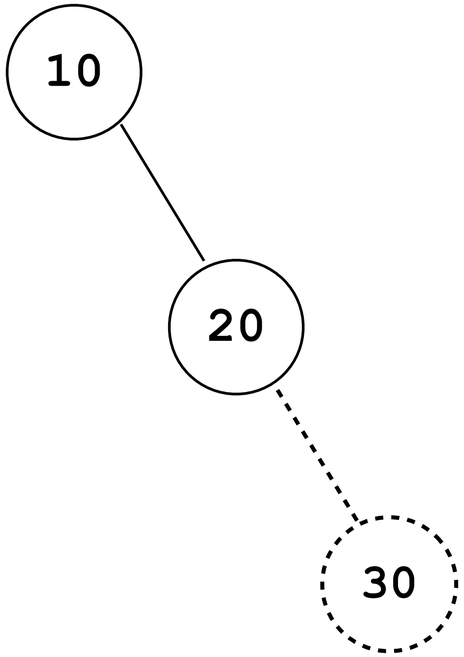
平衡二叉树平衡调整

RR 型失衡：插入在右子树的右子树，向左旋转。



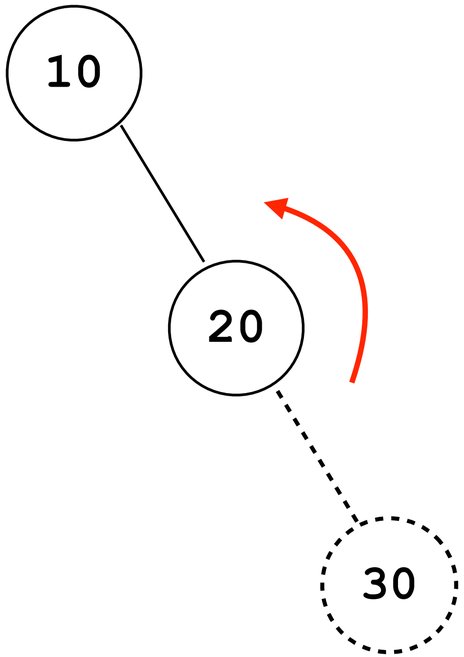
平衡二叉树平衡调整

RR 型失衡：插入在右子树的右子树，向左旋转。



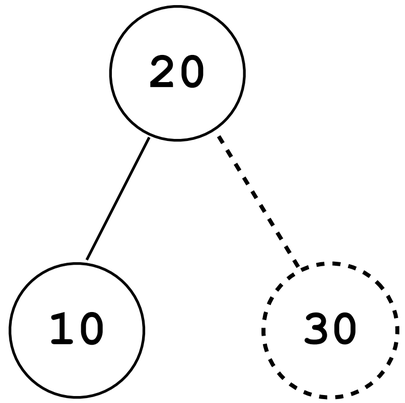
平衡二叉树平衡调整

RR 型失衡：插入在右子树的右子树，向左旋转。



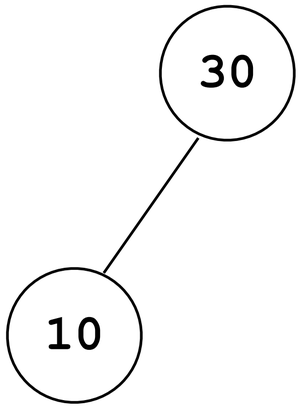
平衡二叉树平衡调整

RR 型失衡：插入在右子树的右子树，向左旋转。



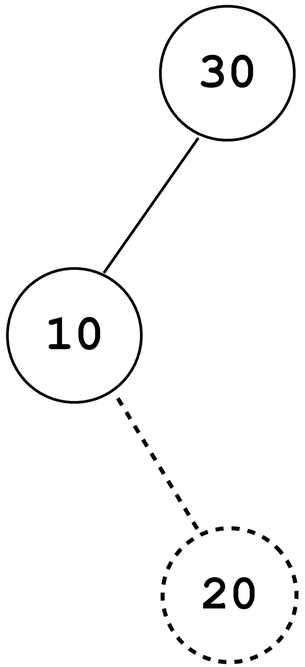
平衡二叉树平衡调整

LR 型失衡：插入在左子树的右子树，先向左旋转再向右旋转。



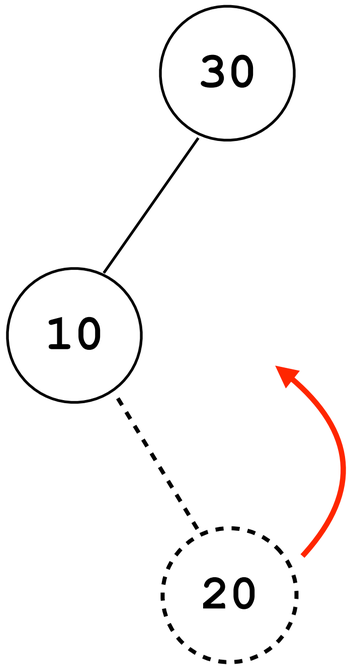
平衡二叉树平衡调整

LR 型失衡：插入在左子树的右子树，先向左旋转再向右旋转。



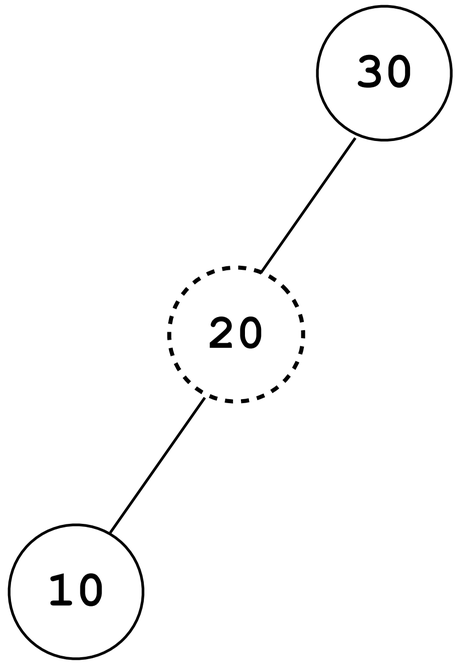
平衡二叉树平衡调整

LR 型失衡：插入在左子树的右子树，先向左旋转再向右旋转。



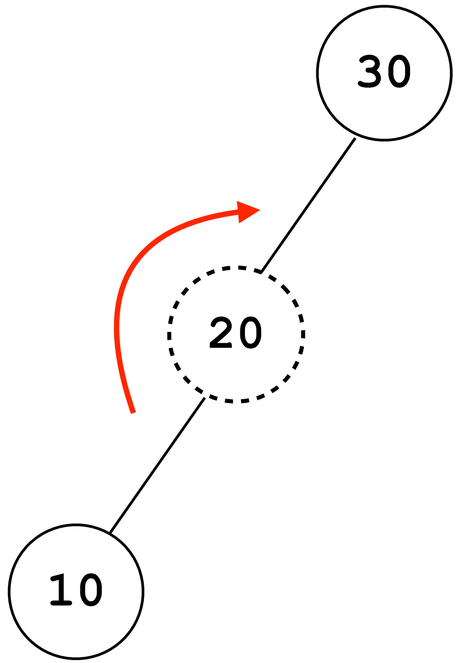
平衡二叉树平衡调整

LR 型失衡：插入在左子树的右子树，先向左旋转再向右旋转。



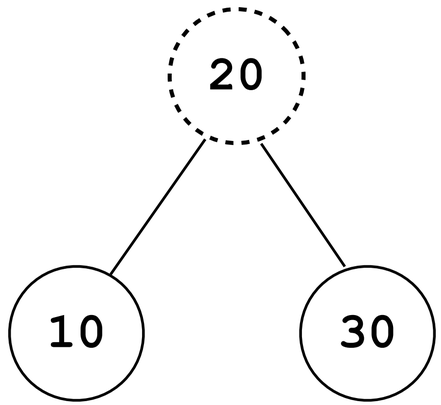
平衡二叉树平衡调整

LR 型失衡：插入在左子树的右子树，先向左旋转再向右旋转。



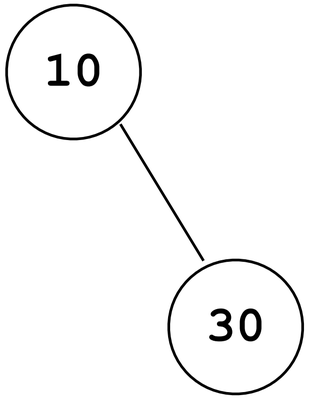
平衡二叉树平衡调整

LR 型失衡：插入在左子树的右子树，先向左旋转再向右旋转。



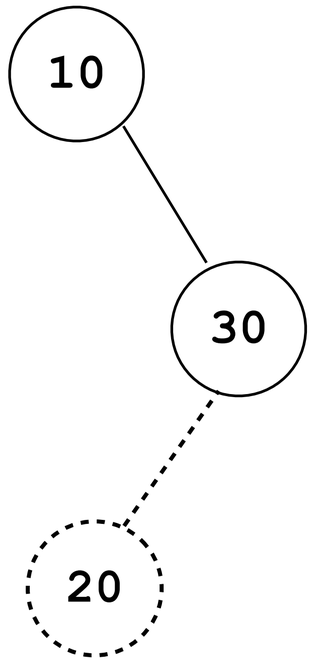
平衡二叉树平衡调整

RL 型失衡：插入在右子树的左子树，先向右旋转再向左旋转。



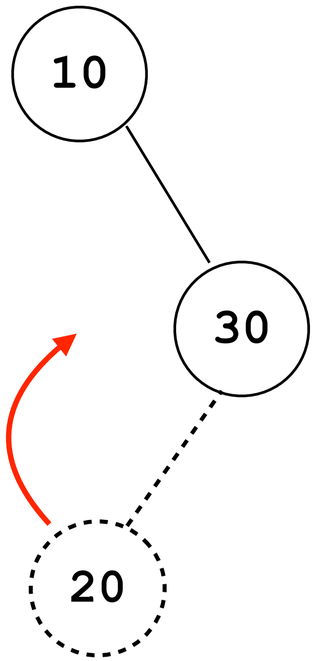
平衡二叉树平衡调整

RL 型失衡：插入在右子树的左子树，先向右旋转再向左旋转。



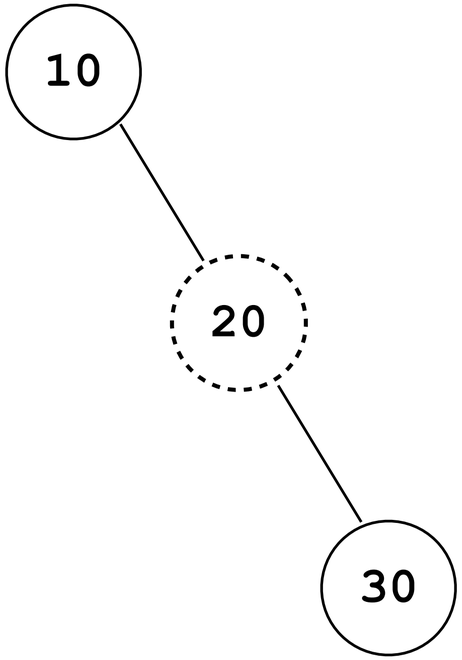
平衡二叉树平衡调整

RL 型失衡：插入在右子树的左子树，先向右旋转再向左旋转。



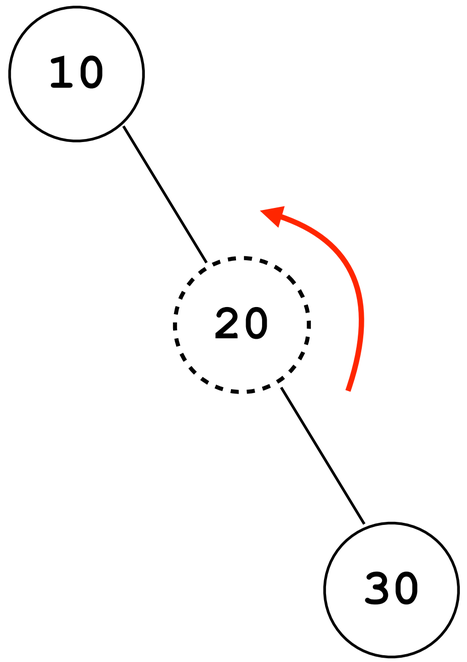
平衡二叉树平衡调整

RL 型失衡：插入在右子树的左子树，先向右旋转再向左旋转。



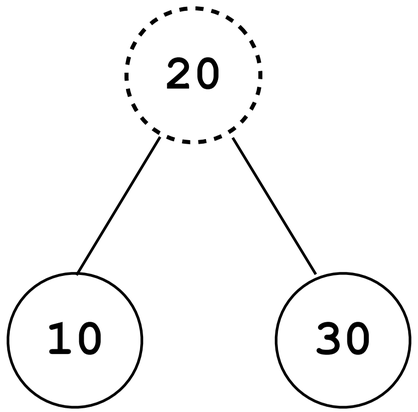
平衡二叉树平衡调整

RL 型失衡：插入在右子树的左子树，先向右旋转再向左旋转。



平衡二叉树平衡调整

RL 型失衡：插入在右子树的左子树，先向右旋转再向左旋转。



平衡二叉树平衡调整

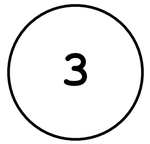
失衡类型	失衡点	调整方式
LL型	祖先结点的左子树	右旋
RR型	祖先结点的右子树	左旋
LR型	左子树的右子树	左旋+右旋
RL型	右子树的左子树	右旋+左旋

平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。

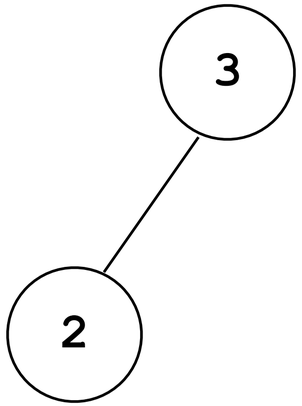
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



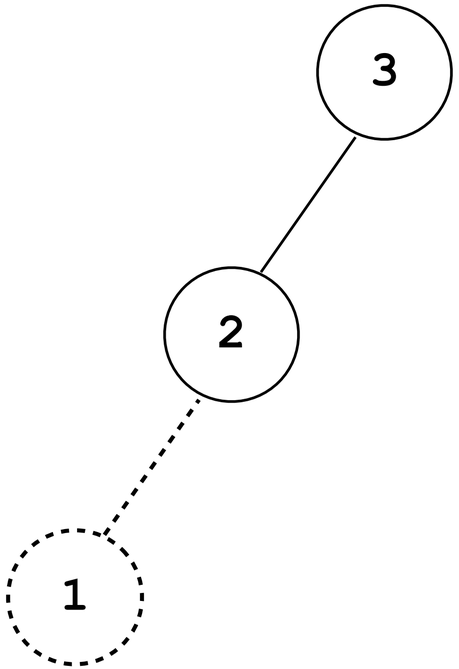
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



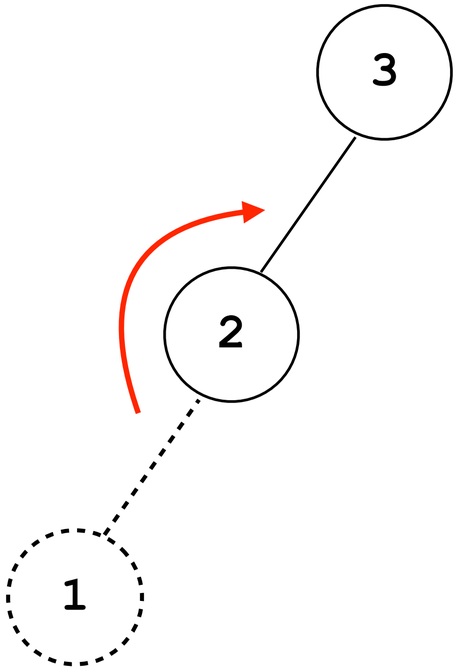
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



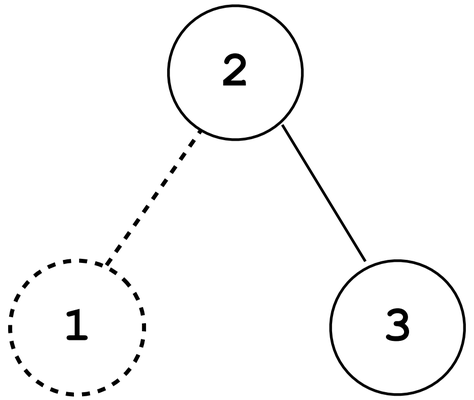
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



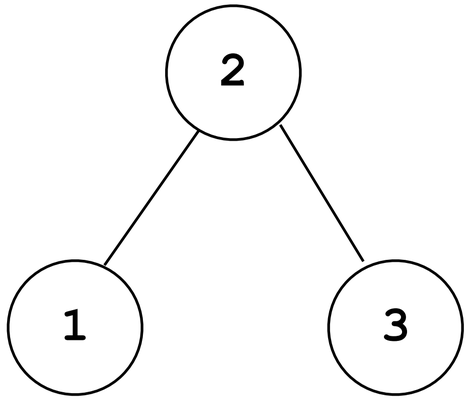
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



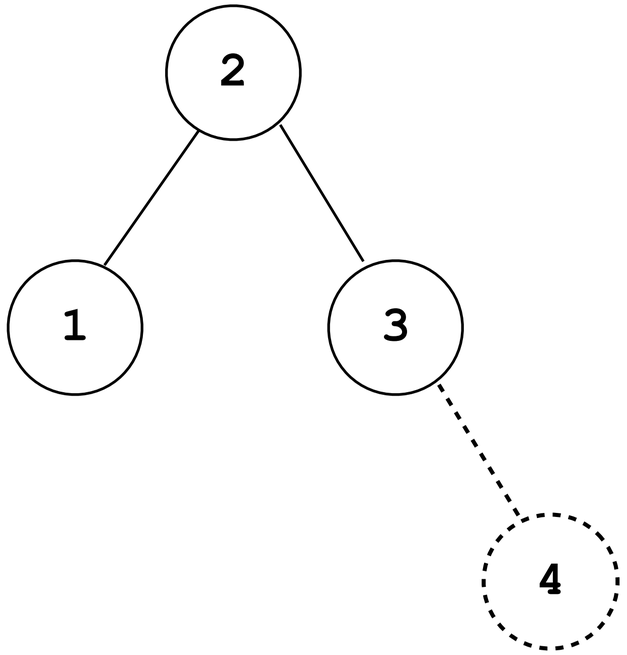
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



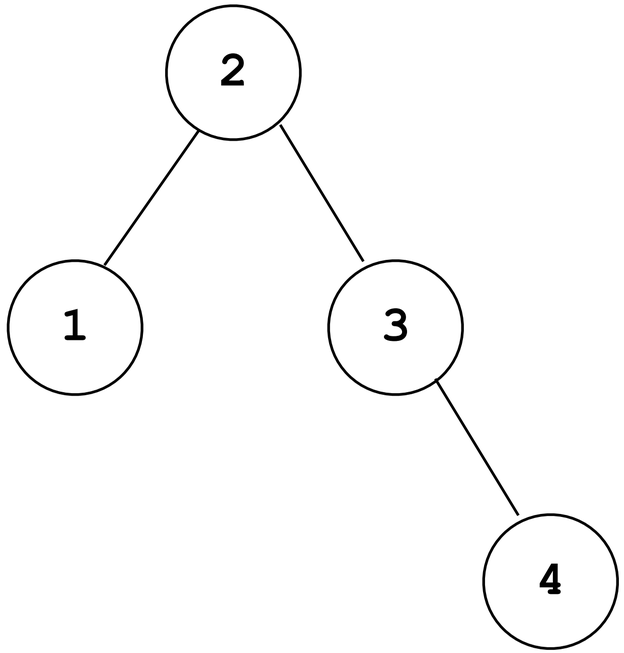
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



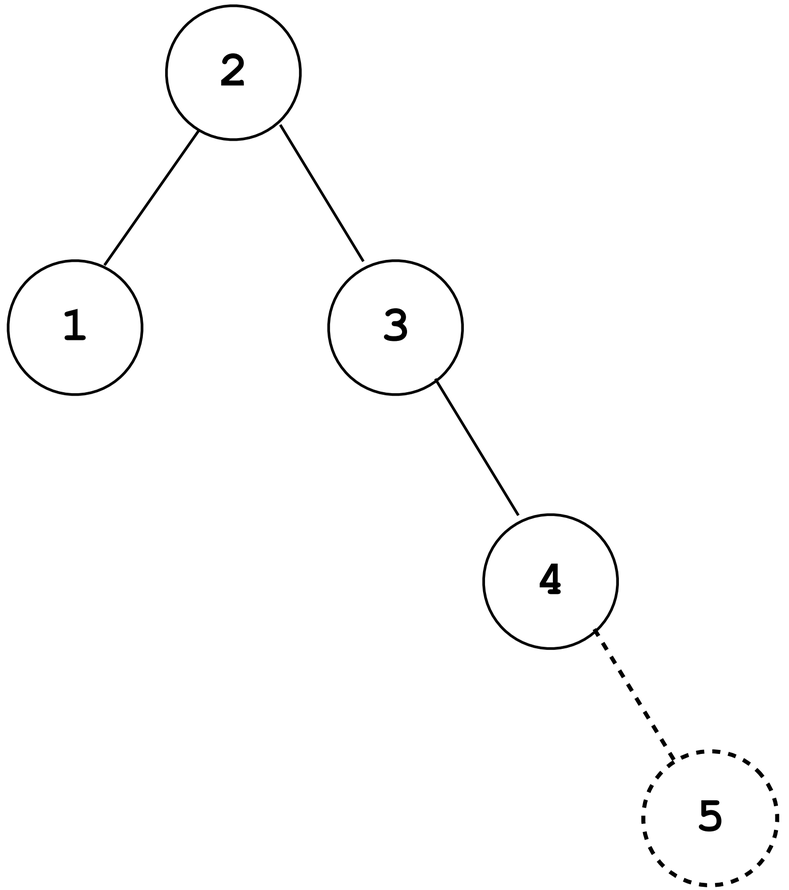
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



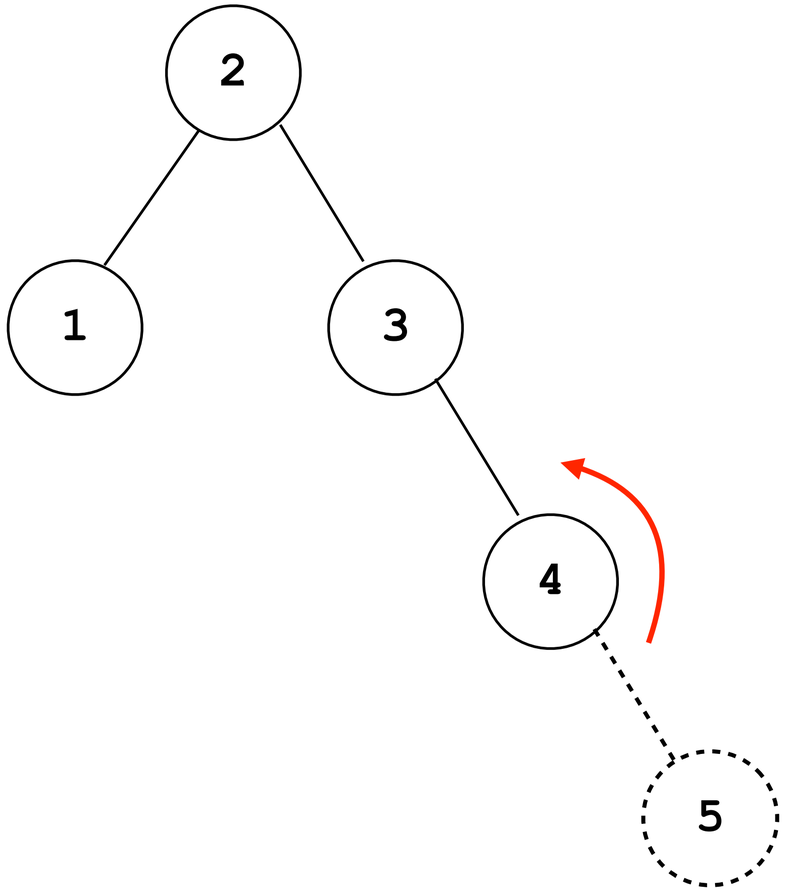
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



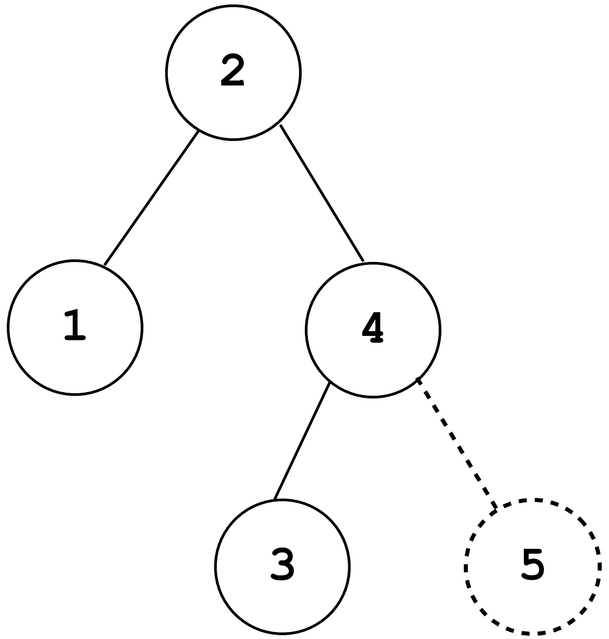
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



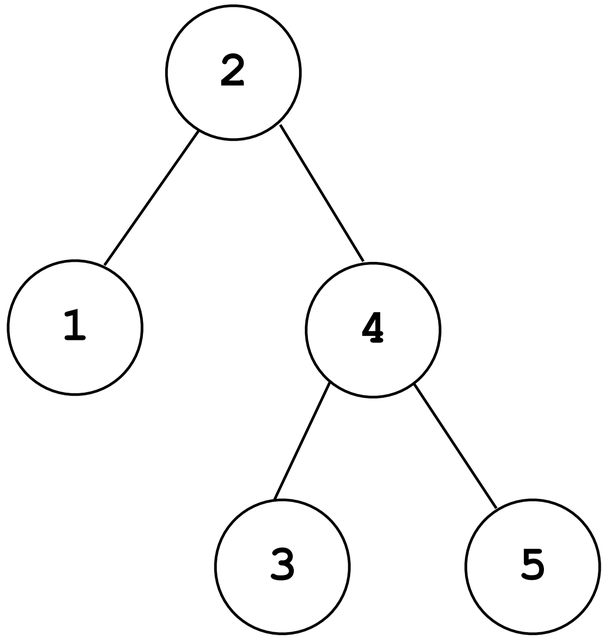
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



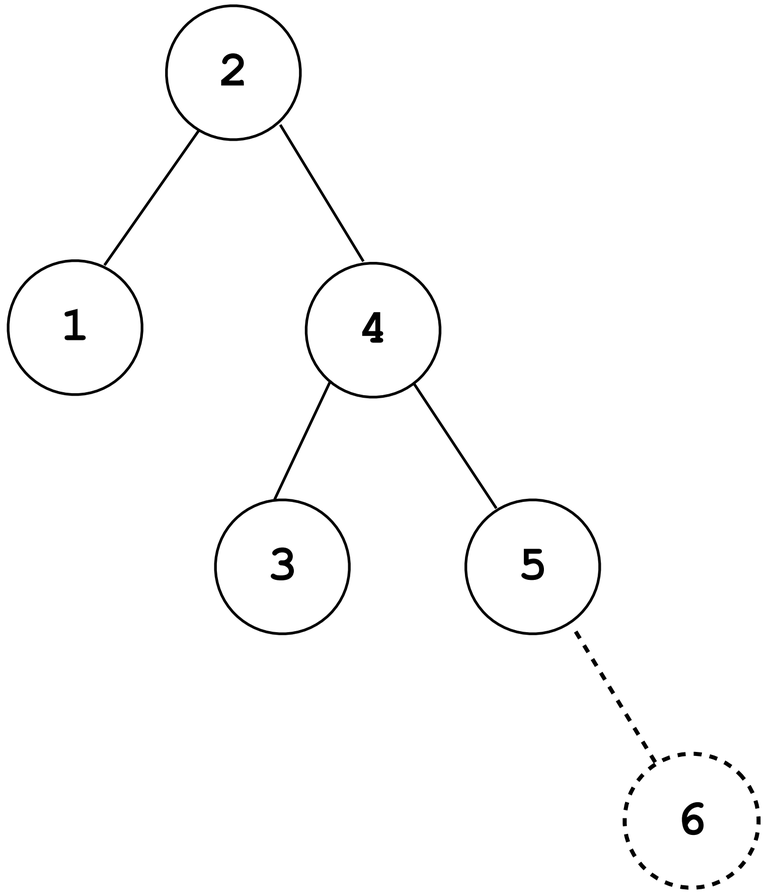
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



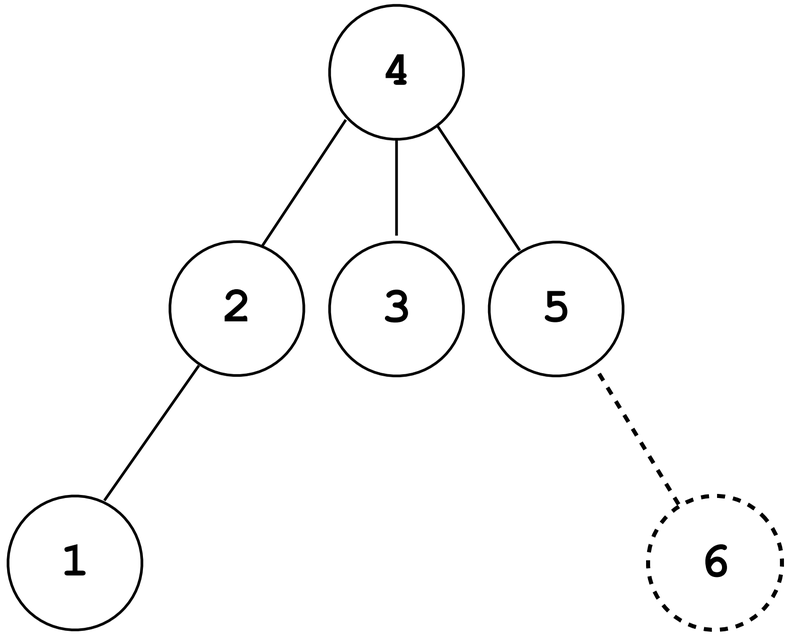
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



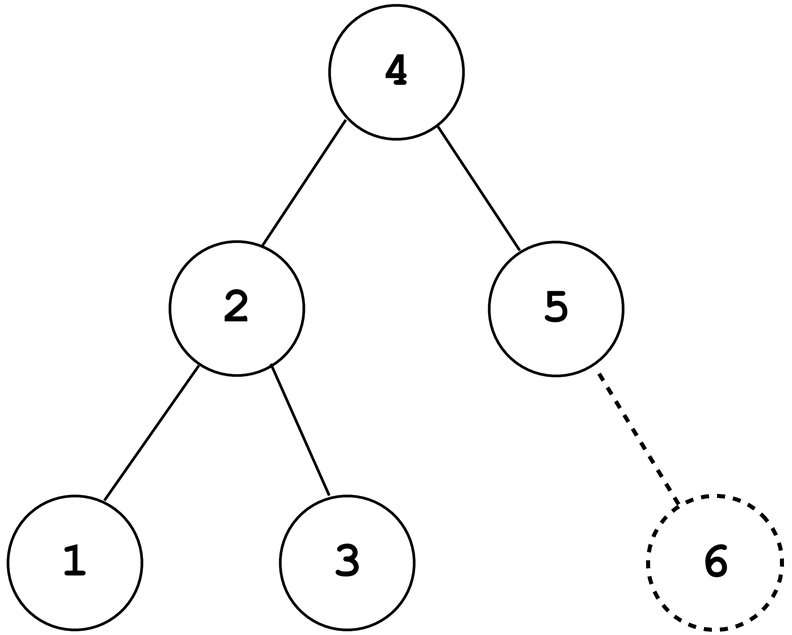
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



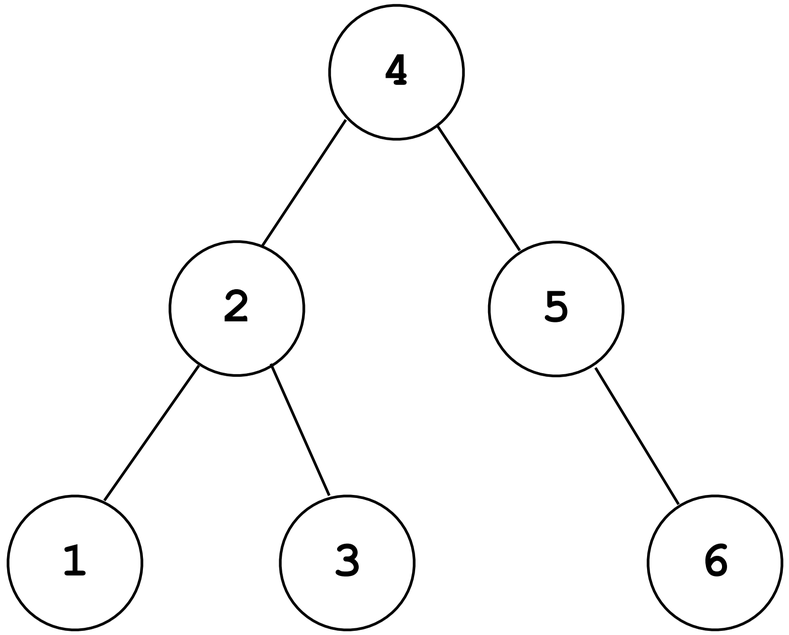
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



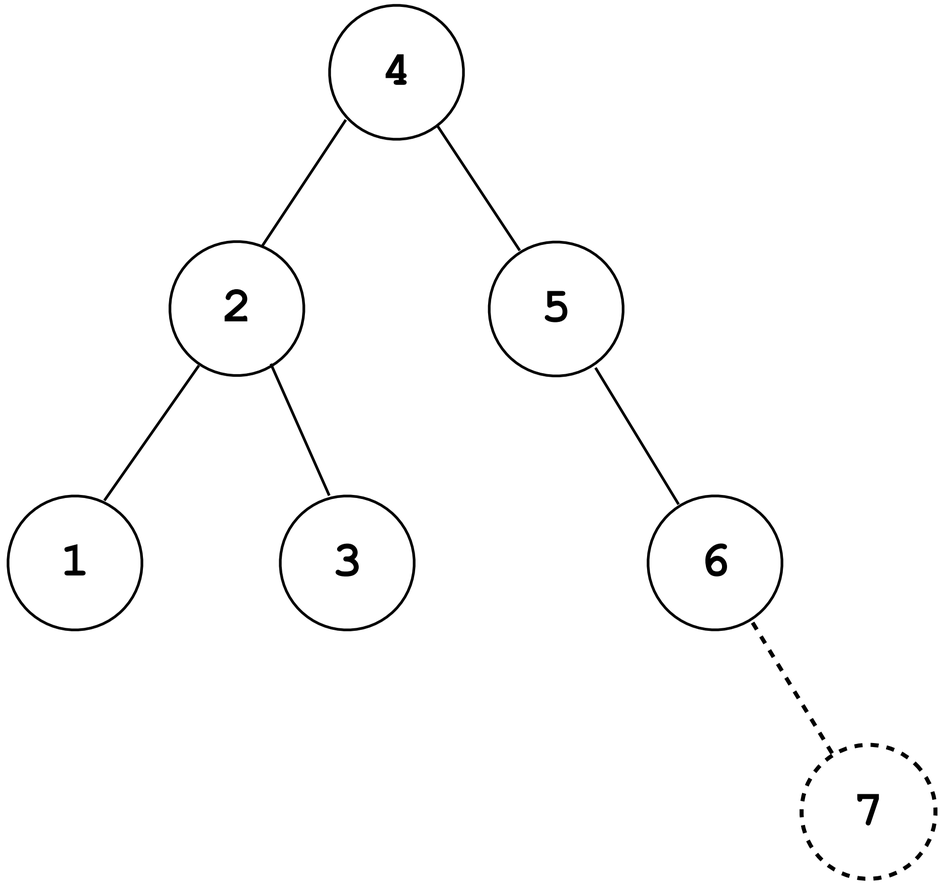
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



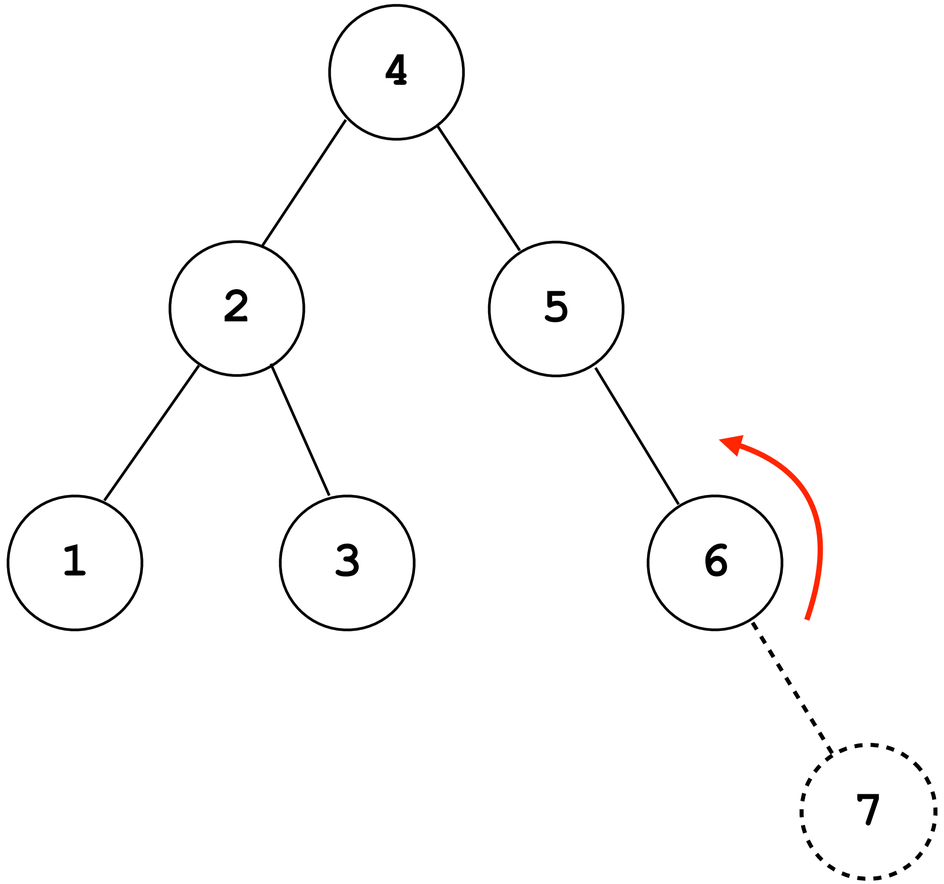
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



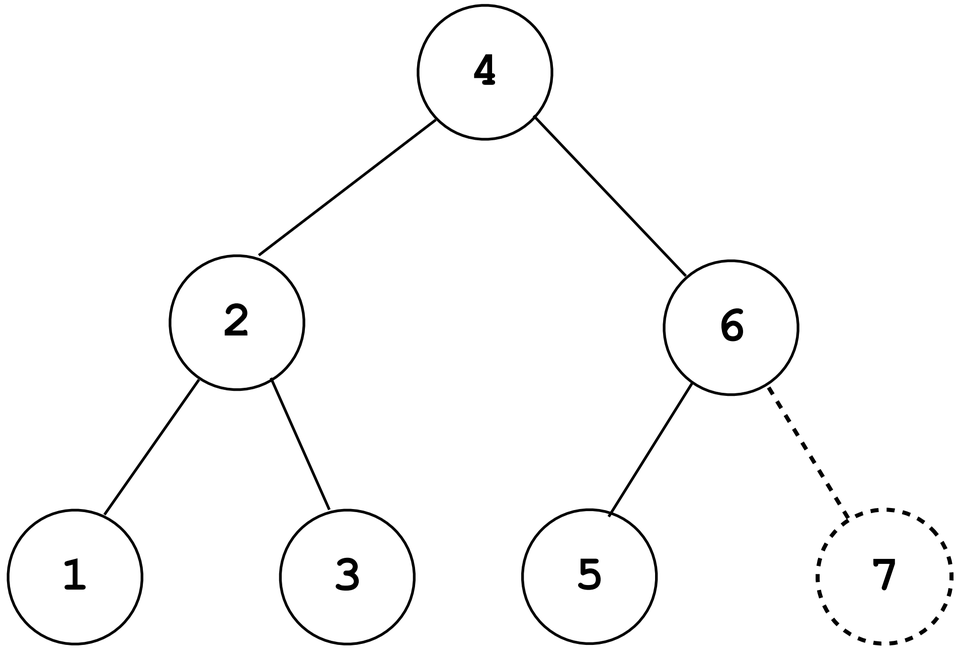
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



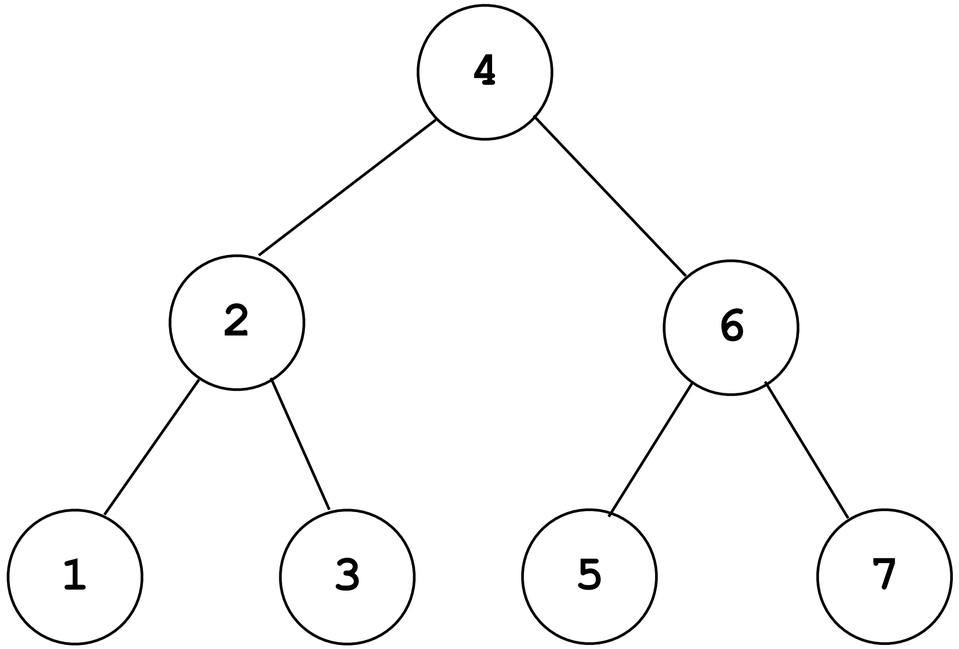
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



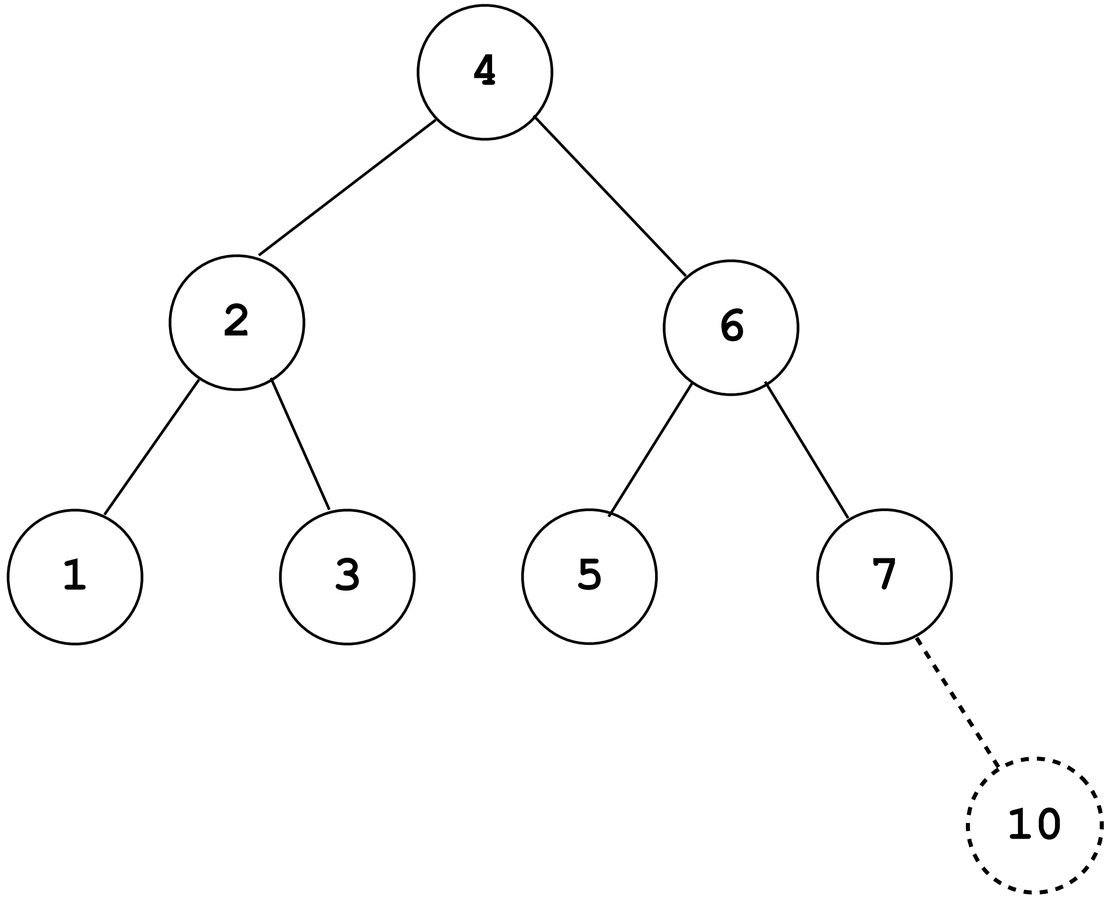
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



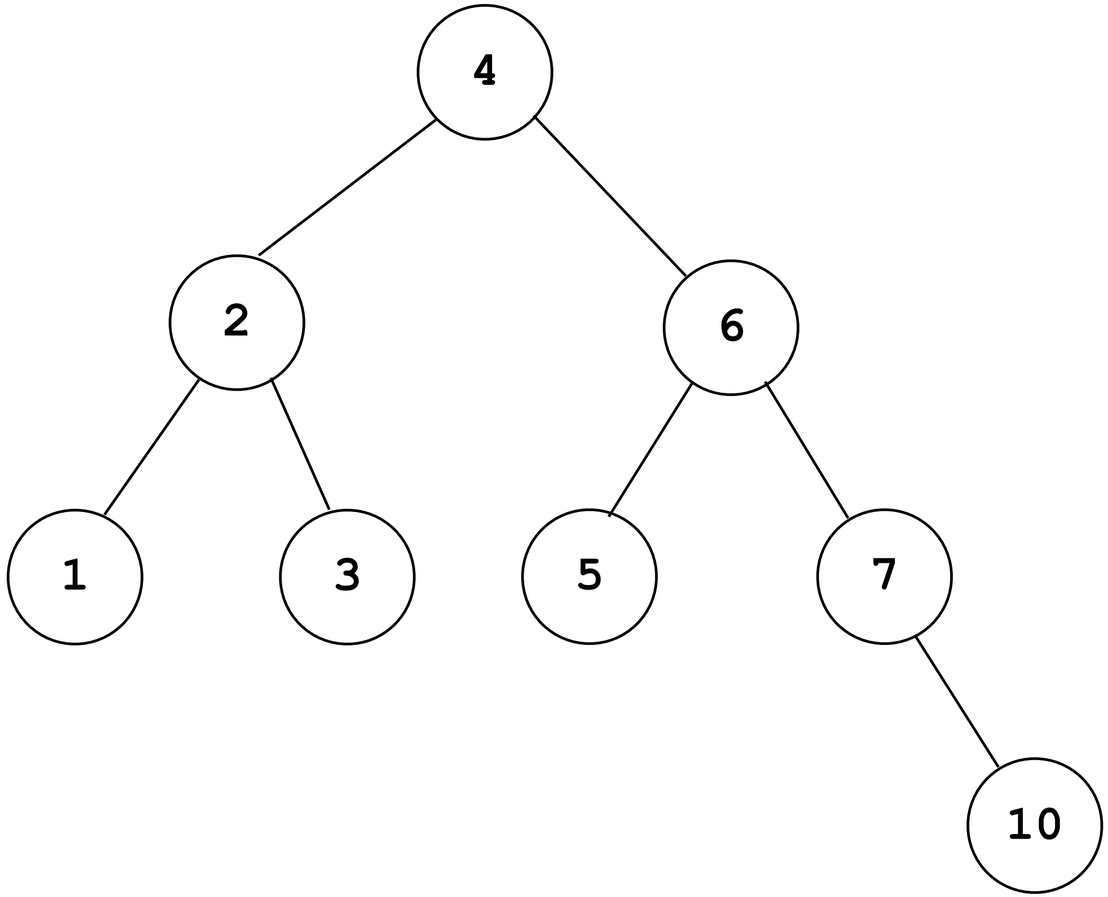
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



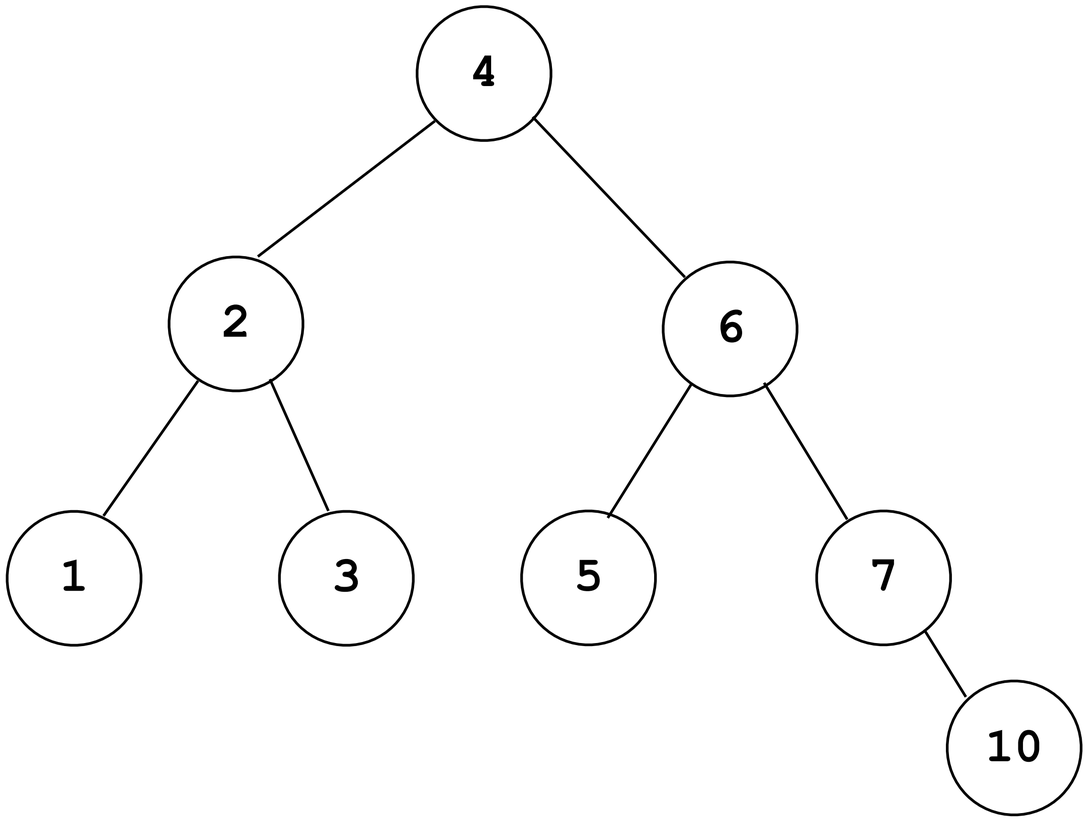
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



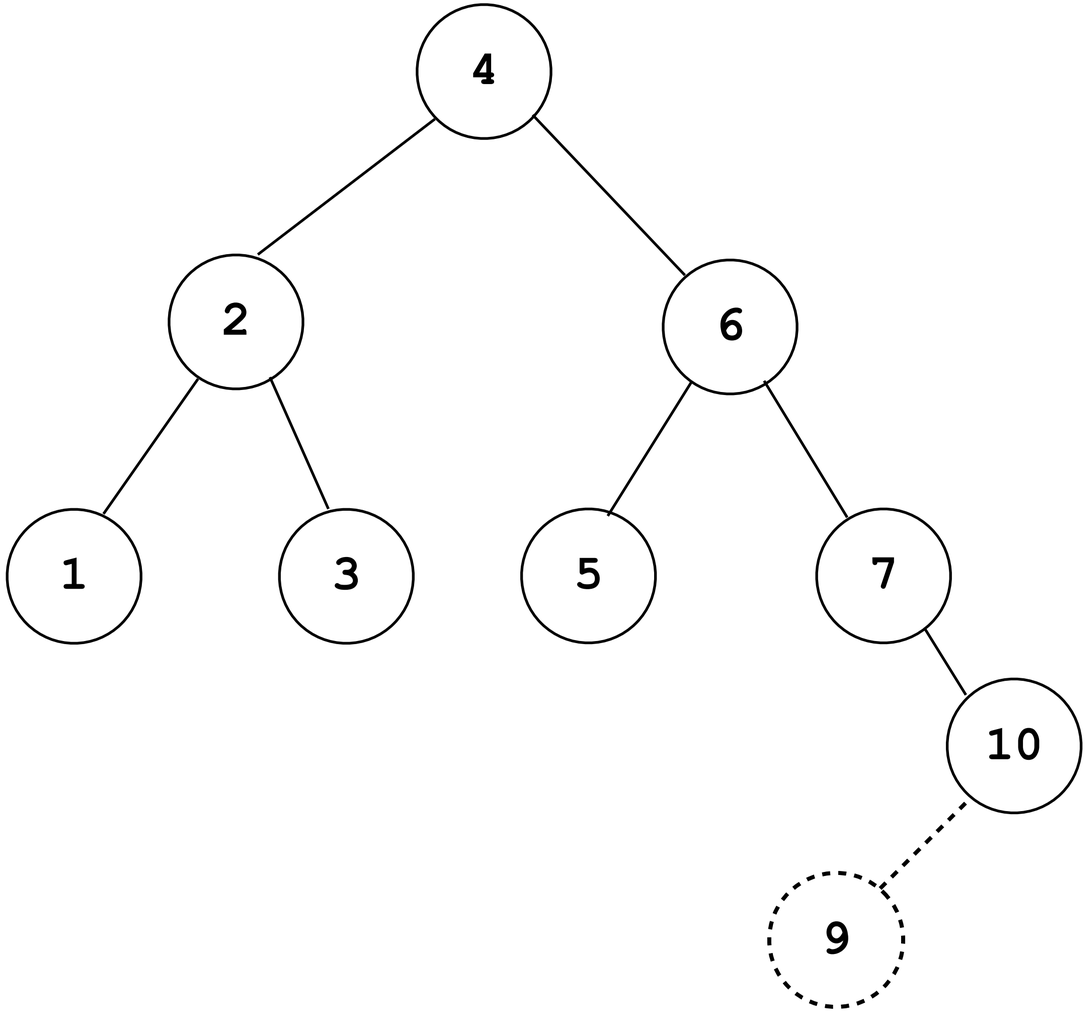
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



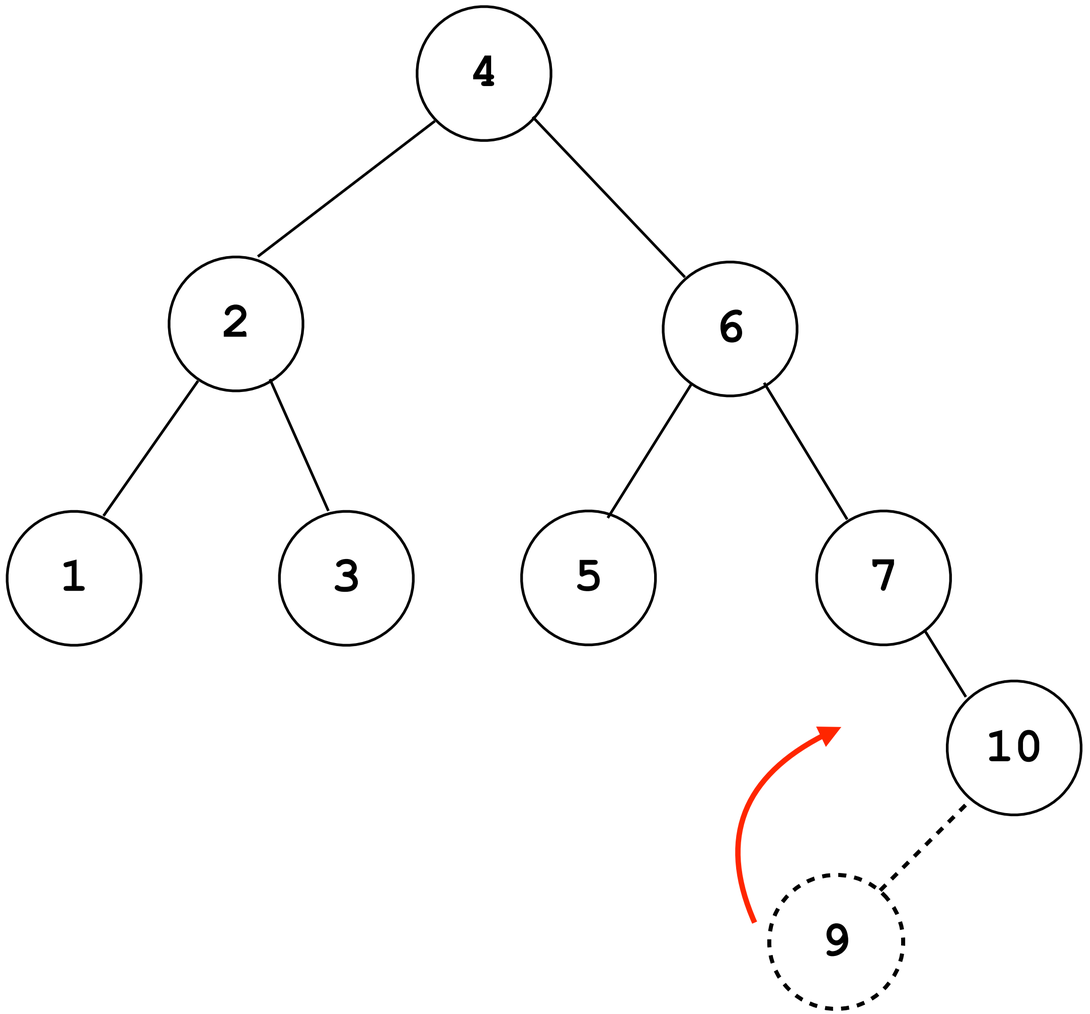
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



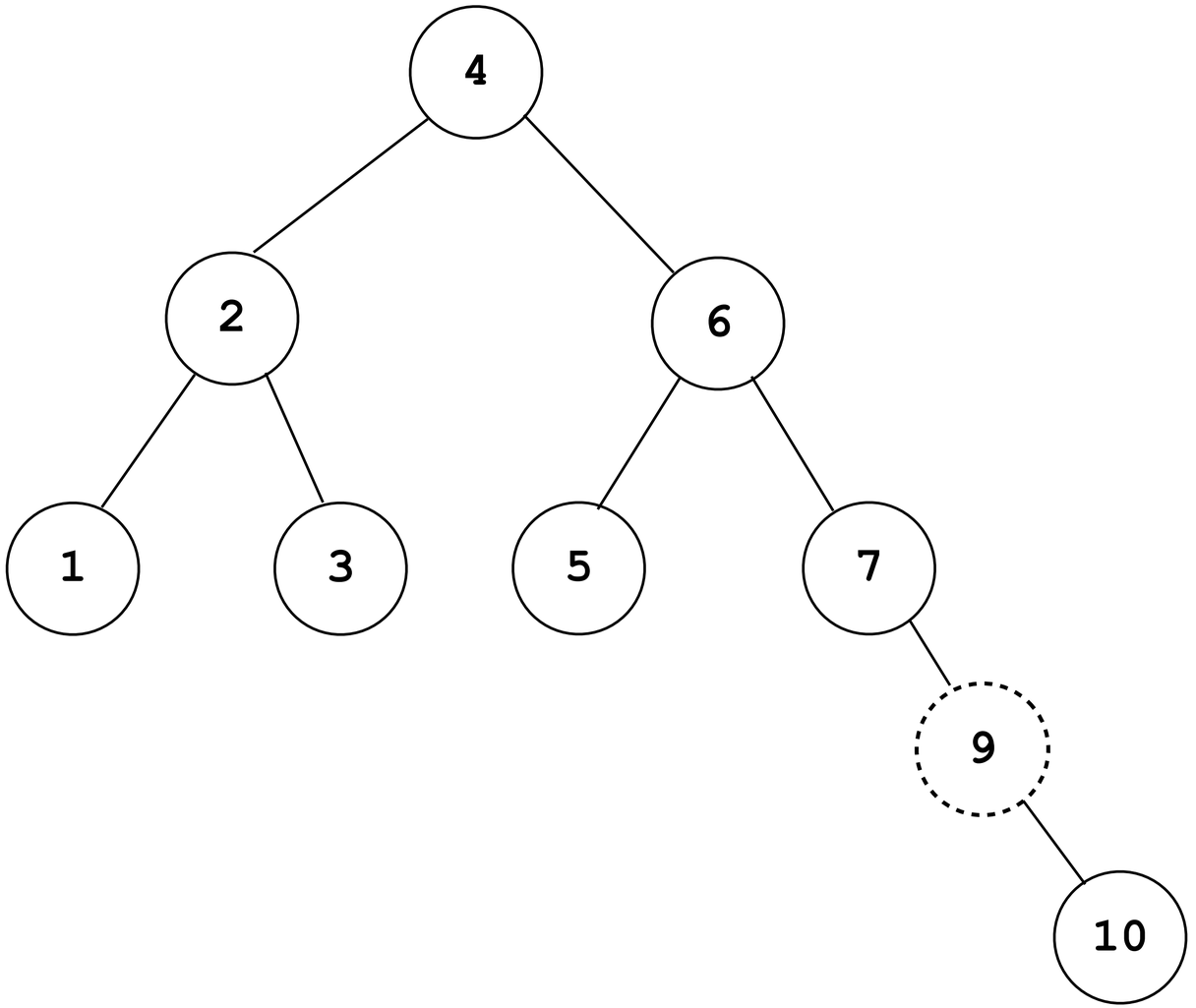
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



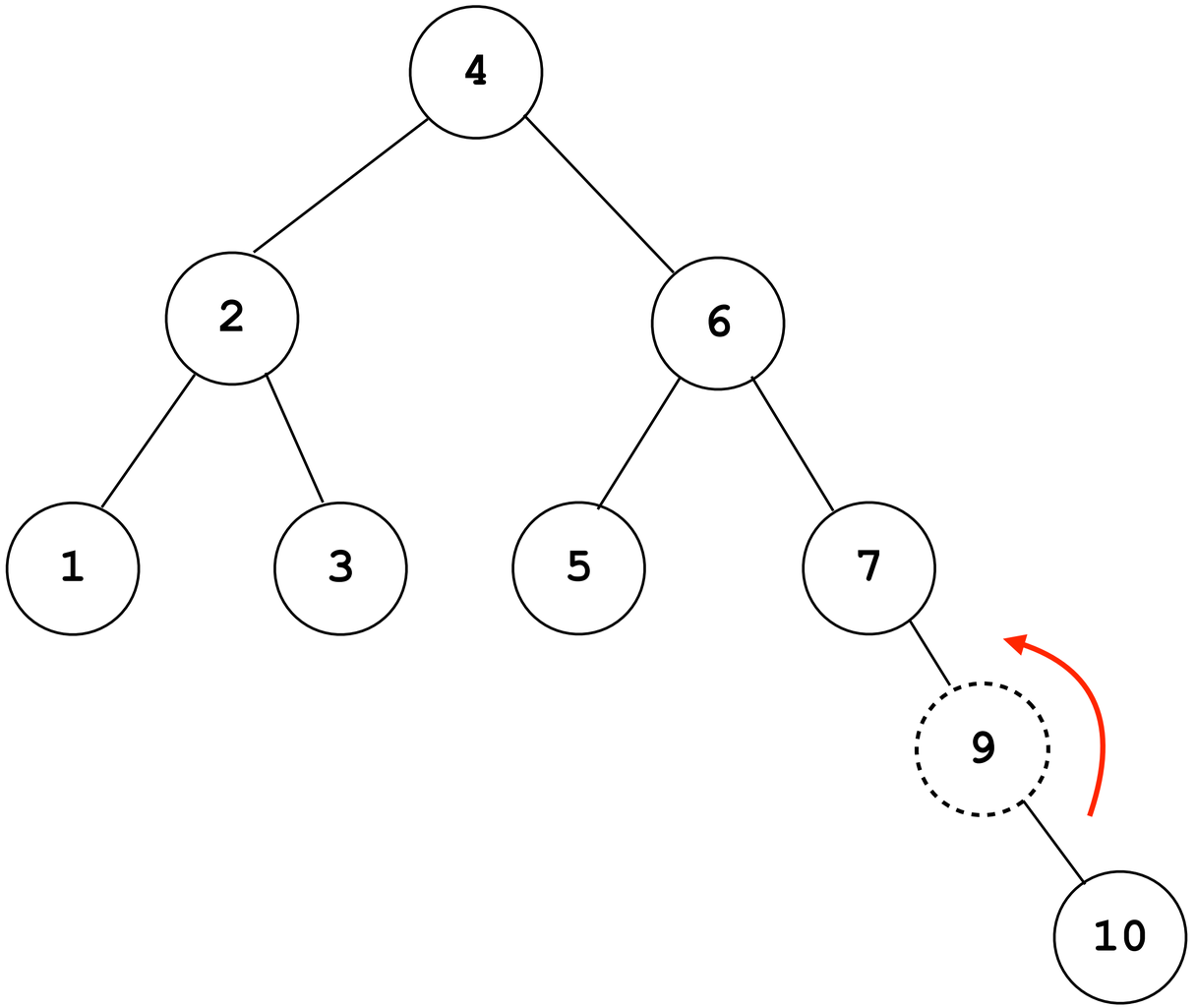
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



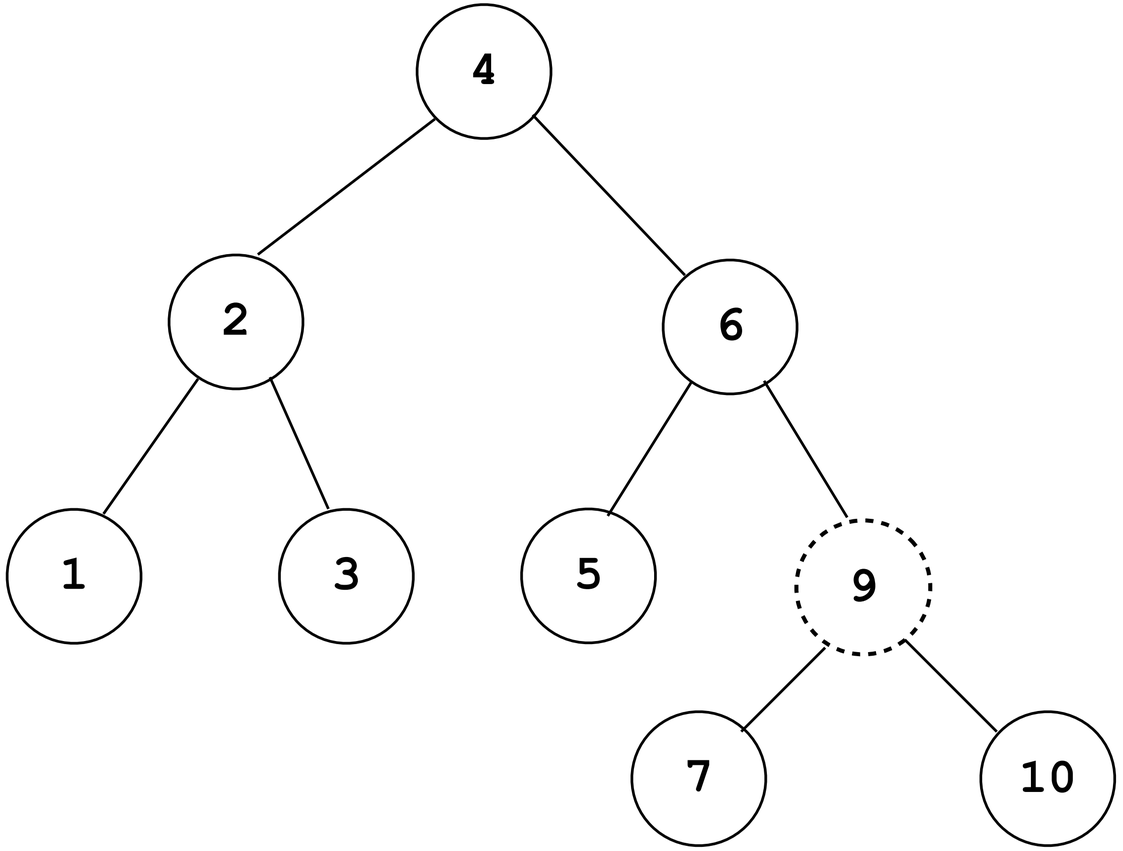
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



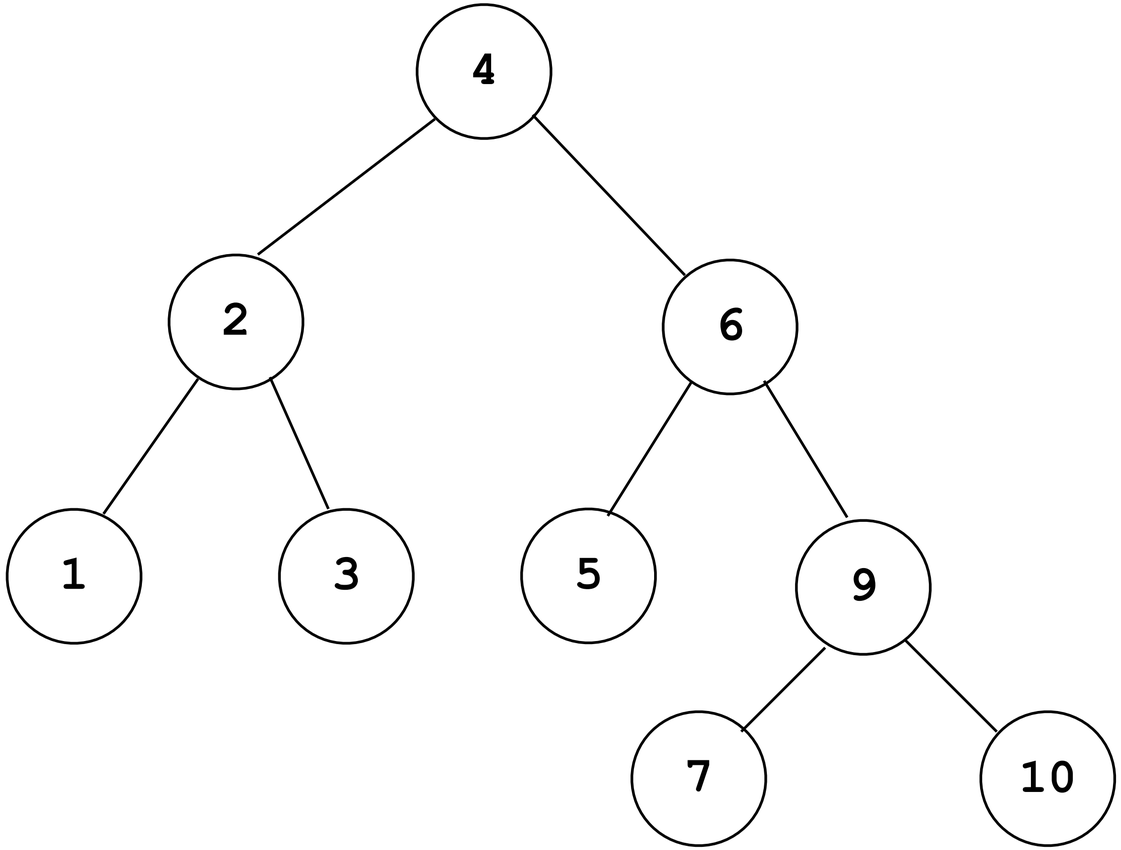
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



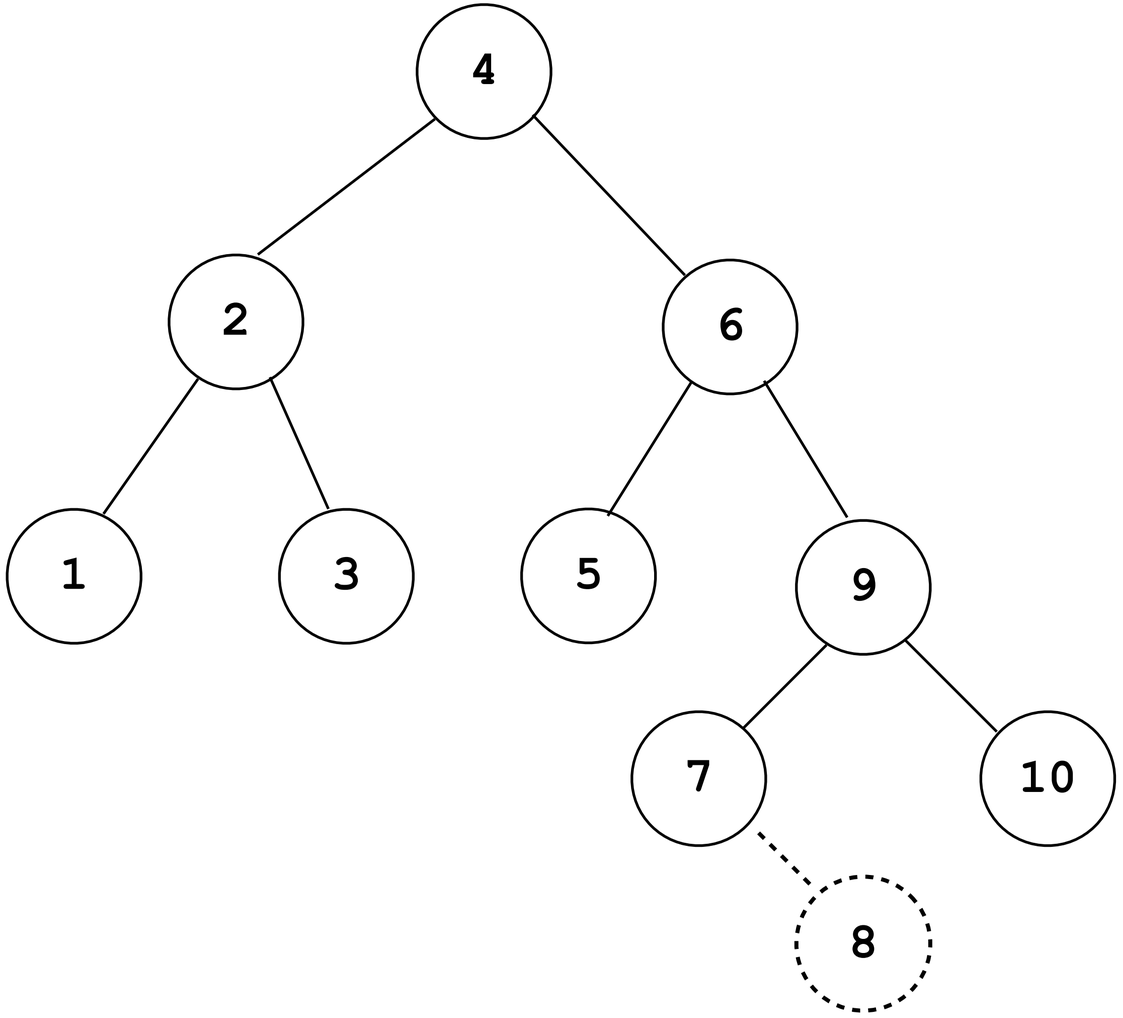
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



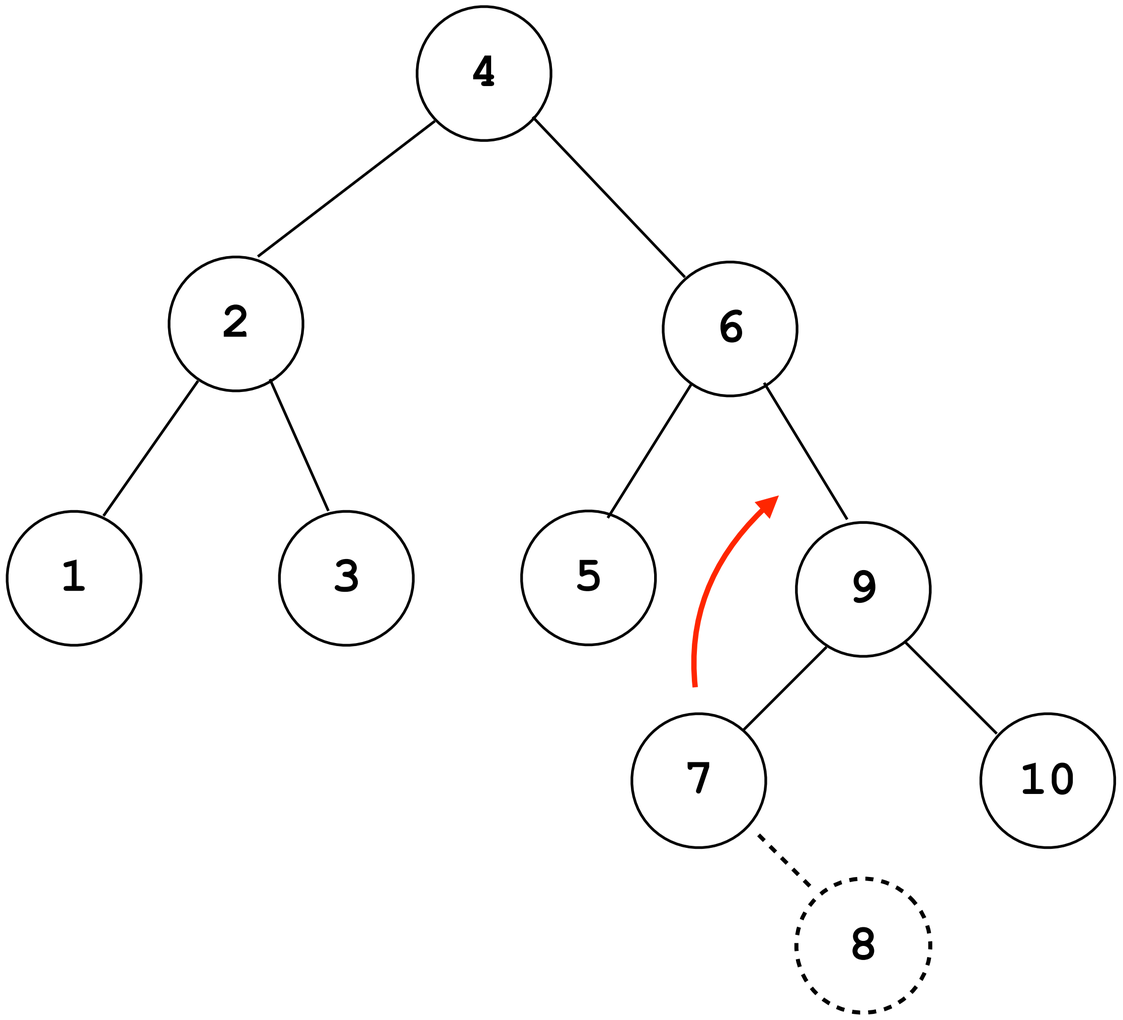
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



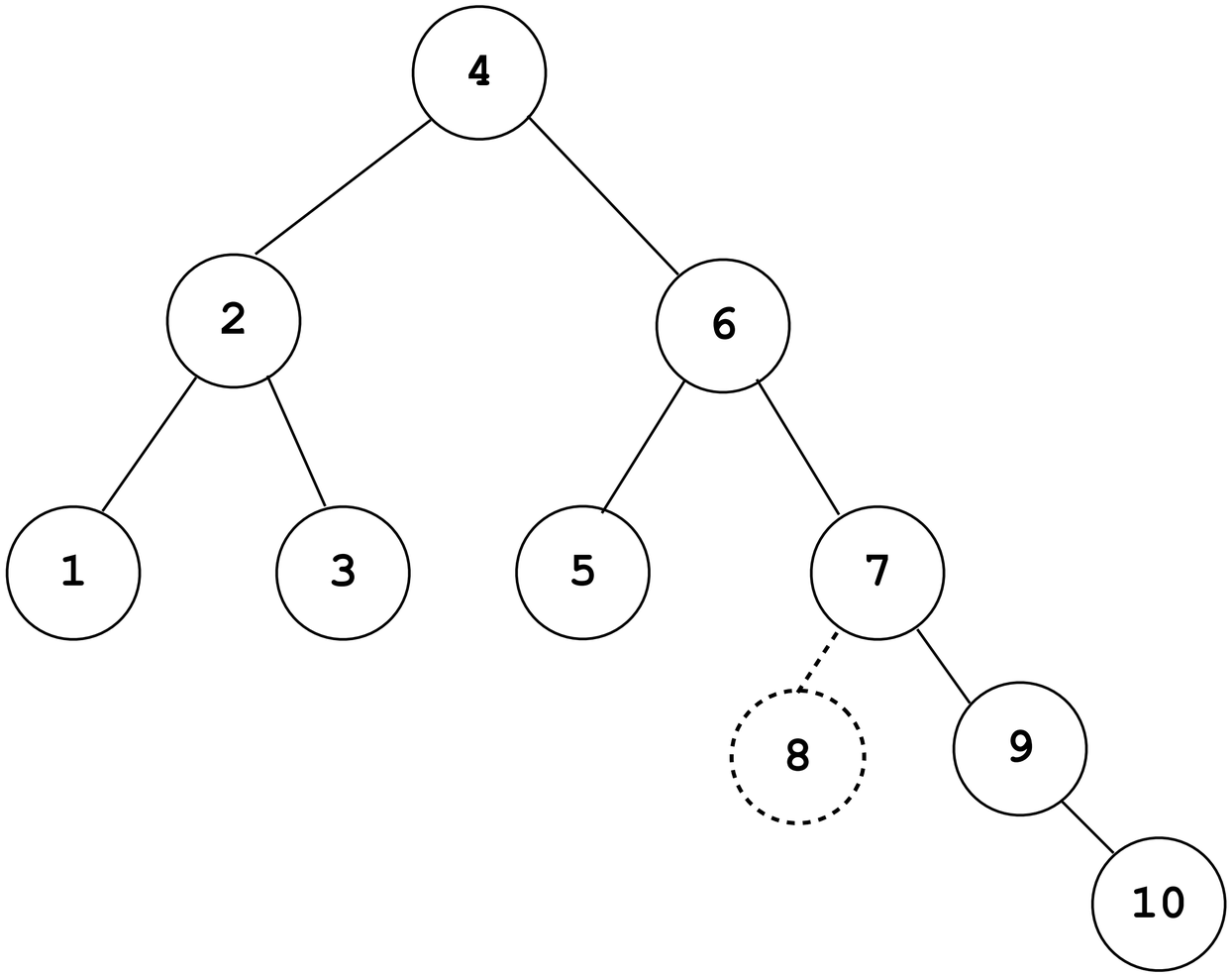
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



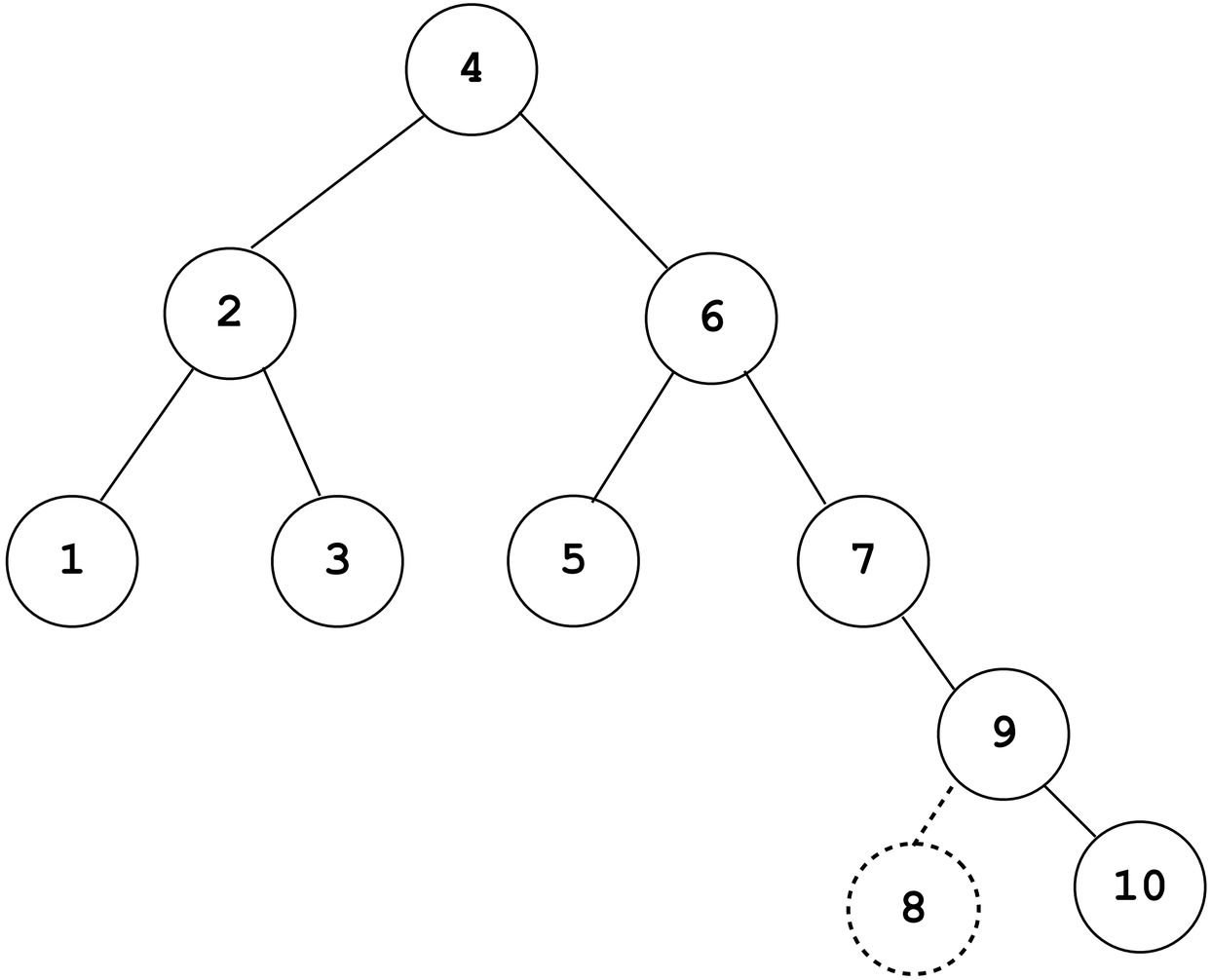
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



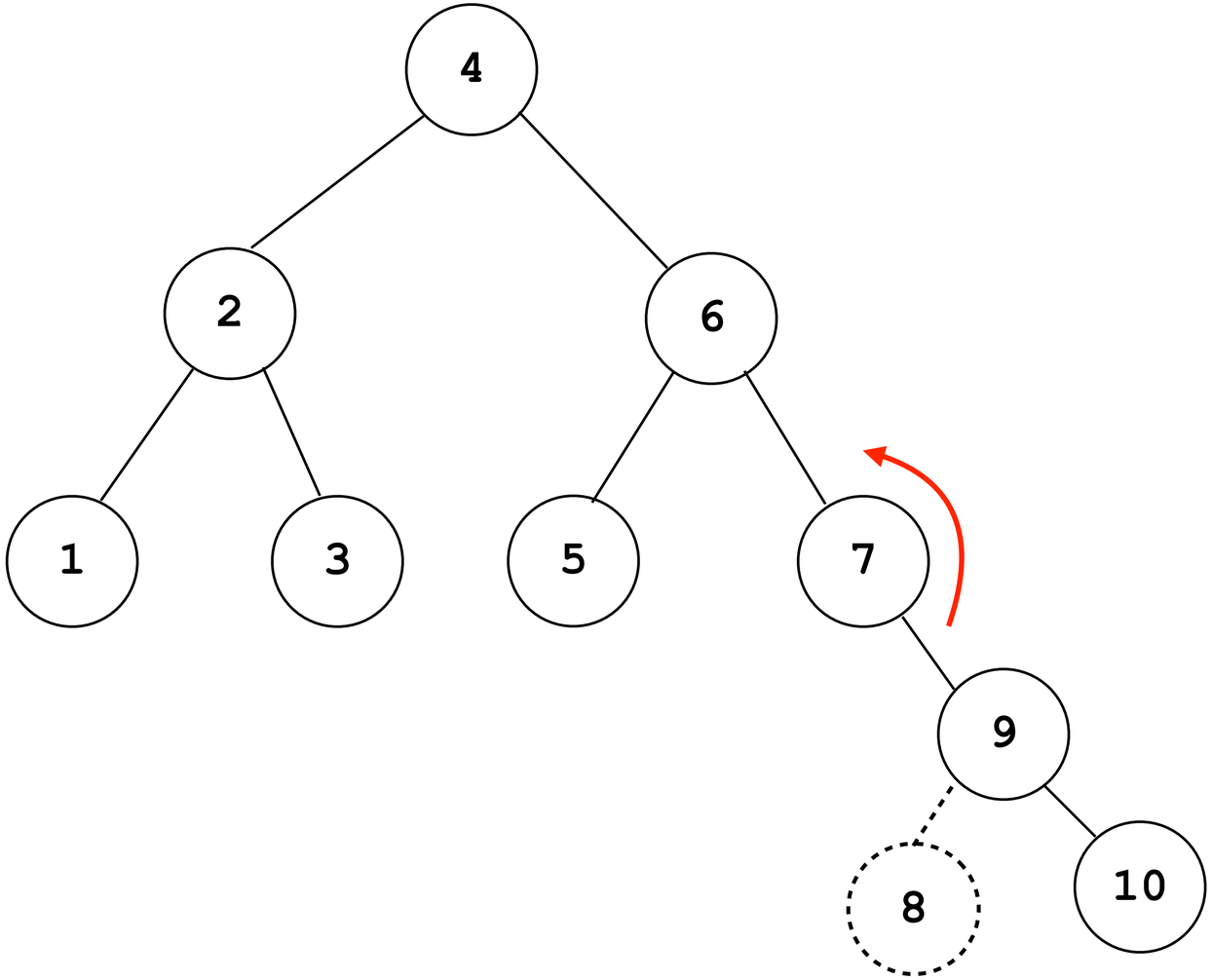
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



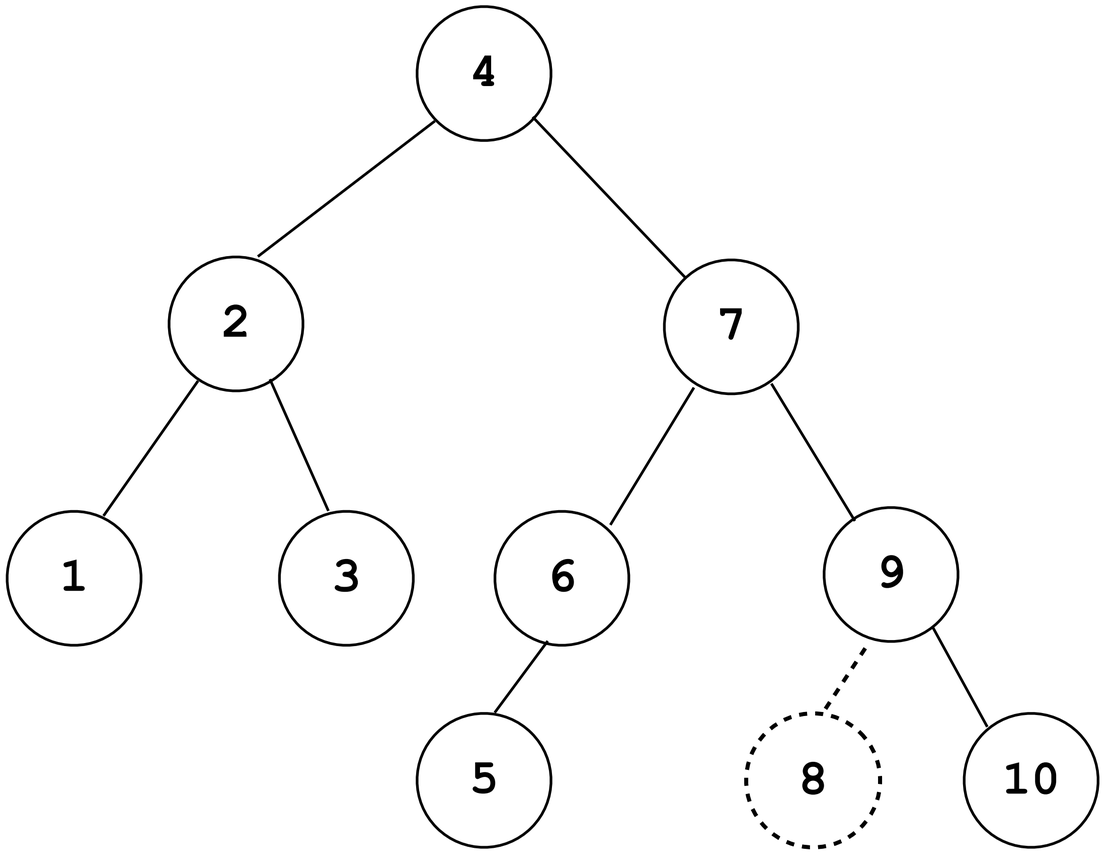
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



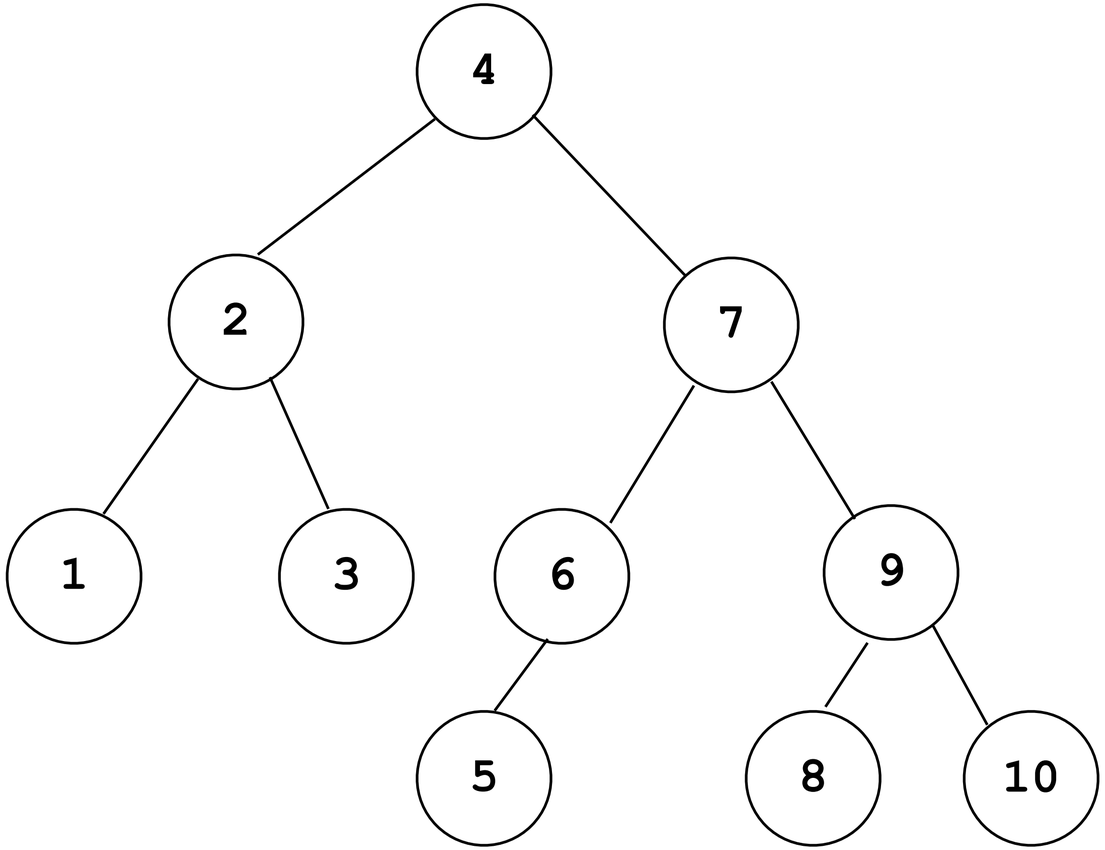
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。



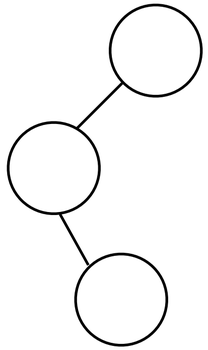
平衡二叉树平衡调整

例 构造一棵平衡二叉树，依次将{3, 2, 1, 4, 5, 6, 7, 10, 9, 8}插入到平衡二叉树中。

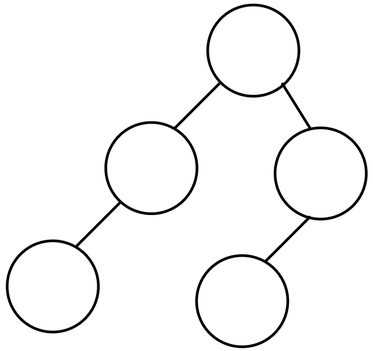


真题解析

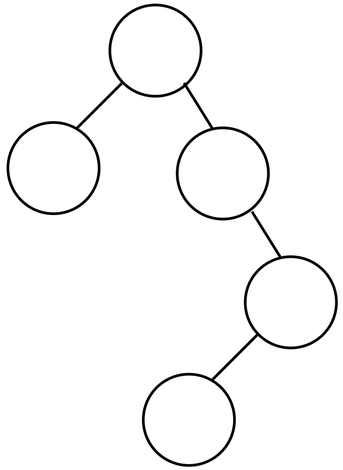
[2009] 4. 下列二叉排序树中，满足平衡二叉树定义的是_____。



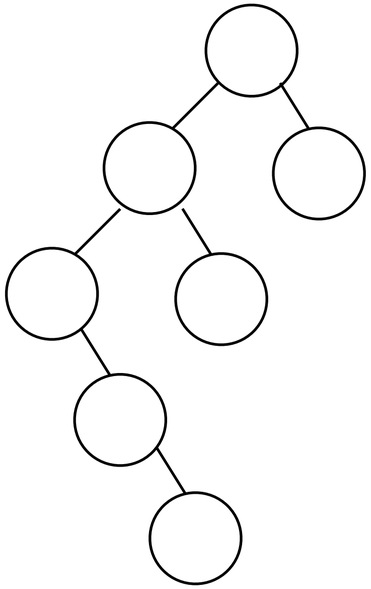
A.



B.



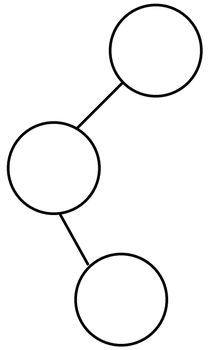
C.



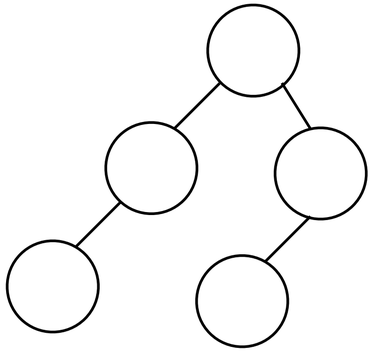
D.

真题解析

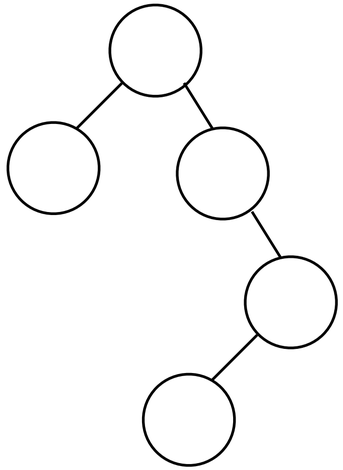
[2009] 4. 下列二叉排序树中，满足平衡二叉树定义的是_____。



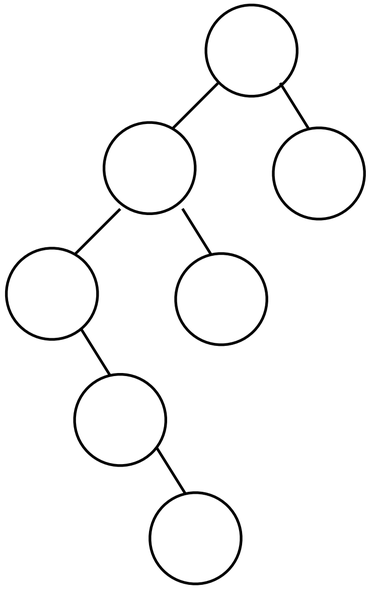
A.



B.



C.



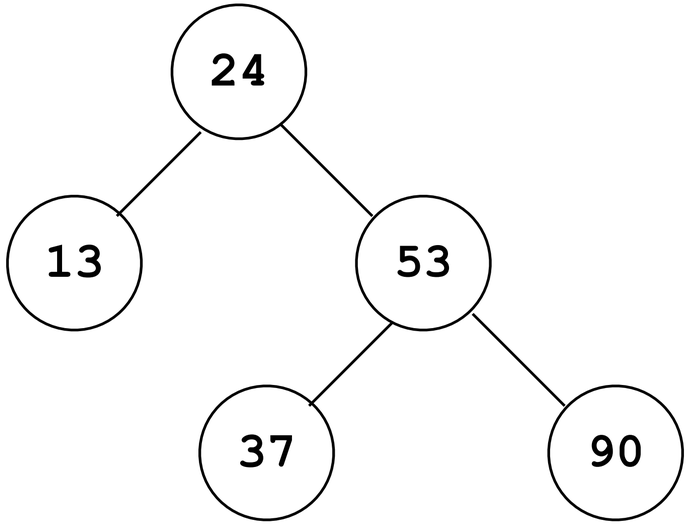
D.

真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。

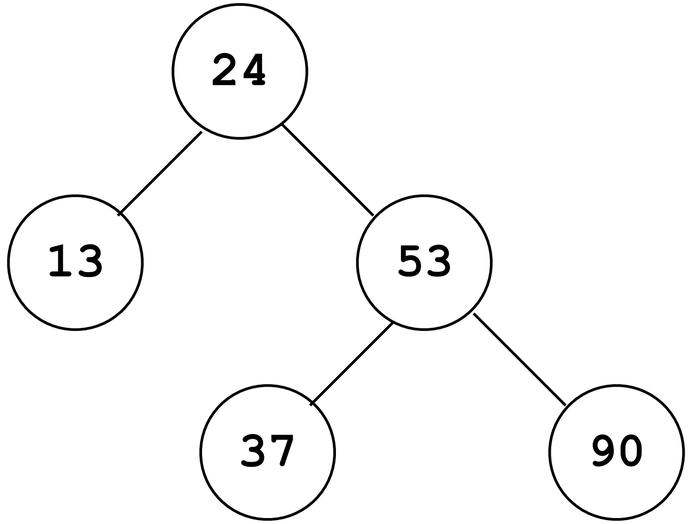
A. 13, 48　　B. 24, 48

C. 24, 53　　D. 24, 90



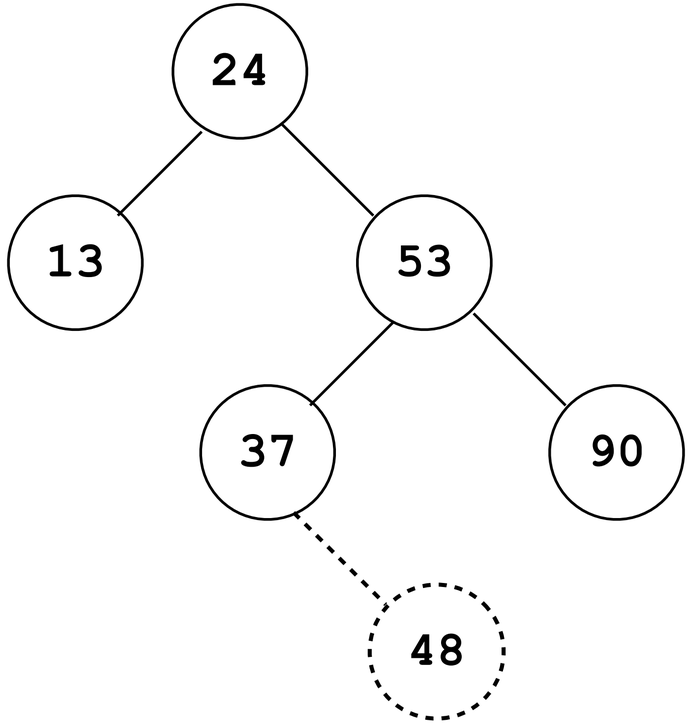
真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。



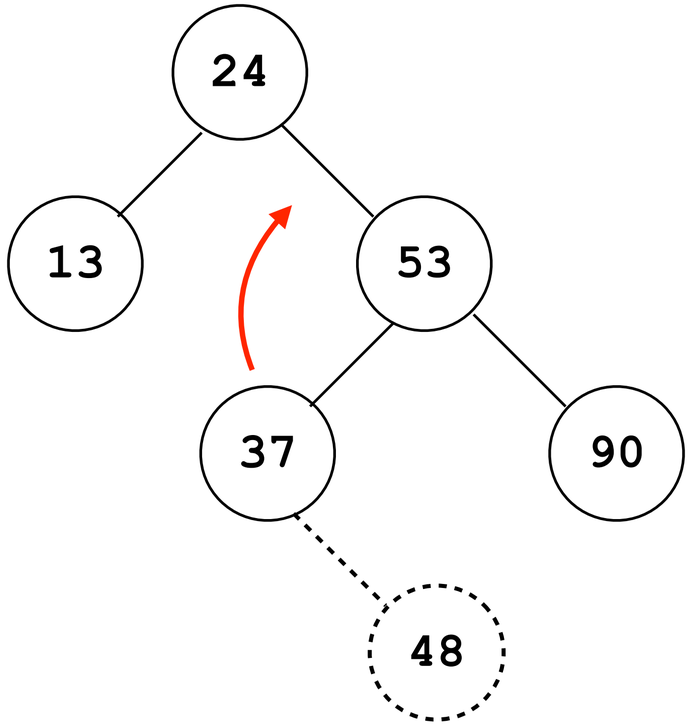
真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。



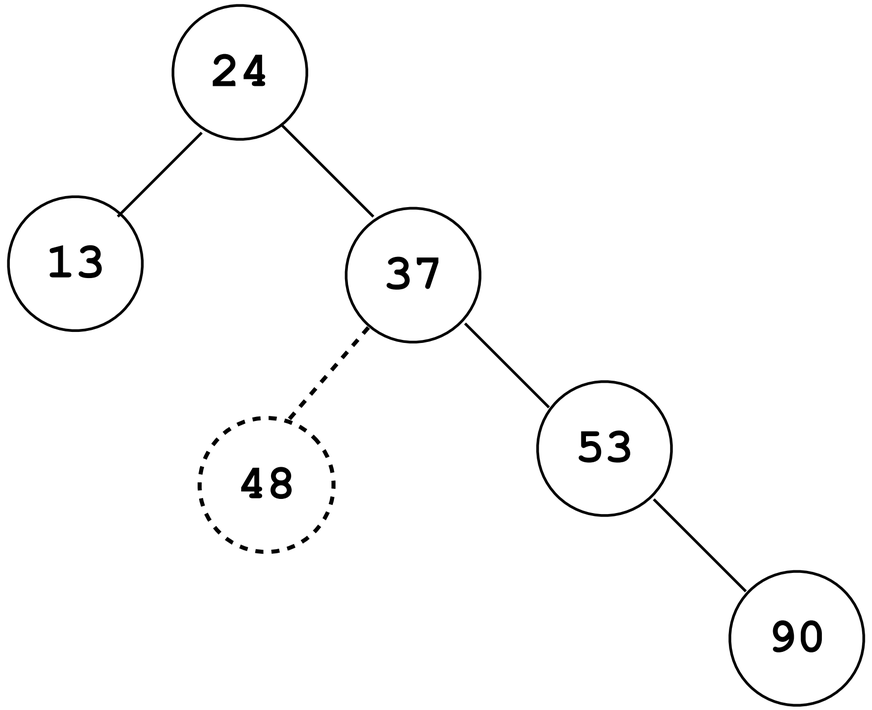
真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。



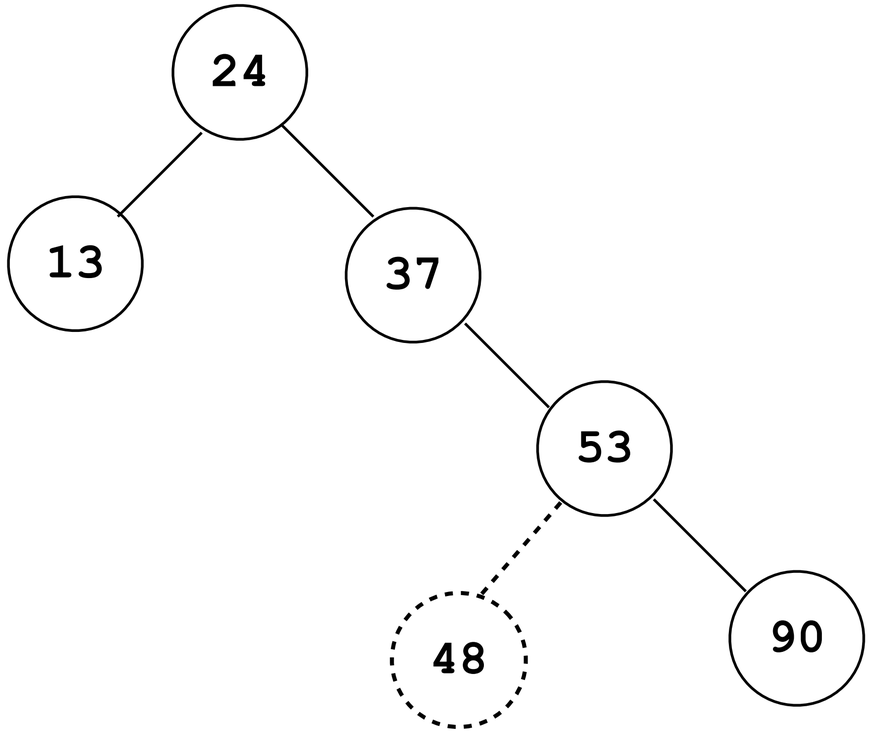
真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。



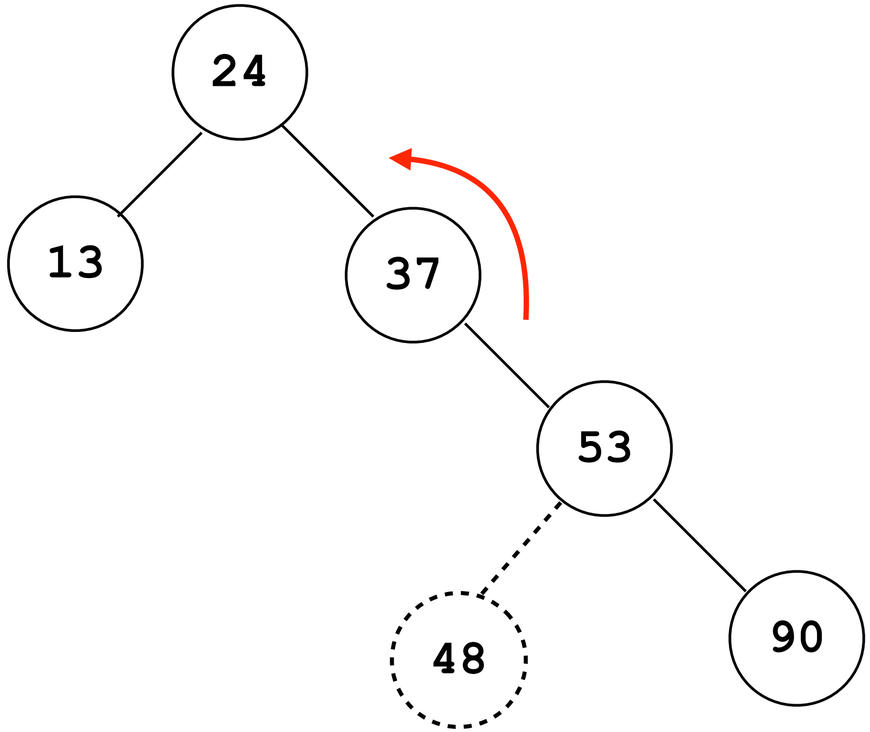
真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。



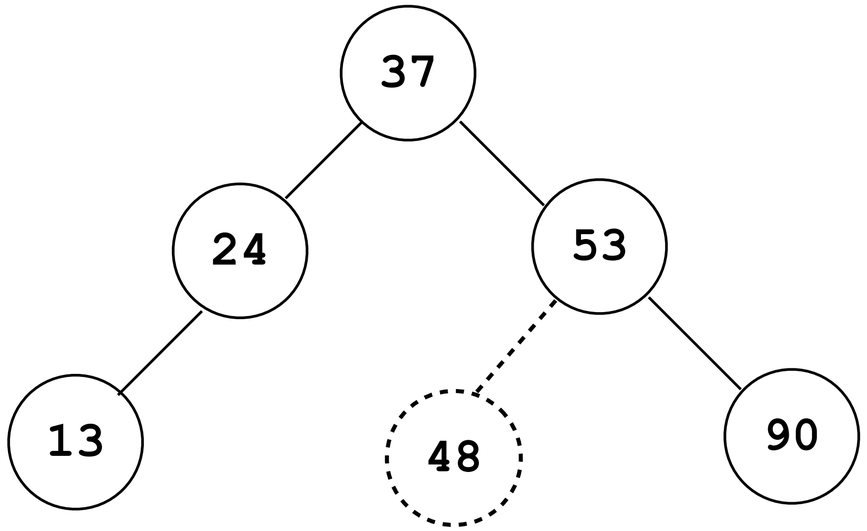
真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。



真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。

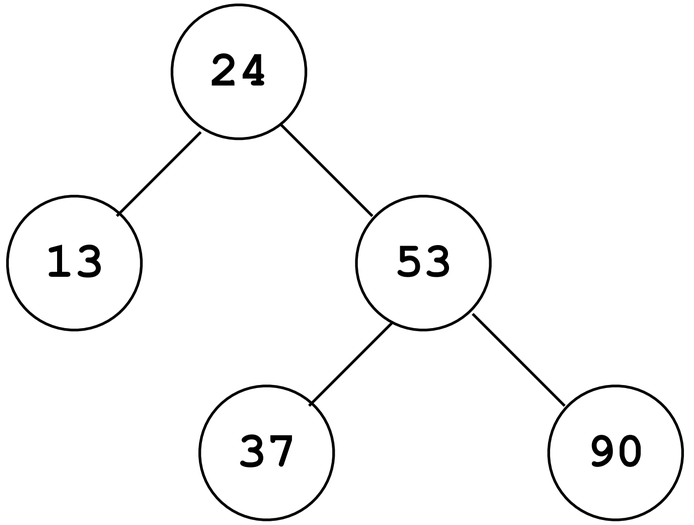


真题解析

[2010] 4. 在图中所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。

A. 13, 48　　B. 24, 48

C. 24, 53　　D. 24, 90



真题解析

[2010] 9. 已知一个长度为 16 的顺序表 L，其元素按关键字有序排列。若采用折半查找法查找一个 L 中不存在的元素，则关键字的比较次数最多的是_____。

A. 4　　B. 5　　C. 6　　D. 7

真题解析

[2010] 9. 已知一个长度为 16 的顺序表 L，其元素按关键字有序排列。若采用折半查找法查找一个 L 中不存在的元素，则关键字的比较次数最多的是_____。

A. 4　　B. 5　　C. 6　　D. 7

$\lfloor \log_2 n \rfloor + 1$

真题解析

[2010] 9. 已知一个长度为 16 的顺序表 L，其元素按关键字有序排列。若采用折半查找法查找一个 L 中不存在的元素，则关键字的比较次数最多的是_____。

A. 4　　B. 5　　C. 6　　D. 7

真题解析

[2011] 7. 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是___。

A. 95, 22, 91, 24, 94, 71

B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34

[2011] 7. 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是___。

A. 95, 22, 91, 24, 94, 71

B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34

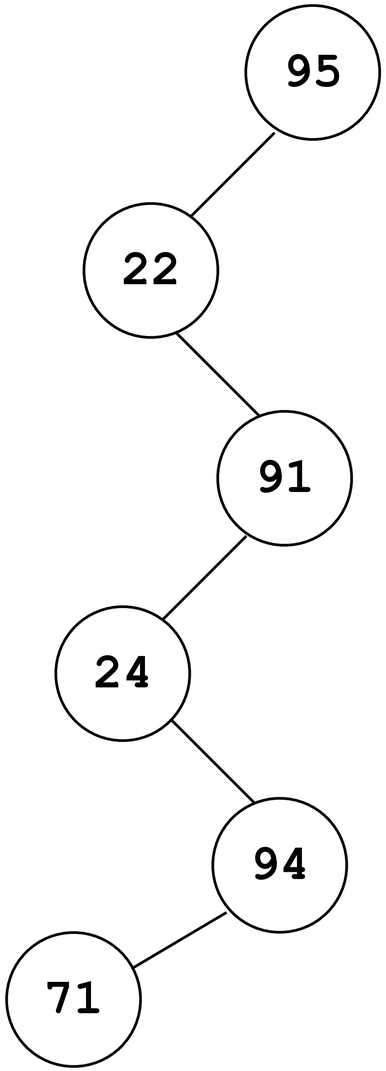
[2011] 7. 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是___。

A. 95, 22, 91, 24, 94, 71

B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34



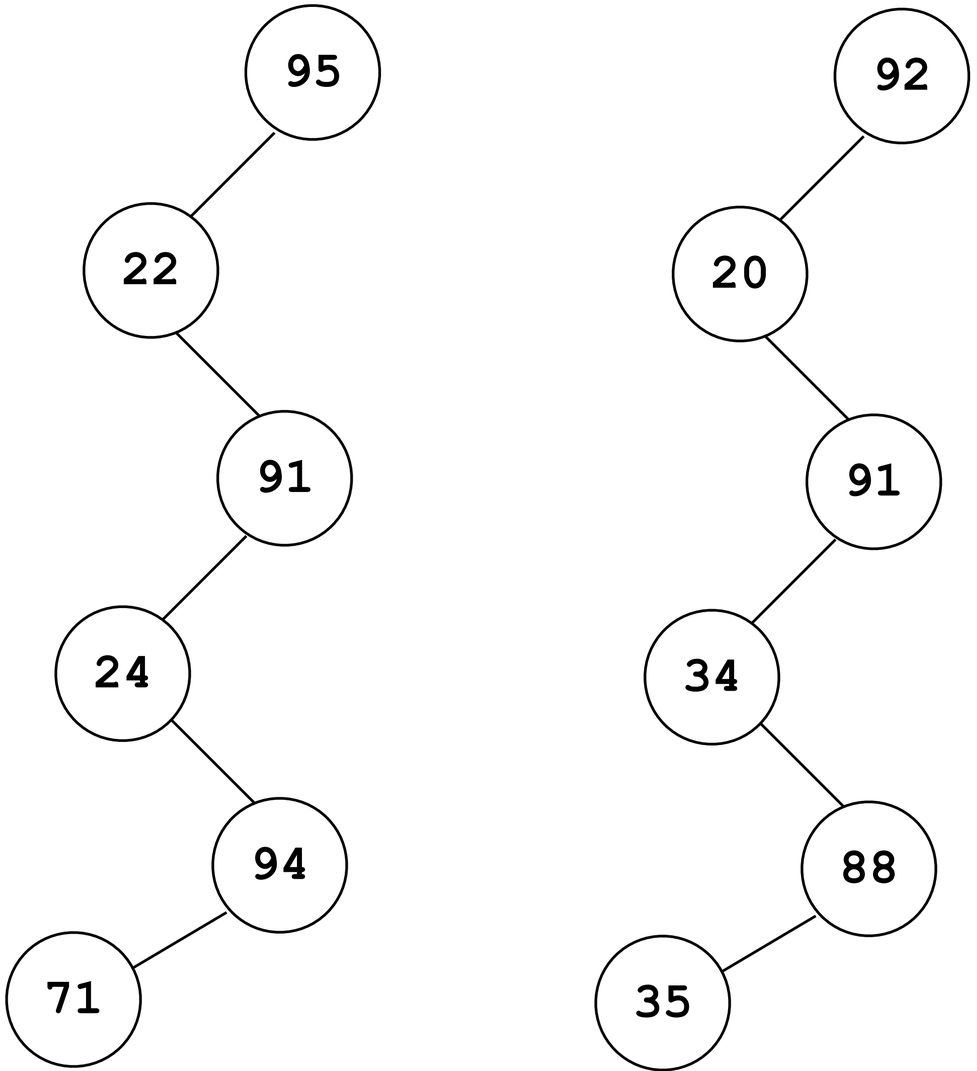
[2011] 7. 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是___。

A. 95, 22, 91, 24, 94, 71

B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34



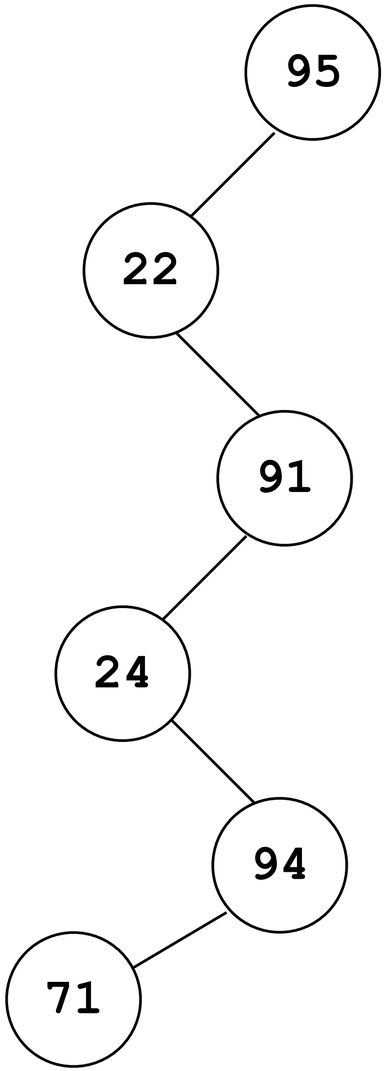
[2011] 7. 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是___。

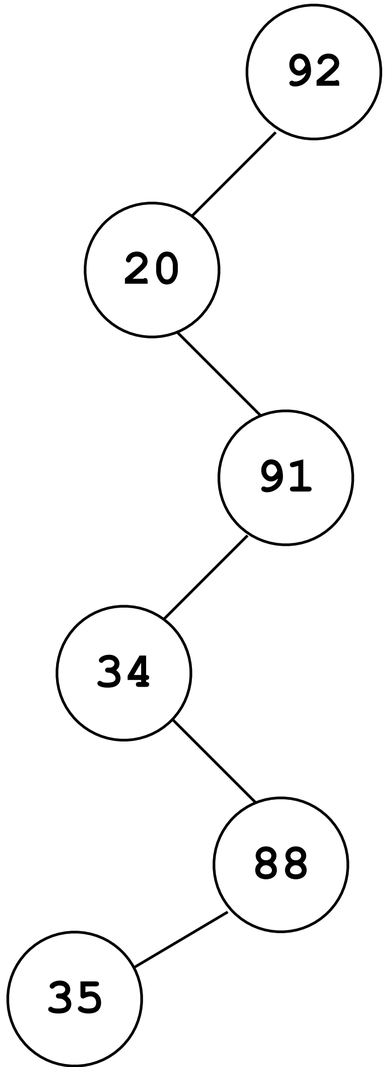
A. 95, 22, 91, 24, 94, 71

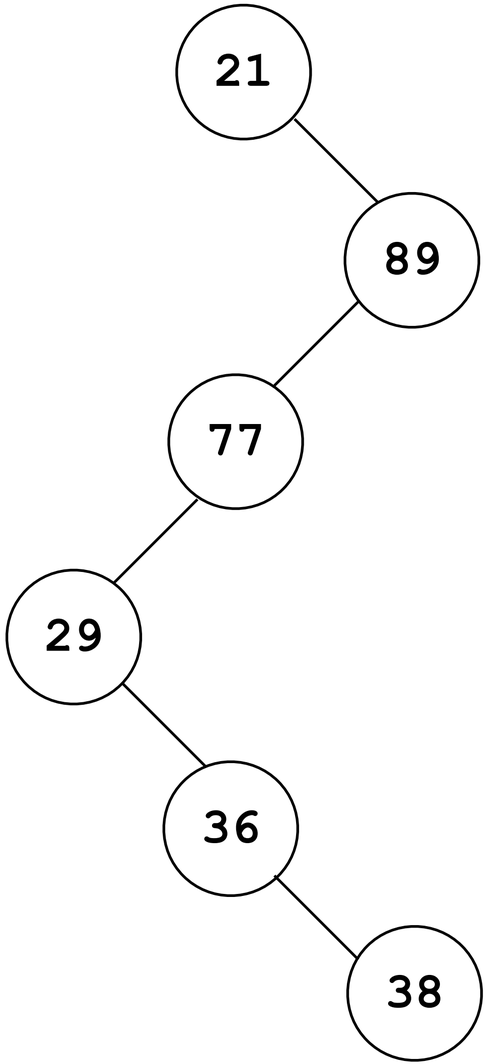
B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34







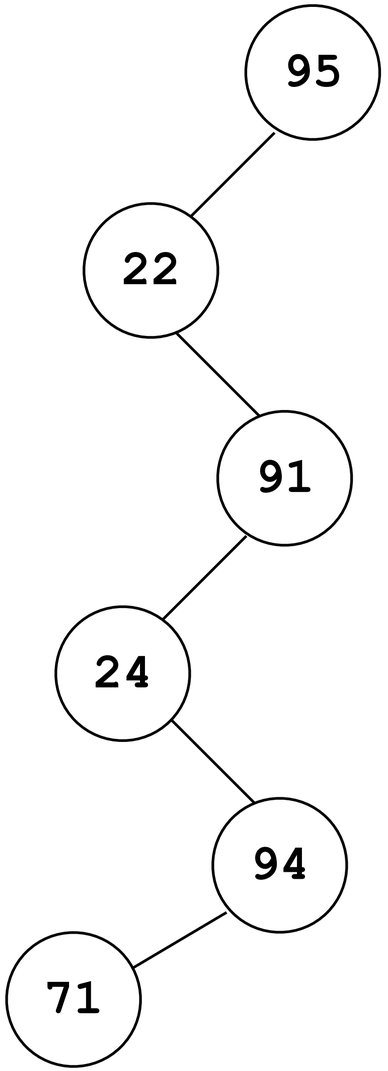
[2011] 7. 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是___。

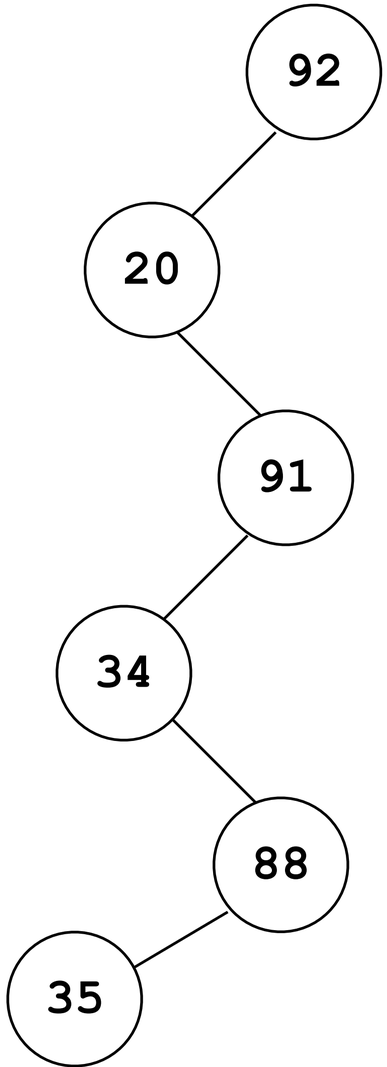
A. 95, 22, 91, 24, 94, 71

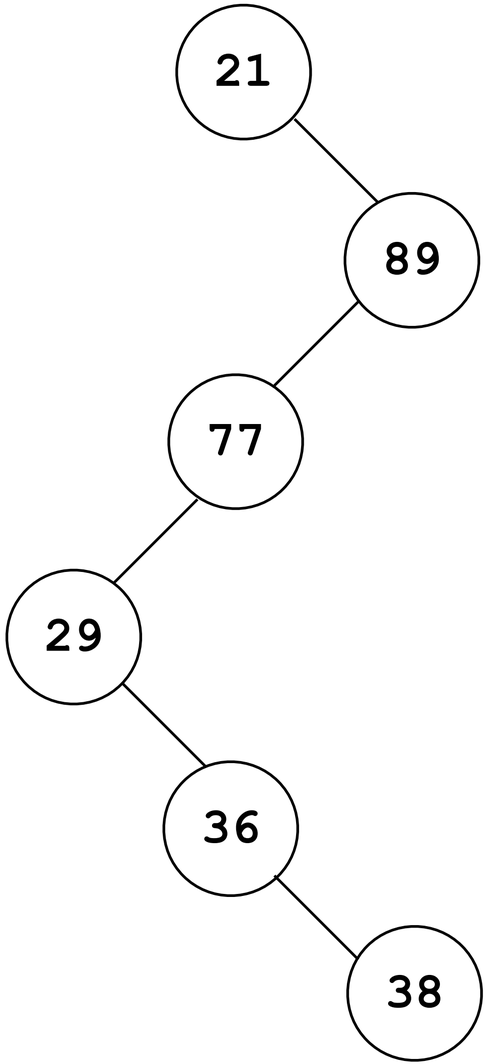
B. 92, 20, 91, 34, 88, 35

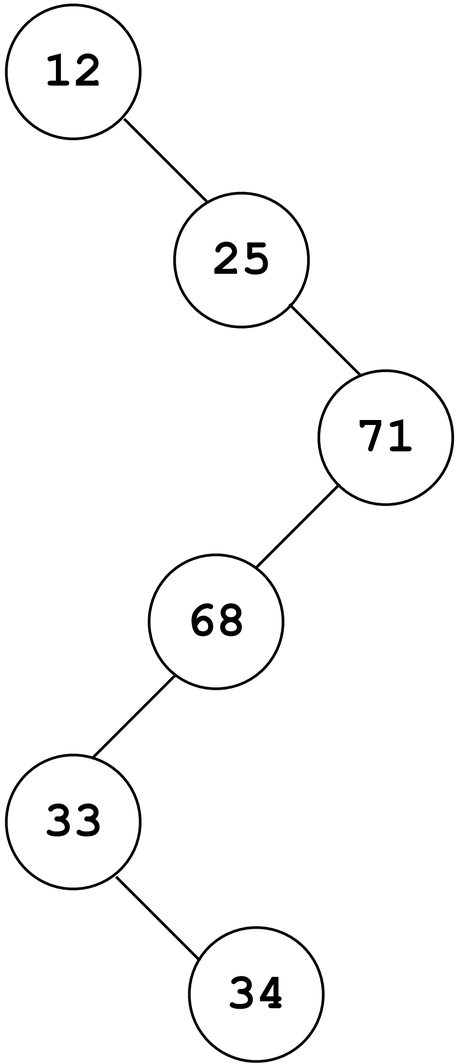
C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34









真题解析

[2011] 7. 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是___。

A. 95, 22, 91, 24, 94, 71

B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34

真题解析

[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

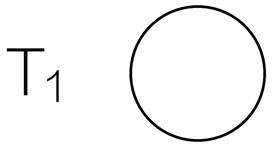
A. 10　　B. 20　　C. 32　　D. 33

[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33

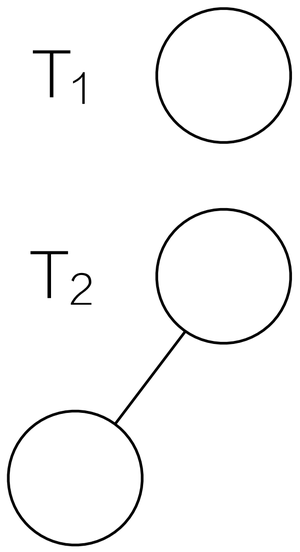
[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33



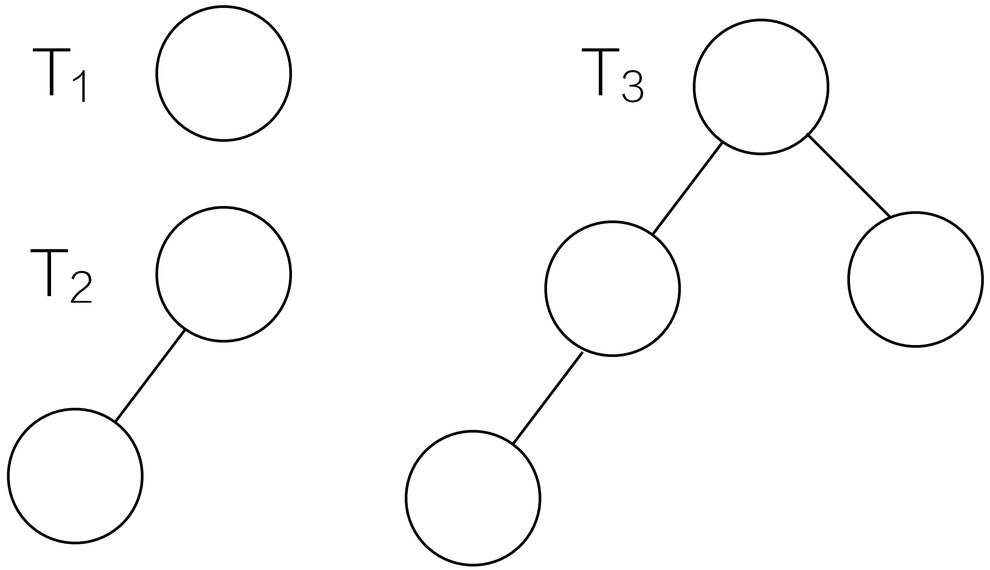
[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33



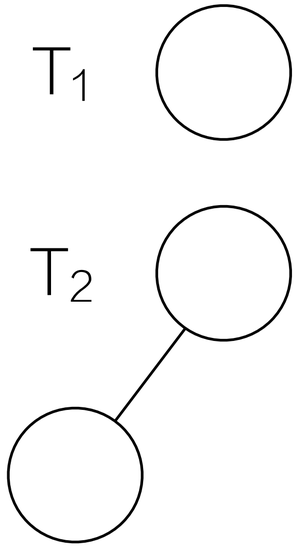
[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

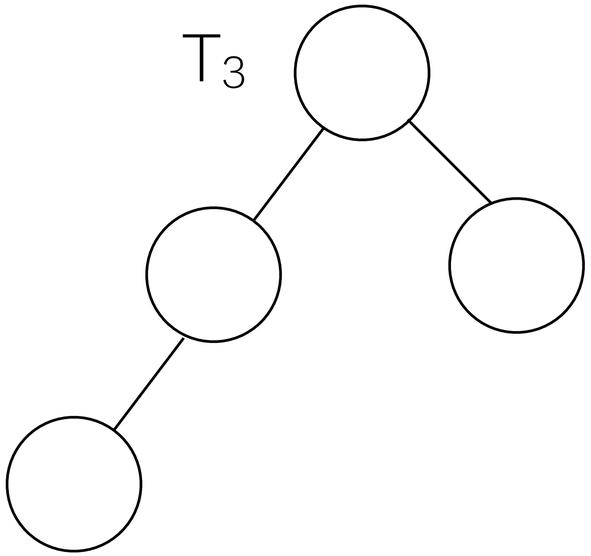
A. 10　　B. 20　　C. 32　　D. 33

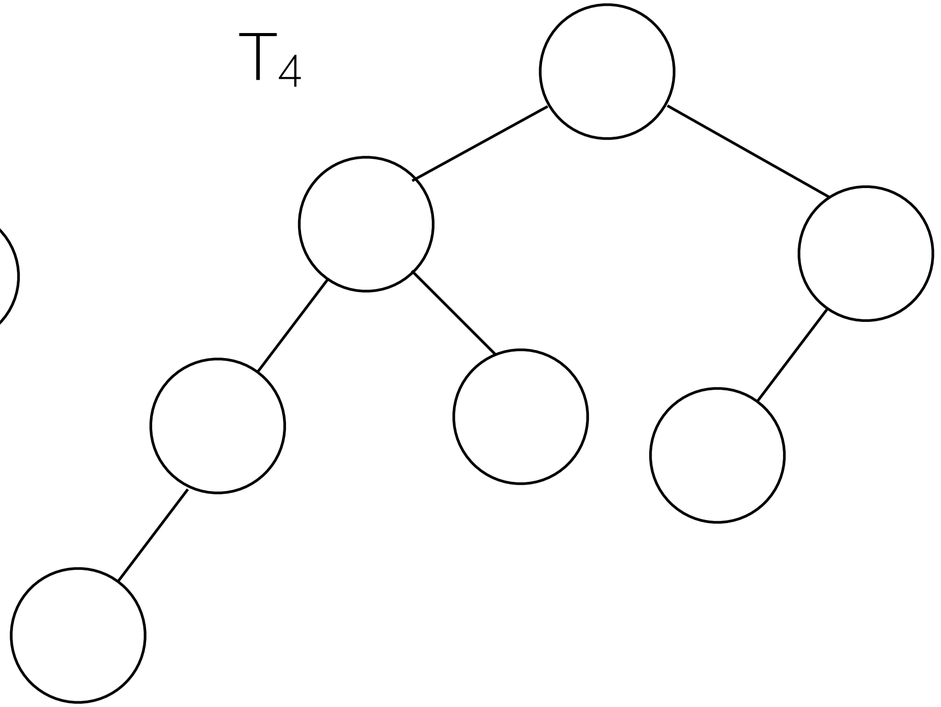


[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33

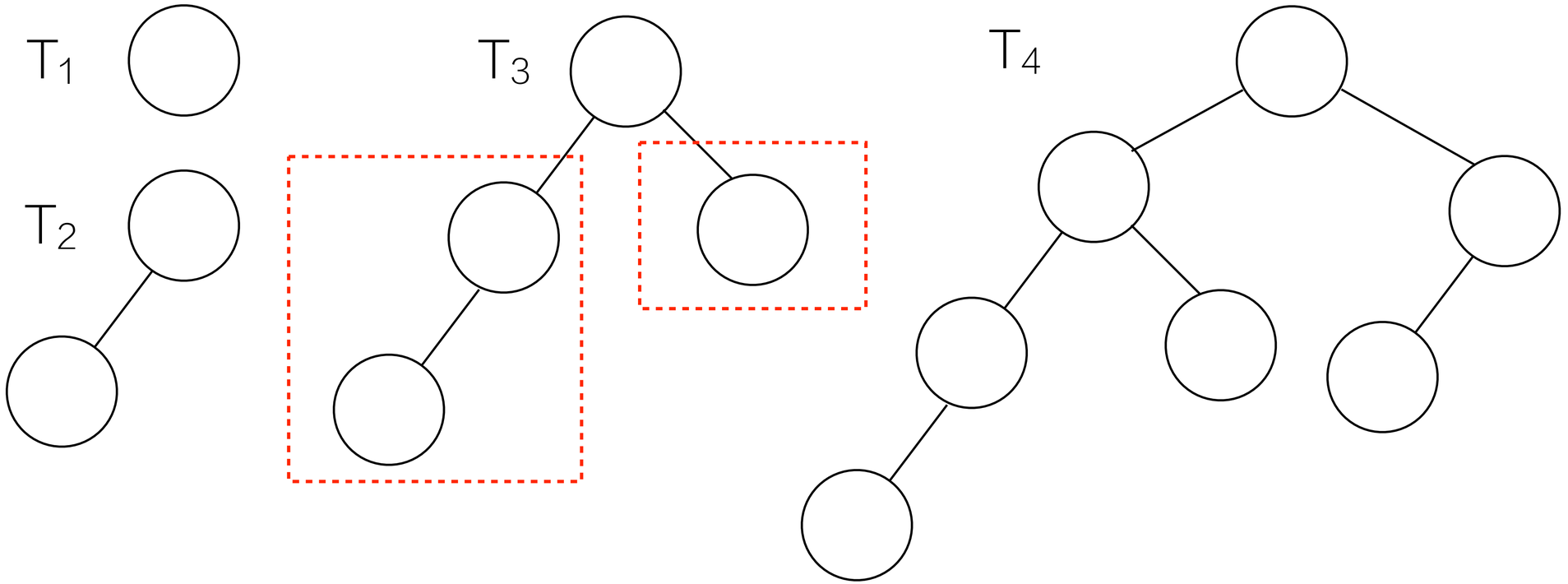






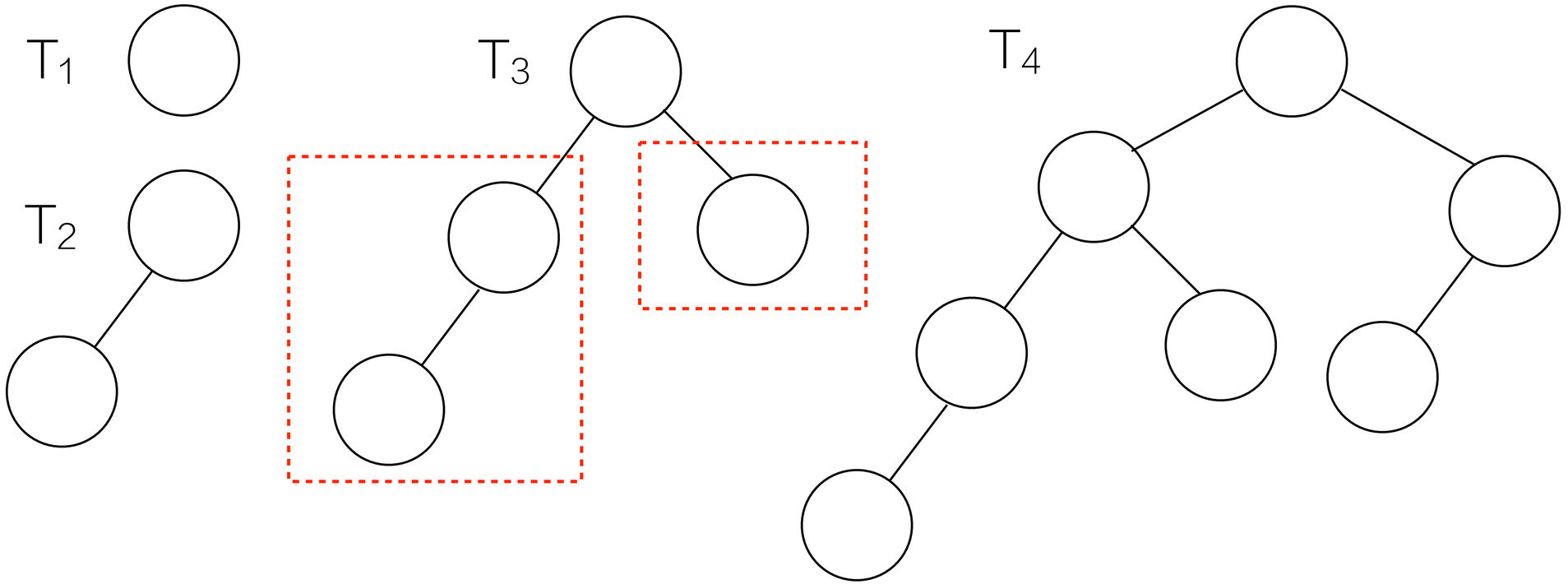
[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33



[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

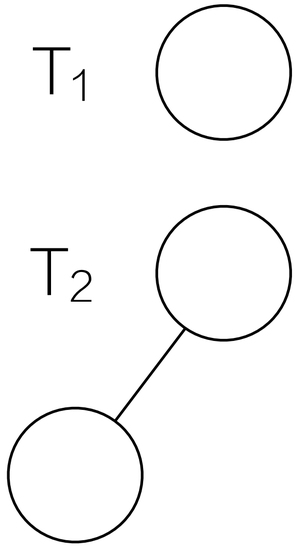
A. 10　　B. 20　　C. 32　　D. 33

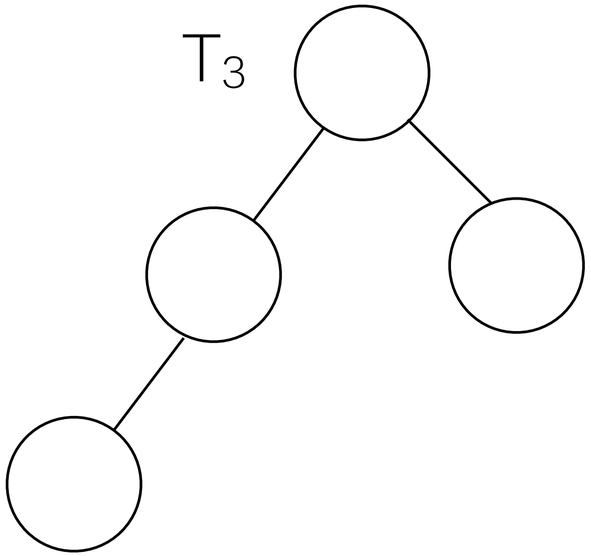


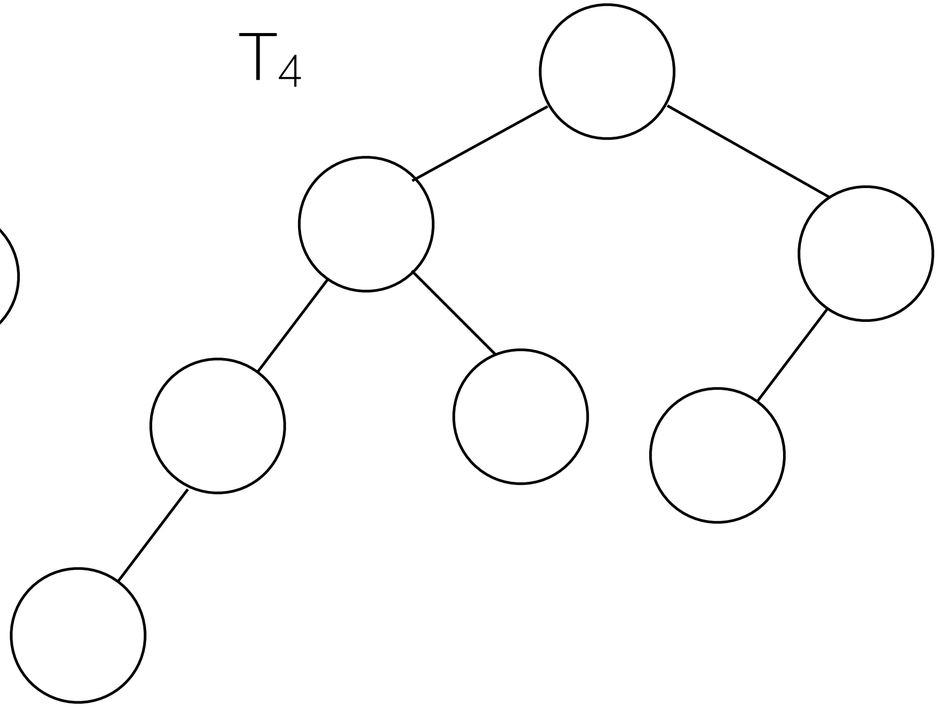
$T_3 = T_2 + T_1 + 1 = 4$

[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33



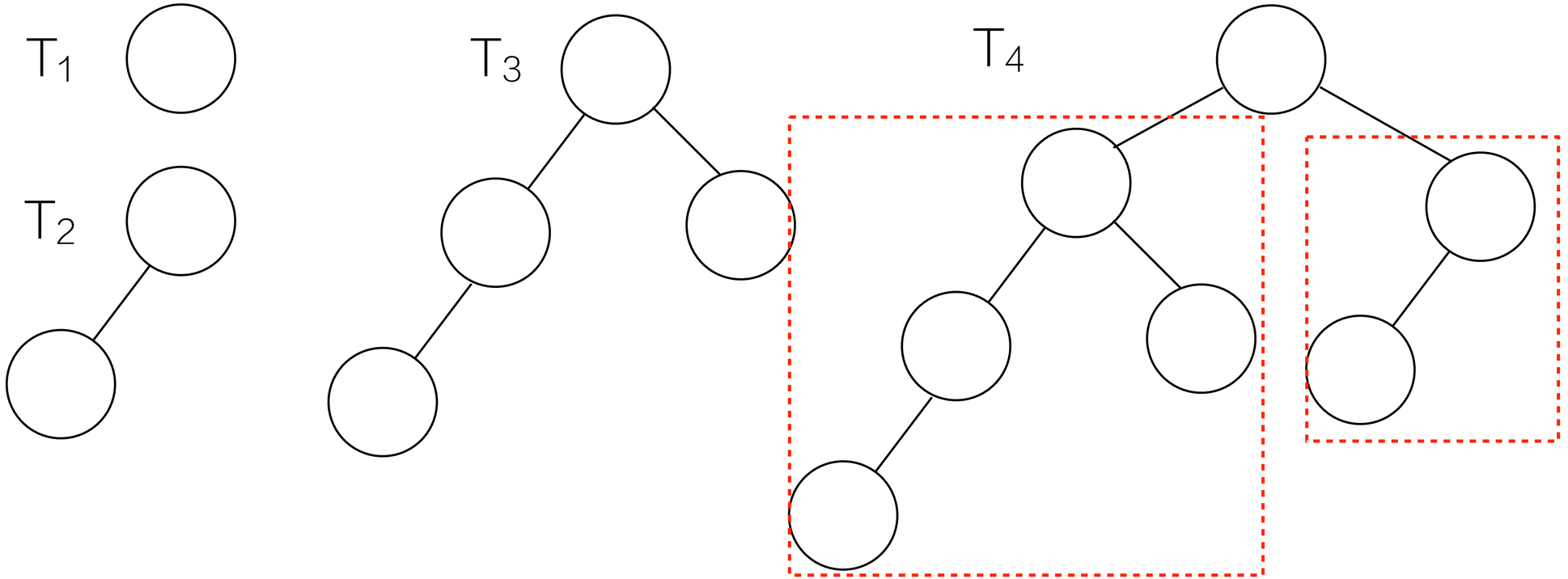




$T_3 = T_2 + T_1 + 1 = 4$

[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33

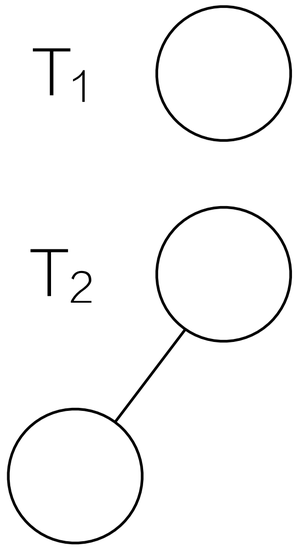


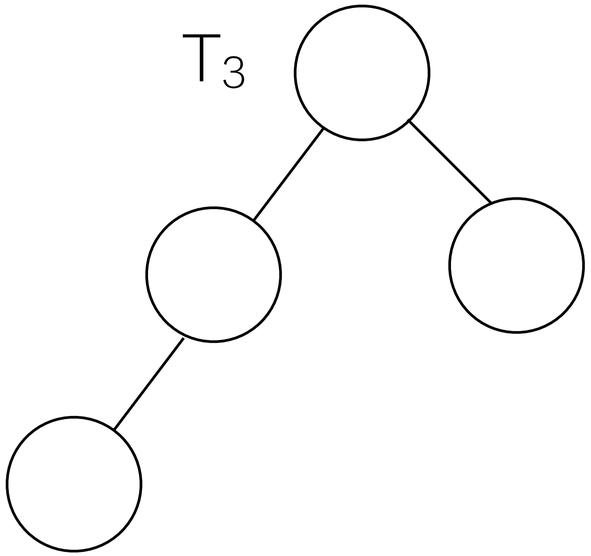
$T_3 = T_2 + T_1 + 1 = 4$

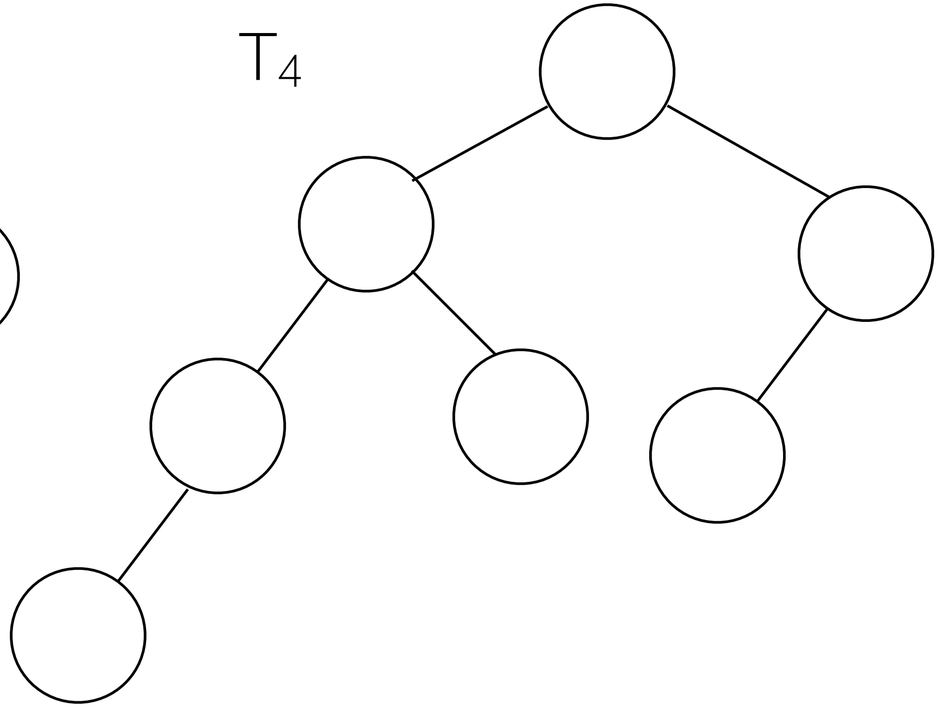
$T_4 = T_3 + T_2 + 1 = 7$

[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33







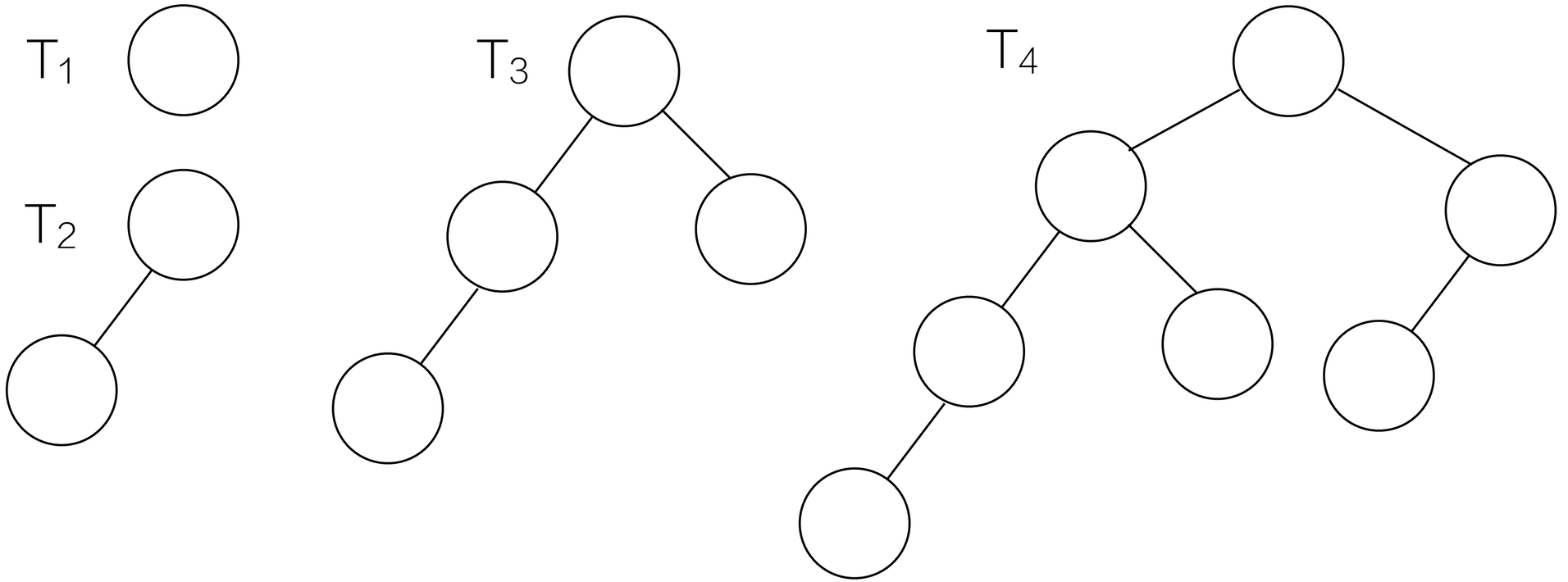
$T_3 = T_2 + T_1 + 1 = 4$

$T_4 = T_3 + T_2 + 1 = 7$

$T_5 = T_4 + T_3 + 1 = 12$

[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33



$T_3 = T_2 + T_1 + 1 = 4$

$T_4 = T_3 + T_2 + 1 = 7$

$T_5 = T_4 + T_3 + 1 = 12$

$T_6 = T_5 + T_4 + 1 = 20$

真题解析

[2012] 4. 若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为_____。

A. 10　　B. 20　　C. 32　　D. 33

真题解析

[2013] 6. 在任意一棵非空二叉排序树 T_1 中，删除某结点 v 之后形成二叉排序树 T_2，再将 v 插入 T_2 形成二叉排序树 T_3。下列关于 T_1 与 T_3 的叙述中，正确的是_____。

I. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 不同

II. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 相同

III. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 不同

IV. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 相同

A. 仅 I、III　　B. 仅 I、IV　　C. 仅 II、III　　D. 仅 II、IV

真题解析

[2013] 6. 在任意一棵非空二叉排序树 T_1 中，删除某结点 v 之后形成二叉排序树 T_2，再将 v 插入 T_2 形成二叉排序树 T_3。下列关于 T_1 与 T_3 的叙述中，正确的是_____。

I. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 不同

II. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 相同

III. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 不同

IV. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 相同

A. 仅 I、III　　B. 仅 I、IV　　C. 仅 II、III　　D. 仅 II、IV

真题解析

[2015] 7. 下列选项中，不能构成折半查找中关键字比较序列的是_____。

A. 500, 200, 450, 180

B. 500, 450, 200, 180

C. 180, 500, 200, 450

D. 180, 200, 500, 450

真题解析

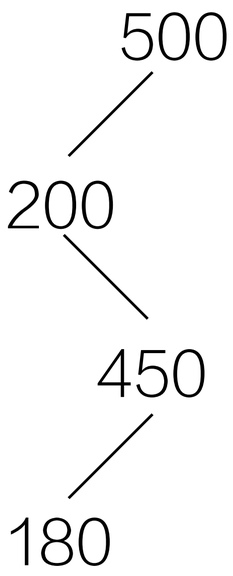
[2015] 7. 下列选项中，不能构成折半查找中关键字比较序列的是_____。

A. 500, 200, 450, 180

B. 500, 450, 200, 180

C. 180, 500, 200, 450

D. 180, 200, 500, 450



真题解析

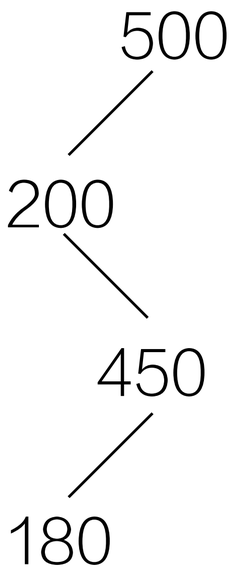
[2015] 7. 下列选项中，不能构成折半查找中关键字比较序列的是_____。

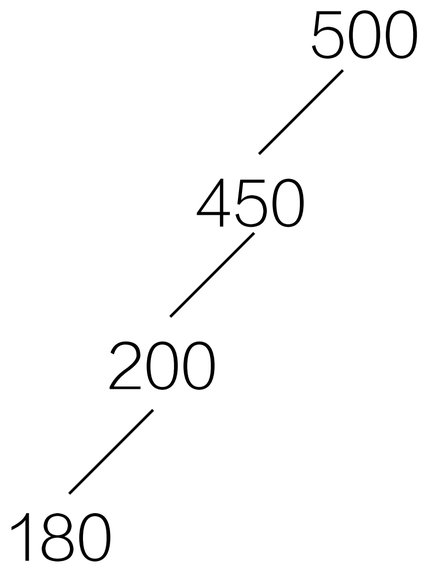
A. 500, 200, 450, 180

B. 500, 450, 200, 180

C. 180, 500, 200, 450

D. 180, 200, 500, 450





真题解析

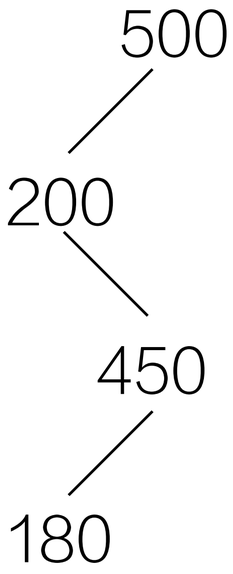
[2015] 7. 下列选项中，不能构成折半查找中关键字比较序列的是_____。

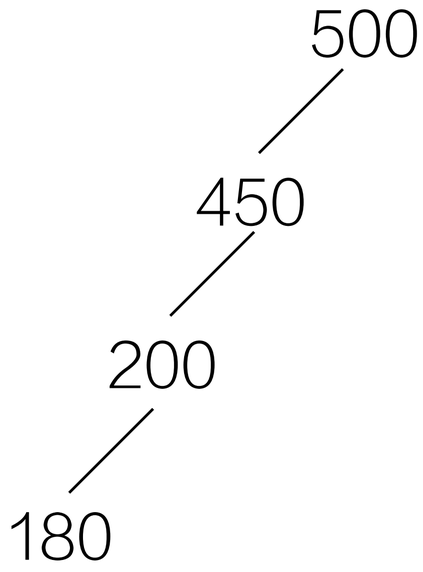
A. 500, 200, 450, 180

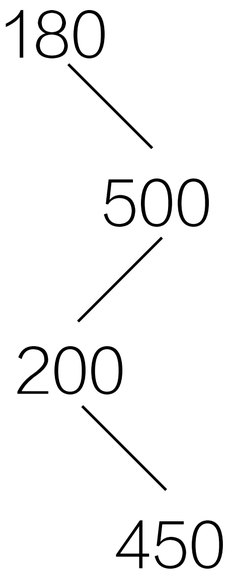
B. 500, 450, 200, 180

C. 180, 500, 200, 450

D. 180, 200, 500, 450







真题解析

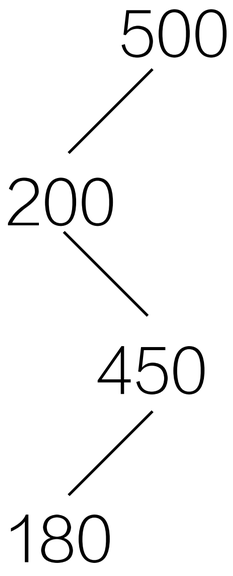
[2015] 7. 下列选项中，不能构成折半查找中关键字比较序列的是_____。

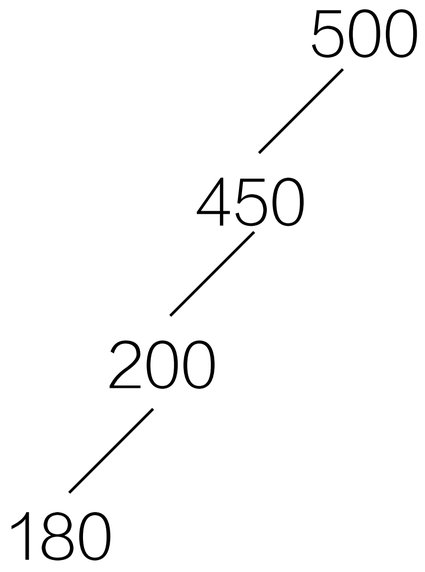
A. 500, 200, 450, 180

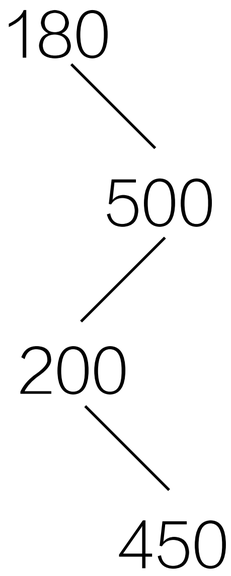
B. 500, 450, 200, 180

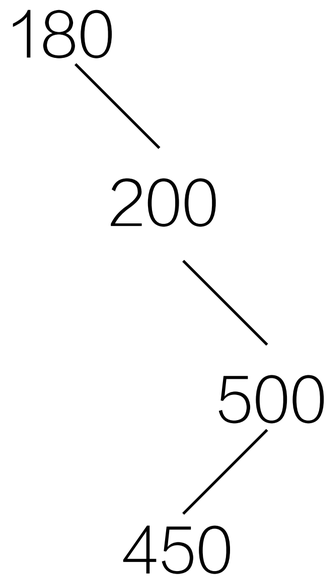
C. 180, 500, 200, 450

D. 180, 200, 500, 450









真题解析

[2015] 7. 下列选项中，不能构成折半查找中关键字比较序列的是_____。

A. 500, 200, 450, 180

B. 500, 450, 200, 180

C. 180, 500, 200, 450

D. 180, 200, 500, 450

真题解析

[2016] 9. 在有 n(n > 1000) 个元素的升序数组 A 中查找关键字 x，查找算法的伪代码如下所示。

```
k = 0;
while(k < n 且 A[k] < x) k = k + 3;
if(k < n 且 A[k] == x) 查找成功;
else if(k - 1 < n 且 A[k - 1] == x) 查找成功;
    else if(k - 2 < n 且 A[k - 2] == x) 查找成功;
        else 查找失败;
```

本算法与折半查找算法相比，有可能具有更少比较次数的情形是_____。

A. 当 x 不在数组中

B. 当 x 接近数组开头处

C. 当 x 接近数组结尾处

D. 当 x 位于数组中间位置

真题解析

[2016] 9. 在有 n(n > 1000) 个元素的升序数组 A 中查找关键字 x，查找算法的伪代码如下所示。

```
k = 0;
while(k < n 且 A[k] < x) k = k + 3;
if(k < n 且 A[k] == x) 查找成功;
else if(k - 1 < n 且 A[k - 1] == x) 查找成功;
    else if(k - 2 < n 且 A[k - 2] == x) 查找成功;
        else 查找失败;
```

本算法与折半查找算法相比，有可能具有更少比较次数的情形是_____。

A. 当 x 不在数组中

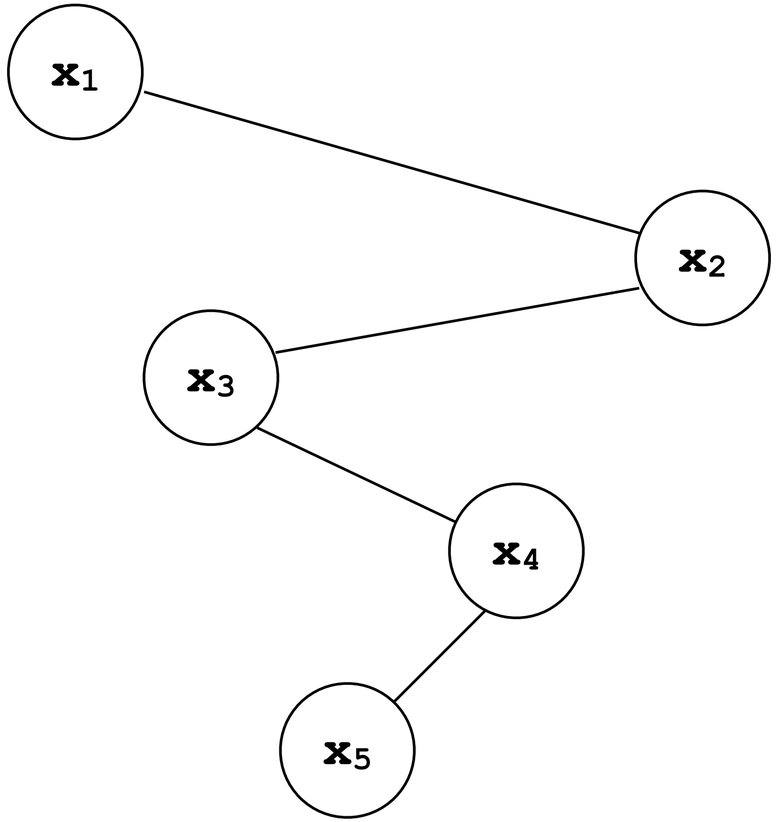
B. 当 x 接近数组开头处

C. 当 x 接近数组结尾处

D. 当 x 位于数组中间位置

真题解析

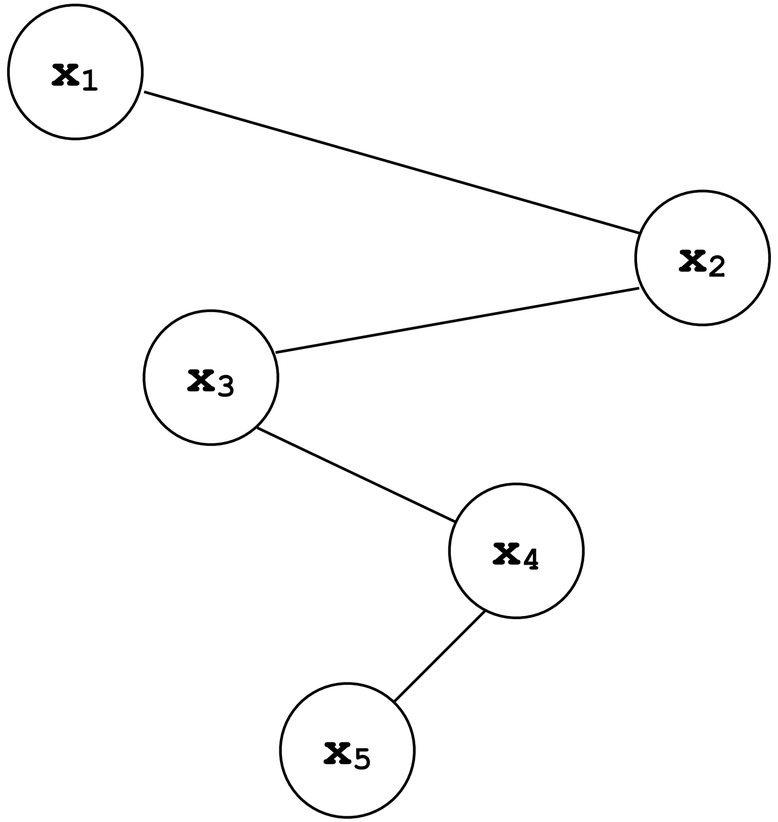
[2018] 6. 已知二叉排序树如下图所示，元素之间应满足的大小关系是_____。



A. $x_1 < x_2 < x_5$　　B. $x_1 < x_4 < x_5$　　C. $x_3 < x_5 < x_4$　　D. $x_4 < x_3 < x_5$

真题解析

[2018] 6. 已知二叉排序树如下图所示，元素之间应满足的大小关系是_____。



A. $x_1 < x_2 < x_5$　　B. $x_1 < x_4 < x_5$　　C. $x_3 < x_5 < x_4$　　D. $x_4 < x_3 < x_5$

真题解析

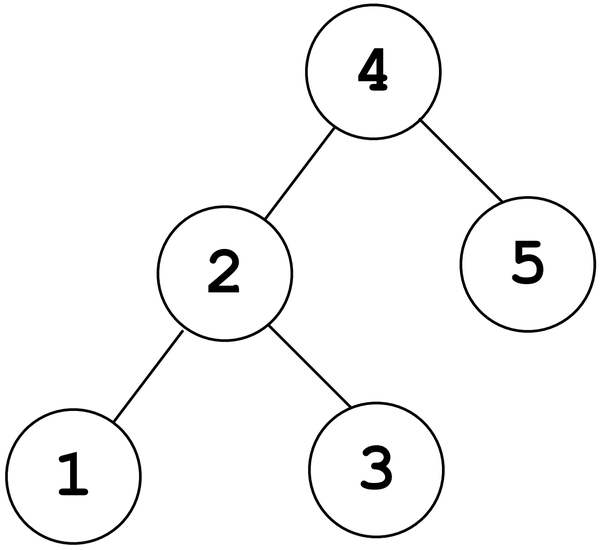
[2020] 5. 下列给定的关键字输入序列中，不能生成如下二叉排序树的是_____。

A. 4, 5, 2, 1, 3

B. 4, 5, 1, 2, 3

C. 4, 2, 5, 3, 1

D. 4, 2, 1, 3, 5



真题解析

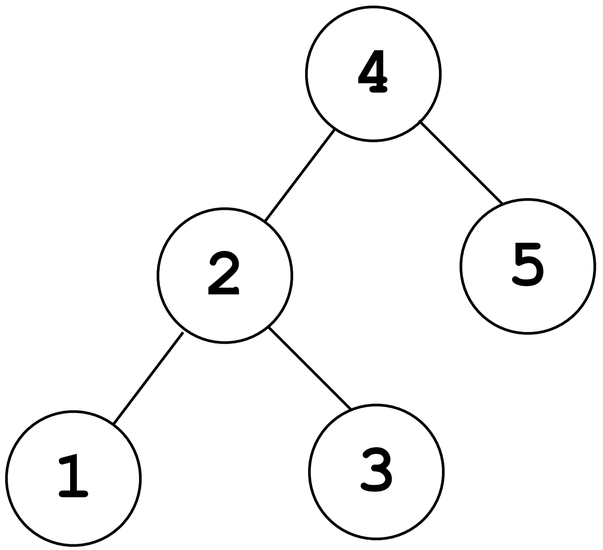
[2020] 5. 下列给定的关键字输入序列中，不能生成如下二叉排序树的是_____。

A. 4, 5, 2, 1, 3

B. 4, 5, 1, 2, 3

C. 4, 2, 5, 3, 1

D. 4, 2, 1, 3, 5



B树（B- 树）

在现实中，很多查找操作并不是在内存中完成的，而是要访问磁盘或数据库。相比内存，磁盘的读写代价非常高，必须尽可能减少查找过程中磁盘的访问次数。此时，二叉排序树就显得有些“力不从心”了，因为其高度容易过大，导致频繁的磁盘访问。

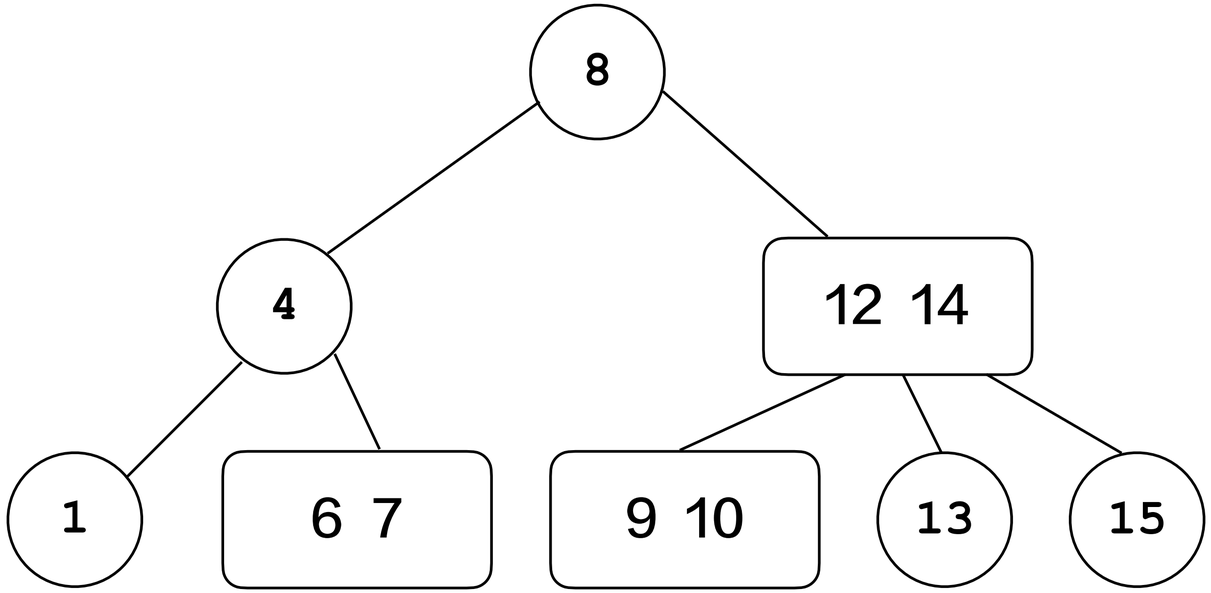
为了解决这个问题，人们提出了一种多路平衡查找树 —— B树（Balanced Tree 或 Bayer Tree）

B树（B- 树）

简单理解，B树是“多路版本”的平衡查找树，或者叫“平衡多叉树”。如果将二叉排序树看成是“每个节点最多两个分支”的树，那么B树允许每个节点有多个关键字和多个孩子，这种结构能有效压缩树的高度。

B树（B- 树）

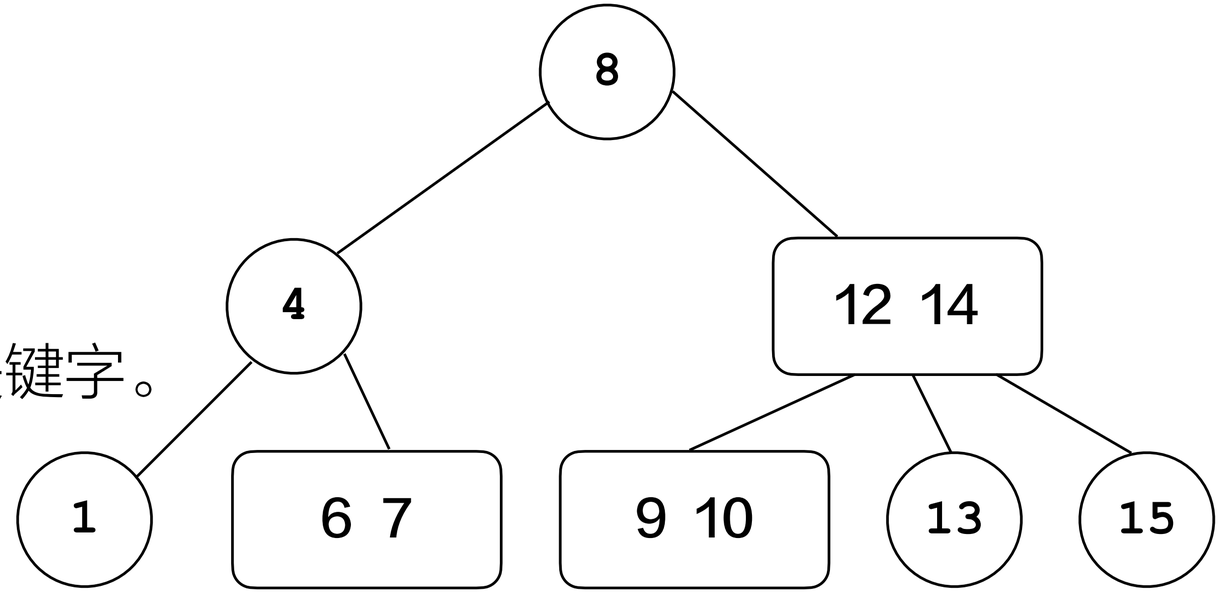
简单理解，B树是“多路版本”的平衡查找树，或者叫“平衡多叉树”。如果将二叉排序树看成是“每个节点最多两个分支”的树，那么B树允许每个节点有多个关键字和多个孩子，这种结构能有效压缩树的高度。



B树（B- 树）

一个m阶的B树满足以下性质：

1. 每个节点最多有 m 个孩子。
2. 每个非根非叶子节点至少有 ⌈m/2⌉ 个孩子。
3. 每个非叶子节点有 k 个孩子，就有 k-1 个关键字。
4. 关键字按照中序遍历是有序的。



3 阶 B 树

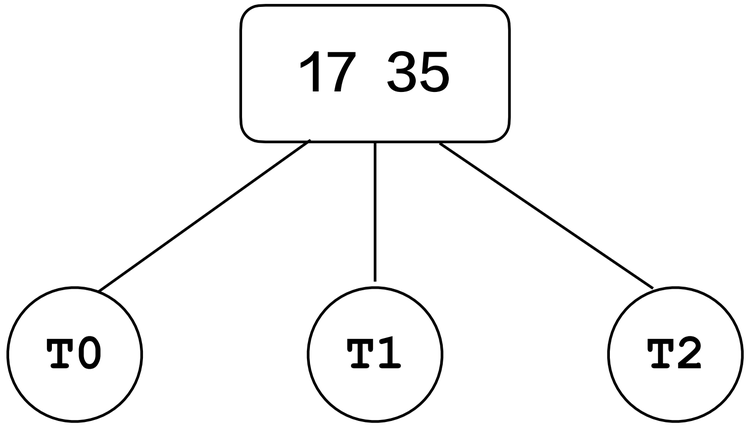
B树（B- 树）

另外还有一点需要注意的是：关键字在节点中是排好序的，孩子在关键字之间穿插，每个孩子对应一个“区间”。

T0：所有值 < 17

T1：所有值 ∈ (17, 35)

T2：所有值 > 35



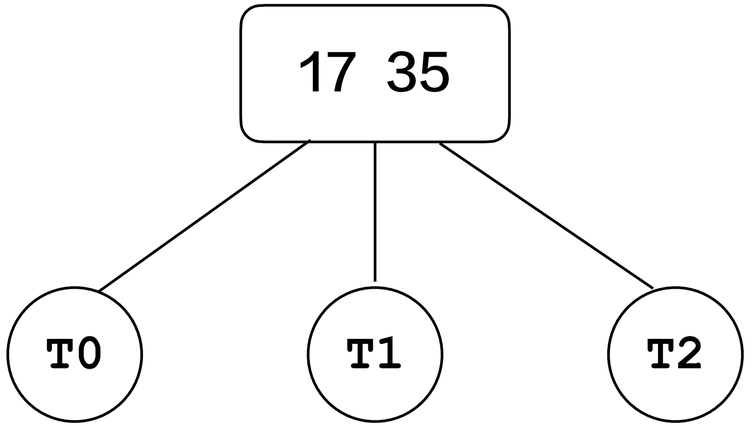
B树（B- 树）

另外还有一点需要注意的是：关键字在节点中是排好序的，孩子在关键字之间穿插，每个孩子对应一个“区间”。

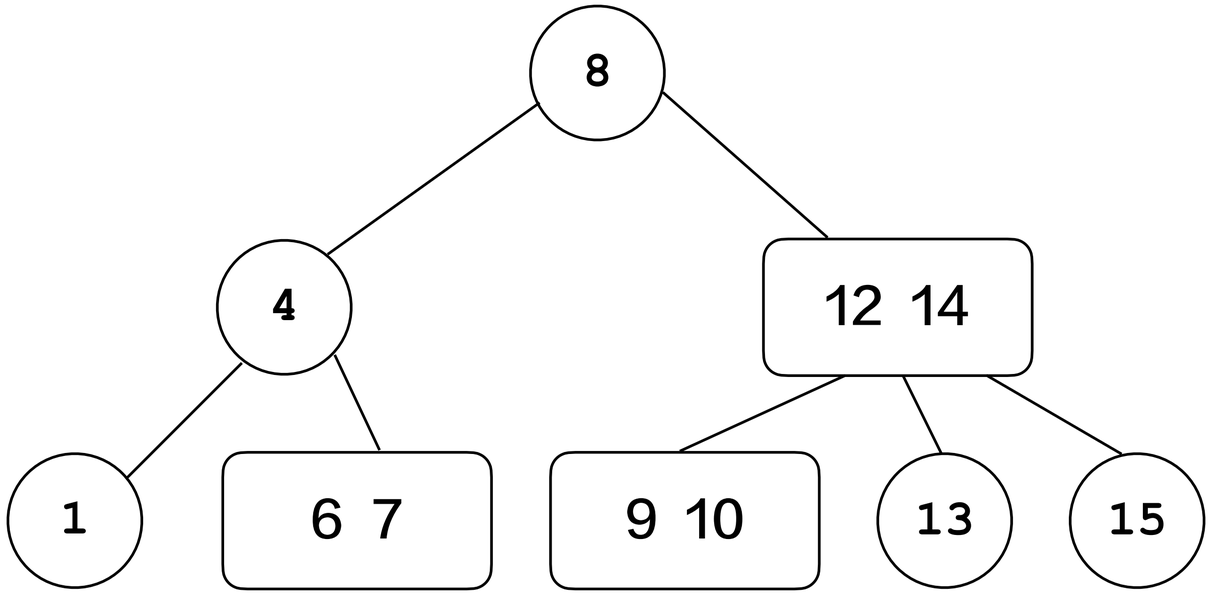
T0：所有值 < 17

T1：所有值 ∈ (17, 35)

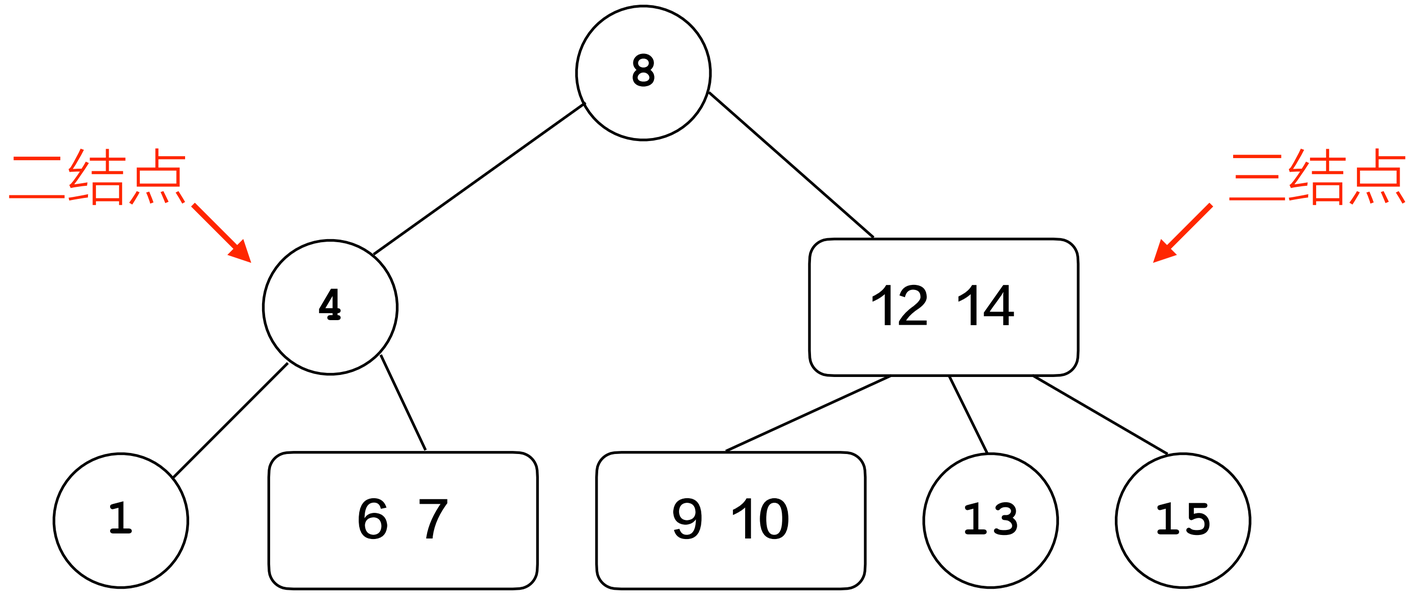
T2：所有值 > 35



3 阶 B 树



3 阶 B 树



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。

B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

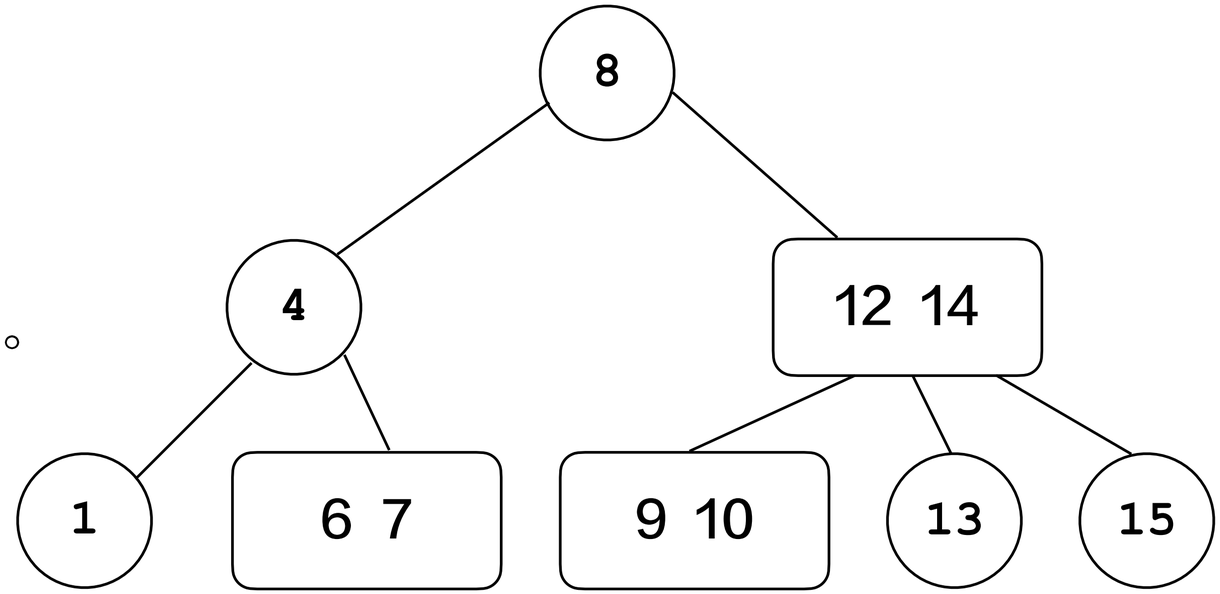
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

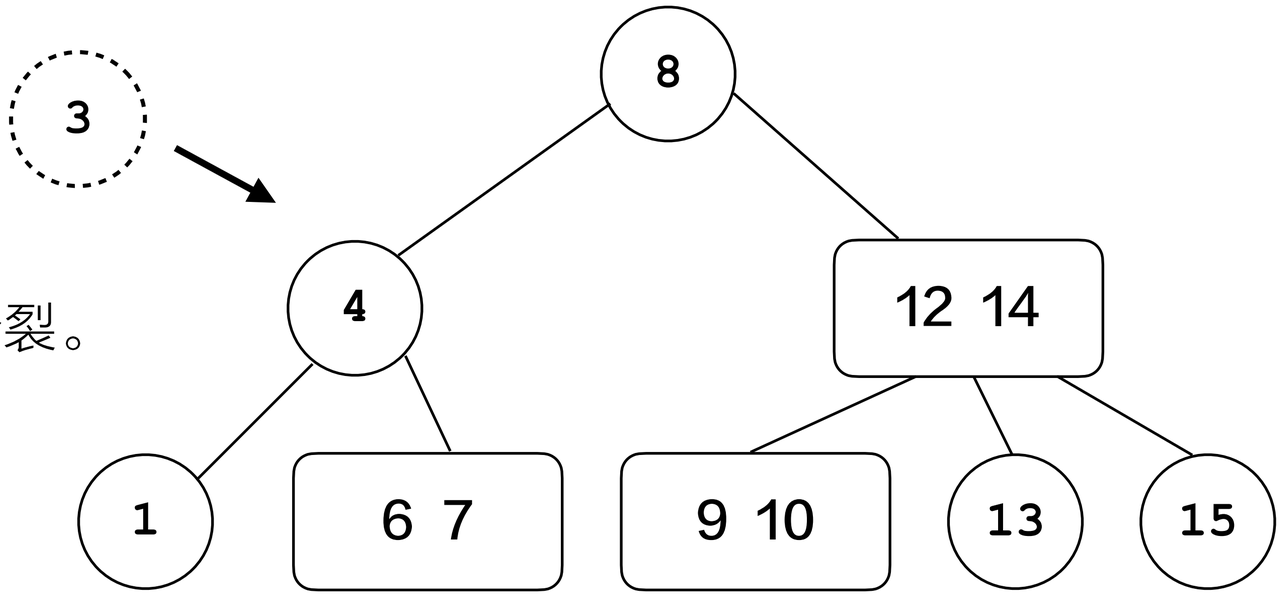
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

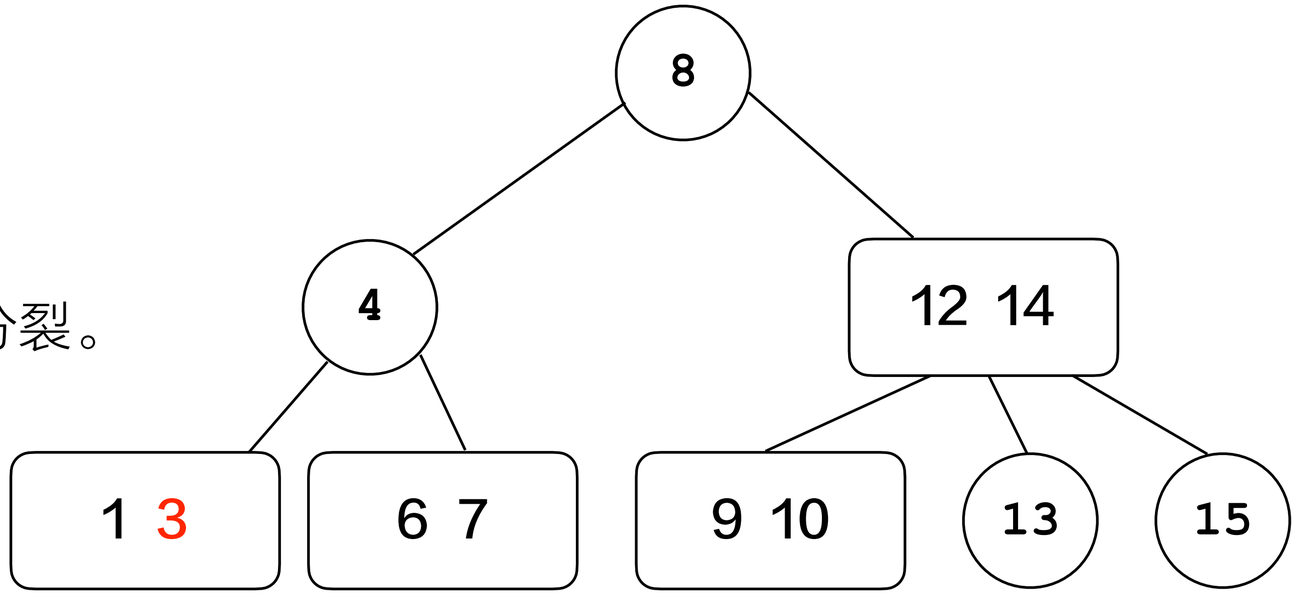
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

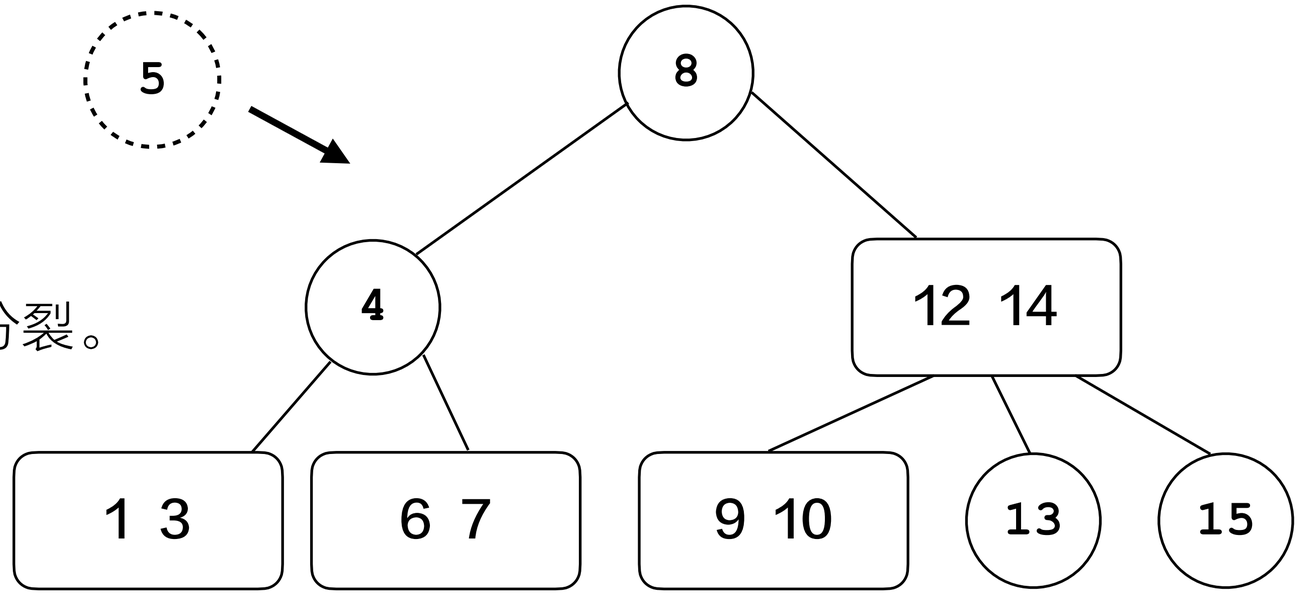
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

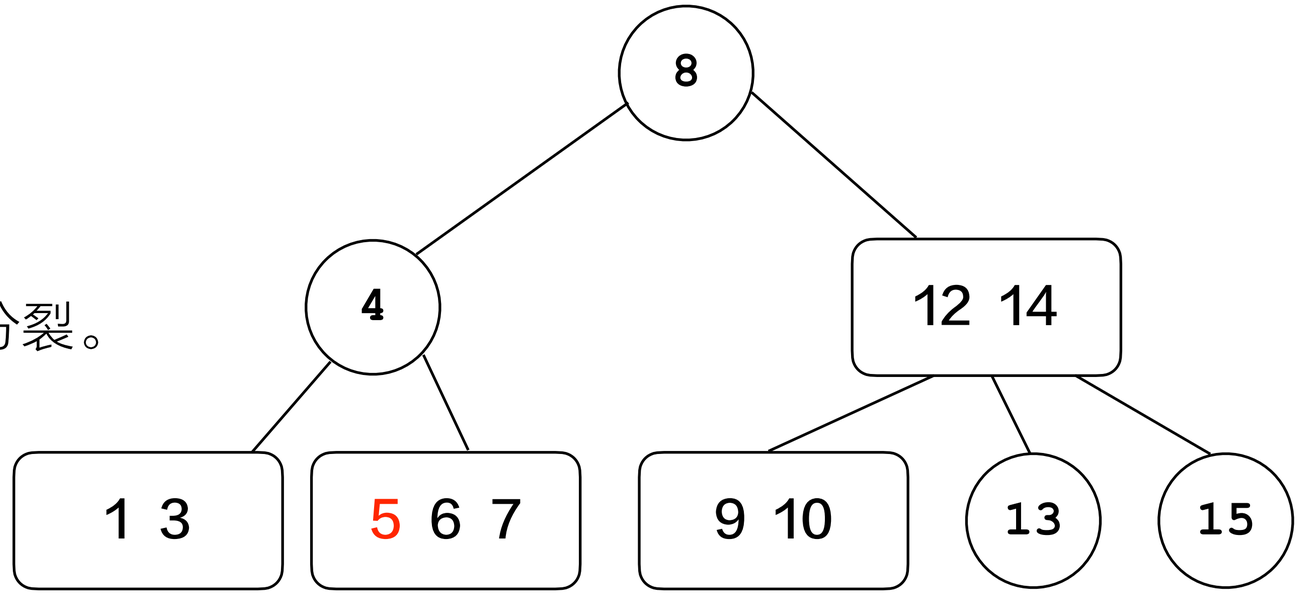
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

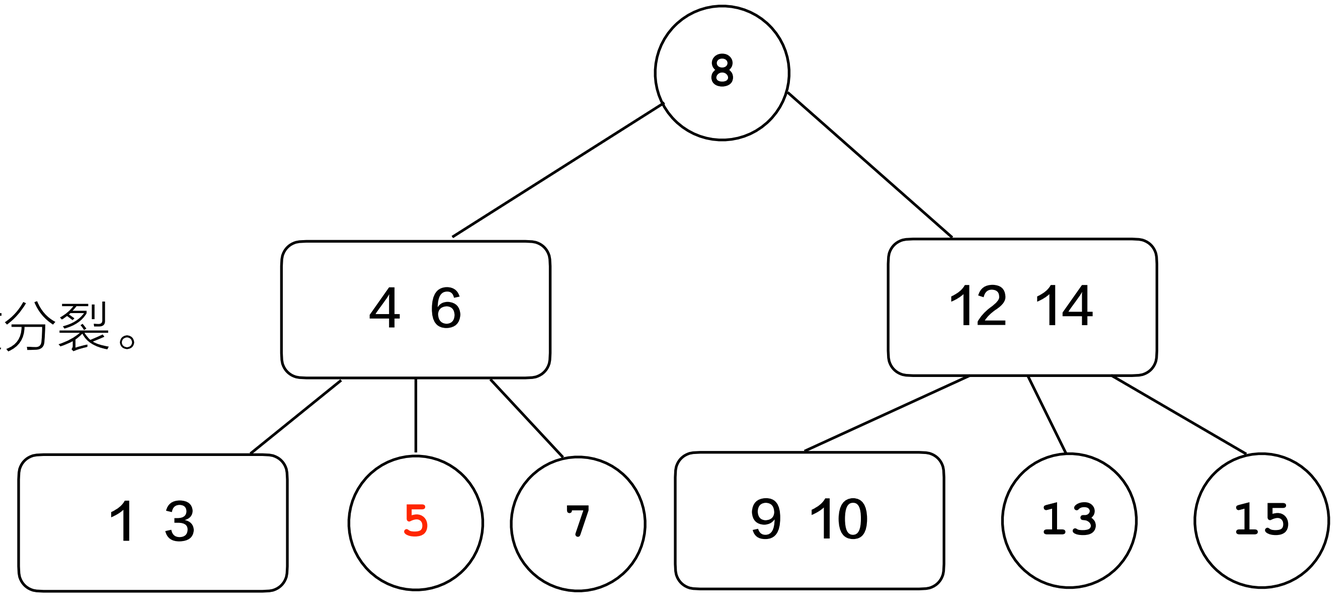
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

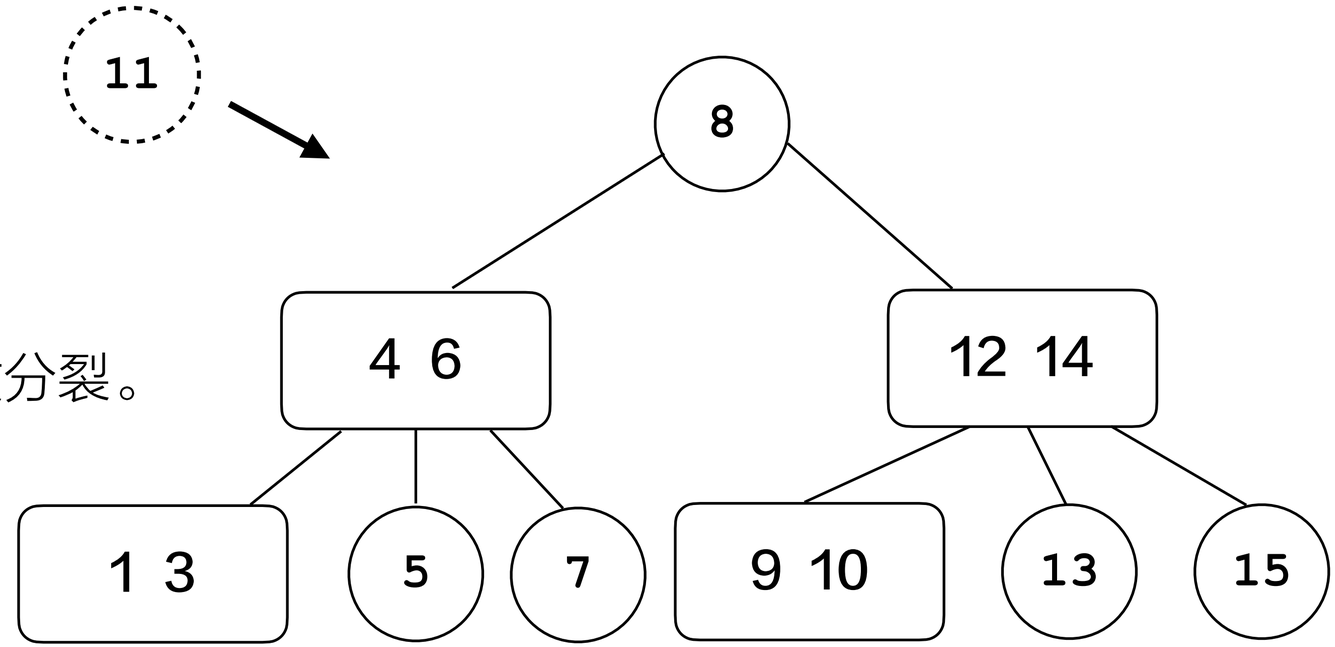
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

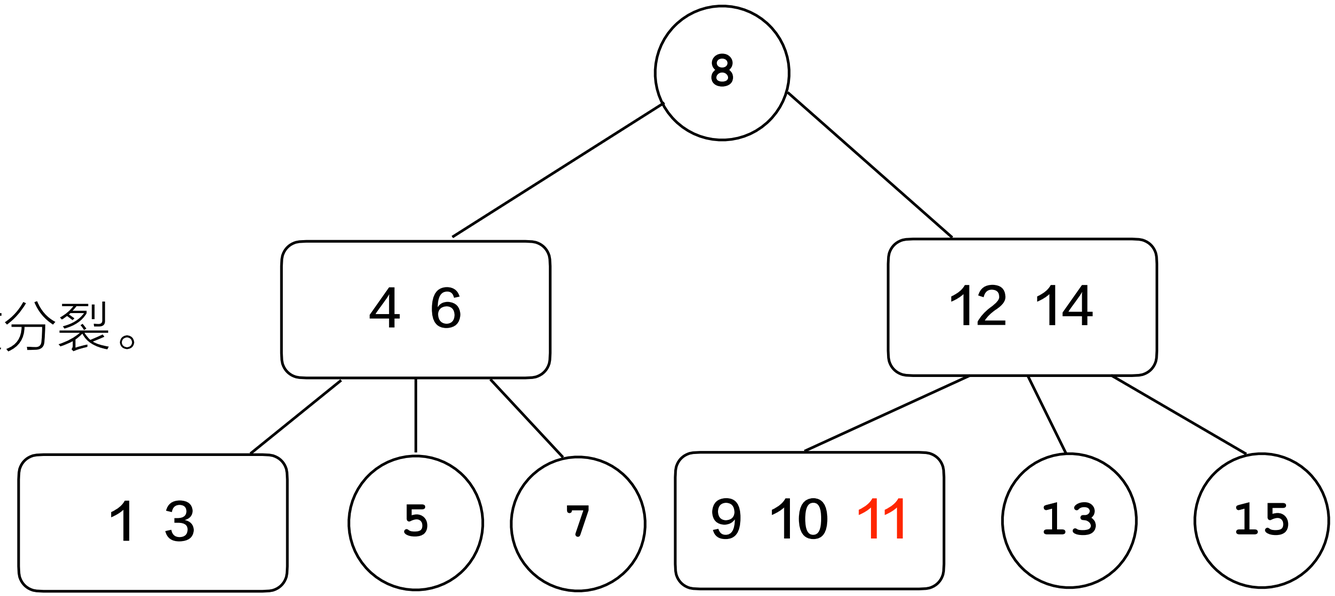
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

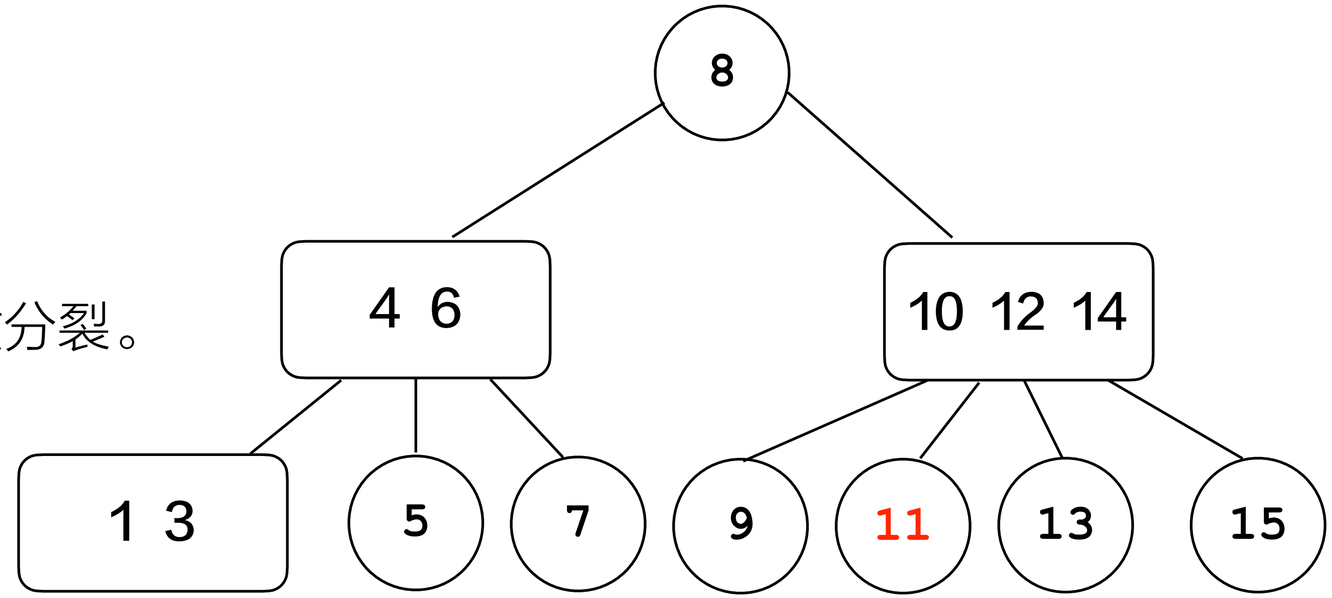
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-插入

（1）查找插入位置（从根开始）

找到要插入的关键字应该落在哪个叶子节点。

（2）将关键字插入该叶子节点的合适位置

如果该节点未满，直接插入，结束。

如果插入后超出最大关键字数（即满了）→ 触发分裂。

（3）分裂：关键字“上移”，节点一分为二

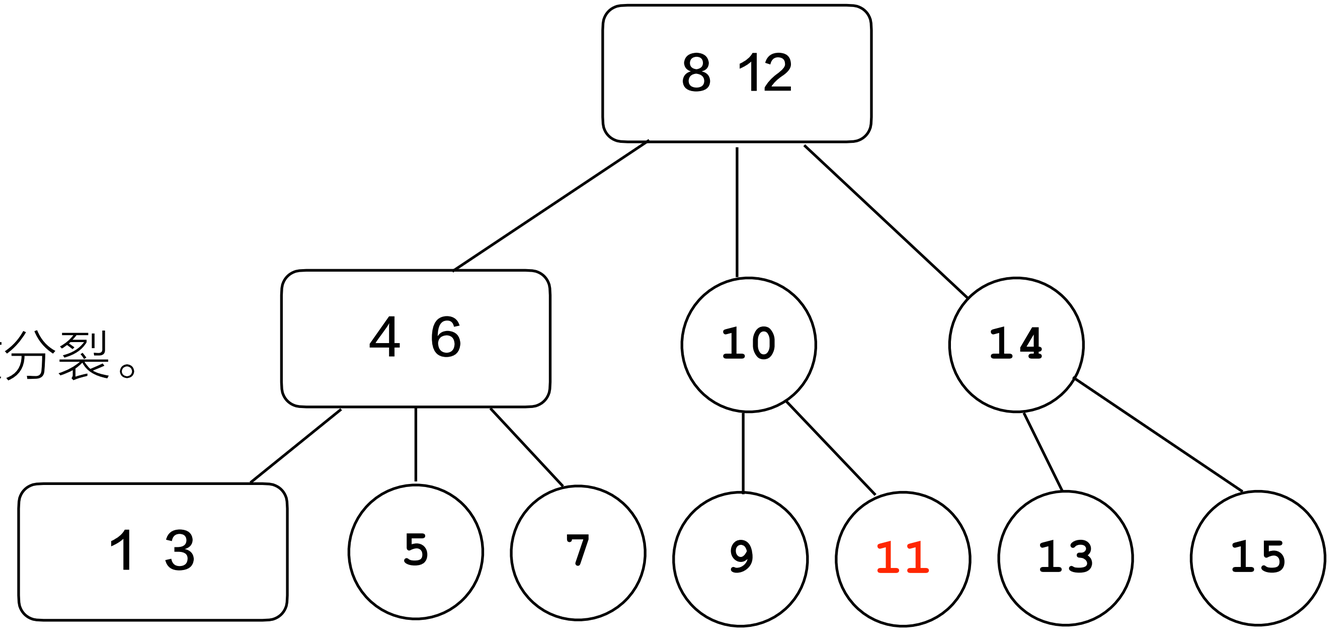
中间的关键字上升到父节点。

分裂产生两个子节点，平分原数据。

父节点接收中间关键字及新子节点指针。

（4）父节点若满，继续向上传递（递归分裂）

最终可能导致根节点分裂 → 树高增加。



B 树的操作-删除

（1）查找目标关键字的位置（从根开始）

从根节点开始查找目标关键字；

若找到关键字：如果该节点是叶子节点，可直接删除；

如果该节点是非叶子节点，需转化为在叶子节点中的删除；

若未找到关键字，递归进入对应子节点继续查找。

B 树的操作-删除

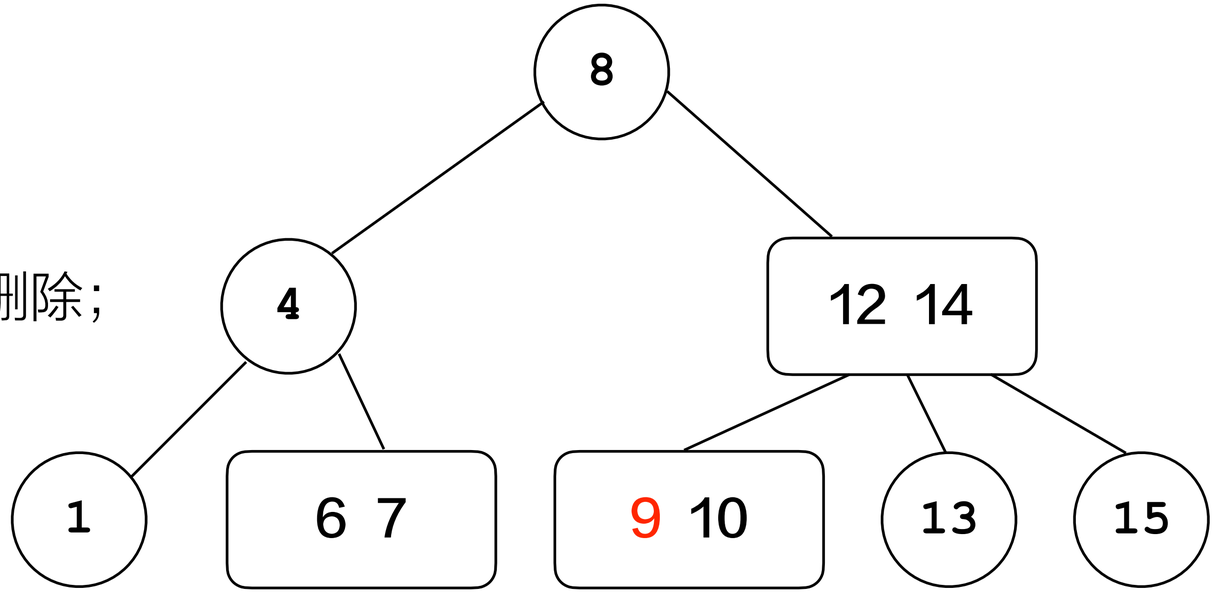
（1）查找目标关键字的位置（从根开始）

从根节点开始查找目标关键字；

若找到关键字：如果该节点是叶子节点，可直接删除；

如果该节点是非叶子节点，需转化为在叶子节点中的删除；

若未找到关键字，递归进入对应子节点继续查找。



B 树的操作-删除

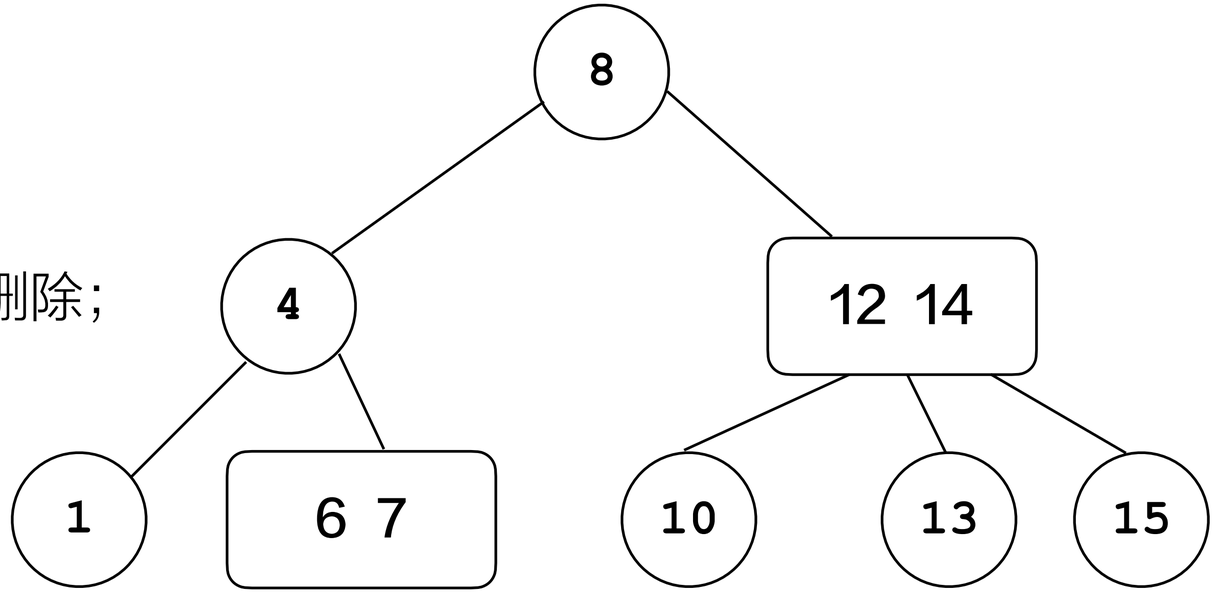
（1）查找目标关键字的位置（从根开始）

从根节点开始查找目标关键字；

若找到关键字：如果该节点是叶子节点，可直接删除；

如果该节点是非叶子节点，需转化为在叶子节点中的删除；

若未找到关键字，递归进入对应子节点继续查找。



B 树的操作–删除

（2）若关键字位于非叶子节点 → 转为删除其前驱或后继关键字

从该节点的左子树中找出最大值（前驱）；

或从右子树中找出最小值（后继）；

用该值替换关键字，然后在叶子节点中递归删除前驱/后继值。

目的：让真正的删除操作都发生在 叶子节点，便于处理。

B 树的操作-删除

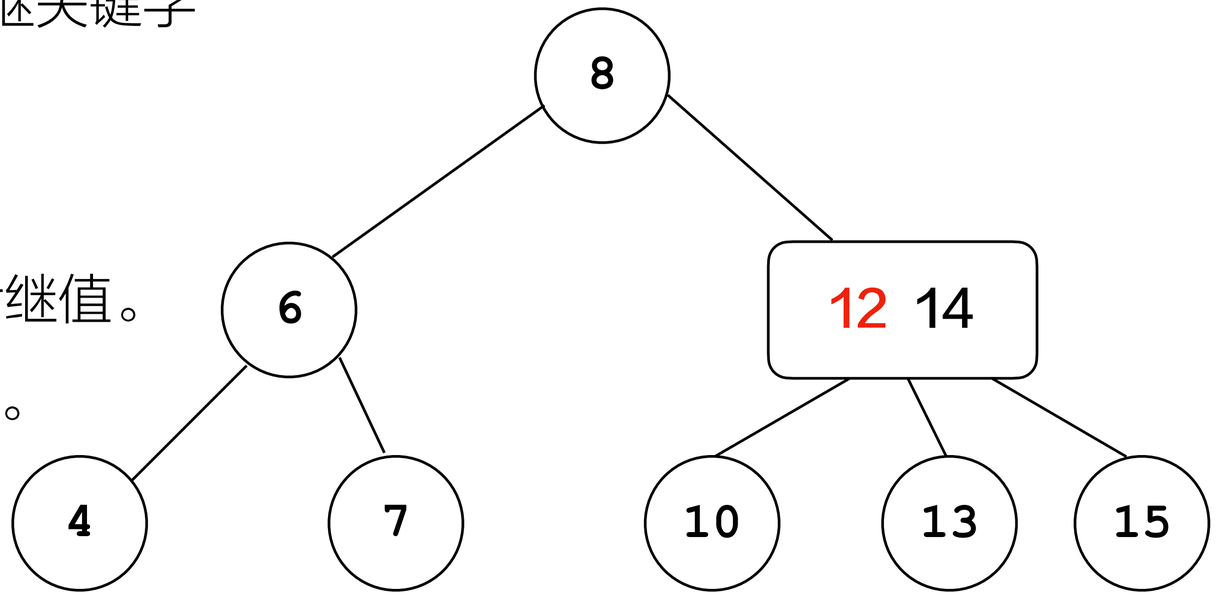
（2）若关键字位于非叶子节点 → 转为删除其前驱或后继关键字

从该节点的左子树中找出最大值（前驱）；

或从右子树中找出最小值（后继）；

用该值替换关键字，然后在叶子节点中递归删除前驱/后继值。

目的：让真正的删除操作都发生在 叶子节点，便于处理。



B 树的操作–删除

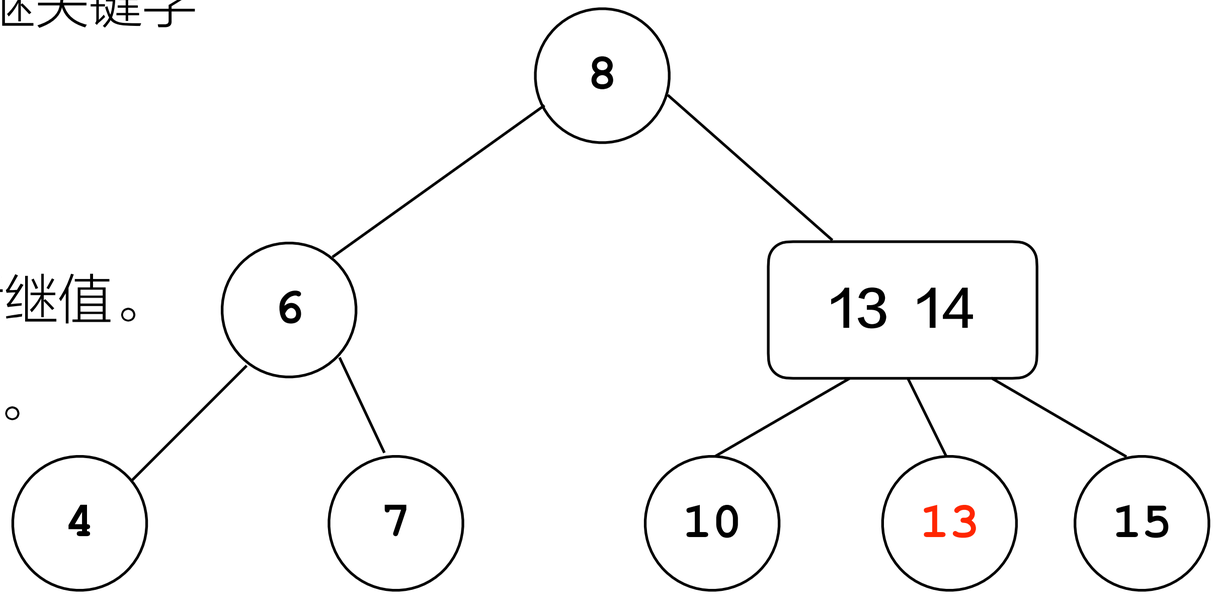
（2）若关键字位于非叶子节点 → 转为删除其前驱或后继关键字

从该节点的左子树中找出最大值（前驱）；

或从右子树中找出最小值（后继）；

用该值替换关键字，然后在叶子节点中递归删除前驱/后继值。

目的：让真正的删除操作都发生在 叶子节点，便于处理。



B 树的操作–删除

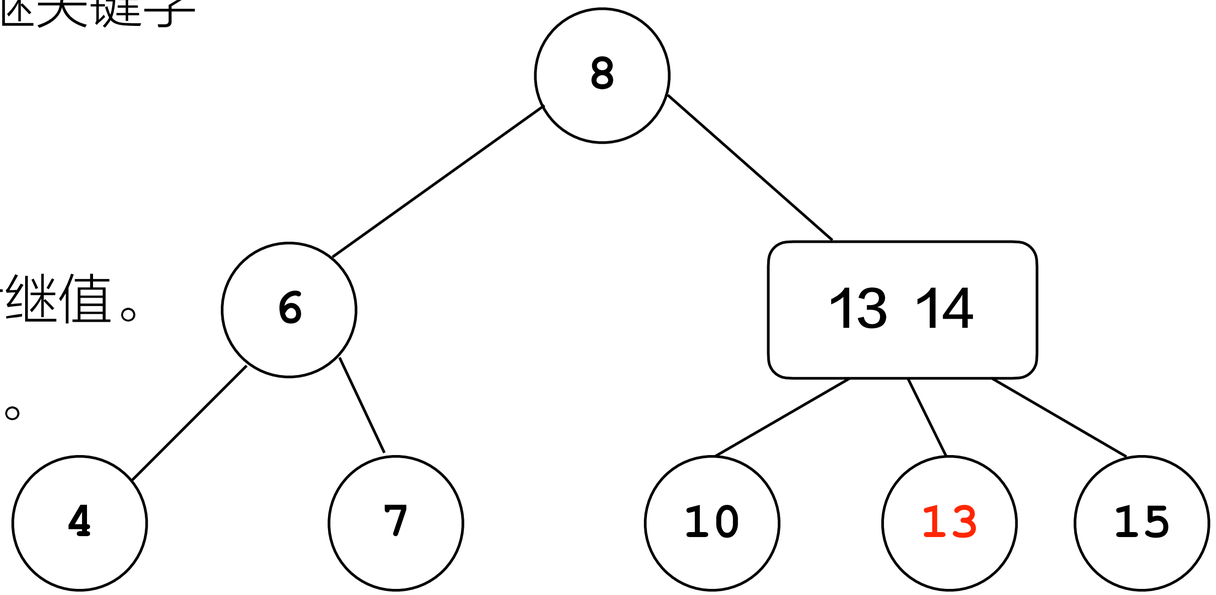
（2）若关键字位于非叶子节点 → 转为删除其前驱或后继关键字

从该节点的左子树中找出最大值（前驱）；

或从右子树中找出最小值（后继）；

用该值替换关键字，然后在叶子节点中递归删除前驱/后继值。

目的：让真正的删除操作都发生在 叶子节点，便于处理。



（3）删除前，判断目标子节点是否安全（关键字个数 ≥ ⌈m/2⌉）

如果不足，可能会导致删除后违反最小关键字约束 → 需要提前“填充”或“合并”，具体见第（4）步。

B 树的操作-删除

（4）不足时如何“填充”目标子节点

假设要进入第i个子节点 C[i] 删除：

情况一：左兄弟 C[i−1] 存在，且关键字数 > ⌈m/2⌉−1 → 借用

从父节点拿一个关键字下调至 C[i]；

左兄弟把自己的最大关键字上调到父节点；

若有子树，调整指针结构。

情况二：右兄弟 C[i+1] 存在，且关键字数 > ⌈m/2⌉−1 → 借用

从父节点拿一个关键字下调至 C[i]；

右兄弟把自己的最小关键字上调到父节点；

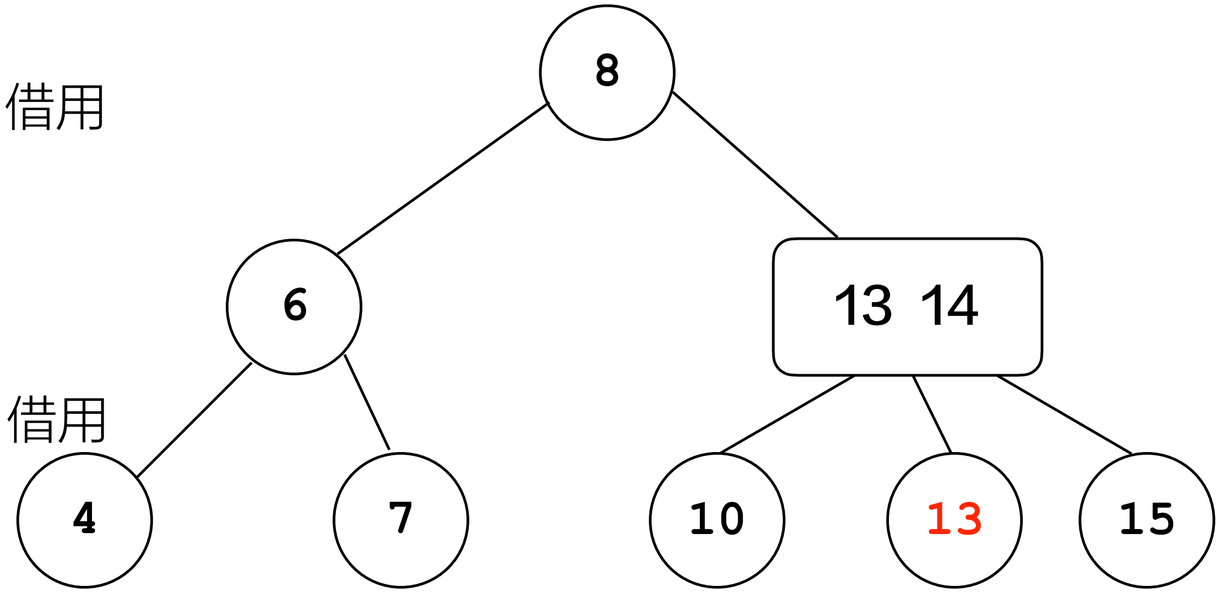
若有子树，调整指针结构。

情况三：两边兄弟都不足 → 合并

将当前子节点与一个兄弟节点合并成一个新节点；

父节点中对应的关键字也下调进入合并节点；

父节点关键字数量减少 → 若也不足，再继续向上传递。



B 树的操作-删除

（4）不足时如何“填充”目标子节点

假设要进入第i个子节点 C[i] 删除：

情况一：左兄弟 C[i−1] 存在，且关键字数 > ⌈m/2⌉−1 → 借用

从父节点拿一个关键字下调至 C[i]；

左兄弟把自己的最大关键字上调到父节点；

若有子树，调整指针结构。

情况二：右兄弟 C[i+1] 存在，且关键字数 > ⌈m/2⌉−1 → 借用

从父节点拿一个关键字下调至 C[i]；

右兄弟把自己的最小关键字上调到父节点；

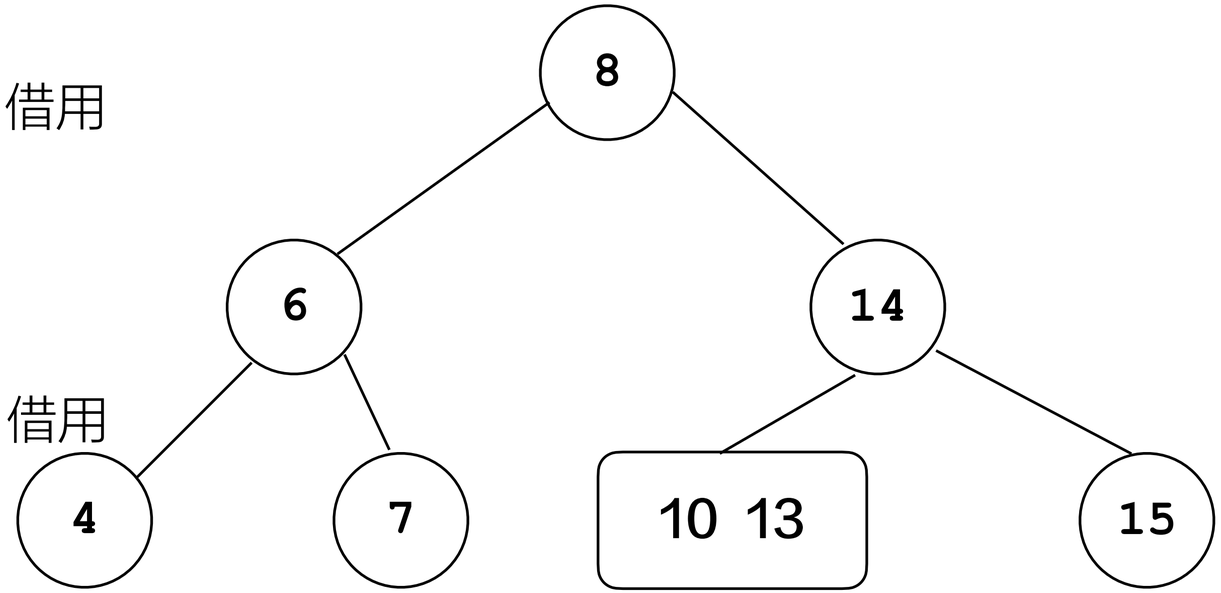
若有子树，调整指针结构。

情况三：两边兄弟都不足 → 合并

将当前子节点与一个兄弟节点合并成一个新节点；

父节点中对应的关键字也下调进入合并节点；

父节点关键字数量减少 → 若也不足，再继续向上传递。



B 树的操作-删除

（4）不足时如何“填充”目标子节点

假设要进入第i个子节点 C[i] 删除：

情况一：左兄弟 C[i−1] 存在，且关键字数 > ⌈m/2⌉−1 → 借用

从父节点拿一个关键字下调至 C[i]；

左兄弟把自己的最大关键字上调到父节点；

若有子树，调整指针结构。

情况二：右兄弟 C[i+1] 存在，且关键字数 > ⌈m/2⌉−1 → 借用

从父节点拿一个关键字下调至 C[i]；

右兄弟把自己的最小关键字上调到父节点；

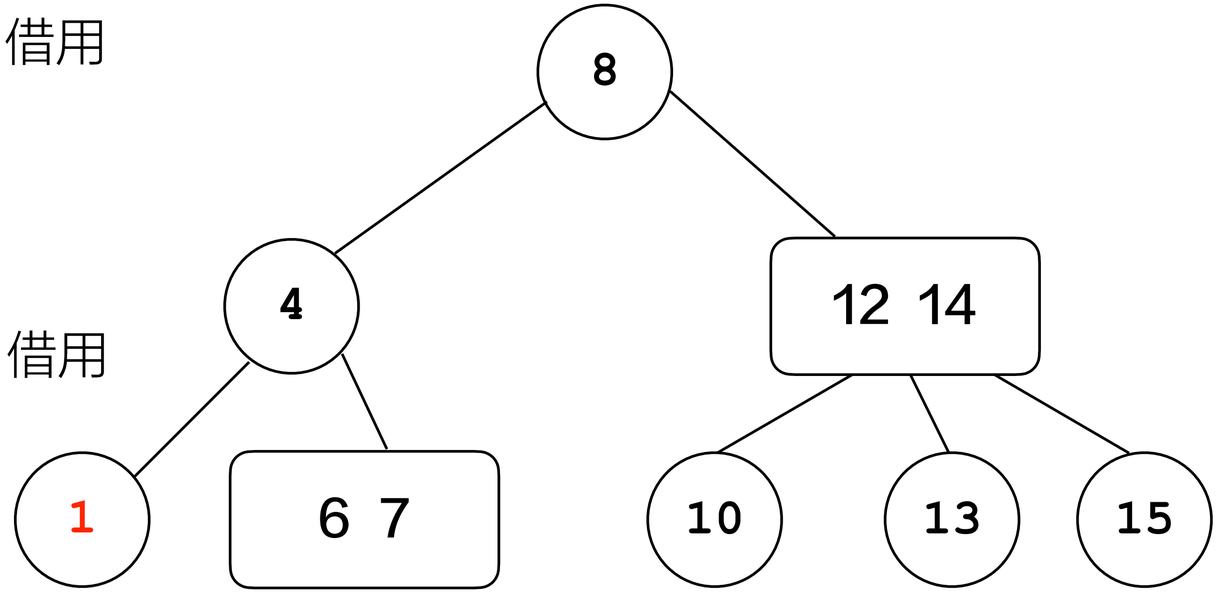
若有子树，调整指针结构。

情况三：两边兄弟都不足 → 合并

将当前子节点与一个兄弟节点合并成一个新节点；

父节点中对应的关键字也下调进入合并节点；

父节点关键字数量减少 → 若也不足，再继续向上传递。



B 树的操作-删除

（4）不足时如何“填充”目标子节点

假设要进入第i个子节点 C[i] 删除：

情况一：左兄弟 C[i−1] 存在，且关键字数 > ⌈m/2⌉−1 → 借用

从父节点拿一个关键字下调至 C[i]；

左兄弟把自己的最大关键字上调到父节点；

若有子树，调整指针结构。

情况二：右兄弟 C[i+1] 存在，且关键字数 > ⌈m/2⌉−1 → 借用

从父节点拿一个关键字下调至 C[i]；

右兄弟把自己的最小关键字上调到父节点；

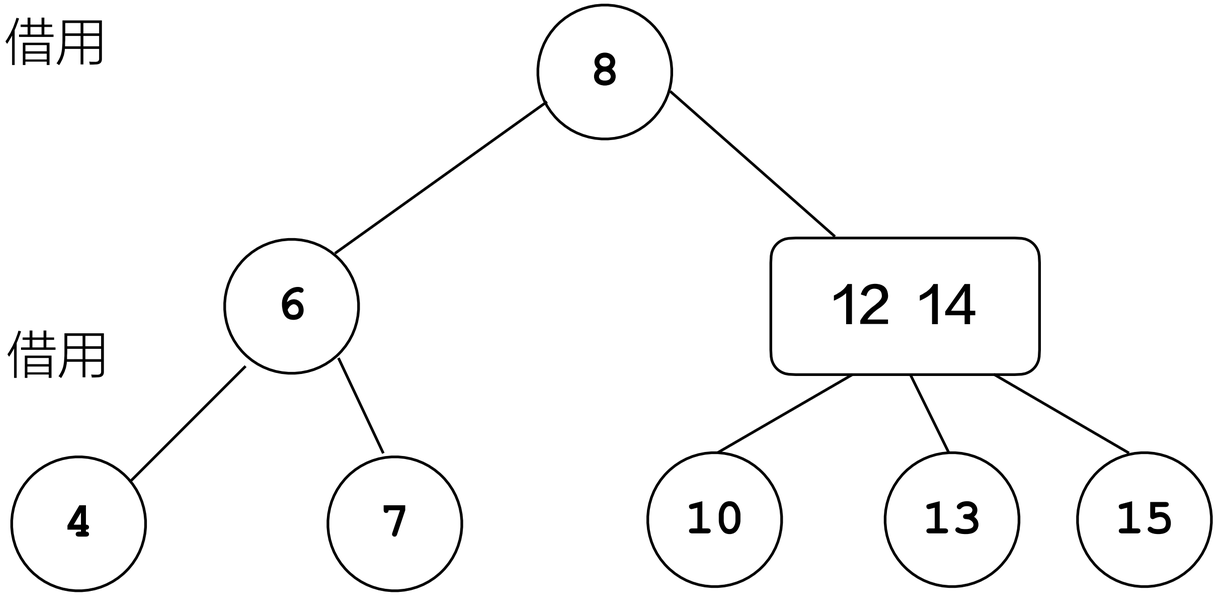
若有子树，调整指针结构。

情况三：两边兄弟都不足 → 合并

将当前子节点与一个兄弟节点合并成一个新节点；

父节点中对应的关键字也下调进入合并节点；

父节点关键字数量减少 → 若也不足，再继续向上传递。



B 树的操作-查找

在学习 B 树时，查找操作反而不是最难的，相比之下，插入和删除的逻辑才是真正体现 B 树结构特性的地方。 B 树的查找过程，其实并不复杂。本质上，它是一种多路查找：每一个结点中不仅有多个关键字，还有多个指向子树的指针。因此，每次在结点中查找时，都不是简单地“往左”或“往右”，而是根据关键字的数量进行“区间判断”，从而决定该走哪一棵子树。

B 树的操作-查找

（1）从根节点开始，逐层向下查找。

在当前结点中，从左到右依次比较关键字：

如果等于某个关键字，查找成功；

如果小于某个关键字，就转入该关键字左边的子树；

如果大于所有关键字，就转入最右边的子树。

（2）继续在子树中重复上述过程，直到查找成功或遇到叶子节点。

若在叶子节点中也没有找到，说明目标关键字不存在于该 B 树中；

若在中途命中关键字，查找结束，返回该位置。

（3）整个过程本质上是“在结点内查找 + 递归进入子树”交替进行，直到命中或结束。

可以将 B 树的查找过程类比为“查电话本”：

使用 B 树的好处

减少磁盘读写次数：由于节点内存储多个关键字，树的高度被显著降低，从而减少了磁盘访问。

适用于大数据量场景：B 树特别适用于数据库索引和文件系统等需要高效读写大量数据的应用。

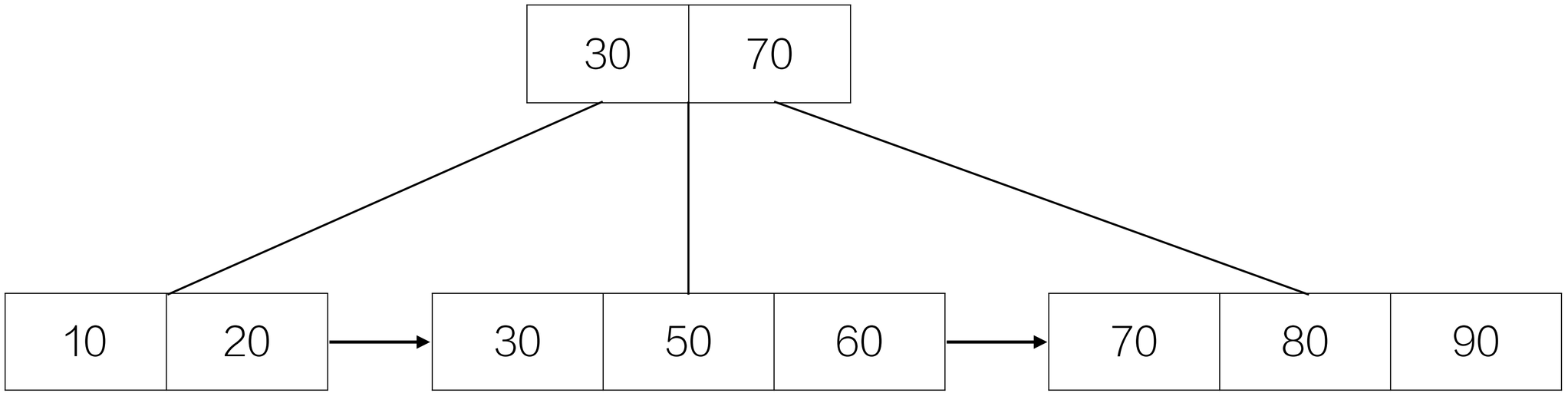
B+ 树

B+树其实是 B 树的一个变种，是在数据库系统、文件系统中应用最广泛的一种平衡查找树结构。它的本质仍然是“分层、多分支、平衡”的，但与 B 树相比，它在结构上做了两处关键性的优化：

（1）所有关键字只出现在叶子结点中，非叶子结点仅用于索引。

（2）所有叶子结点按大小顺序通过指针连接，形成有序链表。

B+ 树



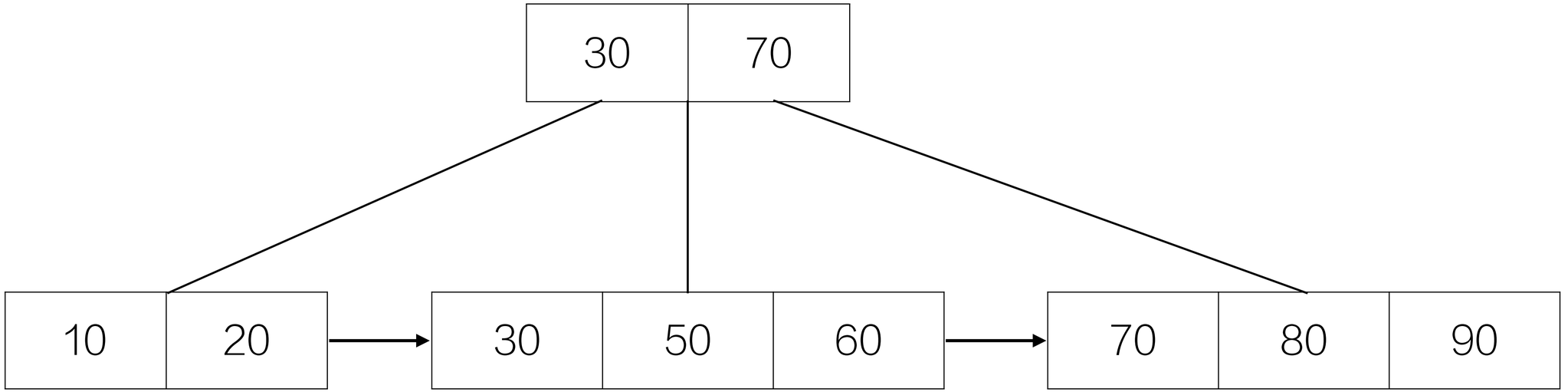
B+ 树查找过程

从根结点出发，比较关键字；

沿着“索引指针”不断下沉，直到找到叶子结点；

在叶子结点中查找目标关键字；

若找不到，也能快速定位范围插入点。



B+ 树的优势

更适合磁盘读取：由于非叶子节点不存储实际数据，单个磁盘块可以容纳更多关键字，从而减少 IO 次数。

范围查找更高效：只需在叶子结点链表中顺序扫描即可，无需回溯中间节点。

查询结果更稳定：所有数据都集中在叶子节点，查找路径一致，避免“某些数据层数较深”的情况。

B 树与 B+ 树

对比维度	B树	B+树
数据位置	所有结点者存储数据	仅叶子结点存储数据
查询路径	路径不等，可能提前结束	路径一致，总是查到叶子结点
范围查找	不方便，需要中序遍历	叶子结点链表连接，查找范围非常高效
适用场景	适用于内存结构	特别适用于磁盘存储、数据库索引结构

真题解析

[2009] 8. 下列叙述中，不符合 m 阶 B 树定义要求的是_____。

A. 根结点最多有 m 棵子树

B. 所有叶结点都在同一层上

C. 各结点内关键字均升序或降序排列

D. 叶结点之间通过指针链接

真题解析

[2009] 8. 下列叙述中，不符合 m 阶 B 树定义要求的是_____。

A. 根结点最多有 m 棵子树

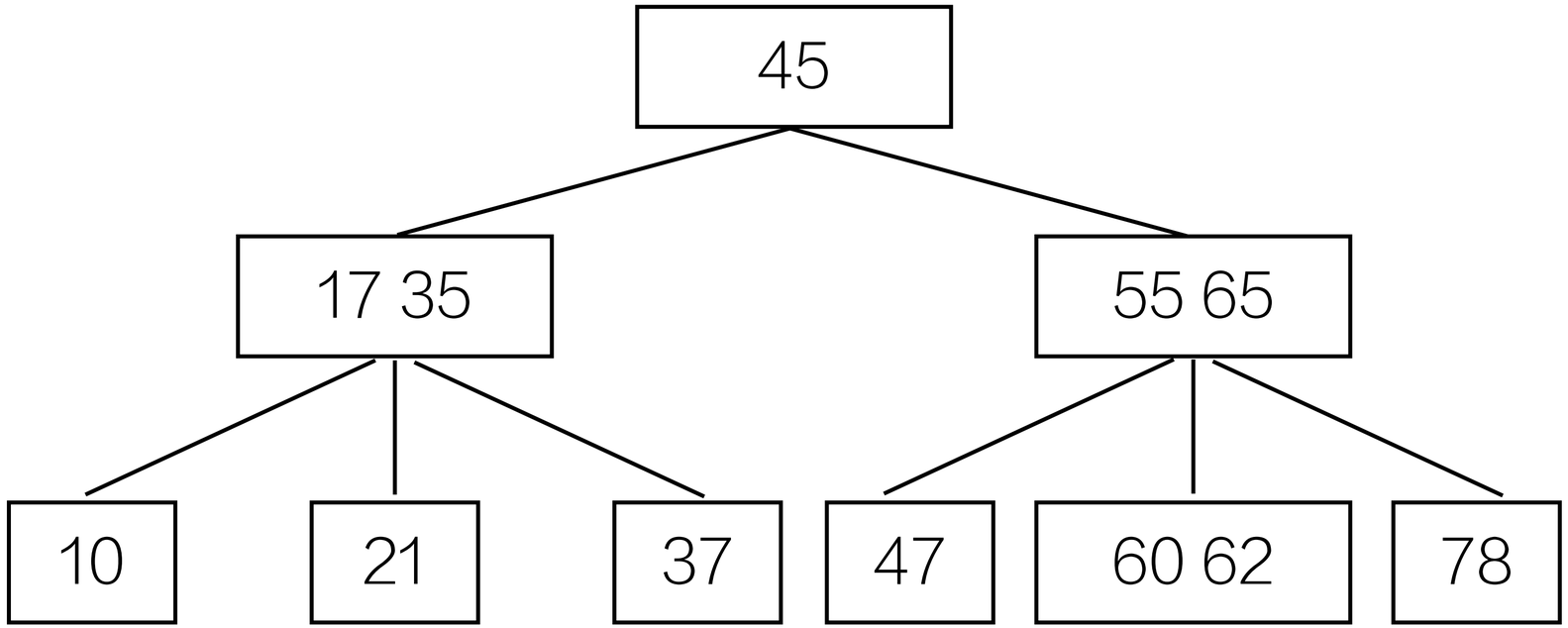
B. 所有叶结点都在同一层上

C. 各结点内关键字均升序或降序排列

D. 叶结点之间通过指针链接

真题解析

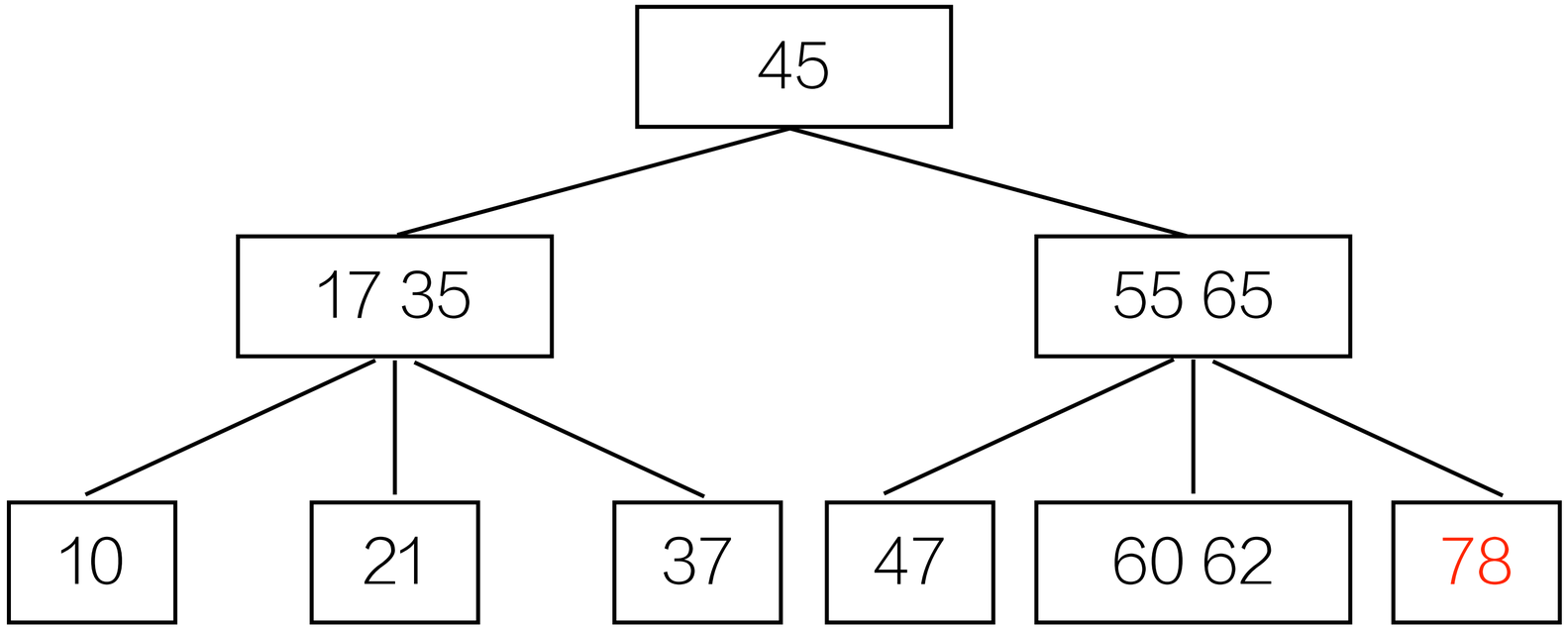
[2012] 9. 已知 3 阶 B- 树，如下图所示。删除关键字 78 得到一棵新 B- 树，其右叶结点中的关键字是_____。



A. 60　　B. 60, 62　　C. 62, 65　　D. 65

真题解析

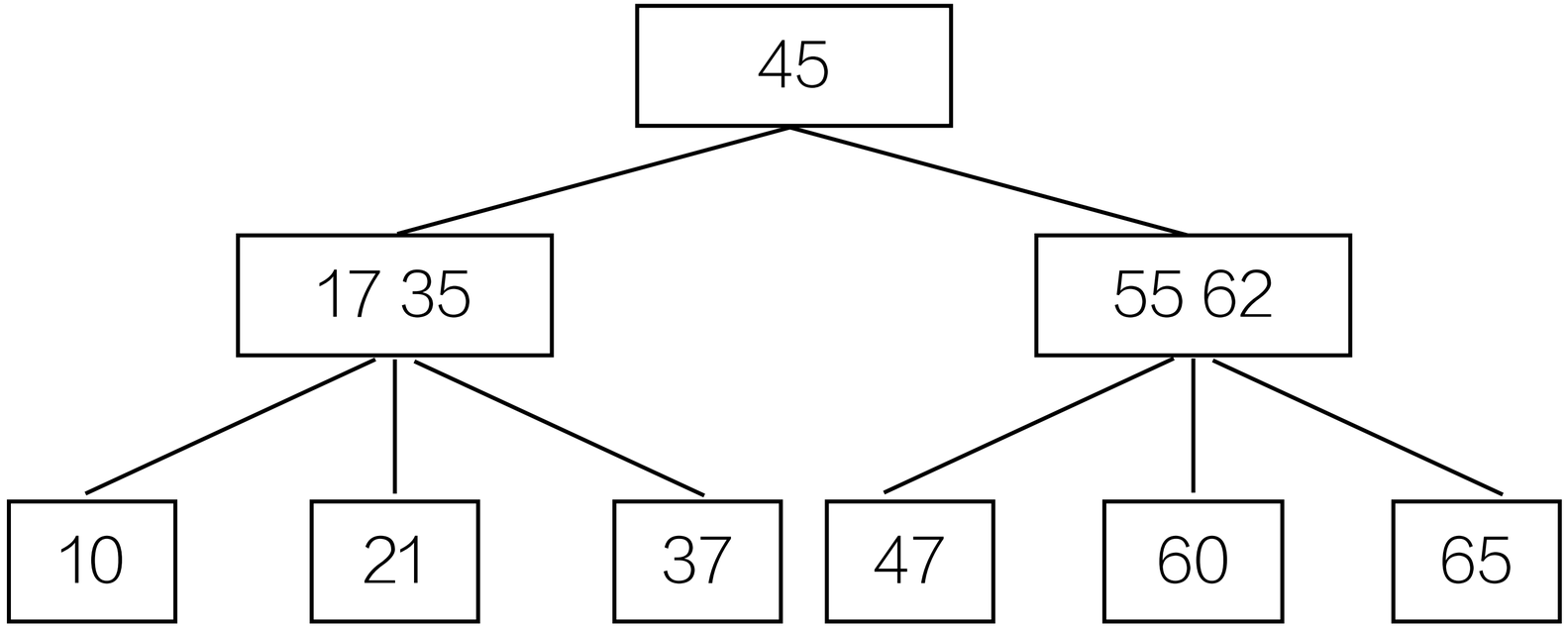
[2012] 9. 已知 3 阶 B- 树，如下图所示。删除关键字 78 得到一棵新 B- 树，其右叶结点中的关键字是_____。



A. 60　　B. 60, 62　　C. 62, 65　　D. 65

真题解析

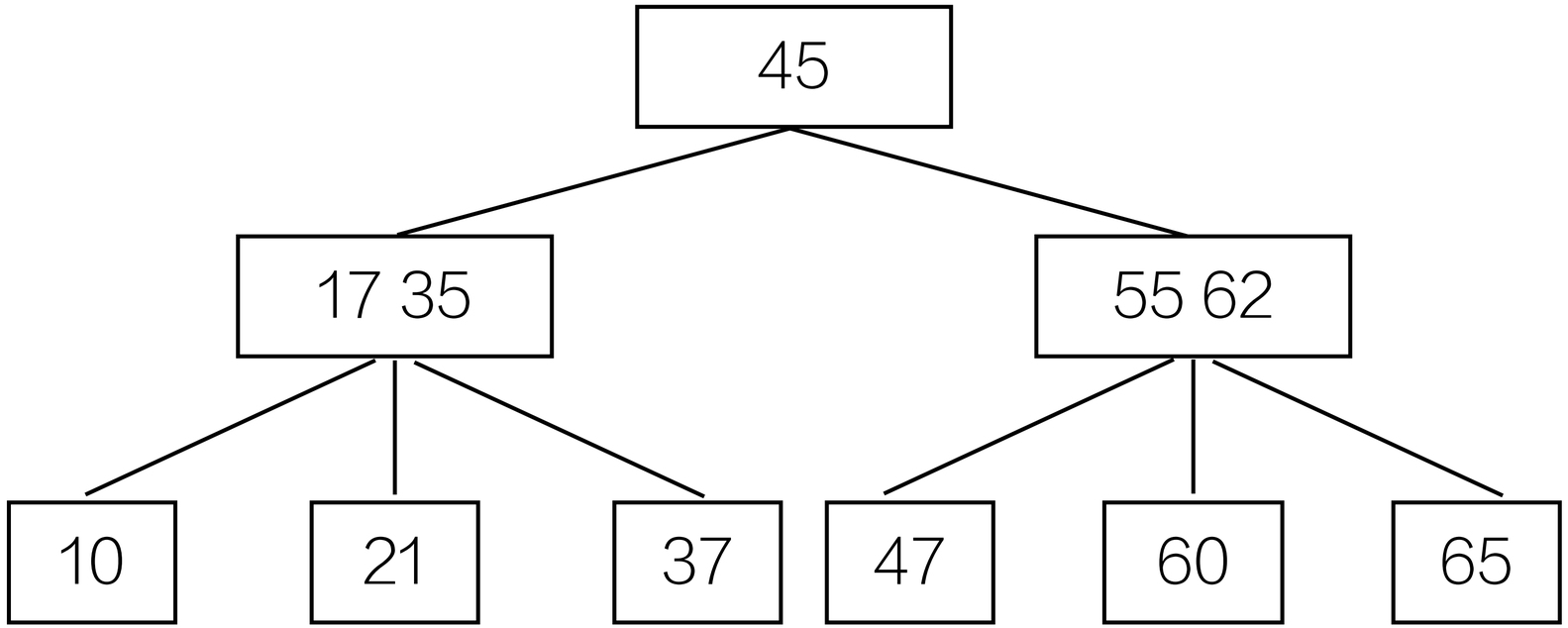
[2012] 9. 已知 3 阶 B- 树，如下图所示。删除关键字 78 得到一棵新 B- 树，其右叶结点中的关键字是_____。



A. 60　　B. 60, 62　　C. 62, 65　　D. 65

真题解析

[2012] 9. 已知 3 阶 B- 树，如下图所示。删除关键字 78 得到一棵新 B- 树，其右叶结点中的关键字是_____。



A. 60　　B. 60, 62　　C. 62, 65　　D. 65

真题解析

[2013] 10. 在一棵高度为 2 的 5 阶 B 树中，所含关键字的个数最少是_____。

A. 5　　B. 7　　C. 8　　D. 14

真题解析

[2013] 10. 在一棵高度为 2 的 5 阶 B 树中，所含关键字的个数最少是_____。

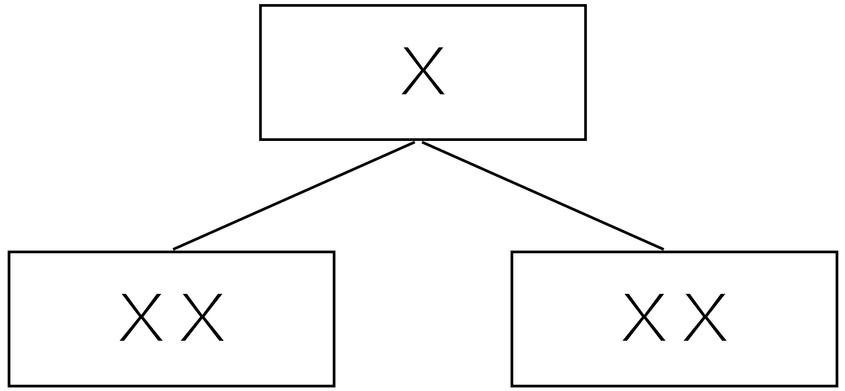
A. 5　　B. 7　　C. 8　　D. 14

XXXX

真题解析

[2013] 10. 在一棵高度为 2 的 5 阶 B 树中，所含关键字的个数最少是_____。

A. 5　　B. 7　　C. 8　　D. 14



真题解析

[2013] 10. 在一棵高度为 2 的 5 阶 B 树中，所含关键字的个数最少是_____。

A. 5　　B. 7　　C. 8　　D. 14

真题解析

[2014] 9. 在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点个数最多是_____。

A. 5　　B. 6　　C. 10　　D. 15

真题解析

[2014] 9. 在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点个数最多是_____。

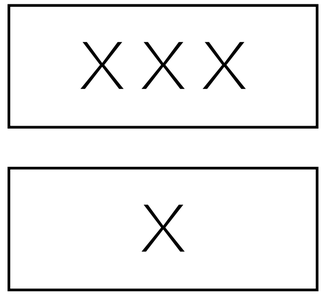
A. 5　　B. 6　　C. 10　　D. 15

XXX

真题解析

[2014] 9. 在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点个数最多是_____。

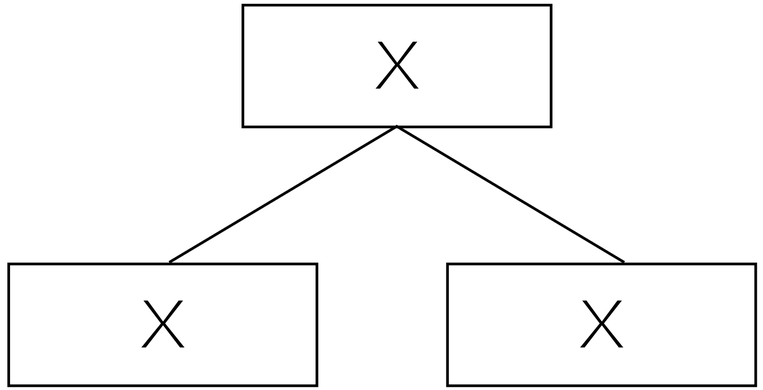
A. 5　　B. 6　　C. 10　　D. 15



真题解析

[2014] 9. 在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点个数最多是_____。

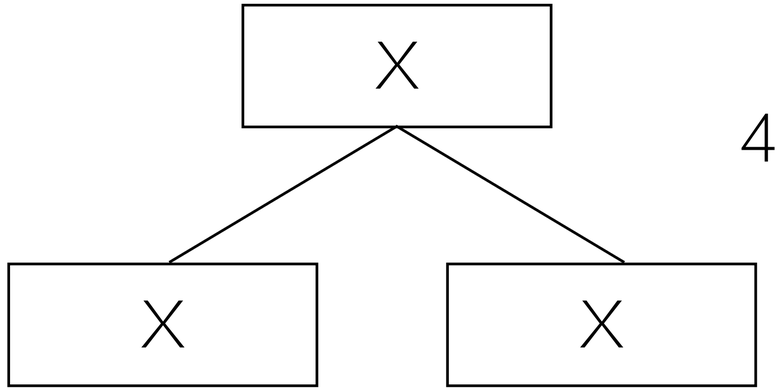
A. 5　　B. 6　　C. 10　　D. 15



真题解析

[2014] 9. 在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点个数最多是_____。

A. 5　　B. 6　　C. 10　　D. 15



4 层满二叉树多少个结点?

真题解析

[2014] 9. 在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点个数最多是_____。

A. 5　　B. 6　　C. 10　　D. 15

真题解析

[2016] 10. B+ 树不同于 B 树的特点之一是_____。

A. 能支持顺序查找

B. 结点中含有关键字

C. 根结点至少有两个分支

D. 所有叶子结点都在同一层

真题解析

[2016] 10. B+ 树不同于 B 树的特点之一是_____。

A. 能支持顺序查找

B. 结点中含有关键字

C. 根结点至少有两个分支

D. 所有叶子结点都在同一层

真题解析

[2017] 9. 下列应用中，适合使用 B+ 树的是_____。

A. 编译器中的记法分析

B. 关系数据库系统中的索引

C. 网络中的路由表快速查找

D. 操作系统的磁盘空闲块管理

真题解析

[2017] 9. 下列应用中，适合使用 B+ 树的是_____。

A. 编译器中的记法分析

B. 关系数据库系统中的索引

C. 网络中的路由表快速查找

D. 操作系统的磁盘空闲块管理

真题解析

[2018] 8. 高度为 5 的 3 阶 B 树含有的关键字个数至和是_____。

A. 15　　B. 31　　C. 62　　D. 242

真题解析

[2018] 8. 高度为 5 的 3 阶 B 树含有的关键字个数至和是_____。

A. 15　　B. 31　　C. 62　　D. 242

真题解析

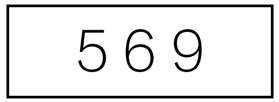
[2020] 10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是_____。

A. 8　　B. 6， 9　　C. 8, 13　　D. 9, 12

真题解析

[2020] 10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是_____。

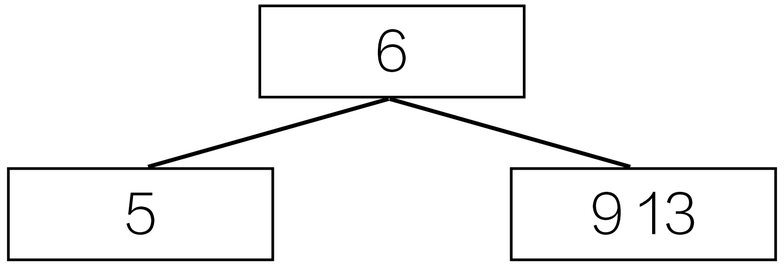
A. 8　　B. 6， 9　　C. 8, 13　　D. 9, 12



真题解析

[2020] 10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是_____。

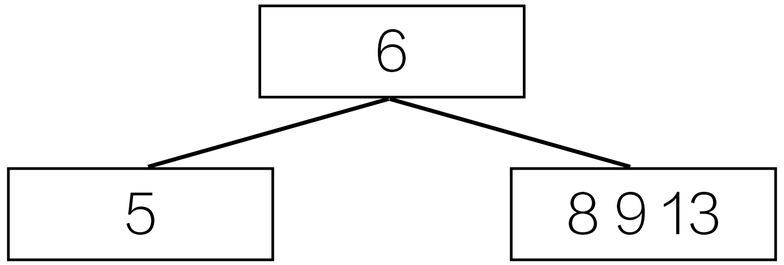
A. 8　　B. 6， 9　　C. 8, 13　　D. 9, 12



真题解析

[2020] 10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是_____。

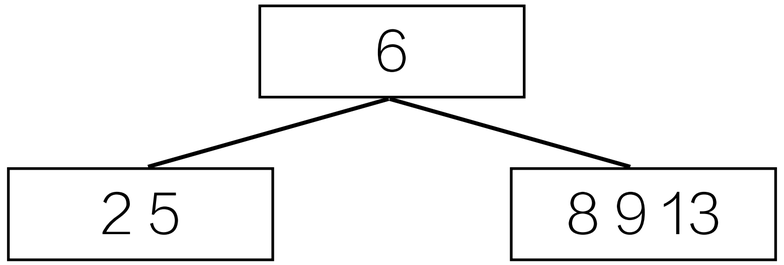
A. 8　　B. 6， 9　　C. 8, 13　　D. 9, 12



真题解析

[2020] 10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是_____。

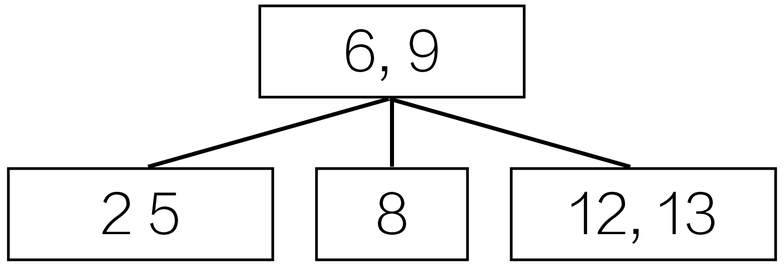
A. 8　　B. 6， 9　　C. 8, 13　　D. 9, 12



真题解析

[2020] 10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是_____。

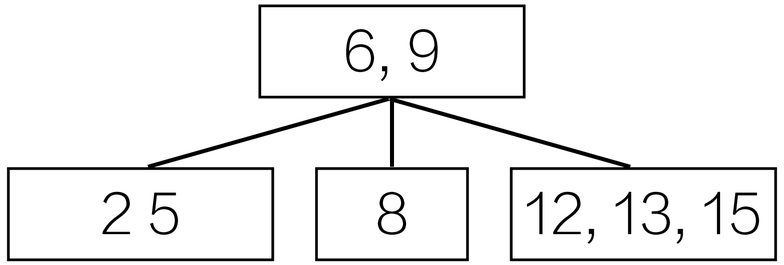
A. 8　　B. 6，　9　　C. 8, 13　　D. 9, 12



真题解析

[2020] 10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是_____。

A. 8　　B. 6， 9　　C. 8, 13　　D. 9, 12



真题解析

[2020] 10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是_____。

A. 8　　B. 6，9　　C. 8, 13　　D. 9, 12

真题解析

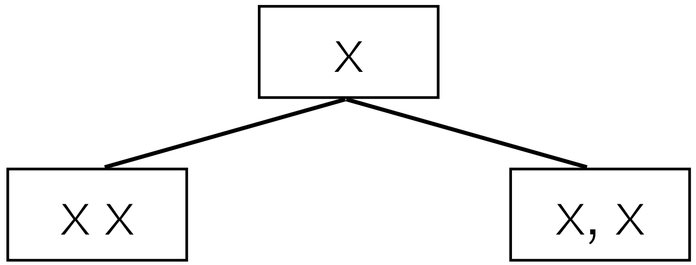
[2021] 9. 在一棵高度为 3 的 3 阶 B 树中，根为第 1 层，若第 2 层中有 4 个关键字，则该树的结点个数最多是_____。

A. 11　　B. 10　　C. 9　　D. 8

真题解析

[2021] 9. 在一棵高度为 3 的 3 阶 B 树中，根为第 1 层，若第 2 层中有 4 个关键字，则该树的结点个数最多是_____。

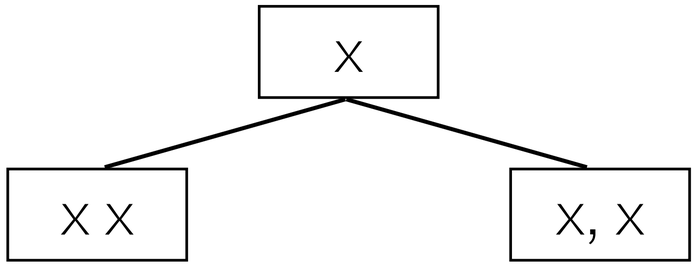
A. 11　　B. 10　　C. 9　　D. 8

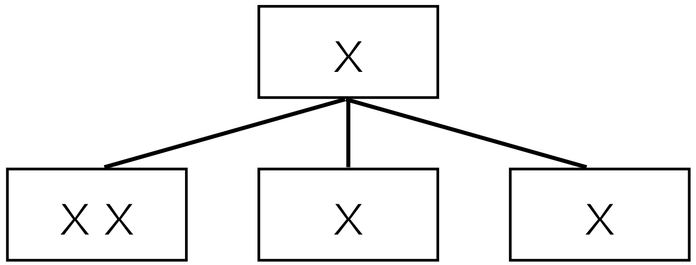


真题解析

[2021] 9. 在一棵高度为 3 的 3 阶 B 树中，根为第 1 层，若第 2 层中有 4 个关键字，则该树的结点个数最多是_____。

A. 11　　B. 10　　C. 9　　D. 8

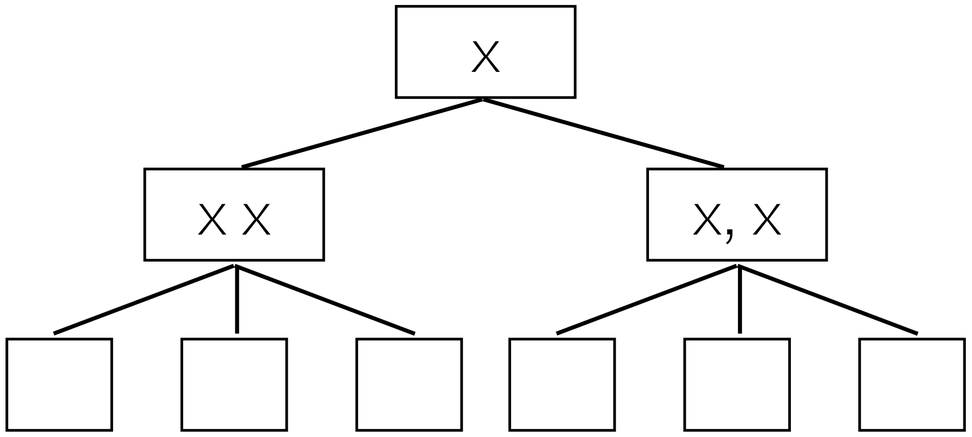


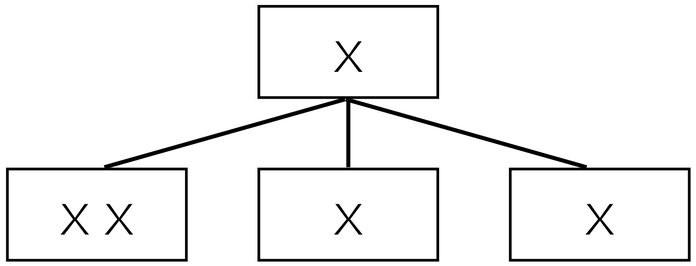


真题解析

[2021] 9. 在一棵高度为 3 的 3 阶 B 树中，根为第 1 层，若第 2 层中有 4 个关键字，则该树的结点个数最多是_____。

A. 11　　B. 10　　C. 9　　D. 8

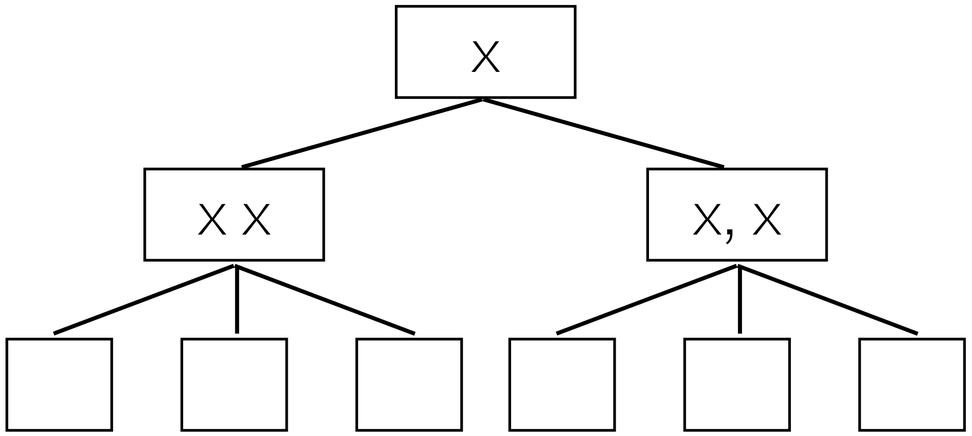


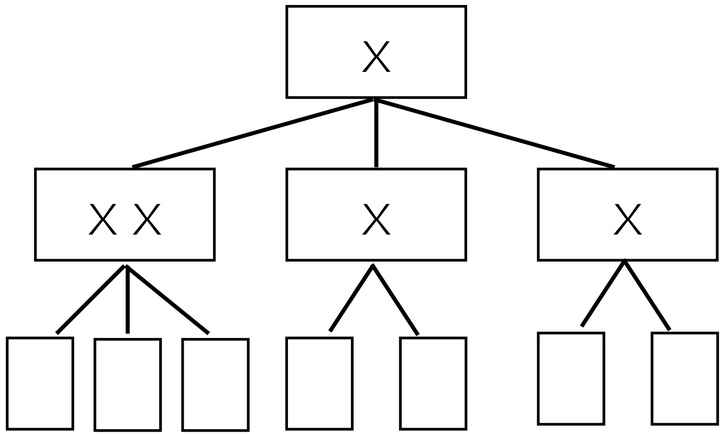


真题解析

[2021] 9. 在一棵高度为 3 的 3 阶 B 树中，根为第 1 层，若第 2 层中有 4 个关键字，则该树的结点个数最多是_____。

A. 11　　B. 10　　C. 9　　D. 8

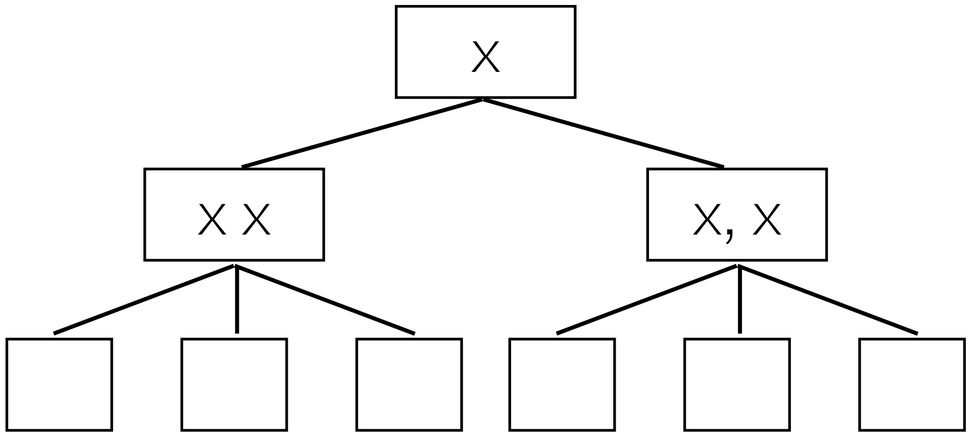




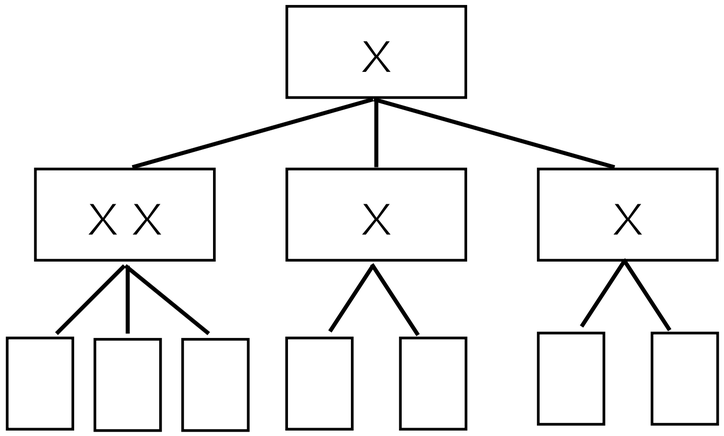
真题解析

[2021] 9. 在一棵高度为 3 的 3 阶 B 树中，根为第 1 层，若第 2 层中有 4 个关键字，则该树的结点个数最多是_____。

A. 11　　B. 10　　C. 9　　D. 8



9



11

真题解析

[2021] 9. 在一棵高度为 3 的 3 阶 B 树中，根为第 1 层，若第 2 层中有 4 个关键字，则该树的结点个数最多是_____。

A. 11　　B. 10　　C. 9　　D. 8

散列表

前面介绍的顺序查找、折半查找，甚至二叉排序树中，查找的效率无不受到“比较次数”的制约。哪怕已经做到了对数级（如折半查找、平衡二叉树），也仍然不能突破这个“比较”的瓶颈。

有没有一种方法，不需要一轮轮地比较，而是一眼就能找到数据在哪里？这听起来像是开挂了，但现实中确实存在这样的结构 —— 散列表（Hash Table），根据音译，通常也被称作哈希表。

散列表

散列表的核心思想非常简单也非常聪明：

用一个“函数”直接计算出数据存储的位置，从而实现快速查找。

就像查词典：你想查“apple”，不需要一页页翻，你根据字母表顺序直接翻到以“A”开头的部分，这其实就是一种“散列”的思想。只不过在散列表中，是用一个“数学函数”来计算这个位置的。

散列表相关概念-散列函数

根据音译，通常也被称作哈希函数，是一种将关键字（Key）映射到散列表中某个地址（位置）的函数。它的作用是根据关键字，快速计算出该元素应该存储的位置。

可以把散列函数想象成一个“指路神器”或“数学变压器”，它接受一个输入（比如学号、名字、手机号），然后通过某种计算规则，把这个输入变成一个储物格编号（地址），告诉你数据该放在哪。

散列表相关概念-散列函数

根据音译，通常也被称作哈希函数，是一种将关键字（Key）映射到散列表中某个地址（位置）的函数。它的作用是根据关键字，快速计算出该元素应该存储的位置。

可以把散列函数想象成一个“指路神器”或“数学变压器”，它接受一个输入（比如学号、名字、手机号），然后通过某种计算规则，把这个输入变成一个储物格编号（地址），告诉你数据该放在哪。

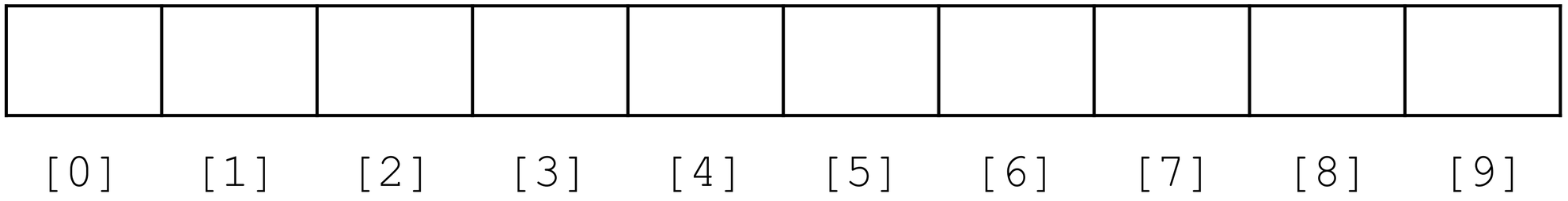
```
H(key) = key % 10
```

散列表相关概念-散列函数

根据音译，通常也被称作哈希函数，是一种将关键字（Key）映射到散列表中某个地址（位置）的函数。它的作用是根据关键字，快速计算出该元素应该存储的位置。

可以把散列函数想象成一个“指路神器”或“数学变压器”，它接受一个输入（比如学号、名字、手机号），然后通过某种计算规则，把这个输入变成一个储物格编号（地址），告诉你数据该放在哪。

```
H(key) = key % 10
```



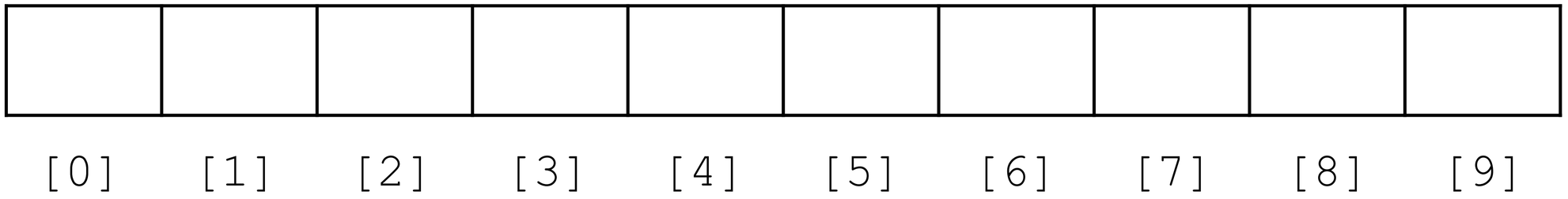
散列表相关概念-散列函数

根据音译，通常也被称作哈希函数，是一种将关键字（Key）映射到散列表中某个地址（位置）的函数。它的作用是根据关键字，快速计算出该元素应该存储的位置。

可以把散列函数想象成一个“指路神器”或“数学变压器”，它接受一个输入（比如学号、名字、手机号），然后通过某种计算规则，把这个输入变成一个储物格编号（地址），告诉你数据该放在哪。

```
H(key) = key % 10
```

key = 23



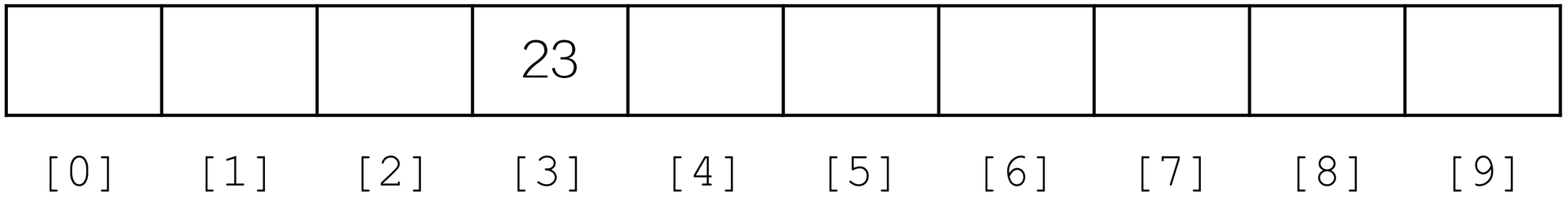
散列表相关概念-散列函数

根据音译，通常也被称作哈希函数，是一种将关键字（Key）映射到散列表中某个地址（位置）的函数。它的作用是根据关键字，快速计算出该元素应该存储的位置。

可以把散列函数想象成一个“指路神器”或“数学变压器”，它接受一个输入（比如学号、名字、手机号），然后通过某种计算规则，把这个输入变成一个储物格编号（地址），告诉你数据该放在哪。

```
H(key) = key % 10
```

key = 23



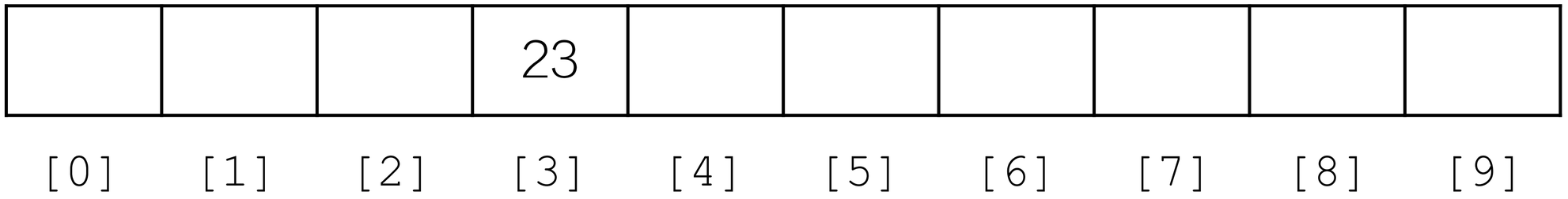
散列表相关概念-散列函数

根据音译，通常也被称作哈希函数，是一种将关键字（Key）映射到散列表中某个地址（位置）的函数。它的作用是根据关键字，快速计算出该元素应该存储的位置。

可以把散列函数想象成一个“指路神器”或“数学变压器”，它接受一个输入（比如学号、名字、手机号），然后通过某种计算规则，把这个输入变成一个储物格编号（地址），告诉你数据该放在哪。

```
H(key) = key % 10
```

key = 97



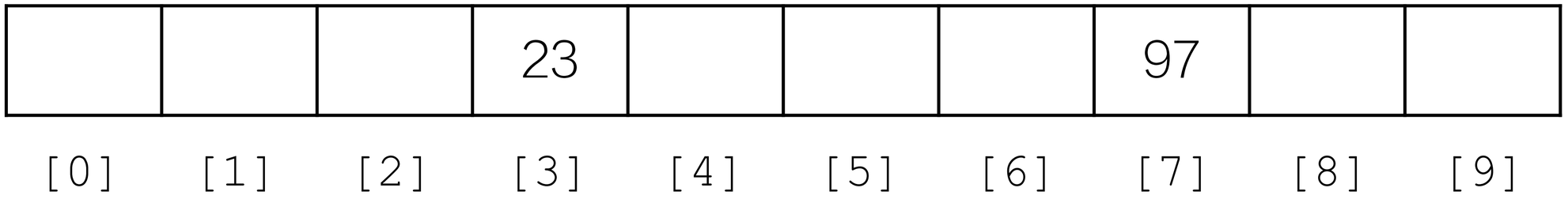
散列表相关概念-散列函数

根据音译，通常也被称作哈希函数，是一种将关键字（Key）映射到散列表中某个地址（位置）的函数。它的作用是根据关键字，快速计算出该元素应该存储的位置。

可以把散列函数想象成一个“指路神器”或“数学变压器”，它接受一个输入（比如学号、名字、手机号），然后通过某种计算规则，把这个输入变成一个储物格编号（地址），告诉你数据该放在哪。

```
H(key) = key % 10
```

key = 97

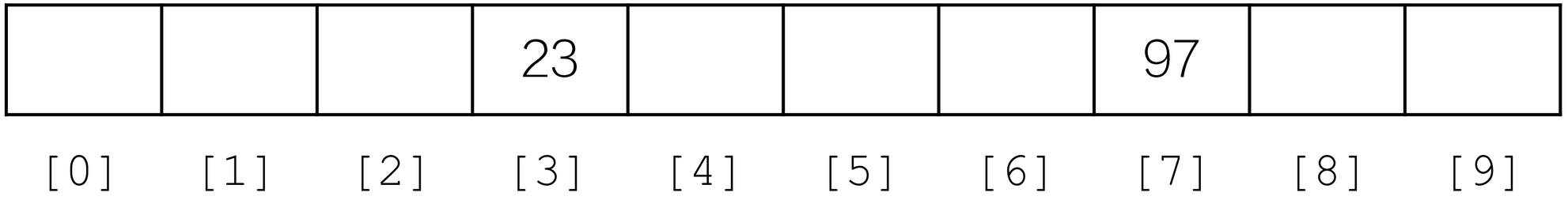


散列表相关概念-散列表

根据音译，通常也被称作哈希表，是一种根据关键字通过散列函数计算出位置，然后将数据存储在该位置的结构。散列表本质上是一个数组，但它不需要像数组那样依次查找，而是直接定位、直接查找。

可以把散列表看成一个“带编号的储物柜”，每个储物格都能放一个或多个数据，当要存放某个数据时，先通过“散列函数”决定放在哪一格，查找的时候也只需“按图索骥”，效率极高。要散列函数设计合理，查找效率可以接近 O(1)，比顺序查找、折半查找还快。

```
H(key) = key % 10
```



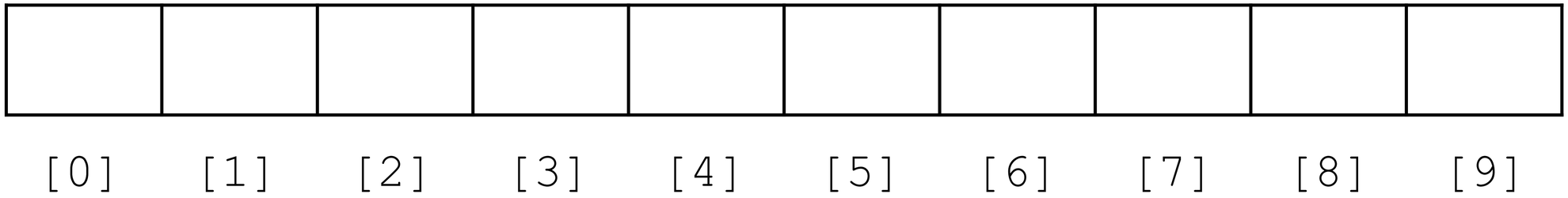
散列表相关概念-冲突

当两个不同的关键字通过散列函数计算后，得到相同的结果值（地址），它们就会被映射到散列表的同一个位置，这种现象称为哈希冲突或散列冲突（简称“冲突”）。

散列表相关概念-冲突

当两个不同的关键字通过散列函数计算后，得到相同的结果值（地址），它们就会被映射到散列表的同一个位置，这种现象称为哈希冲突或散列冲突（简称“冲突”）。

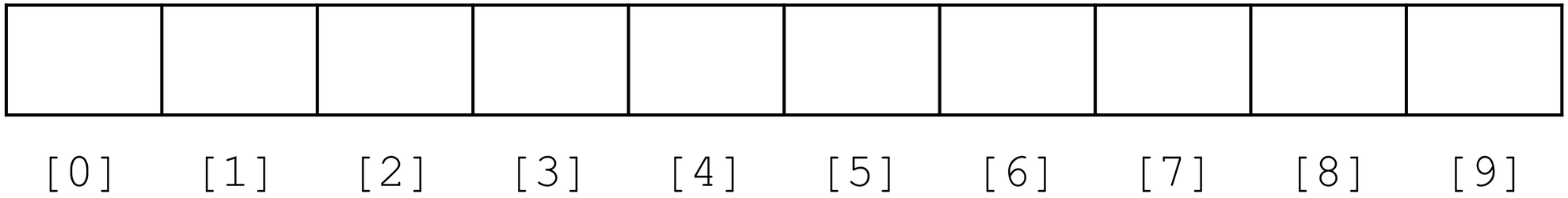
```
H(key) = key % 12
```



散列表相关概念-冲突

当两个不同的关键字通过散列函数计算后，得到相同的结果值（地址），它们就会被映射到散列表的同一个位置，这种现象称为哈希冲突或散列冲突（简称“冲突”）。

```
H(key) = key % 12
```



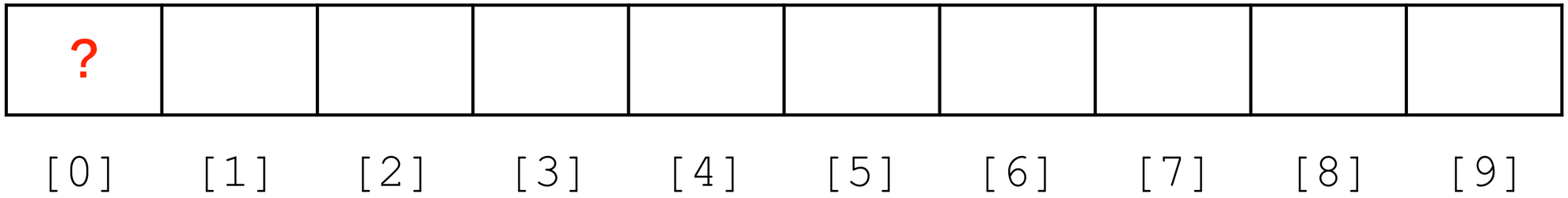
key = 12

key = 144

散列表相关概念-冲突

当两个不同的关键字通过散列函数计算后，得到相同的结果值（地址），它们就会被映射到散列表的同一个位置，这种现象称为哈希冲突或散列冲突（简称“冲突”）。

```
H(key) = key % 12
```



key = 12

key = 144

散列函数构造方法

除留余数法

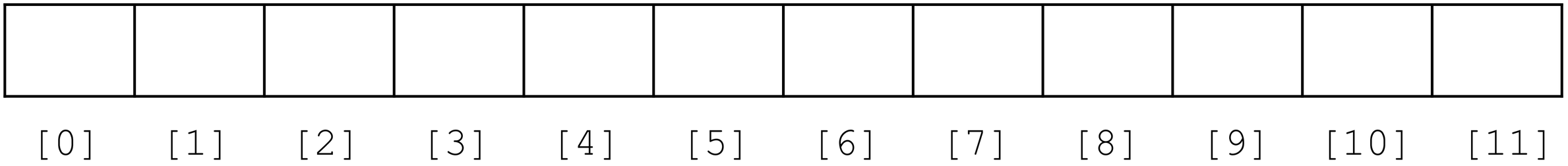
数字分析法

平方取中法

散列函数构造方法-除留余数法

{12, 47, 25, 22, 38, 21, 15, 56, 16, 67, 29, 78}

H(key) = key % p , p=12



散列函数构造方法-除留余数法

{12, 47, 25, 22, 38, 21, 15, 56, 16, 67, 29, 78}

H(key) = key % p , p=12

12	25	38	15	16	29	78	67	56	21	22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

散列函数构造方法-除留余数法

{12, 47, 25, 22, 38, 21, 15, 56, 16, 67, 29, 78}

H(key) = key % p , p=12

12	25	38	15	16	29	78	67	56	21	22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

使用合适的模数很关键，一般建议 p 为一个不太接近 2 的幂的素数，这样可以让关键字在散列表中分布得更均匀，减少冲突。

散列函数构造方法-数字分析法

数字分析法是一种根据关键字中某些“数位”上的数字来构造散列函数的方法。其核心思想是：观察所有关键字在不同数位上数字的分布特征，选择分布较均匀的一位或几位，提取出来作为散列地址。 例如，关键字是一串电话号码或学号，从中“挑几个有代表性的数字”来作为它在散列表中的位置

散列函数构造方法-数字分析法

通常的操作步骤是：

（1）收集全部关键字样本；

（2）统计每一位数字的分布情况；

（3）选择分布最“离散”的几位数字；

（4）将这些数字拼接/组合为散列地址。

散列函数构造方法-数字分析法

假设一组关键字为如下学生的学号

学号（关键字）	千位	百位	十位	个位
7132	7	1	3	2
8433	8	4	3	3
9531	9	5	3	1
7734	7	7	3	4

散列函数构造方法-数字分析法

假设一组关键字为如下学生的学号

学号（关键字）	千位	百位	十位	个位
7132	7	1	3	2
8433	8	4	3	3
9531	9	5	3	1
7734	7	7	3	4

学号（关键字）	百位+个位	散列地址
7132	12	12
8433	43	43
9531	51	51
7734	74	74

散列函数构造方法-平方取中法

平方取中法是指，先对关键字进行平方运算，然后从所得结果中间若干位中取出一部分数字，作为散列地址。散列函数的一般形式为：

$$H(key) = MiddleDigits(key \times key)$$

散列函数构造方法-平方取中法

平方取中法是指，先对关键字进行平方运算，然后从所得结果中间若干位中取出一部分数字，作为散列地址。散列函数的一般形式为：

$$H(key) = MiddleDigits(key \times key)$$

假设关键字是一串数字，先让它“自乘”一遍（平方），再从这个结果的中间部分挖出几位作为“门牌号”（即地址）。这个方法的核心理念是，平方运算会引入更多的组合信息，取中间部分可以“削弱”原始数字的某些集中性，提高地址分布的均匀性。

散列函数构造方法-平方取中法

平方取中法是指，先对关键字进行平方运算，然后从所得结果中间若干位中取出一部分数字，作为散列地址。散列函数的一般形式为：

$$H(key) = MiddleDigits(key \times key)$$

假设关键字是一串数字，先让它“自乘”一遍（平方），再从这个结果的中间部分挖出几位作为“门牌号”（即地址）。这个方法的核心理念是，平方运算会引入更多的组合信息，取中间部分可以“削弱”原始数字的某些集中性，提高地址分布的均匀性。

关键字	平方	中间两位（作为地址）	散列地址
1234	1522756	22	22
4321	18671041	71	71
2468	6090624	90	90
1111	1234321	34	34

处理冲突

开放地址法

再散列法

链地址法

处理冲突-开放地址法

当一个关键字的散列地址已经被占用时，通过某种探测方式，寻找下一个空位，直到找到可以插入的位置为止。可以把整个散列表看作一个连续的“停车位”或“单元房”，每个关键字要停放或入住时，会先查自己的“预定位置”（散列函数给出的地址）：

如果那一格空着，顺利入住；

如果被占了，就换个位置继续试，直到找到空位为止。

开放地址法-线性探测法

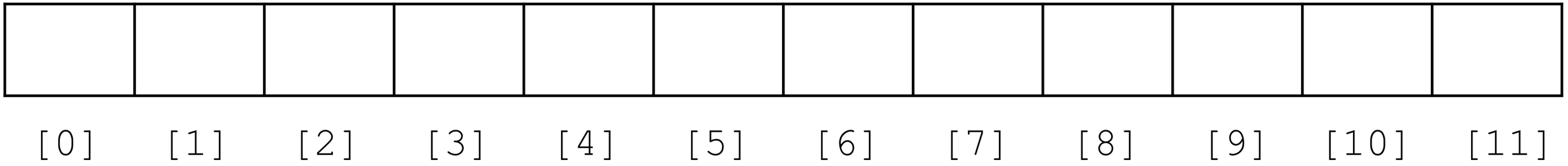
线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

```
{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}
```

```
H(key) = key % 12
```



开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

12	25			16			67	56			
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

37

12	25			16			67	56			
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

```
{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12
```

12	25	37		16			67	56			
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

12	25	37	15	16	29		67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

48

12	25	37	15	16	29		67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

48

12	25	37	15	16	29		67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

48

12	25	37	15	16	29		67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

48

12	25	37	15	16	29		67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

48

12	25	37	15	16	29		67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-线性探测法

线性探测法可以用该公式，Hi = (H(key) + i) % m (i = 1, 2, ..., m-1)，其中 H(key) 为散列表函数，m 为散列表长度，i 是一个增量序列，取值范围为 0～m-1。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

12	25	37	15	16	29	48	67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-二次探测法

二次探测法可以用该公式，$Hi = (H(key) + i^2) \% m$ $(i = 1, -1, 2, -2..., m/2, -(m/2))$，其中 H(key)为散列表函数，m 为散列表长度，i 是一个增量序列。

开放地址法-二次探测法

二次探测法可以用该公式，Hi = (H(key) + i^2) % m (i = 1, -1, 2, -2..., m/2, -(m/2))，其中 H(key)为散列表函数，m 为散列表长度，i 是一个增量序列。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

12	25	37	15	16	29	48	67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-二次探测法

二次探测法可以用该公式，Hi = (H(key) + i^2) % m (i = 1, -1, 2, -2..., m/2, -(m/2))，其中 H(key)为散列表函数，m 为散列表长度，i 是一个增量序列。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

34

12	25	37	15	16	29	48	67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-二次探测法

二次探测法可以用该公式，Hi = (H(key) + i^2) % m (i = 1, -1, 2, -2..., m/2, -(m/2))，其中 H(key)为散列表函数，m 为散列表长度，i 是一个增量序列。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

34

12	25	37	15	16	29	48	67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-二次探测法

二次探测法可以用该公式，Hi = (H(key) + i^2) % m (i = 1, -1, 2, -2..., m/2, -(m/2))，其中 H(key)为散列表函数，m 为散列表长度，i 是一个增量序列。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

34

12	25	37	15	16	29	48	67	56		22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-二次探测法

二次探测法可以用该公式，Hi = (H(key) + i^2) % m (i = 1, -1, 2, -2..., m/2, -(m/2))，其中 H(key)为散列表函数，m 为散列表长度，i 是一个增量序列。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12

12	25	37	15	16	29	48	67	56	34	22	47
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

开放地址法-伪随机探测法

伪随机探测法在每次冲突时，不是线性地、也不是按固定平方规律，而是借助一个伪随机序列来跳跃地探测下一个地址。

$$Hi = (H(key) + i) \% m \quad ， i 是一个随机数$$

处理冲突-再散列法

当多个关键字经过同一个散列函数计算后出现冲突时，我们可以引入多个不同的散列函数，在发生冲突时依次使用备用的散列函数重新计算新的地址。这种处理冲突的方法称为再散列法。

处理冲突-链地址法

将所有散列到同一个地址的关键字，用一个链表（或其它结构）连接起来，每个散列表元素指向一条链表。

处理冲突-链地址法

将所有散列到同一个地址的关键字，用一个链表（或其它结构）连接起来，每个散列表元素指向一条链表。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

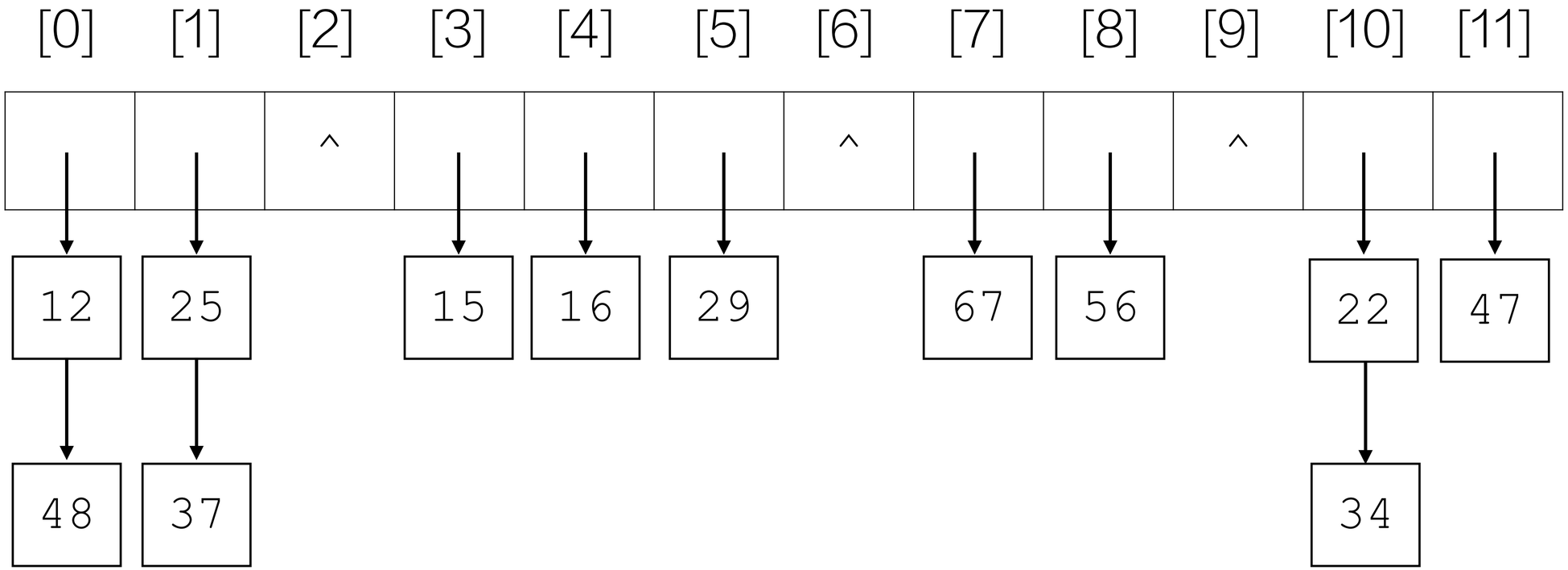
H(key) = key % 12

处理冲突-链地址法

将所有散列到同一个地址的关键字，用一个链表（或其它结构）连接起来，每个散列表元素指向一条链表。

{12, 67, 56, 16, 25, 37, 22, 29, 15, 47, 48, 34}

H(key) = key % 12



散列表查找流程

1. 计算散列地址

对待查关键字 key，使用事先设定好的散列函数 H(key)，计算其散列地址。

2. 访问该地址

如果该地址上存储的就是 key，那么查找成功；

如果该地址为空，说明 key 不存在，查找失败；

如果该地址上是另一个关键字，说明发生了冲突，必须进行“冲突处理”。

3. 冲突处理查找下一个地址

如果找到了目标关键字，则查找成功；

如果查到某个位置为空，或所有探测范围内都查不到，则查找失败。

影响散列表查找性能的因素

散函数是否均匀

处理冲突的方法

散列表的装填因子

影响散列表查找性能的因素

散函数是否均匀

处理冲突的方法

散列表的装填因子

$$\text{装填因子} = \frac{\text{已插入元素个数}}{\text{散列表的长度（槽位总数）}}$$

影响散列表查找性能的因素

散函数是否均匀

处理冲突的方法

散列表的装填因子

$$\text{装填因子} = \frac{\text{已插入元素个数}}{\text{散列表的长度（槽位总数）}}$$

装填因子越大，产生冲突可能性就越大

真题解析

[2011] 9. 为提高散列（Hash）表的查找效率，可以采取的正确措施是_____。

I. 增大装填（载）因子

II. 设计冲突（碰撞）少的散列函数

III. 处理冲突（碰撞）时避免产生聚焦（堆积）现象

A. 仅 I　　B. 仅 II　　C. 仅 I、II　　D. 仅 II、III

真题解析

[2011] 9. 为提高散列（Hash）表的查找效率，可以采取的正确措施是_____。

I. 增大装填（载）因子

II. 设计冲突（碰撞）少的散列函数

III. 处理冲突（碰撞）时避免产生聚焦（堆积）现象

A. 仅 I　　B. 仅 II　　C. 仅 I、II　　D. 仅 II、III

真题解析

[2014] 8. 用哈希（散列）方法处理冲突（碰撞）时可能出现堆积（聚焦）现象。下列选项中，会受堆积现象影响的是_____。

A. 存储效率

B. 散列函数

C. 装填（装载）因子

D. 平均查找长度

真题解析

[2014] 8. 用哈希（散列）方法处理冲突（碰撞）时可能出现堆积（聚焦）现象。下列选项中，会受堆积现象影响的是_____。

A. 存储效率

B. 散列函数

C. 装填（装载）因子

D. 平均查找长度

真题解析

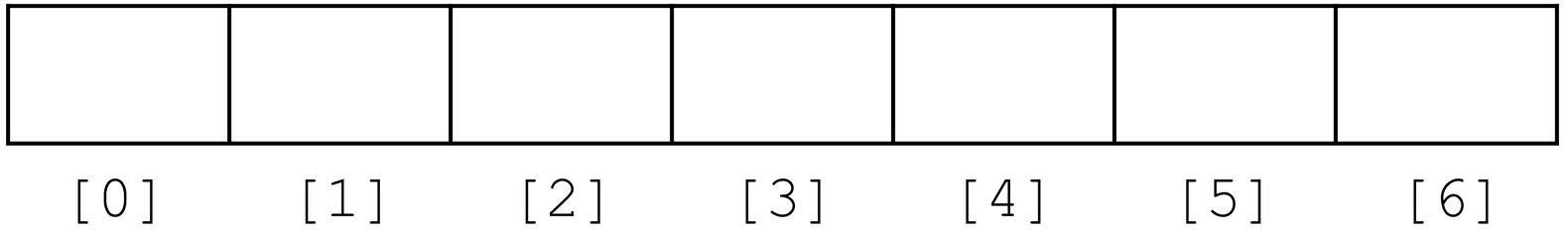
[2018] 9. 现有长度为7、初始为空的散列表 HT，散列函数 H(k) = k % 7，用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入到 HT 后，查找成功的平均查找长度是_____。

A. 1.5　　B. 1.6　　C. 2　　D. 3

真题解析

[2018] 9. 现有长度为7、初始为空的散列表 HT，散列函数 H(k) = k % 7，用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入到 HT 后，查找成功的平均查找长度是_____。

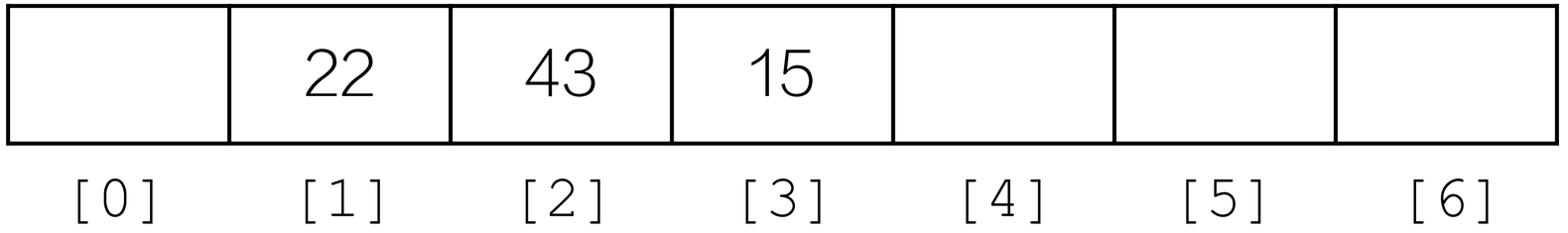
A. 1.5　　B. 1.6　　C. 2　　D. 3



真题解析

[2018] 9. 现有长度为7、初始为空的散列表 HT，散列函数 H(k) = k % 7，用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入到 HT 后，查找成功的平均查找长度是_____。

A. 1.5　　B. 1.6　　C. 2　　D. 3

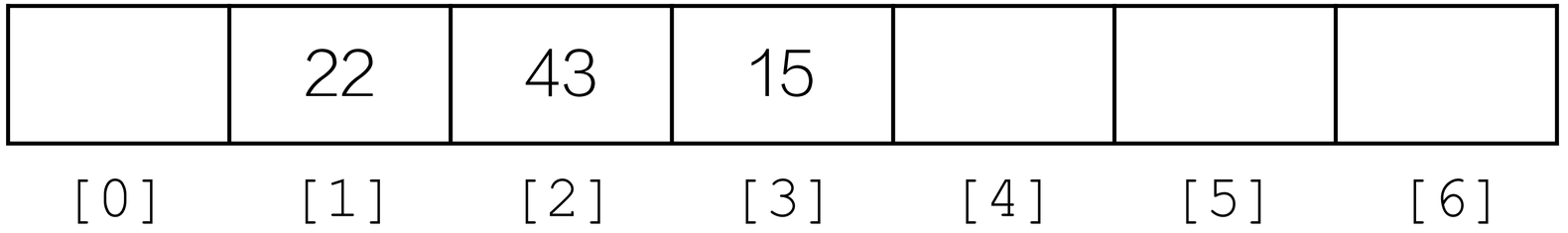


真题解析

[2018] 9. 现有长度为7、初始为空的散列表 HT，散列函数 H(k) = k % 7，用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入到 HT 后，查找成功的平均查找长度是_____。

A. 1.5　　B. 1.6　　C. 2　　D. 3

(1+2+3)/3 = 2



真题解析

[2018] 9. 现有长度为7、初始为空的散列表 HT，散列函数 H(k) = k % 7，用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入到 HT 后，查找成功的平均查找长度是_____。

A. 1.5　　B. 1.6　　C. 2　　D. 3

真题解析

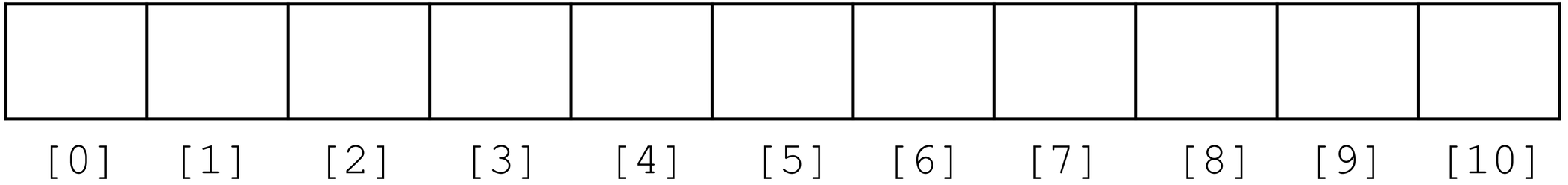
[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

A. 4　　B. 5.25　　C. 6　　D. 6.29

真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

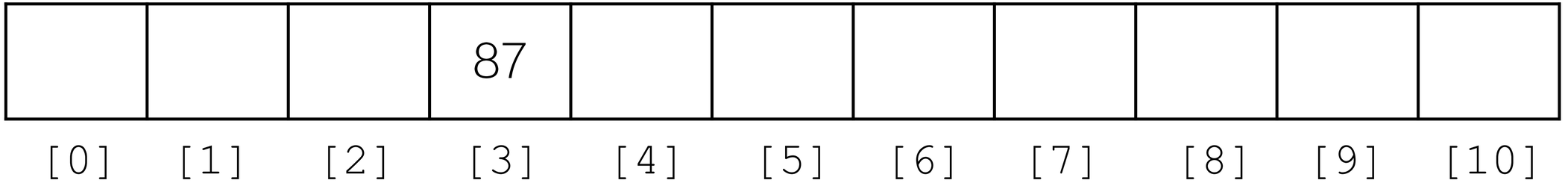
A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

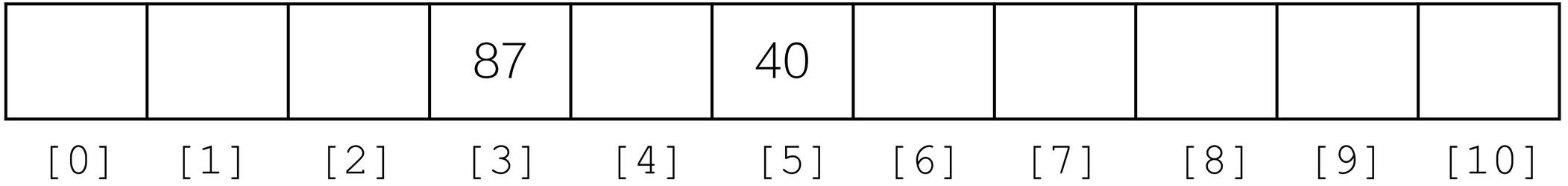
A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

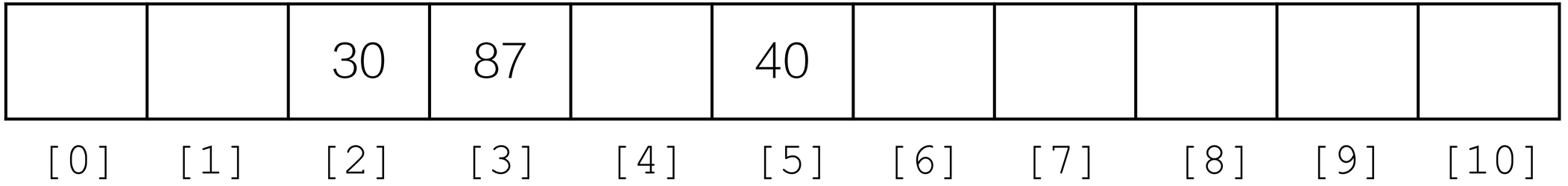
A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

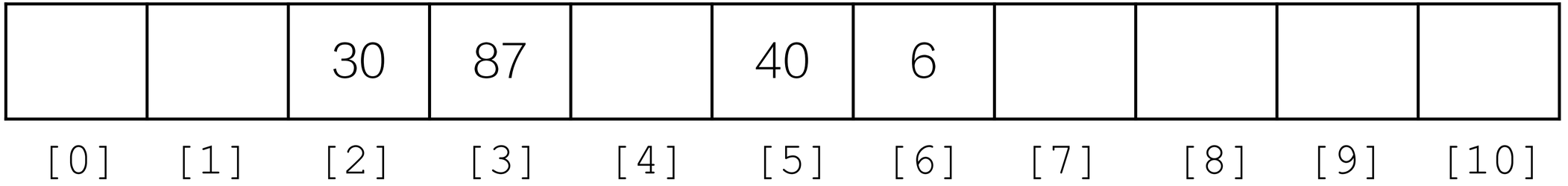
A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

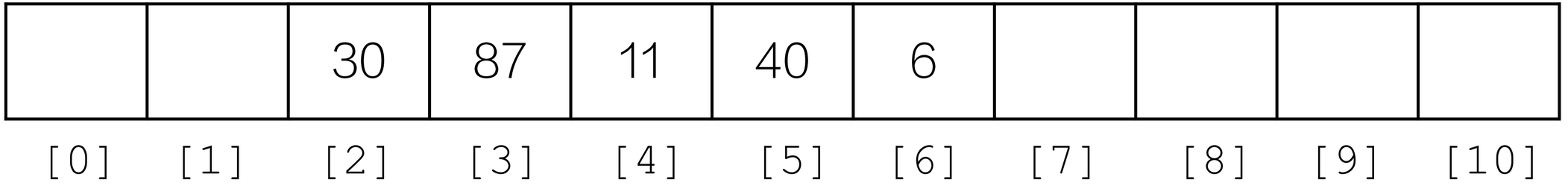
A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

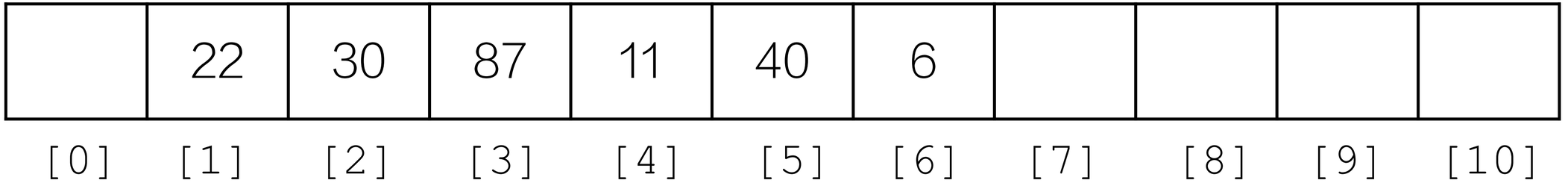
A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

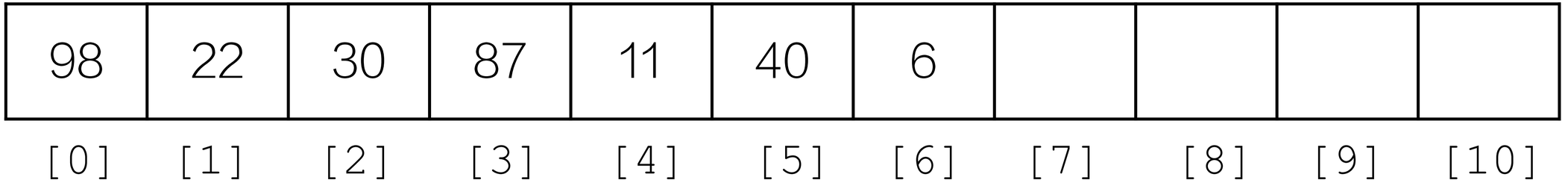
A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

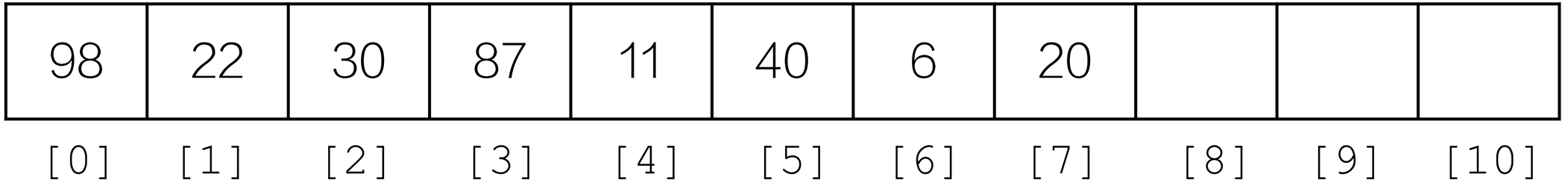
A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

A. 4　　B. 5.25　　C. 6　　D. 6.29



真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

A. 4　　B. 5.25　　C. 6　　D. 6.29

(9+8+7+6+5+4+3)/7 = 6

98	22	30	87	11	40	6	20			
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]

真题解析

[2019] 8. 现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key) = key % 7，采用线性探查（线性探测再散列）法解决冲突。将关键字 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是_____。

A. 4　　B. 5.25　　C. 6　　D. 6.29